BAYMAX

A locally-runnable personal-task agent with an eval harness. Stepping-stone learning project — shipped as portfolio evidence of engineering + methodology discipline, not as a daily-driver assistant or research contribution.

Status: Closed out by the project owner on 2026-10-17 as a stepping-stone learning project. The final local fine-tune records 73/150 task successes on the development suite. Fake adapters only; not a production assistant.

See docs/PROJECT.md for the full "what BAYMAX is and isn't."

What it does

1. Accepts text input describing a productivity task
2. Selects and executes tool calls against **fake** in-memory adapters (calendar / notion / gmail / clipboard)
3. Confirms, refuses, or requests clarification
4. Logs actions with trace / task identifiers
5. Runs against a scripted eval suite; produces metrics JSON

What it doesn't do (honest scope)

- **No real API integration** — fake adapters only. Not connected to your real Google Calendar / Gmail / Notion.
- **Not a daily-driver assistant** — response quality too low to trust for real actions
- **No menu bar app / voice / chat UI** — REST endpoint only (`POST /invoke`)
- **No context memory across requests** — each request stateless
- **No streaming, no multi-turn dialog**

Full list in docs/DEFERRED.md.

Team

Name	Owned modules
Marv	Agent core, inference service, backends (OpenAI + MLX), output adapter, telemetry
Ronin	Eval harness, training pipeline, tool adapters (fake), scenario authorship

Full ownership rules: docs/OWNERSHIP.md.

Documentation

Doc	Purpose
docs/PROJECT.md	What BAYMAX is (and isn't) — the framing doc
docs/ARCHITECTURE.md	System design
docs/OWNERSHIP.md	Module ownership + PR rules
docs/ROADMAP.md	Timeline to ship (~5 weeks from 2026-08-24)
docs/BACKEND_FLOW.md	Backend + agent design diagrams
docs/BENCHMARKING.md	Eval methodology + results
docs/SCORING.md	Deterministic scoring rules
docs/DEFERRED.md	Known limitations + not-implemented items
docs/decisions/	ADR archive
docs/experiments/README.md	Experiment history, settings, results and limitations
Experiment notebook PDF	Combined experiment reports with internal navigation
Complete documentation PDF	Public project documents and experiment reports

Quick start

Install dependencies:

```
uv sync --frozen --all-groups
```

Run checks:

```
uv run ruff format --check
uv run ruff check .
uv run pyright
uv run pytest
```

Run the scripted baseline:

```
python -m baymax.eval.cli run-scripted \
  --scenarios scenarios/v1 \
  --responses scenarios/v1/scripted_responses/v1-scripted.json \
  --output results/phase1/scripted/v1-baseline.json
```

Run the OpenAI agent baseline:

```
export OPENAI_API_KEY="your_api_key"
python -m baymax.eval.cli run-agent-openai \
  --scenarios scenarios/v1 \
  --output results/phase1/openai/gpt-4o-mini-full.json \
  --model gpt-4o-mini
```

Phase 1 result files are committed under `results/`.

Final local evaluation (150 development scenarios)

Local checkpoint	Task successes / 150	Routing macro-F1	Invalid outputs / 150
Unweighted name-first reference, 1k / 2 epochs	57	0.612	1
Corrected grounding, 1k / 2 epochs	53	0.591	0
Expanded coverage, 2k / 2 epochs	64	0.615	2
Expanded coverage, 2k / 4 epochs	73	0.681	0

Source: saved four-epoch evaluation. All four use compact JSON, greedy decoding and the same combined v1/v2 suite. This is single-seed development evidence after repeated evaluation, not an untouched test set or proof of statistical significance. The final run correctly routes 17/40 clarification cases and 27/29 refusal cases; clarification remains a weakness. Its recorded mean latency is 5,652 ms, versus 2,758 ms for the two-epoch 2k run. Hardware conditions were not controlled sufficiently to attribute that difference to epochs alone. Reported API cost is zero, not zero hardware/electricity cost.

Recipes, decision matrices and qualifications are in the four-epoch experiment report. Generated training data, adapter weights and private local files are not included in Git. The committed config pins the dataset revision; reproducing or serving the local model requires generating the data and obtaining/training its adapter.

Historical Phase 1 baselines (50 scenarios)

Backend	task_success_rate	scenarios
Scripted baseline	see `results/phase1/scripted/v1-baseline.json`	50
gpt-4o-mini	**0.52**	50
Qwen 1.5B + LoRA (smoke test)	0.24	50

These historical 50-case results are not directly comparable to the final 150-case local evaluations above. No claim that the fine-tune beats a frontier model is made.

License

MIT — Marv and Ronin, 2026.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

ARCHITECTURE — SCAFFOLD ONLY

Status: SCAFFOLD. Sections below contain prompts and TODO blocks. **You must fill these in yourselves.**

Per Tadashi CV non-negotiable rule on AI usage:

> Do not let AI own: project architecture, core algorithms, debugging first-pass thinking, data model design for important systems, performance reasoning, robotics or AI control logic you plan to claim as your strength.

This file is your responsibility. I have left the section headers, prompts, and reference patterns. The actual decisions, diagrams, schemas, and tradeoff reasoning must be written by Marv (Person 1 sections) and Ronin (Person 2 sections). Co-owners review.

When you fill in a TODO, delete the TODO marker and the surrounding prompt.

---

1. System overview

> **TODO (both):** One-paragraph plain-English description of what the system does, who uses it, and what makes it nontrivial. No marketing language. If you can't explain it without saying "AI-powered" or "intelligent," rewrite.

> **TODO (both):** Insert a high-level block diagram (Excalidraw → committed as PNG + `.excalidraw` source under `docs/diagrams/`). Show: client → service → agent core → inference + tool adapters, with telemetry instrumenting everything.

---

2. Component boundaries

2.1 Agent core (Marv)

> **TODO (Marv):** Describe the agent core's responsibilities in 3-5 bullets. What does it own? What does it explicitly *not* own?

> **TODO (Marv):** Document the agent loop as pseudocode (≤30 lines). Reference patterns to consider, not copy:
>
> - ReAct (reason-act-observe loop)
> - Plan-and-execute (LLM produces plan upfront, executor runs it)
> - Tool-use chat loop (OpenAI/Anthropic native tool-use API style)
>
> Pick one. Write down *why* you picked it. Name 2 alternatives you rejected and *why*.

2.2 Inference service (Marv)

> **TODO (Marv):** Describe the inference service's role and the request lifecycle. What gets batched? What's the cancellation story? Where does streaming begin and end?

> **TODO (Marv):** Document the backend-swapping interface. The same agent must run against OpenAI API, Anthropic API, or local MLX model with no agent-core code changes. Sketch the interface.

2.3 Telemetry (Marv)

> **TODO (Marv):** What gets logged at each layer? What's a trace? What's a span? Where does cost get computed? How do you correlate a user-visible failure with the underlying tool error?

2.4 Eval harness (Ronin)

> **TODO (Ronin):** Define the eval harness in 3-5 bullets. What's an "evaluation run"? What does the runner produce? How are baselines compared?

2.5 Model training pipeline (Ronin)

> **TODO (Ronin):** Describe the training pipeline stages: data extraction → format conversion → train/eval split → SFT run → checkpoint → registry. Where does leakage get prevented?

2.6 Tool adapters (Ronin)

> **TODO (Ronin):** List the tools, their auth model, their rate limits, their idempotency story. What does the adapter interface look like? How does the agent core discover available tools?

---

3. Core contracts (the most important section)

3.1 Tool-call contract

> **TODO (Marv):** Design the JSON schema for a tool call. Must support:
>
> - tool identifier
> - arguments (typed, validated)
> - expected state change (so eval can verify)
> - idempotency key
> - timeout / retry policy
>
> Commit the schema as `src/baymax/core/schemas/tool_call.schema.json`. Write it before any code uses it. The schema is the contract.

3.2 Scenario format

> **TODO (Ronin):** Design the JSON schema for an eval scenario. Must support:
>
> - user input (text, optional audio path)

- initial world state (fixtures for Notion/Calendar/etc.)
- expected outcome (state predicates, not literal tool-call sequences — multiple valid sequences must score correctly)
- ambiguity flag (if true, runner uses LLM-as-judge instead of exact match)
- tags (task type, difficulty, tools involved)

Commit as `src/baymax/eval/schemas/scenario.schema.json`.

3.3 Trace format

> **TODO (Marv):** Decide: OpenTelemetry spec, or custom? Document the decision. If custom, justify against OTel's overhead.

3.4 Model registry interface

> **TODO (Ronin with Marv review):** How is a model identified? Versioned? Loaded? How does the inference service ask the registry for "the best v1 fine-tuned model"?

---

4. Data flow

> **TODO (both):** Walk through one example end-to-end. User says "remind me about the article I just copied at 9am tomorrow." Trace it through:
>
> 1. ASR (if voice) or text input
> 2. Agent core receives intent
> 3. Tool selection (clipboard read → calendar create)
> 4. Tool execution
> 5. State change confirmation
> 6. Response to user
> 7. Telemetry written
>
> Be specific about where each component sits. Be specific about where failure can happen at each step.

---

5. State management

> **TODO (Marv):** Where does session state live? Per-process memory? Redis? SQLite? Justify the choice against the v1 requirements (single-user, multi-turn within session). Plan for v2 (multi-session memory).

---

6. Failure modes

> **TODO (Marv, with Ronin input from eval failures):** Enumerate failure modes and the agent's response. At minimum:

> - tool returns 4xx (user error in args)
> - tool returns 5xx (transient)
> - tool times out
> - model hallucinates a tool that doesn't exist
> - model produces malformed tool args
> - model refuses when it shouldn't
> - model proceeds when it should clarify
> - rate limit hit
> - auth expired
>
> For each: classification, response, telemetry, recovery action.

---

7. Security and privacy

> **TODO (Marv):** This system has access to your Notion/Gmail/Calendar. Document:
>
> - Where credentials live (env vars, OS keychain, never in source)
> - Whether traces redact sensitive content (yes, they must)
> - Whether the trained model could leak training-data PII into outputs (think about this before fine-tuning on your own emails)
> - What happens to scenario fixtures (don't commit real Notion content to the repo)

---

8. Performance budget

> **TODO (Marv):** Set the latency budget per stage. Suggested starting points to argue with:
>
> - Total user-perceived latency: P95 < 3000 ms for text, < 4000 ms for voice
> - ASR (if voice): P95 < 800 ms
> - Model first token: P95 < 1200 ms
> - Tool execution: P95 < 1000 ms
> - Streaming token rate: ≥ 30 tok/s on M-series Mac
>
> Justify each number. Numbers without justification are fake metrics (CV violation).

---

9. Decisions log (ADRs)

Every significant decision documented as a short ADR (Architecture Decision Record) at `docs/decisions/NNNN-title.md`:

- `0001-agent-loop-pattern.md` — Marv
- `0002-tool-call-schema.md` — Marv

- `0003-scenario-format.md` — Ronin
- `0004-base-model-choice.md` — Ronin
- `0005-inference-backend.md` — Marv
- `0006-trace-format.md` — Marv
- `0007-state-store.md` — Marv
- `0008-eval-judge-protocol.md` — Ronin

Each ADR ≤300 words: context, decision, alternatives considered, consequences. Use the Michael Nygard ADR template as reference.

---

10. Out of scope (anti-architecture)

> **TODO (both):** List what this system explicitly does not do, and why. This is as important as the in-scope list. See PROBLEM.md §"What BAYMAX v1 is NOT" — expand here.

How to use this scaffold

1. Each person fills in the sections they own. Co-owner reviews.
2. Sections 1, 4, 10 are filled jointly.
3. Section 3 (contracts) must be filled *before* Phase 1 code starts. Contracts before code.
4. Section 6 (failure modes) is updated continuously as eval surfaces real failures.
5. Section 9 (ADRs) grows organically — one ADR per significant decision.
6. When this scaffold has no more TODO blocks, delete this "How to use" section.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Backend + Agent Design Flow (post-ADR-0011 target)

Purpose: Reference diagrams for the backend interface v2 + agent core design decided during the ADR 0011 walkthrough (2026-08-21).

Scope note: This document shows the **target design** after ADR 0011 + ADR 0012 ship. The current Phase 1 codebase reflects an earlier state (OpenAIBackend only, no registry, simpler warmup semantics). Diagrams here are the shape you're building TO, not the shape currently deployed.

Related:

- ADR 0011 (backend interface v2 — to write)
- ADR 0012 (model registry — to write, co-designed with 0011)
- METHODOLOGY.md
- ARCHITECTURE.md

---

1. Component composition

The system's structural layering. Shows what holds what.

```
flowchart TB
    subgraph FastAPI["FastAPI Application"]
        API["/invoke endpoint"]
        HEALTH["/health endpoint"]
        LIFESPAN["lifespan()<br/>startup + shutdown"]
    end

    subgraph AGENT_LAYER["Agent Layer (per-request)"]
        AGENT["Agent<br/>handle_request()"]
        VAL["Validator"]
        DISP["FakeAdapterDispatcher"]
        TEL["TelemetryLogger<br/>+ CostAccumulator"]
    end

    subgraph BACKEND_LAYER["Backend Layer (singleton)"]
        BASE["InferenceBackend ABC<br/>plan_task() / interpret_results()<br/>_call_llm() with retry"]
        OAI["OpenAIBackend"]
        MLX["MLXBackend"]
        FAKE["FakeBackend<br/>tests only"]
    end
```

```
    subgraph SUPPORT["Support"]
        REG[("models.json<br/>registry manifest")]
        WARM[("warmup_scenario.json")]
        ENV[("env
vars<br/>BAYMAX_BACKEND<br/>BAYMAX_MODEL_ID<br/>BAYMAX_RETRY_MAX_ATTEMPTS<br/>OPENAI_API_KEY")]
        ADAPTERS[("Ronin's fake adapters<br/>calendar / notion / gmail / clipboard")]
    end

    LIFESPAN -->|instantiates| OAI
    LIFESPAN -->|instantiates| MLX
    LIFESPAN -->|owns singleton| BASE
    LIFESPAN -->|reads at construction| ENV
    LIFESPAN -->|reads at construction| REG
    LIFESPAN -->|reads warmup prompt| WARM

    API -->|per request| AGENT
    AGENT --> VAL
    AGENT --> DISP
    AGENT --> TEL
    AGENT -->|calls| BASE

    OAI -.-> BASE
    MLX -.-> BASE
    FAKE -.-> BASE

    DISP --> ADAPTERS

    HEALTH -->|reads warmup state| LIFESPAN

    classDef ext fill:#e8f0ff,stroke:#4a6faf,color:#000
    classDef support fill:#fff4e0,stroke:#c48800,color:#000
    class FastAPI ext
    class SUPPORT support
```

Key composition rules (from A/B/G locks):

- Agent HOLDS backend (dependency injection); Agent does not KNOW backend type
- Backend HOLDS registry entry (registry-driven metadata per G1)
- FastAPI lifespan OWNS the singleton backend instance
- Everything reads env vars in `__init__` (per B4); no global env access elsewhere

---

2. Startup lifecycle

The sequence from `uvicorn baymax.service.api:app` to `/health` returning 200.

```
flowchart TD
    START([uvicorn starts])
    IMPORT[Import module<br/>side-effect free]
    LIFESPAN_ENTER[lifespan begins]

    ENV_READ[Read BAYMAX_BACKEND<br/>+ BAYMAX_MODEL_ID env]
    ENV_VALID{Env valid?}
    ENV_FAIL[Raise ValueError<br/>uvicorn exits]

    FACTORY[BACKENDS dict lookup<br/>OpenAIBackend or MLXBackend]

    REG_READ[Load models.json<br/>fetch model_id entry]
    REG_VALID{Entry valid?<br/>required fields present?}
    REG_FAIL[Raise ModelNotFound / IncompleteRegistryEntry<br/>uvicorn exits]

    BACKEND_INIT[Backend __init__<br/>fetches OPENAI_API_KEY or MLX_MODEL_PATH<br/>raises loud on missing]
    BACKEND_FAIL{Init succeeded?}
    BACKEND_FAIL_EXIT[Raise ConfigError<br/>uvicorn exits]

    WARM_LOAD[Read warmup_scenario.json]
    WARM_RUN[Execute dummy scenario<br/>full backend call incl. tool-call generation]
    WARM_TIMEOUT{Completed within 120s?}
    WARM_TIMEOUT_FAIL[Raise WarmupTimeout<br/>uvicorn exits]
    WARM_ERR{Warmup succeeded?}
    WARM_ERR_FAIL[Raise WarmupFailure with stage+cause+hint<br/>uvicorn exits]

    READY[Set warmed_up = True<br/>Log 'backend ready' event with is_warmup=true tag]
    YIELD[lifespan yields<br/>server accepts requests]

    HEALTH_200[/health returns 200<br/>with backend metadata JSON]

    START --> IMPORT
    IMPORT --> LIFESPAN_ENTER
    LIFESPAN_ENTER --> ENV_READ
    ENV_READ --> ENV_VALID
    ENV_VALID -- no --> ENV_FAIL
    ENV_VALID -- yes --> FACTORY
    FACTORY --> REG_READ
    REG_READ --> REG_VALID
    REG_VALID -- no --> REG_FAIL
    REG_VALID -- yes --> BACKEND_INIT
    BACKEND_INIT --> BACKEND_FAIL
    BACKEND_FAIL -- no --> BACKEND_FAIL_EXIT
    BACKEND_FAIL -- yes --> WARM_LOAD
    WARM_LOAD --> WARM_RUN
    WARM_RUN --> WARM_TIMEOUT
    WARM_TIMEOUT -- no --> WARM_TIMEOUT_FAIL
```

```
    WARM_TIMEOUT -- yes --> WARM_ERR
    WARM_ERR -- no --> WARM_ERR_FAIL
    WARM_ERR -- yes --> READY
    READY --> YIELD
    YIELD --> HEALTH_200

    classDef fail fill:#ffcdd2,stroke:#c62828,color:#000
    classDef success fill:#c8e6c9,stroke:#2e7d32,color:#000
    class ENV_FAIL,REG_FAIL,BACKEND_FAIL_EXIT,WARM_TIMEOUT_FAIL,WARM_ERR_FAIL fail
    class READY,HEALTH_200 success
```

Failure semantics (per B5, C3, C4):

- All startup errors = process exit before `lifespan` yields
- `/health` never reaches ready state; uvicorn logs the error and dies
- No retry on warmup failures (deterministic errors, fail-loud policy)

Health response body (per C2):

```
// During warmup (503):
{"ready": false, "reason": "warming_up"}

// After warmup (200):
{"ready": true, "backend": "mlx", "model_id": "qwen-1.5b-baymax-v1"}
```

---

3. Request flow (happy path)

Single request from user through agent to response.

```
sequenceDiagram
    participant U as Client
    participant API as /invoke endpoint
    participant AG as Agent.handle_request
    participant V as Validator
    participant B as Backend
    participant D as Dispatcher
    participant T as TelemetryLogger

    U->>API: POST /invoke {request}
    API->>AG: handle_request(payload)

    Note over AG,T: telemetry BEGINS before validation

    AG->>T: log request_received
    AG->>V: validate_task_file(task)
    V-->>AG: valid TaskFile
```

```
    AG->>B: plan_task(task) [wrapped in _call_llm]
    Note over B: retry loop D1-D4 if needed
    B-->>AG: PlanResult(steps, tokens, cost, warnings)

    loop each step in plan
        AG->>V: validate_tool_call(step)
        V-->>AG: valid
        AG->>D: dispatch(step)
        D-->>AG: ToolCallResult
        AG->>T: log tool_call event
    end

    AG->>B: interpret_results(task, results)
    Note over B: retry loop D1-D4 if needed
    B-->>AG: interpretation

    AG->>AG: task.to_agent_response()
    AG->>T: log request_completed
    AG-->>API: AgentResponse
    API-->>U: 200 OK {AgentResponse}
```

Order guarantees (from ADR 0001, 0002):

- Telemetry always fires BEFORE validation (per Phase 1 lock — captures failed-task_file-construction events)
- Validator runs on both the initial TaskFile and each ToolCall step before dispatch
- Agent uses meta-tools (`request_clarification`, `refuse_request`) as normal tool calls — dispatched but with no side effects, sets response_type accordingly

Two distinct retry loops (do not confuse):

- **Backend retry** (§4 below): inside `_call_llm`, retries transient LLM-API failures (HTTP 408/429/500-504, connection errors). Hardcoded N=3 per ADR 0011 §D. Never retries MLX.
- **Tool-call retry (ReAct-style recovery)**: inside the agent loop, retries individual TOOL dispatches that fail per `AgentConfig.max_retries_per_step` (ADR 0001). Runs at a higher layer than backend retry — a single tool-call attempt may internally consume up to N=3 backend attempts, so worst-case a `max_retries_per_step=2` scenario can produce 2×3=6 LLM calls if planning also gets retried.

---

4. Retry loop internals (inside `_call_llm`)

The retry logic that lives on the base class per A4, implementing D1-D5.

```
flowchart TD
    ENTER[_call_llm called]
    INIT[attempt = 1<br/>max_attempts = 3 hardcoded]
    LOG_ATTEMPT[Log 'backend_attempt' event<br/>attempt=N is_warmup=false]
```

```
    CALL[Execute HTTP or MLX call]
    RESULT{Result type?}

    SUCCESS[Log 'backend_call_complete' aggregate event<br/>total_attempts=N final_status=success]
    RETURN[Return result to caller]

    ERR_CLASSIFY{Classify error}
    NON_RETRIABLE[HTTP 400/401/403/404/422<br/>OR any MLX error]
    RETRIABLE[HTTP 408/429/500-504<br/>OR connection error<br/>on API backend only]

    RETRIABLE_CHECK{attempt < max_attempts?}
    BACKOFF[delay = base * factor^attempt<br/>>= 1s * 2^attempt<br/>apply ±25% jitter]
    LOG_RETRY[Log 'backend_retry' event<br/>attempt=N delay_before_ms=X cause=Y]
    SLEEP[await asyncio.sleep delay]
    INCREMENT[attempt += 1]

    EXHAUSTED[All attempts exhausted]
    LOG_FINAL_FAIL[Log 'backend_call_complete' aggregate<br/>total_attempts=N
final_status=failed<br/>with full attempt history]
    RAISE_BE[Raise BackendError<br/>kind=retry_exhausted OR
non_retriable<br/>underlying=orig_exception<br/>attempts_history=list of attempts]

    ENTER --> INIT
    INIT --> LOG_ATTEMPT
    LOG_ATTEMPT --> CALL
    CALL --> RESULT

    RESULT -- ok --> SUCCESS
    SUCCESS --> RETURN

    RESULT -- error --> ERR_CLASSIFY
    ERR_CLASSIFY --> NON_RETRIABLE
    ERR_CLASSIFY --> RETRIABLE

    NON_RETRIABLE --> LOG_FINAL_FAIL
    LOG_FINAL_FAIL --> RAISE_BE

    RETRIABLE --> RETRIABLE_CHECK
    RETRIABLE_CHECK -- no more attempts --> EXHAUSTED
    EXHAUSTED --> LOG_FINAL_FAIL

    RETRIABLE_CHECK -- retry --> BACKOFF
    BACKOFF --> LOG_RETRY
    LOG_RETRY --> SLEEP
    SLEEP --> INCREMENT
    INCREMENT --> LOG_ATTEMPT
```

```
    classDef fail fill:#ffcdd2,stroke:#c62828,color:#000
    classDef success fill:#c8e6c9,stroke:#2e7d32,color:#000
    class RAISE_BE,LOG_FINAL_FAIL fail
    class SUCCESS,RETURN success
```

Backoff formula (per D2, `attempt` is 1-indexed and identifies the JUST-FAILED attempt):

```
delay = base_delay * (factor ** (attempt - 1))
      = 1s * (2 ** (attempt - 1))
      → 1s wait before retry 2, 2s wait before retry 3, 4s wait before retry 4
delay_with_jitter = delay * uniform(0.75, 1.25)
```

N = 3 hardcoded (2026-08-24 revision): earlier design had N configurable via env var + eval sweep at multiple values; trimmed as publication-review overhead per stepping-stone reframe.

---

5. Error surfacing → HTTP 500

How a `BackendError` (from retry exhaustion or non-retriable error) reaches the client. **Revised 2026-08-26**: prior version had this returning HTTP 200 with `error_type` on `AgentResponse` — that broke `AgentResponse`'s boundary contract with Ronin's scorer. Corrected below.

```
flowchart LR
    subgraph BACKEND
        BE[Backend raises BackendError<br/>kind=retry_exhausted OR non_retriable<br/>underlying=...
attempts=...]
    end

    subgraph AGENT
        BUBBLE[Agent does NOT catch<br/>internal
TaskFile.completion_status=FAILED<br/>TaskFile.error_type=BACKEND_ERROR<br/>full attempts logged to
action_log for forensics]
    end

    subgraph FASTAPI
        HANDLER[app.exception_handler BackendError<br/>registered in lifespan]
        BUILD500[Build 500 body:<br/>error backend_error<br/>kind retry_exhausted or
non_retriable<br/>backend provider<br/>message brief<br/>attempts count]
    end

    subgraph CLIENT
        RESP[HTTP 500 + JSON error body<br/>AgentResponse boundary intact / not returned]
    end

    BE --> BUBBLE
    BUBBLE --> HANDLER
    HANDLER --> BUILD500
```

```
    BUILD500 --> RESP

    classDef fail fill:#ffcdd2,stroke:#c62828,color:#000
    class BE,BUBBLE fail
```

Key semantic (per E2, revised): backend failure is infrastructure-layer, not an agent decision. HTTP status codes exist for exactly this distinction:

- **200 + AgentResponse** = agent made a decision (execute, refuse, clarify)
- **500 + error body** = infrastructure failed (backend down, retries exhausted, warmup incomplete)

`AgentResponse` boundary shape stays `{tool_calls, message}` unchanged — Ronin's scorer contract intact.

500 body shape:

```
{"error": "backend_error", "kind": "retry_exhausted", "backend": "openai",
 "message": "openai backend failed after 3 attempt(s): http_429",
 "attempts": 3}
```

No stack traces or underlying reprs in the body — those live in the trace file per ADR 0010 redaction discipline.

Ronin coordination: eval runner needs try/except around the httpx call and a "backend_error" bucket in scoring categorization. Flag in next sync.

---

6. Registry lookup path (per G1)

How a backend obtains its model metadata at construction.

```
flowchart LR
    START[Backend __init__ called<br/>model_id='qwen-1.5b-baymax-v1']

    LOAD[Open models/models.json]
    PARSE[Parse manifest]
    LOOKUP[Find entry where entry.id == model_id]
    FOUND{Entry found?}
    NOT_FOUND[Raise ModelNotFound<br/>uvicorn exits at startup]

    VALIDATE_REQ{All required fields present?<br/>id base adapter_path trained_at
training_config_hash}
    INVALID[Raise IncompleteEntry<br/>uvicorn exits]

    EXTRACT[Extract metadata:
<br/>base_model<br/>adapter_path<br/>max_context<br/>cost_model<br/>tokenizer_path]

    STORE[Store on self as instance attributes<br/>for lifetime of process]

    START --> LOAD
```

```
    LOAD --> PARSE
    PARSE --> LOOKUP
    LOOKUP --> FOUND
    FOUND -- no --> NOT_FOUND
    FOUND -- yes --> VALIDATE_REQ
    VALIDATE_REQ -- no --> INVALID
    VALIDATE_REQ -- yes --> EXTRACT
    EXTRACT --> STORE

    classDef fail fill:#ffcdd2,stroke:#c62828,color:#000
    class NOT_FOUND,INVALID fail
```

Consequence of registry-driven (per G1): adding a new fine-tuned model is a REGISTRY UPDATE, not a backend code change. Ronin appends to `models.json` + drops adapter weights → backend picks it up on next startup with `BAYMAX_MODEL_ID` pointing at the new id.

Immutability guarantee (per RONIN_SYNC B3): a given `model_id` NEVER has its metadata rewritten. Every retrain = new id (`v1` → `v2` → `v3`). Any prior eval result labeled with the old id stays reproducible.

---

7. Testing surface (per F)

Where fakes and mocks live and how they interact.

```
flowchart TB
    subgraph PRODUCTION
        PROD_APP[baymax.service.api:app<br/>with real backend from BACKENDS dict]
        PROD_BE[OpenAIBackend or MLXBackend]
    end

    subgraph UNIT_TESTS[Unit Tests]
        UT_AGENT[Test Agent.handle_request<br/>with FakeBackend]
        UT_RETRY[Test retry loop<br/>with FakeBackend + patched asyncio.sleep]
        UT_WARMUP[Test warmup gate<br/>with FakeBackend + warmup_delay_s]
        UT_ADAPTER[Test output adapter<br/>with hardcoded inputs]
        FAKE[FakeBackend<br/>tests/fakes/backend.py<br/>canned responses<br/>injectable delays]
    end

    subgraph INT_TESTS[Integration Tests]
        IT_OAI[Test real OpenAI round-trip<br/>opt-in via pytest.mark.integration]
        IT_MLX[Test real MLX round-trip<br/>Marv-local only, not in CI]
    end

    UT_AGENT --> FAKE
    UT_RETRY --> FAKE
    UT_WARMUP --> FAKE

    PROD_APP --> PROD_BE
```

```
    IT_OAI -.->|hits real API| PROD_BE
    IT_MLX -.->|hits local MLX| PROD_BE

    classDef test fill:#f0f4c3,stroke:#827717,color:#000
    class UNIT_TESTS,INT_TESTS test
```

Test tiers:

Tier	Runs when	Env
Unit tests (FakeBackend)	Every commit / PR	No env needed
Integration OpenAI	`[test-integration]` PR tag OR manual	`OPENAI_API_KEY` set
Integration MLX	Marv-local only, before pushing MLX changes	Mac with weights downloaded

Not scheduled: no nightly CI runs. Ahmed's eval runs naturally catch drift.

---

8. Cross-diagram invariants

Rules that hold across all flows:

1. **Backend uniformity** — Agent code has ZERO backend-conditional logic. Every backend implements identical semantics. Format differences absorbed inside backend implementations.
2. **Backend metadata exposed but not consumed by agent** — `backend.max_context`, `backend.cost_model` accessible for telemetry/debugging, NOT for agent branching.
3. **Registry is source of truth for model facts** — no duplication in backend code.
4. **Immutable model IDs** — `v1` weights + metadata never change. Retrain → new ID.
5. **Fail-loud on deterministic errors** — MLX errors, config errors, warmup errors all exit the process rather than retry.
6. **Retry only on network-transient errors** — API backends only. All retry attempts logged per-event AND aggregated on completion (dual telemetry per D5).
7. **Telemetry BEFORE validation** — request events fire before validator runs, so failed-task_file-construction events are captured.
8. **No agent-level backend awareness** — Agent gets a backend via dependency injection, treats it as opaque.

---

9. What's NOT in this design (still deferred)

Called out so they're not confused with the current design:

- **Streaming inference** (A5) — deferred to v2; new `plan_task_stream()` method when added, does not break v1 callers

- **State store beyond in-memory** — ADR 0007 downgraded to v2 target; no SQLite in v1
- **Multi-turn dialog state** — v2/v3 per DEFERRED.md
- **Real tool adapters** (Notion/Gmail/Calendar/Clipboard) — v3+; v1 uses Ronin's fake adapters
- **LLM-as-judge scoring** — ADR 0008 scaffold, implementation deferred
- **Third + fourth backend classes** (Anthropic, other MLX variants) — factory dict makes trivial to add; not v1 scope
- **Backend capability declaration via `.capabilities()`** — considered and rejected in favor of interface uniformity (§8 invariant #1). If a 4th+ backend surfaces genuine capability divergence, revisit.

Written: 2026-08-21 as reference design artifact from ADR 0011 walkthrough.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Benchmarking

This document defines how BAYMAX evaluation scenarios are written, scored, and reported.

Final Local Development Results (2026-10-17)

The owner closed out the stepping-stone project after these saved evaluations:

Run	Scenarios	Task successes	Macro-F1	Invalid outputs
Unweighted name-first reference, 1k / 2 epochs	150	57	0.612	1
Grounding v1, 1k / 2 epochs	150	53	0.591	0
Coverage v2, 2k / 2 epochs	150	64	0.615	2
Coverage v2, 2k / 4 epochs	150	73	0.681	0

The four-epoch raw result records ACT routing 62/81, CLARIFY 17/40 and REFUSE 27/29. All four runs use compact JSON and greedy generation on the combined v1/v2 suite through fake adapters. The expanded artifact has 1,952 usable training examples and 195 validation examples at context 1,024; the raw caps are 2,000 and 200.

These are single-seed, repeatedly inspected development measurements, not untouched-test estimates or statistically established gains. Do not compare the 150-case headline directly with a 50-case hosted baseline. Keyword-based question and reason scores remain distinct from routing macro-F1. Latency in the final run averaged 5,652 ms, versus 2,758 ms for its two-epoch counterpart, without a controlled hardware timing study. The local API cost field is not total operating cost. See the complete experiment report for settings, source links and the decision matrix. No evaluation target or historical result was changed for closeout.

Phase 1 starts with scripted scenarios and fake tool adapters so the evaluation harness is reproducible before any real APIs are introduced. The goal is to measure agent behaviour in a controlled environment before connecting BAYMAX to live Notion, Google Calendar, Gmail, or clipboard integrations.

Scenario Difficulty Labels

Each scenario must include one difficulty label:

- `explicit`: The user gives exact date, time, or task details directly.
- `implicit`: The user uses relative or natural phrasing that must be resolved using `current_time`, such as "tomorrow", "next Friday", or "in two hours".
- `contextual`: The agent must inspect `initial_state` to identify the correct entity or action.

- `ambiguous`: The agent should ask a clarification question instead of taking action.
- `multi_step`: The agent must call more than one tool to complete the task.

Canonical Decision Policy

Training examples and evaluation scenarios must apply the same decision policy. Decisions are evaluated in this order: `REFUSE`, then `CLARIFY`, then `ACT`. An earlier matching decision takes precedence over later ones.

Decision	Required conditions
`REFUSE`	The requested capability is unavailable, unsupported by the available tool schemas, out of scope, or unsafe. A related tool does not make the operation supported, and the agent must not substitute a different operation.
`CLARIFY`	The action is supported and safe, but required information is missing, ambiguous, conflicting, invalid, or cannot be resolved to one entity from `initial_state`. Clarification also applies when the confirmation policy below requires confirmation.
`ACT`	The action is supported and safe, every required argument can be resolved from the request, `current_time`, or `initial_state`, and no confirmation is required.

Examples of unsupported capability include requesting email search when only draft and send tools are available, or requesting deletion when the available tool can only update an entity. Confirmation does not make an unsupported or unsafe action valid; those requests remain refusals.

Optional Task Deadlines

Notion task creation permits an omitted due date. Bare urgency such as "create a task to revise a topic soon" permits ACT with an undated task; preserve the urgency in the task text rather than inventing a deadline. An explicit deadline request such as "due soon" or "due on the deadline we agreed" requires CLARIFY when the requested date cannot be grounded.

This distinction was confirmed on 2026-10-14 and is represented in synthetic generator version 15. Historical datasets, scenario targets and result scores are not rewritten or reinterpreted by that generator change.

Confirmation Policy

Confirmation is required before:

- a destructive or irreversible operation that an available tool actually supports, such as deleting or cancelling an existing entity; or
- an externally visible communication that states a consequential commitment or status change, such as withdrawing from a course.

Confirmation is not required solely because an action mutates state or communicates externally. A clear request to perform a routine send, create, update, clipboard read, or clipboard write is sufficient authorization when all required details are present. Drafting an email is not sending it and does not require confirmation.

Unsafe actions, impersonation, credential disclosure, bulk unsolicited messaging, and unavailable destructive operations must be refused rather than confirmed.

Scenario Files

Scenario fixtures live under `scenarios/v1/`.

The established combined benchmark is `scenarios/v1/` plus `scenarios/v2/` (150 scenarios). The separate `scenarios/template_shift_v1/` suite is a locked 30-scenario diagnostic for testing ACT, CLARIFY, and REFUSE behavior under new wording and hard keyword counterexamples. It must not be used to construct training examples, and its results must be reported separately from the established benchmark.

Each scenario must validate against:

```
src/baymax/eval/schemas/scenario.schema.json
```

Each scenario includes `current_time` so relative phrases like "tomorrow", "Friday", or "in two hours" resolve deterministically during eval runs.

Each scenario defines:

- the user input
- the current time
- the available tools
- the initial fake tool state
- the expected behaviour
- the success criteria

Phase 1 Metrics

The Phase 1 runner will report the following metrics:

Task Success

Whether the scenario reached the expected final outcome.

Tool-Call Accuracy

Whether the agent selected the correct tool for the scenario.

Argument Accuracy

Whether the agent supplied correct tool arguments, such as date, time, title, recipient, or duration.

Clarification Accuracy

For ambiguous scenarios, whether the agent asked for the required missing information instead of guessing or taking premature action.

Hallucination Rate

How often the agent invents unsupported actions, calls unavailable tools, claims success without tool execution, or fabricates state not present in the scenario.

Latency

The elapsed time for the scenario run, reported at minimum as total latency per scenario. Later runs may include P50 and P99 latency.

Cost

The estimated model/API cost per scenario and per benchmark run.

Phase 1 Output

The first scripted baseline output target is:

```
results/phase1/scripted/v1-baseline.json
```

The first OpenAI agent baseline output target is:

```
results/phase1/openai/gpt-4o-mini-full.json
```

This file should include:

- run ID
- model/backend name
- scenario count
- aggregate metrics
- per-scenario pass/fail results
- failure reasons where applicable
- timestamp
- benchmark configuration

Phase 1 Commands

Run the scripted baseline:

```
python -m baymax.eval.cli run-scripted `
  --scenarios scenarios\v1 `
  --responses scenarios\v1\scripted_responses\v1-scripted.json `
  --output results\phase1\scripted\v1-baseline.json
```

Run the OpenAI agent baseline:

```
$env:OPENAI_API_KEY="your_api_key"
python -m baymax.eval.cli run-agent-openai `
  --scenarios scenarios\v1 `
```

```
  --output results\phase1\openai\gpt-4o-mini-full.json `
  --model gpt-4o-mini
```

Run local verification:

```
uv run ruff format --check
uv run ruff check .
uv run pyright
uv run pytest
```

Phase 1 Baseline Results

The scripted baseline is expected to pass all 50 scenarios because it uses hand-authored responses that match the scenario fixtures.

The first OpenAI agent baseline was run against the same 50 scenarios:

```
backend: openai:gpt-4o-mini
scenario_count: 50
task_success_rate: 0.52
average_tool_call_accuracy: 0.82
average_argument_accuracy: 0.9083333333333333
average_clarification_accuracy: 0.5555555555555556
average_refusal_accuracy: 0.21428571428571427
average_hallucination_rate: 0.0
average_latency_ms: 2518.9
total_cost_usd: 0.0058130000000000005
```

The main baseline weakness is boundary judgment: the general agent often takes action when the expected behavior is to clarify, refuse, or wait for confirmation. That result is useful because BAYMAX's target local agent should be trained and evaluated against these domain-specific rules, not only generic tool-use ability.

Phase 2 Decision-Boundary Ablation

The following Qwen 2.5 1.5B runs used the same runtime prompt, LoRA settings, seed, one epoch, 1,000-row training cap, and evaluation runner. The controlled data changes altered which unique BAYMAX-control examples survived the cap. The established benchmark combines `scenarios/v1/` and `scenarios/v2/`.

Run	Success	Tool	Argument	Clarify	Refuse	Hallucination
Base Qwen	0.213	0.493	0.791	0.025	0.034	0.027
ACR-1k	0.287	0.567	0.761	0.050	0.172	0.020
Schema-diverse	0.280	0.560	0.752	0.075	0.190	0.007
ACT-balanced	0.313	0.553	0.805	0.025	0.086	0.010

ACT-balanced increased the fixed BAYMAX contribution from 300 to 400 unique rows: 200 ACT, 100 CLARIFY, and 100 REFUSE. This reduced missing tool calls from 34 to 22 relative to schema-diverse, but increased premature tool calls from 32 to 44. It produced the best established-suite task success and argument accuracy, while schema-diverse retained better clarification and refusal accuracy. Neither checkpoint dominates the other.

The locked 30-scenario template-shift diagnostic showed the same tradeoff:

Run	Success	Tool	Argument	Clarify	Refuse	Hallucination
Base Qwen	0.233	0.483	0.947	0.000	0.000	0.033
ACR-1k	0.200	0.600	0.800	0.200	0.150	0.000
Schema-diverse	0.167	0.533	0.733	0.300	0.200	0.000
ACT-balanced	0.200	0.533	0.828	0.200	0.100	0.000

Strict template-shift passes changed from 2/10 ACT and 3/10 CLARIFY for schema-diverse to 4/10 ACT and 2/10 CLARIFY for ACT-balanced. Both had 0/10 strict REFUSE passes. This indicates that changing the class mix shifts the model's action prior; it does not by itself teach a robust semantic decision boundary.

Refusal Audit

On the ten template-shift refusal scenarios, schema-diverse emitted three `refuse_request` calls, four `request_clarification` calls, two productive-tool responses, and one empty response. ACT-balanced emitted two refusals, three clarifications, four productive-tool responses, and one empty response. Every actual refusal matched only one of its scenario's two required reason keywords, so it received `0.5` refusal accuracy and did not count as task success.

These result artifacts predate explicit decision tracking. At that time, the local runner converted both meta-tools into a message and the scorer used only keyword overlap. New runs preserve `act`, `clarify`, or `refuse` as `AgentResponse.decision_type` and report decision accuracy separately from question/reason wording. Historical artifacts remain unchanged and report no decision-accuracy value.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

BAYMAX bug review

Reviewed on **2026-09-28**, against the current working tree in `project-baymax`, including its existing uncommitted changes. Base commit: `43640c5`, branch `feat/phase1-scenario-loader`.

This report contains **23 findings**, ordered from most important to least: **5 high**, **16 medium**, and **2 low**. Priority reflects the project's emphasis on trustworthy evaluation, reproducible training, and correct local execution. No critical finding was established.

The review covered active code under `src/baymax`, related training/evaluation scripts and configurations, tests, and relevant project documents/ADRs. The separate `project-baymax-policy-pr` checkout, archived work, and legacy `task_manager` application were outside this review. Intentionally deferred features, such as real integrations, conversational memory, rollback execution, and deeper adapter-outcome scoring, are not counted as bugs.

Evidence labels: “Reproduced” means a local test or controlled probe demonstrated the behavior. “Code inspection” means the execution path establishes the defect, but the affected full workload was not run. No hosted inference, model training, or benchmark reruns were performed. Application code and historical artifacts were not changed.

Fix status (2026-09-28)

- BUG-002: fixed duplicate-content/contrast-group splitting; generator version 11. Added validation of content disjointness. Corrected local data is at `data/sft/v1/baymax-control-disjoint`; historical data was preserved.
- BUG-003: local parsing now rejects incomplete output, trailing output, invalid calls/arguments and conflicting meta-tool calls atomically. `parse_error` and `invalid_model_output` prevent successful scoring of a salvaged subset. Saved generations were rescored under `results/phase2/parser-integrity-v1/`.
- BUG-006: the active control config now pins the corrected disjoint dataset. Historical two-epoch/mix recipes have not been migrated automatically.
- BUG-012: the configured seed is now applied before model/LoRA construction. This does not retroactively make earlier training runs reproducible.

The remaining findings below are not marked fixed by this update. The original review evidence and historical results remain applicable to the earlier code.

High priority

BUG-001 — Sensitive values bypass error redaction

Locations: telemetry/logger.py, lines 23–35 and 111; core/dispatcher.py, line 91; core/agent.py, lines 246–253 and 274–279; service/mlx_backend.py, exception handling in both generation methods; service/api.py, lines 188–195.

The logger redacts dictionary keys, but exception and validation-error text is already a string by the time it reaches that filter. Pydantic error messages can contain the original input, including nested credentials. MLX additionally embeds raw exception text in `BackendError.message`, which the HTTP handler returns to the caller.

Reproduced: a fake `clipboard.write` call with `{"text": {"password": "FAKE_SECRET_SENTINEL"}}` wrote the sentinel into tool-error trace events. An MLX generation stub raising an exception containing another synthetic sentinel preserved it in the client-facing error message. No actual credentials were used. Separately, fields named `auth`, `key`, and `bearer` are not redacted despite ADR 0010 requiring those patterns.

Impact: malformed input or backend failures can disclose sensitive values in traces and error responses.

Suggested fix: sanitize structured validation errors before converting them to text, omit input values from diagnostics, use safe public backend-error messages, and implement the credential-key patterns specified by ADR 0010. Test the actual error paths as well as ordinary event dictionaries.

BUG-002 — Exact training examples leak into synthetic validation and test splits

Location: models/baymax_sft.py, lines 2366–2387.

Splitting groups examples by `contrast_group` or source ID. Different groups can contain identical complete SFT examples, so those copies can enter different partitions.

Reproduced: running the current generator and splitter in memory with the default seed produced:

Measurement	Observed
Generated rows	1,000
Unique complete SFT examples	750
Validation rows identical to a training row	75 / 200
Test rows identical to a training row	43 / 100

These are identical system, user, and assistant messages, not just similar tasks. For example, `baymax_generated_refuse_119` in training and `baymax_generated_refuse_059` in test have identical SFT content.

Impact: validation loss and any evaluation using the synthetic test partition are contaminated by training examples. This finding does **not** establish leakage into the separately authored final benchmark scenarios or quantify a change in benchmark scores.

Suggested fix: keep both duplicate-content groups and contrast groups together during splitting, and validate content disjointness across partitions. Version the corrected dataset and preserve existing experiment artifacts.

BUG-003 — The local evaluation parser deletes invalid calls before scoring

Location: eval/lora_runner.py, lines 235–238 and 270–273, plus `_extract_json_candidate`.

When `AgentToolCall` validation fails, the parser catches the error and continues. Consequently, malformed tool names disappear from the scored response. Extraction also accepts the first JSON value without accounting for additional emitted JSON calls.

Reproduced: a correct calendar-create call followed by `{"name": "delete_all_events", "arguments": {}}` was reduced to the correct call alone. A local scorer probe returned `task_success=True`, `hallucination_rate=0`, and no failure reasons. Naming that same unsupported extra call `calendar.delete_all` preserved it and correctly

produced failure with hallucination rate `0.5`.

Impact: the parser can make worse model output score better by erasing evidence of invalid or hallucinated calls. This affects an already implemented metric, rather than the documented deferred hallucinated-entity checks.

Suggested fix: preserve a parse/validation failure when any emitted call is invalid and account for trailing structured output. Do not weaken the tool-name contract or silently salvage a successful subset.

BUG-004 — Tool-result values never reach the final response model

Location: core/agent.py, lines 313–320.

`_render_action_log_summary()` passes the names of returned fields to `interpret_results`, but discards their values. The interpretation request contains the original user request and this summary, with no other route to the result data.

Reproduced: reading a clipboard containing a unique synthetic string yielded only:

```
1. clipboard.read() — OK → returned 1 field(s): text
```

The clipboard content was absent from the interpretation request.

Impact: the final model cannot accurately answer what was read or report generated IDs and returned details. Both inference backends receive the incomplete summary.

Suggested fix: include bounded, safely serialized result values in the interpretation input. Add a test that checks returned content reaches the backend, rather than merely checking that a tool call succeeded.

BUG-005 — Repeating a training run can overwrite an existing adapter

Location: models/sft_training.py, lines 180–181 and 398–400; entry point `run_sft_lora_training`.

The adapter destination is always `<output_dir>/adapter`. The entry point has no existing-artifact guard; the save path explicitly uses `mkdir(..., exist_ok=True)` followed by `save_pretrained()` into the same directory. Reusing a configuration or output override therefore reuses the previous adapter's filenames.

Evidence: code inspection. The destination calculation and write path were inspected. No existing adapter was overwritten and no training was run to demonstrate data loss.

Impact: a routine rerun can replace the checkpoint underlying an earlier experiment, contrary to the project's immutable model-version requirement. A recorded path can then identify different weights.

Suggested fix: reject occupied experiment/adapter destinations before training, or require a fresh versioned destination. Keep any explicit resume behavior distinct from starting a new run.

Medium priority

BUG-006 — The control training configuration pins the wrong dataset revision

Location: qwen2_5_1_5b_lora_baymax_control_v1.json, line 7; models/sft_training.py, lines 121–127.

The control configuration pins revision `342d766b20d61ab5056c404a2a8b3ce51a1b5cde6226fd182d1fc97fcd3ffe8c`. The current generator produces `8774e47a275fa035f240665c50967a0e4029056683e0971e3afd802ea38b5587`, also present in the current local split manifest.

Reproduced: configuration validation returns `valid=False` with a `data.revision` mismatch. The existing test `test_committed_qwen_lora_configs_are_valid[config_path1]` fails. Rebuilding with the current generator does not resolve this mismatch.

Impact: the supplied control run cannot pass training preflight against data generated by the current code.

Suggested fix: provide a versioned configuration matching the intended current dataset, or reproducibly retain the historical generator/data required by the older configuration. Keep revision validation intact. The untracked two-epoch mix configuration also mismatches its current local manifest; establish the intended mixing recipe before changing that record.

BUG-007 — The HTTP service hides supplied initial state from the planner

Location: service/api.py, lines 247–265; compare eval/agent_runner.py, lines 150–155.

`POST /invoke` initializes adapters from `req.initial_state` but passes only `req.context` to the agent. The evaluation runner includes initial state in planning context; the HTTP service does not.

Reproduced: invoking the service with a populated Notion task list and the default context produced `PlanRequest.context == {}`. The planner never received the supplied task's ID or title.

Impact: requests referring to entities in `initial_state` cannot be planned reliably through the service unless the caller manually duplicates that state in `context`. Service behavior diverges from the evaluated path.

Suggested fix: construct planning context consistently at both entry points, including the provided initial state with a documented precedence rule.

BUG-008 — Runtime failures are mislabeled as model refusal decisions

Location: core/contracts.py, lines 256–262; core/agent.py, lines 203 and 272.

`to_agent_response()` infers the decision from final `response_type`. Plan-validation and exhausted tool-execution failures set `TASK_REFUSAL`, even when the model selected an action plan.

Reproduced: a fake backend returned an ACT plan containing an unavailable tool. Runtime validation rejected it and exported `decision_type="refuse"`; the decision scorer recorded an incorrect refusal rather than the model's selected action.

Impact: decision accuracy and failure analysis mix model policy decisions with runtime errors.

Suggested fix: retain the selected model decision independently from execution outcome and response wording. Exercise both validation and dispatch failures in decision-metric tests.

BUG-009 — One backend failure discards completed evaluation work

Location: eval/agent_runner.py, lines 191–200; eval/cli.py, lines 296–312.

A `BackendError` escapes the scenario loop. The CLI writes its artifact only after that loop returns, so completed results are never saved if a later scenario fails.

Reproduced: a three-scenario fake backend sequence containing a successfully returned clarification, a `BackendError`, and another response made two calls, left the third scenario unrun, and made zero artifact writes.

Impact: transient or exhausted backend failures lose already completed evaluation results and cost records, and provide no structured failure record for the failed scenario.

Suggested fix: preserve partial results and represent infrastructure failures explicitly, consistent with ADR 0011. Do not fabricate an agent response or count a backend outage as a model refusal.

BUG-010 — Local evaluation artifacts omit checkpoint and generation identity

Location: eval/cli.py, lines 360–365 and 422–425.

The local runner uses `adapter_path`, `max_new_tokens`, and `temperature`, but none is passed into the result payload. The backend name identifies only `local-lora:<base_model>`, while `benchmark_config` contains the runner and fake-adapter flag.

Evidence: code inspection and existing artifact headers. Two adapters or generation configurations can produce artifacts with identical configuration metadata. Filename conventions are the remaining clue, rather than a self-contained run record.

Impact: result contents cannot reliably identify or reproduce the model experiment that generated them.

Suggested fix: persist checkpoint identity and effective generation settings, together with relevant dataset/scenario and configuration identity. Add fields to new artifacts without rewriting historical results.

BUG-011 — Missing scenario directories silently shrink an evaluation suite

Location: eval/scenario_loader.py, line 166 and lines 187–197.

The loader calls `directory.glob("*.json")` without checking that each requested path exists and is a directory. A nonexistent directory behaves like an empty suite.

Reproduced: loading `[Path("scenarios/v1"), Path("__review_nonexistent_scenario_directory__")]` returned 50 scenarios normally.

Impact: a typo in a second suite path silently changes benchmark composition while still allowing a normal result artifact.

Suggested fix: validate every supplied path before loading. Preserve the existing, explicitly tested behavior for an empty directory that actually exists.

BUG-012 — The training seed is set after random adapter initialization

Location: models/sft_training.py, lines 356, 386, and 418.

`peft.get_peft_model()` initializes LoRA parameters before the configured seed reaches `Trainer`. There is no earlier RNG initialization. In installed Transformers 5.15.0, the trainer seeds during its constructor, after the adapter has already been created.

Reproduced: constructing adapters on identical copies of a tiny Qwen model under two ambient RNG states, then applying the same configured seed, produced different `q_proj.lora_A` and `v_proj.lora_A` weights. This was an offline initialization probe, without optimization or downloaded weights.

Impact: identical recorded seeds and configurations do not reproduce initial adapter weights.

Suggested fix: seed before model/adapter construction, or use a model initialization callback whose seeding is controlled by the trainer.

BUG-013 — Training configuration paths are Windows-specific

Locations: qwen2_5_1_5b_lora_v1.json, lines 11–18 and runtime paths; equivalent fields in the control, mix, and two-epoch mix configurations; models/training_config.py, lines 35–40 and 138–141.

The four current configurations serialize paths with backslashes, such as `data\sft\v1\xlam\train.jsonl`. Deserialization uses ordinary `Path` handling without translating Windows separators.

Reproduced at the path-semantics level: `PurePosixPath(value).parts` treats that value as one literal filename. No Linux or macOS training run was performed.

Impact: on Linux/macOS, preflight searches for filenames containing backslashes instead of datasets generated under normal directories. Output destinations have the same portability defect.

Suggested fix: serialize portable forward-slash paths and make path/configuration hashing consistent across platforms.

BUG-014 — Three configured backend attempts can make nine HTTP requests

Location: service/inference.py, line 469 and `_call_llm`.

BAYMAX wraps API calls in its three-attempt retry loop, but constructs `AsyncOpenAI` without disabling the SDK's own retries. The installed SDK defaults to two retries per call.

Reproduced: an in-memory HTTP transport returning status 500 received **nine requests**, while BAYMAX reported **three attempts**. Sleeps were mocked and no network request was made.

Impact: rate limits and outages cause more traffic and latency than the documented retry budget; attempt telemetry understates actual HTTP attempts.

Suggested fix: make one layer responsible for retries, for example by passing `max_retries=0` to the SDK client when BAYMAX's wrapper is active.

BUG-015 — The MLX planner never receives tool argument schemas

Location: service/mlx_backend.py, lines 232–248.

`_build_plan_prompt()` includes tool names but ignores `PlanRequest.tool_schemas`. The OpenAI backend and local Hugging Face evaluation path both expose parameter specifications.

Reproduced: a request containing a uniquely named required schema property produced an MLX prompt without that property or any tool parameter schema.

Impact: the local service model must guess required argument names and types, and receives materially different tool information from the other evaluated paths.

Suggested fix: include the supplied tool specifications in the MLX planning prompt, with formatting based on the training interface rather than individual benchmark fixtures. Validate the prompt with a schema-containing request.

BUG-016 — Service backends mishandle malformed model arguments

Locations: service/inference.py, lines 536–579; service/mlx_backend.py, lines 295–307.

Parsed model output is used before its shape is validated. OpenAI meta-tool handling calls `.get()` on decoded values even when they are lists or scalars. MLX does the same for call entries and arguments. OpenAI additionally replaces invalid JSON with `{}` instead of reporting invalid output.

Reproduced: OpenAI `request_clarification` with arguments `[]` raised `AttributeError`. MLX refusal arguments containing a nonempty list or integer did likewise. OpenAI `clipboard__read` with arguments `{not-json` became a valid-looking `clipboard.read({})` plan.

Impact: malformed output causes unstructured request failures or is silently changed into executable output. These failures occur outside the intended generation-error handling.

Suggested fix: validate call objects and argument dictionaries before access, and return a structured parse/backend failure for invalid output. Do not turn invalid arguments into empty arguments.

BUG-017 — MLX JSON extraction breaks on valid strings containing `}]`

Location: service/mlx_backend.py, lines 269–289.

The non-greedy regular expression used to find a JSON array does not understand JSON strings or nesting. It stops at matching characters inside string content.

Reproduced: this valid model output became a clarification request instead of a clipboard write:

```
[{"name": "clipboard.write", "arguments": {"text": "literal }] marker"}}]
```

Impact: ordinary text/code that happens to contain those characters can make a valid tool call fail parsing.

Suggested fix: use JSON-aware decoding that respects strings and nesting. Cover both embedded delimiter text and nested JSON in parser tests.

BUG-018 — Evaluation silently ignores invalid clipboard-read arguments

Location: eval/fake_executor.py, lines 81–82; compare core/dispatcher.py, line 85.

The evaluation dispatcher invokes `self._clipboard.read()` without forwarding arguments. The service dispatcher forwards them and consequently rejects unexpected keyword arguments.

Reproduced: `clipboard.read` with `{"unexpected": "x"}` returned `success=True` in evaluation and `success=False` with a `TypeError` through the service dispatcher.

Impact: the two execution paths disagree for the same malformed model call, despite the shipped tool schema disallowing additional properties.

Suggested fix: forward and validate arguments consistently, preferably through shared dispatch behavior. This is separate from intentionally deferred scoring of tool-execution outcomes.

BUG-019 — The documented HTTP server command lacks its dependency

Locations: pyproject.toml, dependencies; service/api.py, line 11; `uv.lock`.

The service documents `uv run uvicorn baymax.service.api:app --reload --port 8000`, but Uvicorn is neither declared nor present in the lockfile. FastAPI is installed without an extra supplying the server.

Evidence: dependency/lockfile inspection and a local import-availability probe, which found no `uvicorn` module in the project environment. No server startup was attempted.

Impact: installing the documented locked dependencies is insufficient to run the documented service command.

Suggested fix: explicitly provide the server dependency or document and supply a supported installation extra containing it.

BUG-020 — `create_app()` creates an application without the BAYMAX endpoints

Location: service/api.py, lines 166–175, 218, and 237.

The factory registers exception handlers, but `/health` and `/invoke` are registered afterward only on the module-level `app` instance.

Reproduced: `TestClient(create_app()).get("/health")` returned 404. The factory application's routes contained only OpenAPI/documentation endpoints. Existing endpoint tests construct another app and manually re-register routes, which hides this defect.

Impact: callers using the advertised application factory, including a factory-based server launch, get no BAYMAX routes. The documented module-level `:app` launch is not affected by this particular bug.

Suggested fix: register the router/endpoints inside the factory and have the module-level app use that same construction path. Test the actual factory.

BUG-021 — MLX generation blocks the asynchronous service loop

Location: service/mlx_backend.py, lines 160–170 and 200–210.

Both async backend methods directly call synchronous `self._generate(...)`. No executor or other offloading is used, despite ADR 0011 specifying wrapping synchronous inference.

Reproduced with a synchronous generation stub: an independently scheduled async heartbeat ran only after generation finished. No Apple Silicon hardware or MLX model was used for this scheduling probe.

Impact: during local generation, the service event loop cannot process health requests or other waiting asynchronous work. Delay follows generation duration.

Suggested fix: run synchronous inference through a suitably serialized worker/executor, respecting the model runtime's thread-safety requirements.

Low priority

BUG-022 — Result task IDs do not match their runtime traces

Locations: eval/agent_runner.py, line 173; core/agent.py, line 109.

The result synthesizes `<run_id>.<scenario_id>.0`, while the agent records `<run_id>.1.0`. All scenarios given the same run ID therefore also reuse one runtime task ID.

Reproduced: a captured trace contained `review.1.0`, while its corresponding scenario result contained `review.calendar_create_001.0`.

Impact: task-ID-based joins between results and traces fail or become ambiguous. The separate trace ID reduces the severity but does not make the task identifier correct.

Suggested fix: assign each scenario a consistent unique task identity and export the identity actually recorded in telemetry.

BUG-023 — Warmup is calculated before rows without training labels are removed

Location: models/sft_training.py, lines 455–465 and 475–480.

Tokenization removes examples with no trainable assistant labels, but the `warmup_steps` fallback uses the original loaded row count. The fallback is active with installed Transformers 5.15.0, whose `TrainingArguments` no longer accepts `warmup_ratio`.

Reproduced: tokenizing the first 1,000 local xLAM training rows with the cached tokenizer, low-VRAM settings, and a 512-token limit retained 267 rows. Those settings yield 17 optimizer steps, but the fallback computed two warmup steps instead of `ceil(17 × 0.03) = 1`. No training was performed.

Impact: truncation/filtering changes the effective warmup schedule away from the configured ratio.

Suggested fix: calculate warmup from the retained training dataset and effective step count, or use the installed trainer's supported ratio mechanism.

Verification performed

Checks used the existing virtual environment without installing or upgrading dependencies:

Check	Result
`.venv/Scripts/python.exe -m ruff format --check`	Passed; 108 files already formatted
`.venv/Scripts/python.exe -m ruff check .`	Passed
`.venv/Scripts/python.exe -m pyright --pythonpath .venv/Scripts/python.exe`	Passed; 0 errors, 0 warnings
`.venv/Scripts/python.exe -m pytest -q -p no:cacheprovider --basetemp <fresh-workspace-temp-directory>`	**289 passed, 1 failed, 1 skipped**

The failing test is the control-config revision mismatch in BUG-006. The skipped test requires explicitly enabled live OpenAI integration. Initial attempts using `uv` and default temporary paths encountered environment restrictions; the checks above used direct virtual-environment executables, an explicit interpreter for Pyright, and a fresh writable temporary directory for pytest. Those environment errors are not listed as project bugs.

Additional verification used in-memory fake backends, synthetic error values, a mocked HTTP transport, the actual synthetic dataset generator/splitter, installed dependency source, and a cached tokenizer. No paid requests or full training runs were made. Full MLX execution on Apple Silicon and POSIX training were not available in this Windows review.

The passing checks do not establish that the behavioral findings are covered: several reproductions exercise paths the existing tests bypass or do not assert. This report records the defects established during the review; it is not a guarantee that no other bugs remain.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

DEFERRED — Known limitations of the stepping-stone artifact

Framing (2026-08-24): BAYMAX ships as a stepping-stone learning artifact per PROJECT.md. This file documents what's INTENTIONALLY NOT in scope, so anyone reviewing the repo understands the artifact honestly.

Not "future work we're planning" — **not-implemented-and-not-planned** for this project. Previously framed as "deferred to v2/v3"; that framing implied continued development. That's not happening — the project ships at stepping-stone scope and Marv moves on.

If any of these ever get built, it would be a NEW project reusing BAYMAX's architecture as a foundation, not a continuation.

Not in the shipped artifact

Real API integrations

- **Real Google Calendar / Notion / Gmail / system clipboard adapters** — fake in-memory adapters only. No OAuth, no real API auth, no rate limits, no network flakiness handling.
- **Consequence**: BAYMAX cannot be used as a daily-driver assistant. Any successful eval scenario has zero real-world effect.

Larger model

- **Only Qwen 2.5 1.5B + LoRA and OpenAI (gpt-4o-mini) supported.** Larger local models (Qwen 7B, Llama 3.1 8B, etc.) not integrated. Anthropic backend class placeholder mentioned in ADR 0011 but not implemented.
- **Consequence**: fine-tune quality ceiling is small-model tier. Cannot demonstrate what a 7B fine-tune would achieve without additional work.

State persistence

- **In-memory state only per request.** SQLite state store designed in ADR 0007 but never implemented. Redis considered for future scale, never implemented.
- **Consequence**: no idempotency across requests, no cross-run history query, no audit/replay of past runs.

Multi-turn dialog

- **Single-turn only.** No conversational memory across user turns within a session.
- **Consequence**: cannot handle "book another meeting like the one yesterday" or clarification follow-ups that create sub-tasks.

Streaming inference

- **All-at-once responses.** No token-by-token streaming to caller.
- **Consequence**: latency feels awkward for interactive use; fine for batch eval.

LLM-as-judge scoring

- **Deterministic scoring only.** ADR 0008 designed the judge protocol; implementation deferred and now not planned.
- **Consequence**: cannot score ambiguous scenarios where multiple tool-call sequences would be valid.

Sub-task chains / multi-phase task modeling

- **Single-task-per-request.** No parent-child task hierarchy, no draft-then-verify-then-send workflow for irreversible actions.
- **Consequence**: agent cannot handle irreversible-tool patterns (like real Gmail send with verification gate).

Cancellation / barge-in mid-tool-call

- **No mid-flight cancellation.**
- **Consequence**: user request that changes mind mid-execution has no clean abort path.

Wall-clock / per-tool timeouts

- **Only** `max_plan_steps` **(step-count budget).** No real time budgets on tool calls.
- **Consequence**: fake adapters return in microseconds so no meaningful timeout risk; would matter for real adapters.

OpenTelemetry / distributed tracing

- **Custom ndjson traces only.** OTel migration considered in ADR 0006, not implemented.
- **Consequence**: no cross-service trace correlation; single-process traces only. Fine at BAYMAX's scale.

Deployment / production infrastructure

- `uvicorn` **manual start.** No launchd, no menu bar app, no Docker image, no cloud deployment, no health-check integration with any orchestrator.
- **Consequence**: not usable as a persistent background service without additional wrapping.

Hallucinated entity / success detection

- **Basic scorer only.** Not detecting when agent invents email addresses / contacts / calendar events not in `initial_state`. Not detecting when agent claims success without supporting tool execution.
- **Consequence**: scoring is trustworthy for what it measures; blind to hallucination categories.

Per-tool cost accounting for real APIs

- **Aggregate cost tracking only.** No per-tool spend attribution against real API quotas.
- **Consequence**: doesn't apply since no real APIs; would matter if real adapters ever ship.

Sensitivity-based verification (beyond just reversibility)

- **Basic reversible/irreversible flag only** (designed in ADR 0009, not implemented since fakes are always reversible).
- **Consequence**: no per-scenario or per-arg sensitivity gating.

Publication-review discipline items — dropped

Removed 2026-08-24 as publication-review overhead that no longer fits the stepping-stone scope:

- Bootstrapped 95% CI reporting requirement on aggregate metrics
- Pre-registered failure criterion per experiment
- 4-tier eval structure (compositional held-out, adversarial subset)
- Retry sensitivity sweep infrastructure (configurable N via env var, eval at multiple N values)
- Ablation sweep across LoRA rank / data mix / instruction template — reduced to "one basic variant, optional" per revised ROADMAP
- Formal experiment pre-registration files under `docs/experiments/`

Basic statistical honesty (sample size disclosed with every headline number) is retained.

Architecture that WOULD support building any of the above

If Marv or someone else ever wanted to un-defer any of the above, BAYMAX's architecture is designed to allow it cleanly:

- Real adapter integration → matches the existing tool-adapter interface (ADR 0009 reversibility contract)
- Larger model → new backend class in the factory dict (ADR 0011)
- SQLite state store → schema defined in ADR 0007
- Multi-turn → hierarchical task_id already in v1 (ADR 0002)
- Streaming → additive method on backend (ADR 0011 §A5)
- LLM-as-judge → ADR 0008 defines the protocol

That extensibility is part of the artifact's engineering value. It's not going to be exercised by this project.

Career / non-project deferrals (tracked elsewhere)

- **TADASHI-1 flagship** — see `future_work/tadashi_1_flagship.md` (gitignored)
- **Marbles-1 replication** — see `future_work/marbles_1_frequency_diffusion_vla.md` (gitignored)
- **open3d reconstruction** — see `future_work/open3d_world_recon_and_action_sim.md` (gitignored)
- **Master's application prep** — see `future_work/masters_program_targets.md` (gitignored)

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

DPO Experiment

Scope

The owner approved testing a small preference-training stage after the original anchored two-epoch SFT checkpoint. The 2:1 SFT candidate is not the starting point: on combined-150 it improved boundary recall but reduced task success from 44 to 29. Keep the model, inference prompt, strict parser and evaluation targets unchanged. The repeatedly used suites remain development benchmarks, not untouched test sets.

Preference Data

Run `scripts/build_dpo_dataset.py` to build data from the reference training cap's BAYMAX rows. xLAM rows are not used as preference anchors in this first small stage. The reference assistant answer is preserved as `chosen`; `prompt` contains the original system/user messages. Each `rejected` answer addresses that same prompt.

Wrong-decision alternatives come from training-only counterparts in the existing authored contrast groups. If a meta-tool counterpart is absent, an explicit generic wrong-meta-tool answer is used and recorded without a counterpart source ID. If an ACT counterpart is absent, the builder reports that missing direction rather than inventing a productive call. Coverage therefore must be reviewed before training.

Chosen targets must pass the existing output-contract validator. Rejected answers must also pass, except that a correctly formed productive call to an unavailable tool is an intentional capability negative and is separately annotated. Malformed JSON and wrong `question`/`reason` fields are not manufactured as routing negatives. These are rule-derived preferences, not LLM-judge labels or online model generations. Their semantic correctness still needs human sample review.

Only source training rows can provide anchors/counterparts. Reference validation and test inputs are read solely to reject overlap; no evaluation fixtures/results are read by the builder. Generator/source drift, contradictory answers to an identical prompt, and source/provenance mismatch fail explicitly. Exact duplicate prompts with identical chosen answers do not produce repeated pairs.

Artifacts

- `train.jsonl`: conversational `prompt`, `chosen`, `rejected` for a DPO trainer.
- `pairs-review.jsonl`: same pairs plus source IDs, decision directions and negative kinds.
- `manifest.json`: source/reference hashes, seed, pair counts, unique anchor counts, missing productive counterparts and preference directions.

Two different rejections for one prompt are different preferences, not duplicate rows. Inspect class exposure by both pair count and unique-anchor count. Freeze the output after review; existing nonempty output directories are not overwritten.

Training Status

Preference preparation and `scripts/train_dpo_lora.py` are implemented. The owner approved TRL; the optional `dpo` extra pins TRL 1.14.0, PEFT 0.20.0, Transformers 5.15.0, Accelerate 1.14.0 and PyTorch 2.13.0. The installed CUDA PyTorch build is retained by the pip installation below. Do not use `uv sync --all-extras` to replace the laptop's existing training environment.

The first smoke reached reference scoring but failed on TRL 1.15's mandatory Triton fused LM head (`_ChunkedLogProbFunction` was unavailable on native Windows). The runner now pins 1.14 and explicitly sets `use_liger_kernel=False`, retaining the ordinary PyTorch full-logits DPO path. No third-party source is patched and no preference labels, reference checkpoint or DPO objective are changed. Full logits can consume more GPU memory than fused scoring; laptop memory fit still needs the user-run smoke test. Run metadata/hash includes the TRL version and loss backend. Version 1.15 is rejected before model loading with a reinstall instruction.

The initial policy loads the original anchored two-epoch SFT adapter as trainable. TRL copies its weights into a `ref` adapter, not bare Qwen. The runner verifies weight equality and freezes the reference before optimization. Reference log probabilities are precomputed with batch size one, avoiding a second full base model in memory. Only the updated `default` policy adapter is saved, so the current `run-agent-lora` evaluation path can load it normally.

Initial experimental settings: one DPO epoch, sigmoid loss, beta 0.1, learning rate 5e-6, constant scheduler, batch one, gradient accumulation 16, context 1024, seed 1. These are conservative starting settings, not measured optimal values. Existing LoRA rank/targets come from the saved adapter. No DPO validation preferences are constructed; evaluate routing/task success with the existing development suites.

The config pins preference manifest/review and SFT adapter hashes. All chosen contracts and annotated rejected contracts are checked, including intentional unavailable-tool negatives. Context or completion truncation and chat-template prefix mismatches fail explicitly. The artifact report records the selected pair directions, unique prompts, completion token counts and zero truncated pairs. Token IDs are read explicitly from `apply_chat_template(..., return_dict=True)`; Transformers 5.15 returns a dictionary, not a bare token list. Prompt-only rendering adds the assistant header; completed conversations explicitly do not add another. `dpo-run.json` records config/run hashes, library versions and completion status. Existing nonempty outputs and paths overlapping frozen inputs are rejected.

The first Windows artifact's manifest had an LF checksum while its data used CRLF. Only its checksum metadata was corrected; preference rows/review labels were not changed. Future builds explicitly write LF so the byte checksum is correct.

Local Commands

From the repository root, install the optional extra in the existing CUDA venv:

```
.venv\Scripts\python -m pip install -e ".[dpo]"
.venv\Scripts\python scripts\train_dpo_lora.py --dry-run
```

Review `data/dpo/v1/anchored-epochs2-preferences/pairs-review.jsonl` before acknowledging `--reviewed`. Then run a separate memory/tokenization smoke test:

```
.venv\Scripts\python scripts\train_dpo_lora.py `
  --reviewed --max-pairs 16 --max-steps 1 `
  --output-dir tmp\qwen-dpo-smoke-windows
```

After the smoke succeeds, a full run uses a new default output directory:

```
.venv\Scripts\python scripts\train_dpo_lora.py --reviewed
```

Generated preference data and model outputs are ignored by Git. No packages were installed, models loaded, or DPO training/evaluation started by the coding agent. The user completed the Windows smoke and full original-checkpoint DPO run. The smoke retained all 16 pairs; the full run retained all 705 pairs with no truncation. The 2:1-starting-checkpoint experiment below is dry-run validated but not yet trained.

2:1 Starting Checkpoint Comparison

The owner approved repeating DPO from the two-epoch 2:1 SFT checkpoint. Use `configs/dpo/qwen2_5_1_5b_2to1_epochs2_v1.json`. The exact 705 preference pairs, prompt, learning rate, beta, epoch count, seed and generation/evaluation settings stay unchanged. Only the SFT policy and its matching frozen reference change. "2:1" refers to the SFT mixture, not a newly rebalanced preference dataset.

`reference_config` remains the preference dataset's original provenance. `initial_sft_config` separately identifies the DPO policy/reference starting point, pinned by file hash and adapter hashes. The alternate config must have a valid content hash, the same base model/prompt, and identical held-out validation/test file hashes. The original preference manifest and earlier checkpoints/results are not rewritten. Regression tests check alternate checkpoint loading and drift.

```
.venv\Scripts\python scripts\train_dpo_lora.py `
  --config configs\dpo\qwen2_5_1_5b_2to1_epochs2_v1.json `
  --reviewed
```

Output: `models/qwen2_5_1_5b_2to1_epochs2_dpo_v1/adapter`. Compare its results against the 2:1 SFT checkpoint first, then against the original SFT/DPO pair. This is an initialization/reference comparison, not proof that a 2:1 ratio is optimal.

Completing Missing ACT Rejections

The first preference set lacks ACT rejections for 50 clarification and 45 refusal anchors. The owner approved filling these before another optimization experiment. `dpo_act_rejections.py` authors explicit wrong-action completions for existing training families only. Unknown families remain uncovered rather than receiving arbitrary tool calls. Existing training-only ACT counterparts take priority.

Negatives deliberately guess unresolved arguments, act before confirmation, attempt a known but unavailable operation, or substitute an available mutation for an unsupported request. Substituting a reminder for a booking/payment or creating a draft instead of searching is wrong for those prompts, even if the adapter could

execute the call. No new capabilities or SFT examples are introduced. Saved passwords/private contents are never supplied; invented access claims are rejected answers, not chosen targets. Each added pair includes `rejection_reason` in the review file, excluded from the model's training input/output columns.

All authored negatives use actual productive tool schemas, including unavailable tools. Malformed arguments are not exempted merely because a tool is unavailable. Chosen answers, prompts, original pairs and split protection remain unchanged. No evaluation fixtures/results were read to author these negatives. Prior data, checkpoints and result artifacts are preserved; weighting is not changed here.

```
.venv\Scripts\python scripts\build_dpo_dataset.py `
  --complete-act-rejections `
  --output-dir data\dpo\v1\anchored-epochs2-act-rejections-v2
```

A short coverage check against the existing review source IDs found authored negatives for all 95 gaps. Expected new dataset: 800 pairs, 400 unique prompts, CLARIFY>ACT=100 and REFUSE>ACT=100 (both were partial). ACT>CLARIFY=200 and ACT>REFUSE=200 remain unchanged: filling coverage is not full direction balancing. Build/report the new artifact and review its rejection explanations before creating a hash-pinned training config. No rebuild or retraining was run by the coding agent for this change.

The user built the v2 artifact: 800 pairs, 400 unique prompts, 95 authored ACT rejections, and no uncovered directions. The next run starts fresh from the original anchored two-epoch SFT adapter (not the earlier DPO adapter or 2:1 SFT). `configs/dpo/qwen2_5_1_5b_anchored_epochs2_act_rejections_v2.json` pins the new manifest/review hashes and unchanged original SFT adapter hashes. DPO settings remain one epoch, 5e-6 learning rate, beta 0.1 and seed 1. Results/checkpoints use new paths. Adding 95 pairs also increases training exposure at the same epoch count; this is a coverage/data-addition experiment, not a step-matched ablation.

After reviewing the 95 added `rejection_reason` entries:

```
.venv\Scripts\python scripts\train_dpo_lora.py `
  --config configs\dpo\qwen2_5_1_5b_anchored_epochs2_act_rejections_v2.json `
  --reviewed
```

Output: `models/qwen2_5_1_5b_anchored_epochs2_dpo_act_rejections_v2/adapter`.

Two-Epoch Duration Test

Prepared on 2026-10-12; training and evaluation remain owner-run steps. Use `configs/dpo/qwen2_5_1_5b_anchored_epochs2_act_rejections_v2_dpo_epochs2.json`. Only the DPO epoch count and output identity differ from the completed-rejections one-epoch config. Keep all 800 pairs / 400 unique prompts, input hashes, original two-epoch SFT policy/reference adapter, prompt, learning rate 5e-6, beta 0.1, batch one, accumulation 16, context 1024, constant scheduler and seed 1 unchanged. Do not rebuild preference data, add weighting, or resume a DPO adapter.

The planned run presents 1600 pairs over two passes and performs 100 optimizer updates, versus 800 presentations / 50 updates for one epoch on one GPU. Unique prompts remain 400. These are calculated exposures, not completed-run measurements or evidence that two epochs is sufficient.

New adapter: `models/qwen2_5_1_5b_anchored_epochs2_dpo_act_rejections_v2_epochs2/adapter`. New result: `results/phase2/qwen-lora/dpo/anchored-sft2-dpo2-act-rejections-v2-combined-150.json`. Compare first against `anchored-sft2-dpo1-act-rejections-v2-combined-150.json`, with the original anchored two-epoch SFT result as an additional reference. Use the same combined 150 scenarios, greedy decoding and 256-token generation limit. Inspect routing macro-F1, the decision matrix, premature actions, task success and invalid outputs. No benchmark improvement is claimed until those runs complete.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

OWNERSHIP — module ownership and PR rules

Locked: 2026-05-16 **Revisit at:** v1 retro (Jul 26), v2 retro (Sep 6)

Why this exists

Per Tadashi CV: "clear ownership" is a non-negotiable signal for elite hiring. When a senior engineer drills you on "what did *you* build?" the answer must be specific, defensible, and not a co-author claim. This file is the ground truth for that.

The split

Module	Owner	Co-owner / reviewer
Agent core (planner, router, state machine, tool-call contract, retry/recovery)	Marv	Ronin
Inference service (FastAPI/async, batching, MLX/llama.cpp serving, latency optimization)	Marv	Ronin
Telemetry (structured logging, OpenTelemetry traces, cost tracker)	Marv	Ronin
Eval harness (scenario format, runner, metrics, judge protocol, baseline vs candidate runner)	Ronin	Marv
Model training pipeline (data prep, SFT/LoRA, eval-during-training, model registry, ablations)	Ronin	Marv
Tool adapters (Notion, Google Calendar, Gmail, clipboard)	Ronin	Marv
Shared: repo scaffolding · `ARCHITECTURE.md` · final benchmark writeup · demo video	both	—

Person 1 = Marv Person 2 = Ronin

What "owner" means

For your owned modules:

- You make the architectural decisions, document them in `docs/decisions/NNNN-title.md` (ADR format), and defend them in review

- You write the core code yourself (CV: AI fine for boilerplate, glue, docs digestion; not for architecture/algorithms/data models)
- You write the tests
- You write the section in `ARCHITECTURE.md`
- Your CV bullet for this module says "designed and built" — the co-owner's CV bullet for the same module says "contributed to" or omits it

For your co-owned modules:

- You review every PR; the owner cannot merge without your approval
- You can propose changes via PR but cannot merge without the owner's approval
- You ask questions; the owner answers and documents

PR rules

1. `main` is protected. No direct pushes. No force-push to `main`, ever.
2. Every PR closes one tracked GitHub Issue, scoped < 2 days of work.
3. Every PR requires the relevant co-owner's review and approval before merge.
4. CI (ruff + pyright + pytest) must be green to merge.
5. PR description includes: what changed, why, how to test, and explicit "I wrote this" or "AI-assisted with: [scope]" disclosure.
6. Per CV rule: if AI wrote code, the human must be able to explain it line-by-line on request. Don't merge code you can't defend.
7. No `--no-verify`. No skipping hooks.
8. Squash merge to keep `main` history clean.

Conflict resolution

When the two of you disagree on an architecture choice that crosses owned modules:

1. Each writes a short doc (≤300 words) stating their proposal + tradeoff
2. Pick the one with the clearer measurable consequence (e.g. "this design lets us swap inference backends in <100 LOC vs ~1000 LOC")
3. If still tied, the owner of the module most affected gets the call
4. Document the decision in an ADR

Self-honesty audit

Once a month, each person writes a 200-word retro on what they actually built vs what they shipped via AI. The CV explicitly warns: "if you cannot rewrite the critical part yourself if needed, then AI is currently weakening you." Catch this early.

When this file changes

Any scope change to an owned module requires a PR that updates this file. No silent ownership drift.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Phase 2 Architecture

This document defines the Phase 2 architecture for BAYMAX's first local-model training experiment.

Phase 1 built the evaluation harness: 50 scripted scenarios, fake adapters, deterministic scoring, and baseline result files. Phase 2 keeps those scenarios as eval-only seed coverage, expands the benchmark to at least 200 tiered scenarios, and trains on a separate external tool-use dataset.

Goal

Phase 2 answers this question:

```
How much tool-use behavior transfers from public HuggingFace data into BAYMAX's
domain-specific task-agent contract?
```

The main comparison is:

```
base Qwen 2.5 1.5B
vs
Qwen 2.5 1.5B fine-tuned on external tool-use data + BAYMAX output adapter
```

OpenAI remains a Phase 1 reference baseline. The primary Phase 2 claim is not that the local model beats OpenAI overall. The defensible claim is:

```
Fine-tuned Qwen is characterized against base Qwen on an eval-only BAYMAX
benchmark, with tiered scores and confidence intervals.
```

Non-Goals

Phase 2 does not add:

- real Notion, Google Calendar, Gmail, or clipboard APIs
- audio input
- long-term memory
- RAG
- multi-agent orchestration
- Phase 3 scenario expansion
- LLM-as-judge scoring
- production deployment

Phase 2 is a transfer-learning experiment. It should measure what external tool-use data can and cannot teach before BAYMAX invests in a larger domain-specific training set. Underperformance against OpenAI is an expected possible outcome, not a failed phase.

Chosen Approach

Phase 2 uses:

```
external dataset + sample review + output adaptation
```

Meaning:

- Training data comes from a pre-existing HuggingFace-hosted external dataset. The current leading candidate is `Salesforce/xLAM-function-calling-60k`.
- The external dataset is kept in its native task/output format for training.
- The fine-tuned model is expected to emit the external dataset's tool-call format.
- A deterministic output adapter converts model output from the external format into BAYMAX `AgentResponse` before the eval scorer sees it.
- `scenarios/v1` and the expanded Phase 2 eval scenarios remain evaluation data only.

The output adapter is part of the Phase 2 experimental system. Marv owns the adapter implementation and translation table. Ronin consumes the adapter through its contract for eval and reporting. Adapter coverage and adapter failures must be measured separately, because adapter bugs can look like model failures.

Core Separation

The most important Phase 2 boundary is:

```
scenarios/v1 and expanded Phase 2 scenarios = evaluation only
external dataset = training source
output adapter = bridge from external output format to BAYMAX AgentResponse
```

The model must not train on:

```
scenarios/v1/*.json
scenarios/v1/scripted_responses/v1-scripted.json
results/v1-*.json
```

The scripted responses remain useful for Phase 1 scripted baseline validation, but they are not Phase 2 SFT targets.

Author-Split Discipline

Phase 2 must preserve the methodology ownership boundary:

```
Ronin owns data sourcing, SFT training, eval expansion, statistical reporting,
and benchmark analysis.

Marv owns runtime backend work, model registry interface, and output-adapter
translation-table design.
```

Ronin must not hand-author training examples or design the adapter translation table from eval-scenario knowledge. Marv must not inspect eval scenario content while doing training-data-adjacent or adapter design work.

System Flow

```
external HuggingFace tool-use dataset
        |
        v
sample review + dataset manifest
        |
        v
leakage check against all eval scenarios
        |
        v
train/validation/internal_test split
        |
        v
SFT JSONL export in external dataset format
        |
        v
Qwen 2.5 1.5B LoRA training
        |
        v
model emits external tool-call format
        |
        v
output adapter
        |
        v
BAYMAX AgentResponse
        |
        v
existing eval runner + fake adapters + scorer on eval tiers
        |
        v
base Qwen vs fine-tuned Qwen comparison with confidence intervals
```

The existing Phase 1 eval runner, fake adapters, and scorer remain the external measurement path. Phase 2 adds external dataset handling, local-model inference, output adaptation, registry metadata, leakage checks, tiered eval reporting, and comparison reporting with confidence intervals.

Contracts

Phase 2 has two output contracts.

Model Output Contract

The fine-tuned model emits the external dataset's native output format.

The exact shape depends on the chosen dataset. It may look like:

```
{
  "name": "external_calendar_create",
  "arguments": {
    "event": "linear algebra revision",
    "date": "tomorrow",
    "time": "4pm",
    "duration": "2 hours"
  }
}
```

Do not force external training examples into BAYMAX `AgentResponse` during SFT. That would turn Phase 2 into a BAYMAX-specific supervised dataset effort, which is not the chosen acquisition strategy.

BAYMAX Eval Contract

Before scoring, the output adapter must convert valid external model output into the `AgentResponse` shape already used by the eval scorer:

```
{
  "tool_calls": [
    {
      "tool": "calendar.create_event",
      "arguments": {
        "title": "linear algebra revision",
        "start_date": "2026-05-19",
        "start_time": "16:00",
        "duration_minutes": 120
      }
    }
  ],
  "message": null
}
```

Clarification and refusal outputs use no tool calls after adaptation:

```
{
  "tool_calls": [],
  "message": "Who should I send the notes to?"
}
```

The Phase 1 scorer receives only valid BAYMAX `AgentResponse` objects.

External Dataset Selection

The leading candidate is:

```
Salesforce/xLAM-function-calling-60k
```

This is a HuggingFace dataset identity: `Salesforce` is the publisher namespace, and `xLAM-function-calling-60k` is the dataset name.

The selected dataset must be recorded by exact HuggingFace identity and revision:

```
dataset_id
dataset_revision
source_split_names
license
download command
example count
tool-call format description
known schema limitations
```

Suggested manifest path:

```
data/sft/v1/dataset_manifest.json
```

If the raw downloaded dataset is large, do not commit it. Commit the manifest, hashes, split indexes, review report, leakage report, and a small sample file.

Dataset Review

Phase 2 uses sample review, not full manual curation.

Suggested report:

```
data/sft/v1/dataset_review.md
```

The review should record:

```
sample size reviewed, target 5-10% of selected examples
quality issues found
error rate, with categories: correct / wrong-tool / wrong-args / malformed
tool families covered
argument patterns covered
clarification/refusal coverage
dataset license concerns
expected transfer gaps
```

The goal is not to prove every row is perfect. The goal is to understand the dataset's quality and likely mismatch with BAYMAX before training.

Reject or revise the dataset plan if the sampled error rate is greater than 25%.

Coverage Gap Supplement

External tool-use datasets may not teach BAYMAX-specific behavior well enough. The dataset audit must explicitly check for:

```
refusal
clarification
verification
_contains-style partial matching
BAYMAX tool coverage
```

If these are missing, Phase 2 may add a small supplementary set of roughly 100-200 narrowly scoped template-generated examples. This supplement must be documented separately and must not be derived from eval scenario content.

Leakage Check

Because BAYMAX eval scenarios are the external benchmark, the data pipeline must check the external SFT dataset against all eval scenarios.

Minimum leakage checks:

```
training example IDs must not reference scenario IDs
training user text must not exactly match scenario user_input
training output must not be copied from v1 scripted responses
training metadata must not include expected_behavior or success_criteria
```

Useful later checks:

```
near-duplicate user text detection
embedding-similarity report
```

```
structural fingerprint overlap by tool/behavior/difficulty
sample-based human overlap review
```

Suggested leakage report:

```
data/sft/v1/leakage_report.json
```

The leakage report should be committed with the dataset export.

SFT Prompt And Target Format

Use chat `messages` format because Qwen 2.5 Instruct models are trained for instruction/chat-style interaction.

The prompt format should follow the chosen external dataset as much as possible. The target assistant message should be the external dataset's native tool-call output, not BAYMAX `AgentResponse`.

Example JSONL row:

```
{
  "messages": [
    {
      "role": "system",
      "content": "Return only a valid tool call in the dataset format."
    },
    {
      "role": "user",
      "content": "Schedule linear algebra revision tomorrow at 4pm for 2 hours."
    },
    {
      "role": "assistant",
      "content": "{\"name\":\"external_calendar_create\",\"arguments\":{\"event\":\"linear algebra
revision\",\"date\":\"tomorrow\",\"time\":\"4pm\",\"duration\":\"2 hours\"}}"
    }
  ],
  "metadata": {
    "external_example_id": "toolbench_000001",
    "split": "train",
    "dataset_id": "external/tool-use-dataset",
    "tool_family": "calendar"
  }
}
```

Keep model-visible training prompts faithful to the external dataset. BAYMAX runtime prompts may include BAYMAX fields such as `current_time`, `available_tools`, and `initial_state`; the output adapter must handle the format bridge during eval.

BAYMAX Eval Prompt

When evaluating against BAYMAX scenarios, the local model receives BAYMAX runtime context:

```
CURRENT_TIME:
{current_time}

AVAILABLE_TOOLS:
{available_tools_json}

INITIAL_STATE:
{initial_state_json}

USER_INPUT:
{user_input}
```

The model is still expected to emit the external dataset's output format. The adapter then maps that external output to BAYMAX `AgentResponse`.

This mismatch is intentional. Phase 2 measures transfer from public tool-use training into BAYMAX's contract through a deterministic adapter.

Training Split Strategy

The external SFT dataset uses a 70/20/10 split:

```
train: 70%
validation: 20%
internal_test: 10%
```

These splits are for training development only. They are not the final BAYMAX benchmark.

The interim external eval can use:

```
scenarios/v1
```

The Phase 2 exit eval must use an expanded benchmark with at least 200 scenarios and separate tier labels:

```
in-distribution
compositional
OOD
adversarial
```

The training split should preserve visibility across the external dataset's available metadata where possible:

```
tool family
single-tool vs multi-tool
argument complexity
clarification-like examples
refusal-like examples
```

Validation is used for training decisions such as early stopping and LoRA config selection. Internal test is used for sanity-checking the external dataset. `scenarios/v1` is useful for interim checks, but the Phase 2 headline result must be reported on the expanded tiered eval benchmark.

Dataset Manifest

Splits must be saved and reused. Do not re-randomize splits for each run.

Path:

```
data/sft/v1/dataset_manifest.json
```

Suggested shape:

```
{
  "dataset_id": "Salesforce/xLAM-function-calling-60k",
  "dataset_revision": "<huggingface_revision_or_commit>",
  "created_at": "2026-08-04T00:00:00Z",
  "source": {
    "provider": "huggingface",
    "download_command": "huggingface-cli download ...",
    "raw_cache_path": "<local_or_remote_cache_path>",
    "eval_scenario_dirs": ["scenarios/v1", "<expanded_phase2_eval_path>"],
    "eval_scripted_responses": "scenarios/v1/scripted_responses/v1-scripted.json"
  },
  "strategy": "coverage_aware_70_20_10",
  "seed": 42,
  "files": {
    "train": "data/sft/v1/train.jsonl",
    "validation": "data/sft/v1/validation.jsonl",
    "internal_test": "data/sft/v1/internal_test.jsonl",
    "review_report": "data/sft/v1/dataset_review.md",
    "leakage_report": "data/sft/v1/leakage_report.json",
    "sample": "data/sft/v1/sample.jsonl"
  },
  "counts": {
    "total": 0,
    "train": 0,
    "validation": 0,
    "internal_test": 0
```

```
  },
  "coverage": {
    "tool_family": {},
    "output_format": {},
    "single_vs_multi_tool": {},
    "clarification_like": 0,
    "refusal_like": 0
  },
  "hashes": {
    "train_sha256": "",
    "validation_sha256": "",
    "internal_test_sha256": "",
    "sample_sha256": "",
    "eval_scenarios_sha256": ""
  }
}
```

The manifest is the source of truth for the training dataset, split membership, coverage, and hashes.

Output Adapter

The output adapter translates external model output into BAYMAX `AgentResponse`.

Ownership:

```
Marv owns adapter implementation and translation-table design.
Ronin owns eval consumption, adapter failure reporting, and benchmark analysis.
```

The exact implementation path is Marv-owned and should be finalized with the backend/interface ADRs. Ronin-side code should treat the adapter as a dependency behind a stable contract, not design its mapping internals from eval scenarios.

The adapter owns:

```
external JSON parsing
external tool-name to BAYMAX tool-name mapping
external argument-name to BAYMAX argument-name mapping
unit conversions where deterministic
unsupported external tool handling
adapter failure reporting
```

The adapter must not inspect:

```
expected_behavior
success_criteria
scripted response targets
```

Those fields are eval truth. If the adapter reads them, the eval is contaminated.

Adapter Coverage Report

Before training or eval, produce an adapter coverage report:

```
data/sft/v1/adapter_coverage.json
```

It should include:

```
BAYMAX tools with direct external equivalents
BAYMAX tools with partial external equivalents
BAYMAX tools with no external equivalent
argument mappings per tool
unsupported argument fields
clarification/refusal coverage gaps
known brittle mappings
```

This report is mandatory because external datasets usually do not share BAYMAX's tool namespace or safety behavior.

Adapter Test Suite

The adapter needs unit tests before it is used in benchmark runs. Marv owns the adapter test suite. Ronin may inspect adapter outputs during eval failure analysis, but should not design adapter translation cases from specific eval scenario answers.

Tests should cover:

```
one successful mapping per BAYMAX tool
multi-tool mapping
argument renaming
unit conversion
unsupported external tool
missing required argument
clarification-like output if supported by the dataset
refusal-like output if supported by the dataset
schema-invalid adapted output
```

Adapter bugs will otherwise be indistinguishable from model failures.

Training Backend

Primary Phase 2 backend:

```
Transformers + PEFT LoRA
```

Reason:

- works naturally with cloud GPU training
- fits RunPod-style experimentation
- does not require Apple Silicon
- can evaluate base Qwen and fine-tuned Qwen through the same runner boundary

Secondary compatibility backend:

```
MLX / mlx-lm on Marv's Mac
```

MLX compatibility is useful, but it should not block the first Phase 2 training proof. The architecture should allow an MLX backend later by keeping the inference interface backend-neutral.

Training Config

Each training run must have a config that records the training recipe.

Suggested path:

```
models/qwen-1.5b-baymax-v1/training_config.json
```

Suggested shape:

```
{
  "base_model": "Qwen/Qwen2.5-1.5B-Instruct",
  "dataset": {
    "manifest": "data/sft/v1/dataset_manifest.json",
    "train_file": "data/sft/v1/train.jsonl",
    "validation_file": "data/sft/v1/validation.jsonl",
    "internal_test_file": "data/sft/v1/internal_test.jsonl",
    "external_eval_scenario_dirs": ["scenarios/v1", "<expanded_phase2_eval_path>"]
  },
  "adapter": {
    "config_file": "data/sft/v1/adapter_coverage.json",
    "version": "baymax_external_adapter_v1"
  },
  "training": {
    "epochs": 3,
    "learning_rate": 0.0002,
    "batch_size": 4,
    "max_sequence_length": 2048,
    "seed": 42
  },
```

```
  "lora": {
    "rank": 8,
    "alpha": 16,
    "dropout": 0.05,
    "target_modules": ["q_proj", "v_proj"]
  }
}
```

The most important overfitting-related knobs are epochs, learning rate, batch size, LoRA rank, and LoRA dropout.

Model Registry

The model registry records model identity and reproducibility metadata. It does not store benchmark scores.

Canonical registry file:

```
models/models.json
```

Model artifact path:

```
models/qwen-1.5b-baymax-v1/
  training_config.json
  adapter/
```

Model IDs are immutable and monotonically versioned. Never overwrite an existing model ID; a retrain creates the next version.

Suggested `models/models.json` shape:

```
{
  "models": [
    {
      "id": "qwen-1.5b-baymax-v1",
      "base": "Qwen/Qwen2.5-1.5B-Instruct",
      "adapter_path": "models/qwen-1.5b-baymax-v1/adapter",
      "adapter_format": "peft_lora",
      "trained_at": "2026-08-04T12:00:00Z",
      "training_config_file": "models/qwen-1.5b-baymax-v1/training_config.json",
      "training_config_hash": "sha256:<training_config_sha256>",
      "dataset_manifest": "data/sft/v1/dataset_manifest.json",
      "external_dataset_id": "Salesforce/xLAM-function-calling-60k",
      "external_dataset_revision": "<huggingface_revision_or_commit>",
      "output_adapter_version": "baymax_external_adapter_v1",
      "notes": "First external-data LoRA run."
    }
```

```
  ]
}
```

Large checkpoint artifacts may live outside Git if needed, but metadata should be committed so that results remain traceable.

The registry must store both `training_config_file` and `training_config_hash`. The path makes the config easy to find; the SHA-256 hash is the reproducibility invariant. If the file is edited later, the hash mismatch shows that the checkpoint no longer matches the config at that path.

Benchmark metrics live only in `results/`, never in `models/models.json`. The registry identifies models; result files measure them.

Inference Interface

Inference backends must be side-effect free. They decide what to do; they do not execute tools or mutate fake state.

Inputs:

```
current_time
user_input
available_tools
initial_state
```

Output:

```
raw model text
parsed external output if valid
adapted AgentResponse if adapter succeeds
parse_status
adapter_status
latency_ms
cost_usd or local cost estimate
```

Conceptual interface:

```
class InferenceBackend:
    def generate_response(
        self,
        *,
        current_time: str,
        user_input: str,
        available_tools: list[str],
        initial_state: dict,
    ) -> ModelGeneration:
        ...
```

`ModelGeneration` should include:

```
raw_output: str
external_output: dict | None
adapted_response: AgentResponse | None
parse_status: Literal["valid", "external_parse_failed"]
adapter_status: Literal["not_run", "adapted", "adapter_failed", "schema_invalid"]
latency_ms: int
cost_usd: float
```

The eval runner remains responsible for:

```
output adaptation
fake tool execution
final_state capture
Phase 1 scoring
Phase 2 comparison reporting
```

This keeps scripted, OpenAI, Transformers/PEFT, and MLX backends comparable.

Invalid Output Handling

Invalid external output is a model failure in primary metrics.

No JSON repair is allowed in Phase 2 primary evaluation. This measures whether the model learned the external output contract and whether the adapter can bridge that output into BAYMAX.

Failure statuses:

```
external_parse_failed
adapter_failed
schema_invalid
```

If output cannot be parsed as the external format, the runner records raw output and does not run the adapter. If adaptation fails or produces invalid `AgentResponse`, the runner records adapter failure details. The Phase 1 scorer receives only valid adapted `AgentResponse` objects.

Suggested failed-output result shape:

```
{
  "scenario_id": "calendar_create_001",
  "raw_output": "...",
  "parse_status": "valid",
  "adapter_status": "adapter_failed",
  "adapted_response": null,
  "score": null,
```

```
  "failure_reasons": ["adapter_failed"]
}
```

This prevents malformed or unadaptable output from being accidentally treated as a valid empty response.

Comparison Method

Do not change the Phase 1 scorer for Phase 2 model comparisons.

Phase 1 scorer answers:

```
Did this adapted AgentResponse pass according to the deterministic eval contract?
```

Phase 2 comparison answers:

```
How does fine-tuned Qwen + output adapter compare to base Qwen + output
adapter across the locked eval tiers?
```

The comparison layer should read result files and produce:

```
results/v2-qwen-comparison-tiered-eval.json
```

The comparison score is behavior-specific.

Tool-Call Scenarios

Applies to `tool_call` and `tool_calls` expected behavior.

```
35% decision correctness
25% tool-call accuracy
25% argument accuracy
15% safety
```

Clarification Scenarios

```
50% decision correctness
35% clarification accuracy
15% safety
```

Refusal Scenarios

```
50% decision correctness
30% refusal accuracy
20% safety
```

Decision correctness measures whether the adapted `AgentResponse` chose the right mode:

```
act
clarify
refuse
```

Safety should primarily account for hallucinated or unavailable tool calls. Premature action is already captured by decision correctness, but hallucinating unsupported tools should make a wrong decision worse.

Fine-tuned Qwen wins a scenario when its Phase 2 comparison score is higher than base Qwen's comparison score for that same scenario.

Eval Methodology Gates

Phase 2 cannot exit on a single aggregate score over the original 50 scenarios.

The benchmark used for headline Phase 2 claims must have:

```
at least 200 scenarios
tier labels for in-distribution / compositional / OOD / adversarial
separate score tables per tier
bootstrapped 95% confidence intervals for aggregate metrics
diversity metrics over tool combo / intent type / phrasing style / error mode
embedding-similarity checks against existing hand-authored scenarios
```

Before the first SFT eval run, write:

```
docs/experiments/<experiment_id>_pre_registration.md
```

That document must state what outcome counts as "the fine-tune did not work." This prevents post-hoc rationalization after results are known.

Phase 2 Success Criteria

Primary success criterion:

```
Phase 2 produces a research-defensible characterization of transfer from xLAM
style public tool-use data to BAYMAX's eval contract.
```

Exit gates:

```
eval scenario count >= 200
eval tiers reported separately: in-distribution / compositional / OOD / adversarial
aggregate metrics include bootstrapped 95% confidence intervals
pre-registered failure criterion exists before first SFT eval
```

```
adapter test suite is green
METHODOLOGY.md section 11 audit passes
```

Secondary metrics:

```
task_success_rate
average_tool_call_accuracy
average_argument_accuracy
average_clarification_accuracy
average_refusal_accuracy
average_hallucination_rate
adapter_failure_rate
invalid_external_output_rate
schema_invalid_rate
average_latency_ms
cost_usd or estimated local cost
```

The internal test split may be reported as a training sanity check. The 50-scenario `scenarios/v1` set may be used for interim checks. The headline Phase 2 benchmark must be the expanded tiered eval set.

Research Framing

The strongest Phase 2 narrative is:

```
We characterize the transfer ceiling from public tool-use datasets into a
domain-specific task-agent contract. The model trains on external tool-use data,
an output adapter bridges namespace and argument differences, and BAYMAX's
locked eval measures how much of that behavior transfers.
```

Expected weaknesses should be reported directly:

```
external dataset may lack BAYMAX tools
external dataset may lack clarification/refusal behavior
argument schemas may not match BAYMAX tools
adapter mappings may be brittle
small interim eval size limits statistical power
```

A weak fine-tuned result is still useful if the failure analysis is specific.

Expected Artifacts

Committed small artifacts:

```
data/sft/v1/dataset_manifest.json
data/sft/v1/dataset_review.md
```

```
data/sft/v1/leakage_report.json
data/sft/v1/adapter_coverage.json
data/sft/v1/sample.jsonl
docs/experiments/<experiment_id>_pre_registration.md
results/v2-qwen-base-tiered-eval.json
results/v2-qwen-sft-tiered-eval.json
results/v2-qwen-comparison-tiered-eval.json
```

Conditional artifacts:

```
data/sft/v1/train.jsonl
data/sft/v1/validation.jsonl
data/sft/v1/internal_test.jsonl
```

Commit the JSONL files only if they are repo-safe in size. If they are large, store them externally or regenerate them from the manifest and scripts.

Optional diagnostic artifacts:

```
results/v2-qwen-sft-internal-test.json
results/v2-qwen-base-v1-eval.json
results/v2-qwen-sft-v1-eval.json
results/v2-qwen-comparison-v1-eval.json
data/sft/v1/preview.md
results/v2-output-adapter-tests.json
```

Model registry artifacts:

```
models/models.json
models/qwen-1.5b-baymax-v1/training_config.json
models/qwen-1.5b-baymax-v1/adapter/
```

Large adapter/checkpoint files may be excluded from Git if needed, but the registry metadata must remain committed.

Proposed Repo Layout

Reusable tested logic:

```
src/baymax/models/
  external_dataset.py
  sft_export.py
  splits.py
  leakage.py
  registry.py
  comparison.py
```

Marv-owned runtime/adapter code:

```
<adapter path finalized by Marv's backend/interface ADR>
```

Script orchestration:

```
scripts/
  prepare_external_dataset.py
  review_external_dataset.py
  export_sft_data.py
  train_qwen_lora.py
  run_local_eval.py
  compare_model_results.py
```

Data and artifacts:

```
data/
  sft/

models/
  models.json
  qwen-1.5b-baymax-v1/

results/
```

Keep Ronin-owned importable training/eval support code in `src/baymax/models/`. Keep one-off command orchestration in `scripts/`. Keep trained artifacts outside `src/`. Keep output-adapter implementation in Marv-owned runtime code.

Benchmark Reporting

`docs/BENCHMARKING.md` should report:

```
OpenAI Phase 1 baseline on scenarios/v1
base Qwen + adapter result on expanded tiered eval
fine-tuned Qwen + adapter result on expanded tiered eval
Phase 2 comparison report
adapter failure analysis
training dataset internal test sanity check
mean +/- bootstrapped 95% CI for aggregate metrics
```

The writeup must clearly state:

```
scenarios/v1 was never used for training
training data came from an external dataset
```

```
the model emitted the external output format
the output adapter converted external output to BAYMAX AgentResponse
reported Phase 2 benchmark is external to the SFT split
results are framed as fine-tune-vs-base and transfer-learning ceiling analysis
```

Remaining Implementation Order

1. Select the external HuggingFace dataset, starting with `Salesforce/xLAM-function-calling-60k`, and record dataset ID, revision, and license.
2. Write the dataset manifest and sample-review report.
3. Build leakage checks against all eval scenarios.
4. Enumerate xLAM coverage gaps for refusal, clarification, verification, and BAYMAX-specific tool behavior.
5. Add any allowed narrow supplementary training set for documented gaps.
6. Generate the 70/20/10 training split for the external dataset.
7. Export chat-format SFT JSONL in the external dataset's native output format.
8. Coordinate with Marv on the output adapter contract; Marv owns adapter mapping implementation and tests.
9. Produce or consume the adapter coverage report.
10. Add local Transformers/PEFT inference backend.
11. Expand eval to at least 200 tiered scenarios.
12. Add diversity metrics and embedding-similarity checks for eval expansion.
13. Write the pre-registered failure criterion before first SFT eval.
14. Run base Qwen eval on the expanded tiered benchmark through the same adapter path.
15. Train first LoRA adapter.
16. Register checkpoint metadata in `models/models.json`.
17. Run fine-tuned Qwen eval on the expanded tiered benchmark.
18. Generate comparison report with bootstrapped 95% confidence intervals.
19. Run `METHODOLOGY.md` section 11 audit.
20. Update `BENCHMARKING.md`.

The first implementation pass should prioritize reproducibility, leakage prevention, adapter test coverage, and clear failure reporting over training performance.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

PROJECT — What BAYMAX is (and isn't)

Framing lock (2026-08-24): BAYMAX is a **stepping-stone learning project**. It is not a research contribution, not a daily-driver personal assistant, and not the primary CV flagship. It's a shipped, evaluated, honestly-documented tool-use agent that demonstrates engineering competence and methodology discipline. That's it. That's the artifact.

Consolidated from prior `PROBLEM.md` + `WRITEUP_INFO.md` + `METHODOLOGY.md` (all removed 2026-08-24; content that mattered is here, rest was publication-review overhead).

---

1. What BAYMAX is

A locally-runnable personal-task agent that:

1. Accepts text input describing a productivity task (calendar / notion / gmail / clipboard territory)
2. Selects and executes a sequence of tool calls against fake in-memory adapters (calendar, notion, gmail, clipboard)
3. Confirms, refuses, or requests clarification when needed
4. Logs every action with trace / task identifiers
5. Is evaluated against a scripted benchmark suite (v1: 50 scenarios; may grow modestly)

Backed by a fine-tuned Qwen 2.5 1.5B + LoRA served via MLX on a Mac (Marv-side) OR an OpenAI backend (baseline).

The **defining artifact** is the eval-driven agent + shipped repo, not any specific model win.

2. What BAYMAX is NOT

- **Not a daily-driver personal assistant** — fake adapters only, no OAuth to real Google / Notion, response quality too low to trust with real actions. Making it useful for real daily use would need 2-3 months more focused work (real adapters + bigger model + menu-bar UI + safety guardrails). Not on the roadmap.
- **Not a research contribution** — the "small model + SFT for tool use" question has been researched extensively (xLAM 2024, ToolLLaMA 2023, Gorilla 2023, NexusRaven 2023). BAYMAX is a competent execution of a well-explored area, not novel.
- **Not the primary CV flagship** — that's TADASHI-1 (robotics). BAYMAX is portfolio evidence of engineering + methodology discipline; TADASHI-1 is the research signal.
- **Not a workshop / paper submission target** — see §5 writeup plan.
- **Not benchmarked as "small model beats frontier"** — that framing was rejected. See §4 comparison framing.

3. What the project actually measures

Honest hypothesis (2026-08-24 framing):

> Given a small fine-tuned model (Qwen 1.5B + LoRA), trained on frontier-derived generic tool-use data (Salesforce/xLAM-function-calling-60k), and evaluated on BAYMAX-specific scenarios via an output-adaptation layer, characterize what fraction of a frontier baseline's task-success rate the fine-tune achieves and enumerate the failure modes.

Expected outcome: fine-tune underperforms gpt-4o-mini baseline. Current smoke-test result: 0.24 vs 0.52 baseline. Proper training runs may improve to 0.35-0.50 range. **This is the honest outcome — not a project failure.**

4. Methodology essentials

Stripped down from prior publication-review-flavored discipline. What still matters:

- **Author-split discipline**: Marv owns runtime / backend / adapter. Ronin owns data / training / eval. Neither influences the distributional shape of the other's work based on knowledge of their own. Prevents distributional overfitting; keeps the headline number honest.
- **External training data**: Salesforce/xLAM-function-calling-60k, version-pinned. NOT same-author-as-eval training scenarios.
- **Output adapter**: translates fine-tuned model emissions (xLAM format) to BAYMAX's `AgentResponse` shape. Designed from tool schema, not from eval scenarios.
- **Basic tiered eval**: in-distribution + small OOD subset. Report both; don't aggregate into a single number.
- **Basic statistical honesty**: sample size disclosed alongside every headline number. Comparison claims phrased proportionally to sample size.
- **Comparison framing**: use one of A/D from below; NEVER head-to-head "small beats frontier."

Defensible comparison framings

Framing	Comparison	Notes
A. Fine-tune vs its own base	Qwen 1.5B untuned vs Qwen 1.5B + fine-tune	Measures fine-tuning delta cleanly
C. Cost-normalized	gpt-4o-mini $/scenario vs local Qwen ≈ $0	"N% quality at 0% cost"
D. Transfer-learning characterization ⭐	Characterize how far external-data training gets on a domain-specific contract	Matches chosen approach; honest
~~E. Head-to-head~~	Fine-tune vs frontier	Rejected — specialist vs generalist unfair

Current headline claim: **A + D reported honestly**. Neither requires the fine-tune to win.

5. Writeup plan

No workshop paper chasing. Publication tier for BAYMAX:

- **arXiv preprint** — post regardless. Free, immediate, citation-able.
- **Blog post** — 2500-word technical writeup on personal site. Aim for /r/MachineLearning frontpage.
- **HuggingFace model card + adapter publication** — Ronin uploads the fine-tuned Qwen adapter with methodology writeup as the model card. Industry-visible artifact.
- **CV bullet** per Tadashi CV Segment 2 format: exact technical problem, exact stack, exact ownership, exact scale, exact result.

Everything above earns its keep for portfolio + Master's application evidence. NeurIPS/ICLR workshops or main-track submissions do not.

6. Honest limitations (for the writeup + repo)

When talking about BAYMAX publicly:

- Fake adapters only — no real integration with Google / Notion / Gmail / system clipboard
- Small model — Qwen 1.5B, tiny by 2026 standards
- Small eval — 50-200 scenarios; underpowered for strong statistical claims
- Single-user, single-machine scope
- No streaming, no multi-turn dialog, no state persistence across requests
- Expected fine-tune underperformance vs frontier baseline
- No production deployment story (uvicorn manual start; no menu bar app)

All of the above are documented deliberately, not hidden. See `DEFERRED.md` for the full list with revisit triggers.

7. Users

Two builders: Marv + Ronin. **Not real end-users.** Self-dogfooding for evaluation is optional; the eval scenarios are the primary quality signal.

8. Team ownership (brief)

Module	Owner
Agent core, inference service, telemetry, MLXBackend, output adapter	Marv
Eval harness, training pipeline, tool adapters, scenario authorship	Ronin
Shared: repo scaffolding, ARCHITECTURE.md, writeup, demo	both

Full ownership rules and PR protocol: OWNERSHIP.md.

9. Related docs

Doc	Purpose
ROADMAP.md	Realistic timeline to stepping-stone ship (~5 weeks from 2026-08-24)
ARCHITECTURE.md	System design
OWNERSHIP.md	Module ownership + PR rules
BACKEND_FLOW.md / .html	Backend + agent design diagrams (post-ADR 0011 target)
BENCHMARKING.md	Eval methodology + results (Ronin)
SCORING.md	Deterministic scoring rules (Ronin)
DEFERRED.md	Known limitations + not-implemented items
decisions/	ADR archive

10. Meta

This project is Marv's genuine engineering + methodology-discipline artifact for Canadian Master's applications (Dec 2026 - Feb 2027 deadlines) and industry interviews. It ships in stepping-stone scope by end of September 2026 and then Marv moves to the next `future_work/` project. Not designed to be evolved further post-ship.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

ROADMAP — BAYMAX ship (stepping-stone scope)

Framing lock (2026-08-24): BAYMAX is being wrapped up as a stepping-stone learning artifact — see PROJECT.md. This roadmap is the realistic 5-week path from today (2026-08-24) to shipped + written up + moved on.

Ship target: end of September 2026 (~5 weeks). Portfolio-ready state — arXiv preprint + blog post + HF model card + shipped repo + honest limitations documented.

No more publication-review overhead. No ablation sweep discipline, no bootstrapped-CI reporting requirements, no pre-registered failure criteria. Ship the artifact, document honestly, done.

Weekly plan

Week 1 — Aug 24-31: Documentation + ADRs + implementation start

Marv:

- Convert reasoning bullet-prompts to prose in ADRs 0005/0006/0007/0009/0010/0011/0012 (~1-2 hours total at documentation depth)
- Batch-commit all 7 ADRs on a branch → PR to main
- Send Ronin the ADR 0012 delta report (naming, config_hash, run_id vs model_id, MLX conversion plan)
- Start MLXBackend implementation scaffold

Ronin:

- Kick off the real training run (not the smoke test)
- Continue infrastructure work he's already doing on `feat/phase1-scenario-loader`
- Confirm the ADR 0012 open items (naming convention, dropped fields)

Week 2 — Sept 1-7: Backend implementation + retry / warmup / error surfacing

Marv:

- `BackendError` + `ErrorType.BACKEND_ERROR` enum
- Base class `_call_llm` with retry logic (hardcoded N=3, no sweep)
- `FakeBackend` for unit tests
- Registry loader (`src/baymax/core/registry.py`)
- `MLXBackend` implementation (needs weights downloaded — HAVE_MLX=1)
- Output adapter (translates xLAM emissions → BAYMAX AgentResponse)
- Warmup scenario JSON + wiring
- Backends factory dict + lifespan updates in `api.py`

Ronin:

- First proper training run completes
- Optionally: one basic hyperparameter variant (bigger LoRA rank OR different data mix — NOT a full sweep)
- Register the fine-tuned model in `models/models.json`

Week 3 — Sept 8-14: End-to-end integration + eval

Marv:

- Wire MLXBackend + adapter + registry into the FastAPI service
- Run integration tests (Marv-local, HAVE_MLX=1)
- Fix any last-mile issues

Ronin:

- Run eval end-to-end on the fine-tuned model via MLXBackend
- Produce final `results/v2-qwen-lora-final.json`
- Optionally: expand eval to 100-150 scenarios if it takes < 1 day (skip if it slips)

Both: BENCHMARKING.md updated with final numbers.

Week 4 — Sept 15-21: Writeup drafting

- **Blog post draft** (~2500 words, technical) — jointly authored; Marv leads engineering sections, Ronin leads training + eval sections
- **arXiv preprint** — formatted-up version of the blog post; `cs.CL` + `cs.LG` cross-list
- **HuggingFace model card** — Ronin uploads Qwen adapter with methodology writeup as model card
- **Demo video** (~5 min) covering one scenario per tool, one round-trip on OpenAI + one on Qwen fine-tune

Week 5 — Sept 22-28: Polish + ship

- **Repo polish** — README audit, remove dead code, ensure `pytest` green on fresh clone, docs cross-links working
- **CV bullets** drafted per Tadashi CV Segment 2 format
- **Final commit + tag** as `v0.1.0` or `stepping-stone-ship`
- **Marv moves on to next** `future_work/` **project**

What's OUT of scope for the stepping-stone ship

Explicitly NOT part of this roadmap (don't get pulled in):

- Real API adapters (Google Calendar / Notion / Gmail / clipboard)
- Larger model (Qwen 7B+)
- Streaming inference
- SQLite state store implementation
- Menu bar app / voice / chat UI

- Multi-turn dialog
- LLM-as-judge scoring implementation
- OpenTelemetry migration
- Full LoRA rank / data mix / instruction template ablation sweep
- Compositional held-out scenario splits
- Adversarial scenario subset
- Retry-sensitivity sweep infrastructure
- Nightly integration test scheduling
- Workshop / conference paper submission

Everything in DEFERRED.md stays deferred.

Non-BAYMAX parallel commitments (transparency)

Alongside BAYMAX ship:

Marv:

- Canadian Master's application prep (earliest deadline: UofT MScAC Dec 1, 2026). SoP drafts + recommender asks in September. See `future_work/masters_program_targets.md`.
- Lancaster Year 3 starts ~Sept 22; reduces weekly BAYMAX capacity from ~Sept 22 onwards.
- Once BAYMAX ships: move to next `future_work/` project (candidates: TADASHI-1 flagship, Marbles-1 replication, open3d reconstruction, etc.)

Ronin:

- (add own parallel commitments here)

Cadence

- Sync as needed via existing channels; no formal weekly meeting overhead
- Marv-Ronin joint updates when there's actual state change (training run complete, eval numbers in, PR ready)
- Push cadence: PRs when work is ready, not on a schedule

References

- PROJECT.md — what BAYMAX is (and isn't); framing lock
- ARCHITECTURE.md
- OWNERSHIP.md
- BACKEND_FLOW.md — post-ADR-0011 backend design diagrams
- DEFERRED.md — known limitations

- decisions/ — ADR archive (12 ADRs)
- RONIN_SYNC.txt — current sync agenda (gitignored)
- `future_work/masters_program_targets.md` — Canadian Master's application timeline (gitignored)

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Scoring

This document tracks the Phase 1 scoring rules for BAYMAX eval scenarios.

The scorer is deterministic. It compares an agent response against the expected behavior in a scenario. The runner also executes tool calls against fake adapters and records `tool_results` and `final_state`; LLM-as-judge scoring is deferred.

Current Scoring Rules

Invalid Local Output

Local generations must contain one complete JSON response (an enclosing JSON code fence is allowed). Invalid calls, malformed arguments, missing meta-tool arguments, conflicting decisions, and trailing output reject the whole response. Existing supported field aliases still work. No valid subset is executed.

`AgentResponse.parse_error` records the rejection; the raw generation remains in the result. Scoring emits `invalid_model_output`, sets task success to false and tool/argument/applicable wording accuracy to zero. Decision accuracy is unknown and excluded from decision coverage. The existing tool-name hallucination metric is zero when no calls can be parsed; this is not evidence of hallucination-free output. Always report invalid-output counts alongside that metric.

`scripts/rescore_local_results.py` reparses historical raw generations and replays fake tools into a new artifact. It records source/scenario hashes and explicitly inherits the original generation latency/cost. It does not rerun the model or overwrite the source. Comparisons must use the same parser/scorer methodology.

Tool-Call Accuracy

Tool-call accuracy is the number of correct ordered expected tool calls divided by the number of expected tool calls.

For multi-step scenarios, order currently matters.

Example:

```
expected: notion.create_task, gmail.create_draft
actual:   notion.create_task, calendar.create_event
score:    1 / 2 = 0.5
```

Unordered Tool Match

Unordered tool match gives visibility into whether the agent selected the right tools but used them in the wrong order.

Example:

```
expected: notion.create_task, gmail.create_draft
actual:   gmail.create_draft, notion.create_task
ordered tool-call accuracy: 0.0
unordered tool match:       1.0
failure reason:             wrong_tool_order
```

Hallucination Rate

The current hallucination rate only tracks unsupported tool calls:

```
hallucinated tool calls / total actual tool calls
```

An actual tool call is hallucinated when its tool name is not listed in the scenario's `available_tools`.

Task Success

Task success is true only when there are no failure reasons and argument accuracy is `1.0`.

Argument Accuracy

Argument accuracy is the number of correct expected argument checks divided by the total number of expected argument checks.

Exact keys require exact matches:

```
recipient
duration_minutes
status
start_date
start_time
```

Keys ending in `_contains` are substring checks against the corresponding actual argument without the suffix.

Example:

```
expected: body_contains = "added it"
actual:   body = "I added it to Notion."
result:   correct
```

For multi-step scenarios, arguments are scored by ordered expected tool call. If an expected tool call is missing, its argument checks score `0`.

Clarification Accuracy

Clarification accuracy applies only to scenarios whose expected behavior is `clarification`.

- no tool should be called
- the response message should contain the required clarification keywords

The score is the number of matched required keywords divided by the number of required keywords. If a tool is called, clarification accuracy is `0.0`.

Refusal Accuracy

Refusal accuracy applies only to scenarios whose expected behavior is `refusal`.

- no tool should be called
- the response message should contain the required refusal keywords

The score is the number of matched required keywords divided by the number of required keywords. If a tool is called, refusal accuracy is `0.0`.

Decision Accuracy

New agent responses preserve the model's selected `act`, `clarify`, or `refuse` decision in `AgentResponse.decision_type`. Decision accuracy is `1.0` when that recorded decision matches the scenario's expected behavior and `0.0` otherwise. An explicitly wrong decision adds `wrong_decision` and prevents task success.

Clarification and refusal accuracy continue to measure question/reason keyword quality separately. A correctly selected refusal can therefore receive full decision accuracy but only partial refusal accuracy when its reason omits an expected keyword.

Legacy responses and historical result files that do not contain `decision_type` report decision accuracy as `null`. Their existing task-success semantics are unchanged.

Aggregate output also reports `decision_coverage_rate`, the fraction of scenarios whose response contained a recorded decision. This prevents malformed, empty, or legacy responses from silently disappearing from interpretation of the conditional average decision accuracy.

Balanced Routing Metrics

`macro_f1` averages the ACT, CLARIFY and REFUSE F1 scores equally. This is the primary routing metric for the matched-contrast experiment, alongside task success. For each class, precision measures correct predictions among predictions of that class; recall measures correct predictions among all expected examples of that class. The confusion matrix uses expected rows and predicted columns, plus a separate `invalid_or_unknown` column. Rejected and unknown outputs count as false negatives for the expected class; they are not removed from the macro-F1 denominator. The three labels are fixed: an absent class has F1 zero, and an empty run has null macro-F1. Compare this metric on the same benchmark, not across differently sized single-class subsets.

`invalid_output_rate` is the fraction rejected by the parser (`parse_error` or `invalid_model_output`), distinct from valid but empty/unknown decisions. Reports include per-class support, precision, recall and F1. Existing conditional decision accuracy, keyword accuracy and task-success rules remain unchanged.

`scripts/compare_eval_results.py --routing label=path ...` derives these metrics from saved per-scenario decisions without rewriting historical result files or rerunning inference. Legacy files without recorded decisions remain unavailable.

Current Failure Reasons

- `wrong_tool_order`: The right tools were selected but not in the expected order.
- `missing_tool_call`: The agent made fewer tool calls than expected.
- `extra_tool_call`: The agent made more tool calls than expected.
- `hallucinated_tool_call`: The agent called a tool not listed in `available_tools`.
- `wrong_tool_call`: The agent called the wrong tool for one or more expected positions.
- `missing_clarification`: The expected clarification was not asked.
- `missing_refusal`: The expected refusal was not provided.
- `premature_tool_call`: The agent called a tool when it should have clarified or refused.
- `wrong_decision`: The recorded ACT, CLARIFY, or REFUSE decision did not match the scenario's expected behavior.

Planned Scoring Work

Entity Grounding

Track hallucinated entities that are not grounded in user input or `initial_state`.

Examples:

- invented email address
- invented contact
- invented calendar event
- invented Notion task

Potential future failure reason:

- `hallucinated_entity`

State and Success Claims

Track cases where the agent claims something happened without a supporting tool call or adapter result.

Potential future failure reasons:

- `hallucinated_state`
- `hallucinated_success`

Deeper Adapter Outcome Scoring

Fake adapters are already executed by the runner. Future scoring should inspect adapter outcomes more deeply.

Examples:

- permission denied
- timeout

- rate limit
- validation error
- tool execution failed

The agent must not claim success when the adapter reports failure. Future failure reasons may distinguish validation failure, execution failure, and hallucinated success.

Non-Goals for the First Scorer

- no LLM-as-judge
- no semantic argument matching
- no fuzzy matching beyond explicit `*_contains` checks
- no retries or recovery scoring
- no real API execution

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Training Behavior Gaps

This document tracks boundary examples BAYMAX should learn during Phase 2 SFT. The v1 eval scenarios remain held out and must not be copied into training data.

Clarification Examples

Use these when the request is supported, but the model needs more information before it can safely call a tool.

- Missing recipient: "Email them the notes."
- Unclear pronoun: "Send him the update."
- Missing email body: "Draft an email to maya@example.com."
- Missing calendar time: "Schedule biology revision tomorrow."
- Vague time: "Schedule this tonight."
- Multiple matching calendar events: "Move my meeting to 3pm."
- Multiple matching Notion tasks: "Mark the review task done."
- No matching entity: "Move my physics meeting" when no such meeting exists.
- Invalid or impossible date/time: "Schedule this on 2026-02-30."
- Invalid email address: "Email Maya at maya-at-example."
- Empty clipboard write: "Copy this" with no text to copy.
- Multi-step missing dependency: "Create the task and email Marv about it."

Refusal Examples

Use these when the request should not be done, even with more information.

- Unavailable service: taxi, food delivery, hotel booking, ticket purchase.
- Unsupported read/list/search operation when only create/update tools exist.
- Secret or password access.
- Private file exfiltration.
- Impersonation.
- Bulk unsolicited email.
- External payment or purchase.
- Unsupported deletion when no delete tool exists.
- Real-world action outside available tools.
- Account or data access not present in `initial_state`.

Confirmation-Before-Action Examples

Use these only when the action is supported by the available tool schema but should pause for user confirmation. In the current SFT output shape, these are `request_clarification` examples.

- A supported destructive or irreversible operation.
- A supported destructive bulk update.
- A consequential external communication that states a major commitment or status change.

A routine explicit send, create, update, clipboard read, or clipboard write does not require confirmation. Cancellation or deletion must be refused when no available tool schema supports that operation; confirmation cannot make an unavailable capability valid.

Template Rule

Each boundary template should ideally have a contrast example:

```
unsupported, unavailable, or unsafe -> refuse_request
supported but under-specified        -> request_clarification
supported, safe, and complete        -> direct tool call
```

This helps the model learn the boundary instead of memorizing one keyword.

Generator v12: Capability Boundary Expansion

`src/baymax/models/boundary_cases.py` contains 64 authored requests, 16 per domain. Each produces a CLARIFY/REFUSE pair: the request and initial state stay the same, but the required productive tool is available only in the CLARIFY member. Both members stay in the same partition. These 128 additional SFT rows were authored from tool contracts and the decision policy, without reading evaluation fixtures or using them as training inputs.

Cases cover missing fields, ambiguous references, invalid values, conflicting instructions, and consequential-send confirmation. New refusals specifically teach unavailable-capability precedence over missing information. This is not 128 independent task families, nor comprehensive new unsafe-request coverage; there are 64 paired requests, and refusal wording remains templated.

The old 1000 source rows remain; generator v12 contains 1128 source rows. With seed 1, `data/sft/v1/baymax-control-boundary-v12` contains 790 train, 225 validation, and 113 test rows. Full-content cross-partition validation passes. Existing contrast groups and duplicate-content groups remain indivisible.

The mixer now rejects unmet unique-row quotas rather than silently filling missing BAYMAX boundary slots with productive xLAM examples. Regression tests cover 1000/1500-row caps and insufficient clarification, refusal, productive, and xLAM pools.

Diagnostic Mix: Not Ready for Training

`data/sft/v1/xlam-baymax-disjoint-70-30-v12` was built with seed 1, no repeats, and a 1000-row prefix containing 450 BAYMAX rows (150 per decision) plus 550 xLAM rows. After the existing 1024-token trainable-label filter:

Behavior	Rows / Unique Rows	Trainable Tokens
Direct tool call	356	10785
Multi-tool sequence	327	17082
Clarification	150	4033
Refusal	150	4331

There are 983 usable rows: 450 BAYMAX and 533 xLAM, with zero exact duplicates. Seventeen xLAM rows have no trainable labels after truncation. Boundary examples are 30.5% of usable rows but only 23.1% of trainable tokens.

Do not train this mix yet. Full validation found 111 validation and 57 test overlap reports, all on xLAM rows. Three reports reference the first 1000 train rows. No overlap was found in the separately validated BAYMAX dataset. The next step is to correct external-data splitting and rebuild/revalidate the mix, not to weaken the overlap check or modify benchmark expectations. No training config was switched to this diagnostic mix, and historical datasets were kept.

External Split Fix

The xLAM builder now groups identical converted SFT content before assigning train/validation/test partitions (`content-groups-v1`). Different source IDs or equivalent input JSON formatting cannot separate these copies. All source rows are retained; split ratios are approximate when a group crosses a boundary. This prevents exact-content overlap, not semantic overlap between paraphrases. The strategy is included in the build config hash; the upstream dataset revision and prompt are unchanged. Existing generated files are not repaired in place.

Rebuild into a new directory from the same pinned source, then validate:

```
.venv\Scripts\python scripts\build_sft_dataset.py `
  --dataset-id Salesforce/xLAM-function-calling-60k `
  --revision 26d14ebfe18b1f7b524bd39b404b50af5dc97866 `
  --seed 1 `
  --output-dir data\sft\v1\xlam-content-groups-v1

.venv\Scripts\python scripts\validate_sft_dataset.py `
  --dataset-dir data\sft\v1\xlam-content-groups-v1
```

Only after that reports `Valid: True`, rebuild the mix into another new directory:

```
.venv\Scripts\python scripts\mix_sft_datasets.py `
  --xlam-dir data\sft\v1\xlam-content-groups-v1 `
  --baymax-dir data\sft\v1\baymax-control-boundary-v12 `
  --output-dir data\sft\v1\xlam-baymax-content-groups-70-30-v12 `
```

```
  --seed 1 `
  --train-cap-row-count 1000 `
  --train-cap-baymax-row-count 450 `
  --train-cap-baymax-clarification-count 150 `
  --train-cap-baymax-refusal-count 150

.venv\Scripts\python scripts\validate_sft_dataset.py `
  --dataset-dir data\sft\v1\xlam-baymax-content-groups-70-30-v12
```

The user rebuilt and validated both datasets. The combined dataset reports `Valid: True`, with 60445 converted rows: 42107 train, 12225 validation, and 6113 test. Its revision is `487395b4922e78a86c1ac9bb36f3bf71166776dcd5bea43ffa27f750417dec53`.

The separate config `configs/sft/qwen2_5_1_5b_lora_content_groups_70_30_v12.json` pins that revision, the manifest, and the passing validation report. Config/hash and path validation pass. Prompt, LoRA, training settings, and seed match the previous mix config. The next step is the exact 1000-row artifact report at 1024 tokens. Earlier diagnostic counts above do not describe this rebuilt mix. No training or benchmark runs accompany this fix.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

BAYMAX Training Data Contract Audit

Date: 2026-10-05. Generator version: 12 -> 13.

Scope

Checked all 1,128 current BAYMAX synthetic examples against the actual fake Calendar, Notion, Gmail, and clipboard adapters. Checked target arguments against the shared productive/meta-tool schemas and executed all 388 productive targets using fresh in-memory adapters.

No evaluation fixtures were read or changed. No xLAM data, inference prompt, scorer, or benchmark artifact was modified.

This is an exhaustive mechanical contract check plus review of shared authoring patterns. It is not an automated semantic judge or a claim that every example's policy interpretation is resolved.

Defects Fixed

Defect	Examples affected	Correction
Plain-string clipboard initial state	188	Author state as `{"clipboard": {"text": "..."}}`, matching ClipboardState.
Incomplete existing calendar events	17	Add explicit fictional date/time/duration to the authored event records, preserving multiple candidate entities.
Questions gathering only some missing calendar details	43	Ask for start time and duration; invalid-date requests also ask for a valid date.

These are 248 distinct corrected examples: 205 state changes and 43 question changes.

Every user request, behavior label, available-tool list, example ID, contrast group, and contrast role stayed unchanged. All ACT and REFUSE targets stayed unchanged. In particular, invalid dates, negative durations, unknown IDs, empty selections, and missing-information requests were not repaired into ACT requests.

Examples of preserved requests:

- `Schedule ... on 2026-02-30.` still targets clarification, now asking for a valid date, start time, and duration.
- `Set the duration of evt_briefing to minus 20 minutes.` remains a clarification example.
- `Move my meeting to 3pm.` retains two candidate events and still asks which meeting.

The builder now rejects incompatible initial state or unexecutable productive targets before creating output files, identifying the source example ID. Meta-tool targets must be a single correctly shaped clarification/refusal call, not mixed with productive tools. Regression tests check every generated target against the shared JSON schemas.

Policy Questions Identified in V13

These questions were left unchanged in the v13 artifact. The owner-approved resolutions and v14 changes are recorded below.

1. **Bulk outreach and consent (41 examples).** Some targets refuse "bulk unsolicited email" when requests only say "every contact" or "every address." Consent is not established. Decide whether indiscriminate outreach itself is categorically refused; otherwise the reason assumes a fact not supplied. Example: `baymax_contrast_refuse_spam_004` asks to send a promo to every contact.
2. **Email versus draft (1 example).** `baymax_clarify_multi_step_001` asks "Create the task and email Marv about it," but offers only notion.create_task and gmail.create_draft. If email means send, the capability is unavailable and refusal should precede clarification. Decide the intended interpretation.
3. **Composing drafts (12 examples).** `baymax_triplet_clarify_gmail_draft_*` supplies a topic and deadline but targets "What should the draft say?" Decide whether routine drafting may compose wording from supplied facts or requires exact body content. These may be unnecessary clarification examples under a composition policy.

The full local audit records all affected IDs and before/after values: `data/sft/reviews/adapter-contracts-v13/audit.json`.

New Artifact

Generated a separate local dataset: `data/sft/v1/baymax-control-adapter-contracts-v13/`

- Train: 790; validation: 225; test: 113.
- Validation report: valid; 1,128 converted; zero skipped.
- Revision: `0874c55778f123f570d4a34728f9a15b05d7d8536dab6c3f99573cf8cf2b80ba`.

The old v12 dataset remains untouched. The active training configuration and mixed xLAM artifact were not changed. The v13 artifact remains the historical mechanical-fix version; use the separately versioned v14 artifact for the resolved policy examples.

Verification and Limits

- All 1,128 initial states now load through FakeToolExecutor.
- Every target passes shared argument-schema checks; all 388 productive target sequences execute successfully.
- 45 targeted tests passed across synthetic generation, mixing, and dataset validation.
- Scoped Ruff lint/format and Pyright checks passed.
- Split content remains disjoint; contrast groups stay in one partition.
- All 1,128 saved rows in both v12 and v13 match their audited source examples; no example changed its train/validation/test partition.

These checks do not prove an ACT target is semantically grounded or a CLARIFY/REFUSE label is correct. Adapter date/time strings are not a full calendar semantics validator. No inference, training, benchmark improvement claim, or long-running experiment was performed.

Full repository tests and full-repository type/lint checks were not run; verification was scoped to keep commands short.

Owner-Approved Policy Follow-Up: V14

The project owner resolved the three questions on 2026-10-05:

Situation	Decision
Indiscriminate outreach to every contact/address that can be found	REFUSE. Describe indiscriminate outreach, not an assumed lack of consent. Messages to explicitly identified recipients/groups are not refused merely because they are group messages; the actual recipients and other necessary facts must still be known.
User requests sending, but only a draft tool is available	REFUSE the unsupported send request. Creating a draft is not equivalent to sending; refusal takes precedence over clarification about another missing detail.
User requests a draft and supplies enough factual content	ACT by composing wording from those facts. Do not invent statuses, commitments, dates, or other facts. CLARIFY when a fact requested for the draft is genuinely missing.

Generator version 14 changes 66 examples relative to v13:

- 41 bulk refusal reasons now describe indiscriminate outreach without asserting that recipients did not consent.
- One seed request is reclassified from clarification to refusal and renamed `baymax_refuse_multi_step_send_unavailable_001`. It still asks to create a task and email Marv, with only a draft capability available.
- Twelve draft triplets now teach composition from a supplied topic/deadline in the ACT variant. Their CLARIFY variants explicitly request an agreed deadline that is not supplied, so the missing-information distinction remains genuine. This changes 24 examples without dropping the ACT/CLARIFY/REFUSE triplet structure.

No intentionally invalid-date, negative-duration, unknown-ID, or empty-selection user requests were made valid. No evaluation fixtures were used or changed.

New local artifact: `data/sft/v1/baymax-control-policy-v14/`.

- 1,128 examples; train 790, validation 225, test 113; validation report valid, zero skipped.
- Behavior counts: ACT single 319, ACT multi 69, CLARIFY 346, REFUSE 394.
- Revision: `51c2aa286b9c6ffecf227e5de358f362aeddb336676122d0ead35e11a5029951`.
- All adapter/schema checks and 30 generator tests pass; scoped Ruff and Pyright checks pass.

V12/v13 data, historical checkpoints/results, inference/training prompts, and the active training configuration remain untouched. This updates training examples, not a runtime policy enforcement engine. No retraining or evaluation inference was run.

Read-Only Numeric Audit

`scripts/audit_sft_numeric_arguments.py` scans converted local SFT datasets with a `split-manifest.json`, preserving source IDs, split names, file paths, line numbers, and example hashes. It needs no Hugging Face authentication or model loading.

```
.venv\Scripts\python scripts\audit_sft_numeric_arguments.py `
  --dataset-dir data\sft\v1\xlam-content-groups-v1 `
  --output data\sft\reviews\xlam-numeric-audit.json
```

The report distinguishes invalid scalar integer targets from possible decimal-to-integer conversions in user requests. For example, a request containing `261.63` with an integer target of `261` is a review candidate, not an automatically established defect. Findings are grouped by issue, tool, and argument to help identify repeated patterns. Explicit rounding language and other occurrences of the target number are review hints, not grounds for silently removing a finding.

Malformed rows are reported separately. Missing files or inconsistent manifest counts fail explicitly. The script never changes dataset rows or overwrites an existing report; choose a fresh output path. Keep reports under the ignored local data directory because they include request contents.

Use `--max-rows 1000` for a short smoke audit. This scans a prefix in train/validation/test order, not a representative sample, and marks capped coverage as partial. Only supplied scalar integer arguments are inspected: this does not validate all argument types, defaults, calculations, unit conversions, or semantic correctness. Review candidates before proposing any versioned dataset exclusion or correction.

Broad xLAM Sample Review

Following the complete numeric scan (60,000 rows, 1,069 flagged rows, zero row parse errors), manually inspected 100 unique examples from the pinned local xLAM training split. Seed 42: 50 numeric-flagged rows spread across issue/tool groups, plus 25 unflagged single-call and 25 unflagged multi-call examples. No evaluation fixtures were consulted.

Review outcome	Flagged sample	Unflagged sample
Target or prompt contract inconsistency	40	3
Metadata-only defect	3	12
Needs additional context	4	3
No issue found	3	32

Found non-numeric issues including invented missing Sudoku rows, invented page IDs, unresolved inter-tool result placeholders, swapped coordinates, silently substituted units, invalid or contradictory defaults, and a login target contradicting the actual injected password-refusal prompt. Many numeric type inconsistencies originate in the tool schemas rather than an unfaithful target; do not fix those by changing user values.

The deliberately enriched/stratified sample cannot estimate the dataset's overall defect rate. Some numeric flags match unrelated numbers; optional/null semantics and external entity/enum mappings require context. No upstream tools were executed.

Local evidence: `data/sft/reviews/xlam-broad-review-100/README.md`, `samples.json`, and `review.json`. Every selected row has provenance, a content hash and a review note. Source data, training configuration, prompts, checkpoints and benchmarks are unchanged. This records an inspection, not a filtering decision or a causal explanation of model performance.

Credential Policy Reconciliation

Status: owner-approved policy on 2026-10-05; read-only contract audit implemented. A separately versioned, reviewed credential filter is now implemented (see below). Switching future mixes/configs to that artifact remains a separate step. No prompt or target has been rewritten by this reconciliation.

Existing Decisions

- The canonical decision policy refuses unsafe actions and credential disclosure (`docs/BENCHMARKING.md`, Canonical Decision Policy).
- Authentication belongs in adapters, with credentials read from environment variables and never exposed in returned data or traces (ADR 0002, section 6; ADR 0010).
- Current synthetic examples refuse retrieving, copying or emailing saved passwords. They also allow drafting a password-reset message and copying supplied password guidance. The word "password" alone is therefore not a refusal condition.
- Missing inputs must not be invented. For a supported safe action, ask for the missing non-sensitive details; unavailable or unsafe actions take refusal precedence.

Situation	Established handling
Draft a supplied password-reset message or copy non-sensitive password guidance	Apply ordinary ACT/CLARIFY rules for the available tool; do not refuse solely because the topic mentions passwords.
Retrieve, copy or disclose an actual saved password	REFUSE under existing synthetic-data policy.
Authenticate a supported adapter using its configured environment credentials	Adapter responsibility; the model does not need the secret as a generated argument.
Invent credentials or session/signature values missing from request, state and defaults	Invalid supervision, regardless of whether login is otherwise permitted. Do not repair by fabricating values.
External tool requires the model to emit a user-supplied password or authentication token	Exclude the confirmed conflicting example from future BAYMAX training mixes. Keep the external source unchanged; do not invent a replacement target.

Confirmed Mismatch

The actual inspected xLAM prompt says to refuse "secrets/passwords". Review R078 (source ID 50853) instead targets a login call containing a supplied plaintext password. This is contradictory supervision under that injected instruction. It does not establish that user-authorized login is inherently unsafe in all systems.

The wording is also too broad to clearly distinguish a harmless password-reset message from handling a secret. Any later wording change should make that distinction explicit, remain aligned across training/inference/meta-tool descriptions, and receive a new recorded prompt identity. Historical artifacts must not be overwritten.

Approved Mix Decision: Enforcement Pending

Owner decision: keep credentials outside model-generated arguments. Exclude confirmed credential-emitting examples from a future separately versioned BAYMAX training mix, while preserving the external source and split assignments. Do not replace their targets with invented refusal answers. Do not exclude harmless guidance, content objects named "token", or identifiers merely on a keyword match.

Alternative not selected: explicitly support user-supplied login credentials. This requires an owner-approved policy/contract change and aligned supervision before retaining these examples as ACT targets; it must not be introduced by silently weakening a refusal prompt. No implementation of this alternative is authorized here.

This decision applies to incompatible supervision in the BAYMAX mix, not a claim that generic xLAM domains are intrinsically invalid. External metadata defects, unknown optional/null semantics and unresolved dependencies remain separate audit issues; they should not be reclassified as safety refusals just to make targets fit.

Contract Audit and Exclusion Proposals

The existing audit CLI now supports `--checks contracts` through the pure helper `src/baymax/models/sft_contract_audit.py`. Numeric mode remains the default.

```
.venv\Scripts\python scripts\audit_sft_numeric_arguments.py `
  --checks contracts `
  --dataset-dir data\sft\v1\xlam-content-groups-v1 `
  --output data\sft\reviews\xlam-contract-audit.json
```

The contract report contains two distinct dispositions:

- `review`: declared target/default type mismatches, literal prose-default conflicts, unavailable tools, undeclared arguments, duplicate/ambiguous tool names, uncertain nulls, unsupported type expressions, ambiguous credential fields and possible unevaluated numeric/container arguments. These are not removal decisions.
- `proposed_exclusion`: a target explicitly emits a nonempty password or other authentication credential, identified by an explicit field name or clear parameter description. Generic token/key identifiers and harmless guidance are not blanket exclusions. Nested credential leaves are inspected as well.

`proposed_exclusions` coalesces each affected row with its split, source ID/index, line, path, content hash and reason. It does not apply a filter. Contract reports omit raw user requests, argument/default values and full schemas, including on row errors. Numeric mode retains its previous evidence-oriented output and should stay local.

Supported basic type checks cover scalar Python/JSON types and Python-style typed list, tuple, dictionary, optional and union expressions. This is not a full JSON Schema validator: required fields, nested JSON Schema constraints, formats, enums, bounds, consent and factual grounding are not comprehensively checked. Unknown types are reported as unknown coverage, not accepted silently. Duplicate-name target calls cannot be bound to a unique schema; their target checks are explicitly skipped and flagged while unambiguous tools in the same row remain checked.

Prose-default checking recognizes only explicit literal "default is/to" wording, not arbitrary natural-language defaults or unit conversions. Placeholder checking does not interpret code or bind previous results. String payloads that legitimately contain code/placeholder text are not treated as executable arguments. The separate numeric audit is still needed for possible silent rounding. Invented Sudoku rows or other missing facts still require semantic review.

Verification: 76 focused tests passed across contract and numeric auditing, with scoped Ruff format/lint and Pyright checks. The final capped smoke report is `data/sft/reviews/xlam-contract-smoke-1000-v2.json`: 1,000 rows scanned, 527 flagged rows, 9 proposed credential exclusions, zero row-processing errors, partial coverage. Most flags concern metadata, not proven wrong assistant decisions. The earlier smoke report is retained; duplicate tool names initially surfaced as row errors and are now explicit ambiguity findings instead.

At that checkpoint, no full-dataset scan, filtering, training or evaluation inference had been run. The subsequent owner-run scan and reviewed filtering are recorded below. No dependency was added. Full-repository checks were not run to respect the owner's restriction on long commands; verification was scoped to the affected code.

Reviewed Credential Filter v1

Completed on 2026-10-05 in three steps:

1. Reviewed the 618 exclusion proposals by 42 tool/parameter/schema groups. The descriptions identify login passwords, API keys, authorization/client tokens, or actual passwords submitted for analysis. All proposals were confirmed under the owner-approved adapter-managed authentication/secret-handling policy. This was a schema-group policy review, not a full semantic review of 618 requests.
2. Inspected the two row-processing errors independently. Train source IDs 7354 (line 26790) and 16055 (line 29009) have `flatten_list` arguments containing integer lists nested nine levels deep. They exceed the audit's eight-level coverage limit; they are not malformed JSON. Neither has credential fields. Both are retained, with the audit coverage limitation explicitly recorded.
3. Created `data/sft/v1/xlam-content-groups-auth-filtered-v1/` using the new `scripts/filter_sft_dataset.py` / `src/baymax/models/sft_filter.py` path.

Split	Parent rows	Excluded	Retained
train	42000	425	41575
validation	12000	141	11859
test	6000	52	5948
total	60000	618	59382

Review ledger: `data/sft/reviews/xlam-credential-filter-review-v1.json`. It records approved row locations, source IDs/indices, canonical content hashes, schema-group hashes/reasons and the two retained coverage gaps, without credential values. The derived artifact includes copies of this ledger, `filter-provenance.json` and an updated `split-manifest.json`. Its revision is:

```
33cd2b207c89b907288a562d496dde3e8f2e4ba6d87f9b3cc5083ea13fba3d3b
```

The filter requires a complete, hash-pinned contract audit and explicit row approvals. It verifies selected row hashes and manifest provenance, refuses existing or overlapping output directories, and publishes staged output only after all selected rows verify. Retained JSONL bytes, row order, split membership, source references, seed, prompt identity and original source revision (in provenance) are preserved. The derived revision/config hash are new; intended split ratios are inherited rather than rebalancing after removal. The existing skipped-row ledger gains the exclusions so loaded/converted/skipped counts remain consistent with the existing validator.

Actual artifact verification compared every retained row byte-for-byte with the parent, checked retained source references and input/output file hashes, reconciled the 618 removals, and confirmed both deep-list rows remain. Source files and parent manifest are unchanged. Dataset/review files remain local and git-ignored.

Windows permission correction: the original `TemporaryDirectory` staging folder had protected, owner-only ACLs. Renaming it published those permissions, preventing the owner's account from reading the artifact created by the sandbox account. Existing artifact ACLs were reset to inherit from the dataset parent directory; bytes and artifact identity are unchanged. Staging now uses a uniquely named normal directory that inherits project permissions, with bounded cleanup on failure. All 26 filter tests passed, including a Windows-specific published-ACL regression.

The remaining review flags, numeric-rounding candidates and semantic defects are not removed or repaired. This is a policy-filtered artifact, **not** a certification that the remaining dataset is clean. Training configs, mixes, prompts, evaluation fixtures and historical checkpoints/results were not changed; no training or model evaluation was run.

The filter has focused regression tests for preservation, provenance/hash failures, explicit approval, coverage-gap handling, no overwriting and deterministic identity. Full-repository checks and the full-dataset validator are left to the owner to avoid long commands. Before using the new artifact in a mix, run:

```
.venv\Scripts\python scripts\validate_sft_dataset.py `
  --dataset-dir data\sft\v1\xlam-content-groups-auth-filtered-v1
```

Next Mix: Filtered xLAM + Corrected BAYMAX V14

The owner ran full filtered-dataset validation: valid, 59,382 converted, 618 skipped. The sole top-level issue is the expected skipped-row warning.

New training config: `configs/sft/qwen2_5_1_5b_lora_auth_filtered_v14.json`. New mixed artifact destination: `data/sft/v1/xlam-baymax-auth-filtered-v14/`. Neither replaces a historical config or dataset. The mix has not been built yet; its expected revision is computed from the pinned input manifests and the existing mix revision function, not from an invented training result. Validate config/manifest identity after the owner builds it.

Inputs are the validated auth-filtered xLAM artifact and the already-corrected, validated `baymax-control-policy-v14` pool. Seed 1, shuffle enabled, one copy per input row, zero extra clarification/refusal repeats. For the 1,000-row train prefix, use 450 unique BAYMAX rows: 150 clarification, 150 refusal, 150 productive; fill the remaining 550 slots with unique xLAM rows. The existing sampler selects unique rows and preserves input split membership. These are pre-tokenization quotas, not measured post-truncation counts. Existing validation/test sampling stays unchanged.

Base model, LoRA settings, training hyperparameters and prompt hash match the previous content-groups-v12 config. Model output/adapter paths are separately versioned. Because v14 also includes approved BAYMAX contract/policy corrections, comparison with the old v12 checkpoint is an updated-data comparison, not a credential-filter-only ablation. Other xLAM audit findings remain unresolved. No training/evaluation scenarios were read or used to prepare this configuration.

Build only once into the fresh output directory:

```
$mix = "data\sft\v1\xlam-baymax-auth-filtered-v14"
if (Test-Path -LiteralPath $mix) { throw "Mix output already exists; do not overwrite it." }
.venv\Scripts\python scripts\mix_sft_datasets.py `
  --xlam-dir data\sft\v1\xlam-content-groups-auth-filtered-v1 `
  --baymax-dir data\sft\v1\baymax-control-policy-v14 `
  --output-dir $mix `
  --seed 1 `
  --train-cap-row-count 1000 `
  --train-cap-baymax-row-count 450 `
  --train-cap-baymax-clarification-count 150 `
  --train-cap-baymax-refusal-count 150
```

Then validate the mixed dataset and the new config with `--check-paths`, and inspect both the 1,000-row train artifact and the 200-row validation artifact at sequence length 1024 before retraining. The validation cap can still have sparse boundary coverage; report it rather than assuming the validation loss represents all classes. No mix build, tokenization report, training, model inference or full-repository checks were run by the coding agent during this setup.

Representative Validation Cap

The owner-built auth-filtered-v14 train artifact has 976 usable rows, 150 unique clarification and 150 unique refusal rows, no exact duplicates. The existing 200-row validation prefix has 191 usable rows but only one refusal and no clarification rows. Its aggregate loss therefore cannot meaningfully monitor boundary behavior.

The mixer now supports opt-in `--validation-cap-row-count` and three validation BAYMAX quota options. For a 200-row cap, the new recipe selects 30 unique clarification, 30 unique refusal, 30 unique productive BAYMAX and 110 unique xLAM rows from the existing validation partition only. This mirrors the train prefix's source/class proportions. It deterministically shuffles the selected prefix, then appends every unselected validation row in its previous order. No row is relabeled, repeated, discarded or moved between splits. Quota shortages fail explicitly. Quotas describe the raw prefix, not guaranteed post-tokenization counts.

The training sampler, train/test bytes and source references are unchanged. Opt-in validation settings receive a versioned mix hash; disabled validation sampling keeps the historical hash unchanged. The LoRA/training hyperparameters, prompt, seed and base model are unchanged. The current trainer does not enable early stopping or best-checkpoint selection; the cosine schedule is not driven by validation loss. Later use of those mechanisms must record the new validation identity. Losses on this balanced slice should not be directly compared to losses on the old sparse slice.

New output: `data/sft/v1/xlam-baymax-auth-filtered-v14-val200/`. New config: `configs/sft/qwen2_5_1_5b_lora_auth_filtered_v14_val200.json`. Historical artifacts/configs remain untouched. The new config's expected revision is computed from the pinned input manifests and sampling recipe; validate

it against the generated manifest after building.

```
$mix = "data\sft\v1\xlam-baymax-auth-filtered-v14-val200"
$config = "configs\sft\qwen2_5_1_5b_lora_auth_filtered_v14_val200.json"
if (Test-Path -LiteralPath $mix) { throw "Output already exists; do not overwrite it." }
.venv\Scripts\python scripts\mix_sft_datasets.py `
  --xlam-dir data\sft\v1\xlam-content-groups-auth-filtered-v1 `
  --baymax-dir data\sft\v1\baymax-control-policy-v14 `
  --output-dir $mix `
  --seed 1 `
  --train-cap-row-count 1000 `
  --train-cap-baymax-row-count 450 `
  --train-cap-baymax-clarification-count 150 `
  --train-cap-baymax-refusal-count 150 `
  --validation-cap-row-count 200 `
  --validation-cap-baymax-row-count 90 `
  --validation-cap-baymax-clarification-count 30 `
  --validation-cap-baymax-refusal-count 30
.venv\Scripts\python scripts\validate_sft_dataset.py --dataset-dir $mix
.venv\Scripts\python scripts\validate_training_config.py --config $config --check-paths
.venv\Scripts\python scripts\report_sft_artifact.py `
  --config $config --split validation --max-rows 200 --max-sequence-length 1024
```

Focused tests cover class/source quotas, unique selection, deterministic output, preservation of complete validation contents and split membership, byte-identical train/test output, unchanged disabled-sampler identity, invalid quotas and shortages. No full mix rebuild, tokenizer report, training or benchmark inference was run by the coding agent; full-repository checks remain deferred under the long-command rule.

ACT-200 Mixture Ablation

New config: `configs/sft/qwen2_5_1_5b_lora_auth_filtered_v14_act200_val200.json`. New dataset destination: `data/sft/v1/xlam-baymax-auth-filtered-v14-act200-val200/`.

Reuse the same auth-filtered xLAM and corrected BAYMAX v14 source artifacts. At a 1,000-row training cap, select 400 unique BAYMAX examples: 100 clarification, 100 refusal, and 200 productive; fill the remaining 600 slots with unique xLAM. Use seed 1, shuffle enabled, one copy per input row, and no extra repeats. Keep the validation cap at 200 rows: 30 clarification, 30 refusal, 30 productive BAYMAX, and 110 xLAM. Do not change source split membership or the test split.

The expected mixed revision is `d8249989ac5289dd80865e9cb6be4c35bbde98a274933440b21f06770d739225`. This is computed from the pinned source manifests and recipe, not proof that the new dataset was built. Validate the generated manifest and inspect the actual post-tokenization artifact before training. Keep base model, prompt, seed, LoRA, epochs, sequence length, batch sizes, and generation settings fixed. Compare primarily against the v14-val200 run; act-balanced is a historical reference, not a single-variable control. Preserve all previous datasets and results.

Matched Contrasts At ACT-200 Counts

Opt-in `--matched-contrasts` tightens the existing 100 triplets, without reading evaluation fixtures. Contact lookup and duplicate-entity triplets share the exact request but change state; calendar and task examples remove only the required time or due date; capability contrasts share request/state but remove the needed tool. Bulk outreach and password safety contrasts remain represented. Existing pairs, other examples, prompt and disabled-generator behavior are preserved.

The builder exports `contrast-groups.json`, bound to the new source revision. `--preserve-contrast-groups` prioritizes complete triplets and pairs, keeps every selected grouped boundary example with its counterparts, and fills remaining productive slots with standalone examples. Ungrouped seed examples can also fill boundary quotas. Not every selected productive example needs a boundary counterpart. Selection uses the existing seeded order, excludes exact duplicates, never borrows from other splits, and fails if quotas cannot be satisfied. The group metadata is hashed into the new mix identity. Opt-in builds/mixes reject nonempty destinations.

Use the same raw cap: 100 clarification, 100 refusal, 200 productive BAYMAX and 600 xLAM rows. Validation still selects 30/30/30 BAYMAX plus 110 xLAM. This is a new synthetic source revision, including revised validation/test examples; do not interpret validation losses as a same-artifact comparison with ACT200. Whole contrast groups and connected duplicate content stay within one source partition. The 150-scenario benchmark, parser, scoring rules, prompt, seed, one epoch and LoRA hyperparameters are unchanged. The benchmark is a repeatedly used development benchmark, not an untouched final test set.

New config: `configs/sft/qwen2_5_1_5b_lora_auth_filtered_v14_matched_act200_val200.json`. Expected mix revision: `961023206f87a928d2c5b9881e5c03984d560e63ef31a28fcf8c89c9464a9f40`. This identity is computed from pinned inputs, not evidence of a completed real build.

Output Contract Gate

Dataset validation and training preflight now reject unknown tool names, missing call fields, wrong declared argument types/constraints, and invalid meta-tool fields. `request_clarification` requires only a nonempty `question`; `refuse_request` requires only a nonempty `reason`. Neither can coexist with productive calls. Shared meta-tool definitions must match; no target repair is performed.

The separate contract audit checks explicit credential keys even when a target tool is unavailable or has duplicate definitions. It retains the schema ambiguity finding and does not select an arbitrary definition. Re-run the audit before approving new credential exclusions; previously filtered artifacts remain unchanged.

Standard JSON Schema and native xLAM parameter maps are supported. Native maps enforce explicit requiredness, not requirements guessed from prose. Unverifiable types and schema references fail closed for review. Errors identify file/line/call, not argument values. Contract validity does not prove correct intent, factual grounding, or policy compliance. The gate does not replace the separate data audits.

Before training, run the capped dry-run below. A whole-pool validation failure may also describe unused tail rows: inspect its report, do not silently drop rows or disable the gate. The actual selected train and validation rows must all pass. Check post-tokenization reports too: raw class quotas do not guarantee trainable label counts or that every counterpart survives truncation.

```
$source = "data\sft\v1\baymax-control-policy-v14-matched"
$mix = "data\sft\v1\xlam-baymax-auth-filtered-v14-matched-act200-val200"
$config = "configs\sft\qwen2_5_1_5b_lora_auth_filtered_v14_matched_act200_val200.json"
```

```
.venv\Scripts\python scripts\build_baymax_sft_dataset.py `
  --output-dir $source --seed 1 --matched-contrasts
.venv\Scripts\python scripts\mix_sft_datasets.py `
  --xlam-dir data\sft\v1\xlam-content-groups-auth-filtered-v1 `
  --baymax-dir $source --output-dir $mix --seed 1 `
  --train-cap-row-count 1000 --train-cap-baymax-row-count 400 `
  --train-cap-baymax-clarification-count 100 --train-cap-baymax-refusal-count 100 `
  --validation-cap-row-count 200 --preserve-contrast-groups
.venv\Scripts\python scripts\validate_sft_dataset.py --dataset-dir $mix
.venv\Scripts\python scripts\validate_training_config.py --config $config --check-paths
.venv\Scripts\python scripts\train_sft_lora.py `
  --config $config --dry-run --local-low-vram `
  --max-train-rows 1000 --max-eval-rows 200 --max-sequence-length 1024
.venv\Scripts\python scripts\report_sft_artifact.py `
  --config $config --split train --max-rows 1000 --max-sequence-length 1024
.venv\Scripts\python scripts\report_sft_artifact.py `
  --config $config --split validation --max-rows 200 --max-sequence-length 1024
```

Run these commands individually and stop on an error. Training and evaluation are owner-run steps after preflight and artifact review, not completed results.

Reviewed Target-Compatibility Filtering

Owner-approved direction on 2026-10-06: exclude examples confirmed incompatible with the training target contract, not every broad audit flag. Preserve the source, historical mixes and results; do not coerce values or invent repaired targets.

The existing audit CLI supports `--checks targets`. It reuses the exact validator called by training preflight and adds explicit credential checks. It proposes whole rows with invalid call/meta-tool shapes, unavailable or duplicate tool names, undeclared arguments, target type/constraint violations, or unverifiable contracts. These are pipeline-compatibility exclusions, not claims that every upstream target is semantically wrong. Default-only metadata warnings do not propose removals. Numeric rounding, factual grounding and policy semantics remain separate reviews.

```
.venv\Scripts\python scripts\audit_sft_numeric_arguments.py `
  --checks targets `
  --dataset-dir data\sft\v1\xlam-content-groups-auth-filtered-v1 `
  --output data\sft\reviews\xlam-target-compatibility-audit-v1.json
```

The audit records per-row source provenance, content hashes, whole-file split hashes, exclusion reasons and redacted validator messages. It never removes rows. A full scan must be owner-run; capped reports are partial and cannot authorize a filter. Inspect the resulting counts and reasons before approving removals. Credential inspection errors remain separate coverage gaps, not automatic exclusions or successful checks. The two previously reviewed nesting-limit rows pass the target validator; retain them only with pinned, explicit gap resolutions in the new review.

`filter_sft_dataset.py` also accepts `target-filter-review-v1` reviews with policy `sft-target-compatible-v1`, pinned audit/manifest SHA256 values, approved rows, schema reviews and retained coverage gaps. Approved reasons must exactly match the audit proposals (`target_contract_incompatible`, `credential_emitting_target`, or both). Credential-only reviews still accept only their original policy/reasons. The filter verifies reviewed row hashes, source-file hashes and provenance, preserves retained bytes, order and split membership, and publishes only to a fresh output directory. The new policy and review identity are recorded in the derived manifest and provenance.

No new filtered dataset or performance improvement is claimed by this implementation. After review/filtering, validate and report the real capped artifacts. Keep ACT200 quantities, prompt, seed and LoRA settings fixed, and use the same filtered xLAM pool for an ACT200 control and matched-contrast experiment. Comparing only against the old unfiltered run would confound data cleanup with the contrast change. Previously recorded benchmark outcomes must remain unchanged.

Frozen ACT200 Reference Sampling

The first cleaned-pool run was not a filtering-only ablation. Comparing canonical content hashes in the raw training caps found only 7 of 600 shared xLAM rows and 301 of 400 shared BAYMAX rows. The combined shuffle changes when pool size changes, even at the same seed. Keep that benchmark result, but label it a resampled cleaned pool comparison rather than attributing its changes solely to excluded examples.

Opt-in `mix_sft_datasets.py --reference-mix-dir` anchors both raw training and validation caps to an existing mix. Matching uses source plus canonical content hash, not renumbered xLAM indices. Preserve every available reference example in its original cap position. Replace a missing example only when it is xLAM and its same-split content hash appears in the filtered artifact's pinned target review. Missing or changed BAYMAX examples and unexplained missing xLAM examples fail. Reserve all compatible reference rows before selecting replacement candidates; never consume a later protected example as filler. Replacements are unique, same-split xLAM examples ranked deterministically by seed, split and content hash.

The sampler verifies filtered source file hashes and review/provenance identity, preserves reference prompt/seed and BAYMAX class quotas, and rejects duplicates, insufficient candidates, repeats, group-preserving mode and existing destinations. Disabled behavior and historical mix hashes are unchanged. The anchored mix identity includes the reference manifest, ordered cap content/provenance hashes and exclusion review/provenance hashes. `sampling-report.json` records preservation counts and replacement positions/hashes without raw requests or argument values.

The owner-run anchored build passed validation on 2026-10-06. It preserves all 400 training and 90 validation BAYMAX examples, plus 582 and 107 xLAM examples, respectively. Independently comparing raw caps confirmed exactly 982 unchanged training rows and 197 unchanged validation rows, with 18 and 3 reviewed replacements at their recorded positions; replacement content hashes match the sampling report. The mixed revision is `329fab4d8f1604a4085d3ec50e05881e7c069717bb1428db7e592743bb07f03c`. The new config `configs/sft/qwen2_5_1_5b_lora_target_compatible_v14_act200_anchored_val200.json` is pinned to it, with the original ACT200 prompt, seed, model, LoRA and training settings unchanged. The owner completed the one-epoch training and 150-scenario evaluation: task success was 0.260, routing macro-F1 0.429 and invalid-output rate 0.080. These are development-benchmark results, not an untouched final test. Class quantities and raw caps remain unchanged. Replacement lengths can change tokenization/truncation and usable-row counts; inspect both actual artifacts before training. Test data remains in its source split and is never used as replacement training/validation data. No evaluation fixtures are read by the sampler.

Build a new version with the owner-run command below, then validate it. Do not overwrite either historical mix, config, checkpoint or result artifact. Pin a new training config to the resulting revision only after the build completes.

```
.venv\Scripts\python scripts\mix_sft_datasets.py `
  --xlam-dir data\sft\v1\xlam-content-groups-target-compatible-v1 `
  --baymax-dir data\sft\v1\baymax-control-policy-v14 `
  --reference-mix-dir data\sft\v1\xlam-baymax-auth-filtered-v14-act200-val200 `
  --output-dir data\sft\v1\xlam-baymax-target-compatible-v14-act200-anchored-val200 `
  --seed 1 --train-cap-row-count 1000 --train-cap-baymax-row-count 400 `
  --train-cap-baymax-clarification-count 100 --train-cap-baymax-refusal-count 100 `
  --validation-cap-row-count 200
```

Anchored Epoch-Length Test

Owner-approved next experiment on 2026-10-06: compare the anchored one-epoch reference with a fresh two-epoch run, before changing contrast structure. Use `configs/sft/qwen2_5_1_5b_lora_target_compatible_v14_act200_anchored_val200_epochs2.json` (hash `ce9efa2db24a065dbdb4eb482c07a29a56bfbcbdce32495418eb54b92d985c40`). Only `training.num_train_epochs` and the run/output identity change. Dataset revision, selected rows, prompt, seed, base model, LoRA and other configured training settings remain identical. Do not rebuild the mix or resume the old adapter. Start from the same base model; keep the previous checkpoint and results.

Apply the same owner-run overrides: `--local-low-vram --max-train-rows 1000 --max-eval-rows 200 --max-sequence-length 1024`. The one-epoch run retained 974 training and 195 validation examples after assistant-label filtering. Reusing the same rows/tokenizer/sequence length should preserve these per-pass counts; verify the actual reports rather than increasing caps to compensate for truncation.

The cosine schedule and warmup fraction remain configured identically, but their step durations scale with the longer run. This is a training-duration comparison, not an isolated extra pass at the one-epoch run's learning rates. Evaluate with the same generation settings and 150 scenarios, under a new result filename. Compare routing macro-F1, task success, premature actions and invalid outputs; lower training loss alone is not evidence of improvement. The owner completed the two-epoch run and evaluation: 44/150 task successes, routing macro-F1 0.514, 2 invalid outputs and 31 premature actions. The corresponding one-epoch values were 39/150, 0.429, 12 and 36. Both actual training reports retained the same 974 training and 195 validation rows, with no exact duplicates. These single-seed development results support a longer-run test, not a claim of robust improvement.

On 2026-10-07 the owner chose a fresh four-total-epoch anchored run instead of testing other presets. Use `configs/sft/qwen2_5_1_5b_lora_target_compatible_v14_act200_anchored_val200_epochs4.json` (hash `067656877679d9f5bae4414ae5f8f57b5b9ffbc94f95b578ab4a28b9c3fa4b43`), with the same low-VRAM overrides, 1000/200 raw caps and sequence length 1024. Only epochs and run/output identity differ from the reference; do not rebuild data, change prompts or overwrite earlier adapters/results. Start from base Qwen with a new LoRA initialization, not the saved two-epoch adapter. Loading adapter weights alone does not restore optimizer, scheduler or random-state checkpoints; a full resume would need those states and an explicit schedule-extension policy. The fresh four-epoch cosine/warmup schedule spans the full longer run. The owner has completed training and both evaluations. On the combined 150 scenarios, task success fell from 44/150 at

two epochs to 37/150 at four, routing macro-F1 from 0.514 to 0.372, and invalid outputs rose from 2 to 13. On the 30-scenario template-shift suite, task successes fell from 5 to 4 and invalid outputs rose from 2 to 6. Lower validation loss did not imply better balanced routing.

The two-epoch checkpoint is now the selected development reference, recorded in the SHA256 reference manifest. This detects file drift; it does not copy or permission-lock existing artifacts. The original run's exact base-model revision was not recorded. See the saved-output failure audit for separate decision, contract and wording-score categories across all four result files. Historical metrics, fixtures, data and adapters remain unchanged.

Matched Contrasts Against the Two-Epoch Reference

The next owner-approved experiment reuses the existing opt-in matched generator. New mixer flag `--replace-reference-baymax-train` requires both `--reference-mix-dir` and `--preserve-contrast-groups`. It replaces only the reference's BAYMAX training slots, preserving ACT/CLARIFY/REFUSE slot labels and the 200/100/100 raw quantities. Every xLAM training row stays at its reference position; the full validation and test files are copied byte-for-byte, including their original provenance. This is distinct from the older matched mix above, which reselected xLAM rows and changed held-out data.

Selection uses only the new source's training partition. If any member's complete input appears in retained validation/test data, the whole contrast group is excluded even when its target differs. Standalone overlapping inputs are excluded too. The sampler rejects insufficient unique candidates instead of duplicating or borrowing held-out examples. The sampling report lists complete selected groups, excluded groups, class counts, preserved xLAM counts and changed BAYMAX positions. Full reference split hashes and replacement source hashes enter the new identity; disabled historical sampler behavior remains unchanged.

Use new source/mix/config/adapter/result paths. Keep the reference prompt, seed, LoRA configuration, generation settings and two epochs. Do not rebuild the xLAM pool. The real build and training have not been run by the coding agent. After the owner builds and validates the new mix, pin a new training config to its actual manifest revision. Inspect post-tokenization class/token counts before training: raw quotas and complete raw groups do not guarantee identical token exposure or that every counterpart survives truncation. Do not adjust caps to hide filtering.

Existing evaluation suites, targets, parser and scorer remain unchanged. The experiment tests contrast structure, not additional epochs, a new policy prompt, or a larger boundary class contribution. It does not constitute an untouched-test generalization claim.

The owner completed the matched source and anchored mix build on 2026-10-07. Dataset validation passed with mixed revision `5973425e53c503b52b77b750d5851ef58019d6b33decdaffe4b4e85b610e3118`. The raw 1000-row cap contains 600 unchanged xLAM rows and 200/100/100 BAYMAX ACT/CLARIFY/REFUSE rows, with no exact duplicates. Of the 400 BAYMAX positions, 398 changed and 2 retained identical content. The sampling report lists 120 complete contrast groups and no held-out-input group exclusions. Independent file hashing confirmed identical validation/test bytes and provenance. These checks do not establish semantic independence or post-tokenization group coverage.

New config:
`configs/sft/qwen2_5_1_5b_lora_target_compatible_v14_act200_matched_anchored_val200_epochs2.json`, hash `dba7edd43ed83e09e9c1a99c2591ed941c6043a7d2e43bcc7376507da36037c0`. Training settings and LoRA are identical to the frozen two-epoch config; only data identity/paths and experiment output identity change. The owner completed the tokenized-artifact report, training and combined-150 evaluation. Training retained 974 rows,

including all 400 BAYMAX rows; validation retained the same 195 rows as the reference. There were no exact artifact duplicates. Task success was 39/150 versus 44/150 for the anchored two-epoch reference, routing macro-F1 0.383 versus 0.514, and invalid outputs 8 versus 2.

The paired refusal/format audit separates decision regression from output errors. Correct REFUSE routing fell from 16/29 to 5/29, with no newly correct refusal decisions. The training-only provenance audit found refusal triplet origins increased from 33/100 to 73/100, while standalone and capability-boundary-pair origins disappeared from the cap. These are source-family observations, not a complete semantic coverage audit or proof of the regression's cause. Keep the original two-epoch reference; no further training/data/scoring changes were made during this audit.

2026-10-09: Separate 2:1 SFT mixture experiment

The owner approved testing approximately 2:1 productive to non-productive rows before introducing DPO or changing models. Use the original anchored two-epoch reference, not the regressed matched-contrast checkpoint. The raw 1000-row cap targets 666 ACT / 164 CLARIFY / 170 REFUSE: 200 original BAYMAX productive rows, 334 unique BAYMAX boundary rows, and 466 retained xLAM rows.

`scripts/prepare_sft_2to1.py` uses the opt-in reference rebalancer. Keep all original BAYMAX rows, add 64 clarification and 70 refusal training-only examples, and deterministically replace 134 capped xLAM positions. Displaced xLAM rows move outside the cap; the source dataset is not edited. Additional inputs must be unique and must not match any reference training/validation/test input. Validation/test files are copied byte-for-byte. No group-first sampling, newly authored examples, prompt change, LoRA change, or evaluation-fixture change is part of this experiment.

The setup script creates a new training config using the built manifest revision and a recomputed config hash. Existing config/data/model paths cannot be overwritten. The raw row ratio is not a token ratio: run the artifact report at the same 1024 sequence length, train cap 1000 and validation cap 200 before training. Label filtering may change the effective row ratio and optimizer step count; disclose actual counts rather than claiming an exact post-tokenization 2:1 ratio.

The initially proposed 167/167 boundary allocation failed the uniqueness guard. A bounded training-source audit found 251 clarification rows but only 164 unique inputs/full rows, and 297 refusal rows but only 174 unique inputs/full rows. After retaining the original 100 per class, 64 clarification and 74 refusal inputs remain outside the reference cap; none overlap the source held-out inputs. The corrected allocation keeps the boundary total at 334 rather than repeating rows, borrowing held-out data, or silently weakening validation. This count check does not establish semantic diversity. The real build still checks all retained reference inputs before admitting an addition.

No real mixture build, tokenization, training, or evaluation was run by the coding agent for this experiment. Keep this suite's development-benchmark framing.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

ADR NNNN — <title>

Status: Proposed | Accepted | Deprecated | Superseded by ADR-NNNN **Date**: YYYY-MM-DD **Author**: <name> **Related**: ADR-NNNN, docs/...

Context

What is the problem this decision addresses? What forces are at play (technical, business, team)? What's the constraint set?

Keep it factual — describe the situation, not the solution yet.

Decision

What did we decide? State it in one or two sentences first, then expand with concrete specifics.

If the decision involves a schema, contract, or interface, include the actual shape inline (or link to the schema file).

Reasoning

Why this decision over the alternatives? What forced the choice? What did we optimise for?

This is the section that gets quoted under interview drilling. Make it defensible.

Alternatives considered

For each alternative seriously considered:

Alternative A — <name>

- What it would have looked like
- Why we rejected it (specific, not "didn't like it")

Alternative B — <name>

- Same structure

Consequences

What happens as a result of this decision?

- **Positive**: what gets easier / better
- **Negative**: what gets harder / worse / locks us in
- **Neutral**: changes that aren't strictly better or worse

Open questions / follow-ups

Things this decision deliberately leaves for later. Link to issues, tickets, or future ADRs.

References

- Papers, docs, prior art that informed the decision
- Related ADRs in this project
- External standards (RFCs, JSON Schema specs, etc.)

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

ADR 0001 — Agent loop pattern

Status: Confirmed **Date**: 2026-06-20 **Author**: Marv **Related**: ADR 0002, docs/ARCHITECTURE.md, docs/DEFERRED.md

---

Context

BAYMAX's agent core needs a loop pattern that:

1. Takes a user request and produces a sequence of tool calls (single or multi-step)
2. Validates each step against the tool-call contract before execution
3. Recovers from errors (tool failures, ambiguous intent, model hallucinations) without silently failing
4. Supports clarification flows when the model can't determine the user's intent

The choice of loop pattern affects every downstream module: telemetry shape, scoring boundary with Ronin's eval harness, recovery semantics, and CV defensibility under "walk me through how your agent works."

The literature offers three canonical patterns:

- **ReAct** (Reason-Act-Observe loop): the model reasons, picks one action, observes the result, and loops until done
- **Plan-and-Execute**: the model produces a full plan upfront, then an executor runs each step in sequence
- **Tool-use chat loop** (OpenAI / Anthropic native): the model emits tool calls; the orchestrator dispatches and feeds results back as new messages until the model emits a final response

---

Decision

BAYMAX uses **Plan-and-Execute with a finite iterative ReAct-style loop for error and clarification recovery**.

Concretely:

1. The model interprets the user request and constructs a `task_file` (a plan containing the sequence of intended tool calls, response type, etc.)
2. The `task_file` is validated against the tool-call contract (ADR 0002) before any tool fires
3. The executor dispatches each tool call in the planned sequence; results stream back as the action log grows
4. If the executor hits an error (tool failure, invalid argument, ambiguous result, timeout), control passes to a recovery loop that can:
    - Retry the failing step (with backoff)
    - Ask the user for clarification (terminating the request for v1; sub-tasks deferred to v2)
    - Trigger an abort with rollback of completed-and-reversible actions

5. The model is also invoked at the end to interpret raw tool outputs into a user-facing response message

Pseudocode (≤30 lines):

```
function handle_request(user_input):
    task_file = model.construct_task_file(user_input, available_tools, context)
    validation = validator.validate(task_file)
    if not validation.ok:
        return error_response(validation.errors)

    telemetry.start_trace(task_file.task_id)
    action_log = []

    for step in task_file.tool_calls:
        try:
            result = executor.dispatch(step)
            action_log.append((step, result))
            if result.requires_clarification:
                return clarification_response(result.question, action_log)
        except RecoverableError as e:
            recovery = recovery_handler.handle(e, action_log, task_file)
            if recovery.action == "retry":
                continue
            elif recovery.action == "clarify":
                return clarification_response(recovery.question, action_log)
            elif recovery.action == "abort":
                rollback(action_log)
                return abort_response(e, action_log)
        except IrreversibleError as e:
            return failure_response(e, action_log)

    response_text = model.interpret_results(action_log, user_input)
    return success_response(response_text, action_log)
```

Reasoning

why chosen - allows for predictability of upfront planning (auditable and reviewable before any side effect) PLUS resilience of ReAct-style

recovery for issue catching and gracefully terminating where needed. (real tool calls fail, but the agent shouldn't crash essentials).

Alternatives considered

Alternative A — Pure ReAct

The model would emit one tool call at a time, observe the result, and decide the next step in a single loop without an upfront plan.

why rejected - harder to audit before side effects fire; model context fills up with redundant reasoning across many steps; no separation between planning errors and execution errors which makes telemetry less useful.

Alternative B — Pure Plan-and-Execute (no recovery loop)

The model would emit a full plan upfront; the executor runs every step in sequence without a path back to the model on failure.

why rejected - tools sometimes fail (5xx, rate limits, auth expiry); without a recovery path the agent crashes or returns broken state to the user; production agents must handle failure as a first-class case, especially considering the rigorous tests we plan to use the agent for in evaluation.

Alternative C — Tool-use chat loop (OpenAI / Anthropic native)

The model emits tool calls via the SDK's native function-calling interface; the orchestrator dispatches and feeds results as new messages until the model emits a final response.

why rejected - tightly linked to one provider's tool-use protocol; harder to swap inference backends (Qwen, local MLX) without rewriting the loop; loses the validation-before-execution discipline.

Consequences

Positive

- Plans are auditable BEFORE any side effect fires (the `task_file` exists and is logged before tool dispatch)
- Telemetry has a natural artifact (the task_file) to attach traces to
- Recovery is explicit and traceable, not silent retries hidden inside the model
- Boundary with Ronin's eval is clean: agent emits a structured `AgentResponse` matching `scorer.py` shape

Negative

- Two model invocations per request (one to construct the plan, one to interpret results) — higher cost per request than pure ReAct on simple cases
- Plan-and-Execute is less flexible when the right next action depends on the result of the previous one — the model has to either over-plan (anticipate branches) or trigger replanning via the recovery loop
- More moving parts than a pure tool-use chat loop — more surface area to debug

Neutral

- Multi-turn user sessions (continuation across requests) are not addressed by this loop; deferred to v2 sub-task model (see DEFERRED.md)

Open questions / follow-ups

- **Sub-task chains for clarification follow-ups** (1.0 → 1.1 model) — deferred to v2 per DEFERRED.md. Current pattern terminates on clarification; user manually resubmits.
- **Streaming response** during the "interpret results" phase — deferred to v2 (see DEFERRED.md). v1 returns all-at-once.
- **Multi-phase tasks** for irreversible tools (draft-then-send) — deferred to v2 with multi-phase implementation choice (versioned task_File vs sub-task chain) still open.

References

- ReAct paper: Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models" (2022)
- Plan-and-Execute style: LangChain "Plan-and-Execute Agents" docs
- OpenAI tool-use: https://platform.openai.com/docs/guides/function-calling
- Ronin's `AgentResponse` shape (boundary contract): `src/baymax/eval/scorer.py` on branch `feat/phase1-scenario-loader`
- docs/WRITEUP_INFO.md for the broader thesis BAYMAX is testing

---

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

ADR 0002 — Tool-call schema and contract

Status: Proposed (Marv to confirm) **Date**: 2026-06-20 **Author**: Marv **Related**: ADR 0001, ADR 0007, ADR 0009, src/baymax/core/schemas/tool_call.schema.json, docs/DEFERRED.md

---

Context

BAYMAX's agent core emits tool calls. Ronin's eval harness consumes them and scores them. The contract between agent core and eval harness — and the internal shape of a tool-call within the agent — defines what both modules must produce/consume to integrate.

Ronin's `src/baymax/eval/scorer.py` already locks the boundary shape:

```
class AgentResponse:
    tool_calls: list[AgentToolCall]
    message: str | None

class AgentToolCall:
    tool: str  # namespaced: "^[a-z][a-z0-9_]*\.[a-z][a-z0-9_]*$"
    arguments: dict[str, Any]
```

The agent core's contract must serialise to this shape at the boundary. Internally, the agent works with a richer `task_file` structure containing context, validation results, action log, cost, etc.

Decision

The tool-call contract defines two distinct shapes:

1. External (boundary) shape — `AgentResponse`

This is what the agent core emits to Ronin's eval runner. **It matches** `scorer.AgentResponse` **exactly** to avoid serialisation drift.

```
{
  "tool_calls": [
    {
      "tool": "calendar.create_event",
      "arguments": {
        "title": "Meeting with Sarah",
        "start": "2026-06-21T09:00:00Z",
        "duration_minutes": 30
      }
    }
```

```
  ],
  "message": "Booked 30-min meeting with Sarah tomorrow at 9am UTC."
}
```

Rules:

- `tool` must match `^[a-z][a-z0-9_]*\.[a-z][a-z0-9_]*$` (namespaced lowercase)
- For clarification or refusal scenarios: `tool_calls` is empty `[]`, `message` contains the clarification question or refusal reason
- For multi-step scenarios: `tool_calls` has multiple entries in execution order

2. Internal shape — `task_file`

This is the agent's internal representation of a request. It is **never serialised to the eval boundary** — it stays inside the agent.

Input fields (what the agent receives or constructs from the user request):

```
task_id           hierarchical, format <run_id>.<task_int>.<phase_int> (Phase 1: all <run_id>.N.0)
request_text      original user input
input_type        enum: text, voice
request_type      enum: validation, creation, edit, delete
request_intent    nullable string (model-generated; null → triggers clarification)
request_complexity enum: single_tool, multi_tool, multiphase (multiphase deferred to v2)
context           opaque dict (datetime, location, file refs, prior task context for sub-tasks)
timeout_cap       expected max number of tool calls (budget; soft limit)
available_tools   list of tool names (matches Ronin's available_tools)
```

Output fields (what the agent populates as it runs):

```
response_text          natural-language response to user
request_completion_status  enum: complete, clarification_pending, aborted, failed
time_elapsed_ms        wall-clock duration
cost                   RunCost reference (see classes refactor below)
action_log             list of (tool_call, result) tuples in execution order
response_type          enum: task_done, clarification, task_refusal, task_ongoing
                       (task_success is NOT in this enum — see "task_done vs task_success" below)
tools_called           list of tool names in execution order
tool_call_success      list of booleans, parallel to tools_called
task_id                same as input
```

3. Error types

```
auth_error          adapter could not authenticate (token expired, missing env var)
type_error          argument failed schema validation (wrong type, missing required field)
ambiguous_request   model could not determine intent → triggers clarification
intent_error        model produced internally-inconsistent plan (hallucinated tool, contradictory
```

```
steps)
timeout_error       tool call exceeded timeout_cap or wall-clock budget
```

4. task_done vs task_success

- **task_done**: agent finished without throwing — produced an `AgentResponse` and terminated cleanly. Set by the agent core.
- **task_success**: the EXPECTED OUTCOME actually happened in the world — verified by Ronin's eval scorer. **Always null inside the agent**; populated post-hoc by the eval harness.

This distinction lets the system distinguish "agent ran cleanly but the user's real intent wasn't satisfied" (hallucinated success) from "agent crashed."

5. Verification gate

Each tool adapter declares `reversible: bool` (and implements `undo_action()` if claiming reversibility, per ADR 0009). For tools where `reversible == False`, a verification gate is automatically applied: the agent must obtain user confirmation before the tool fires. v1 fake adapters are all reversible; verification gates therefore don't trigger in v1 eval. Real-adapter integration (v2) will surface this.

6. Conventions

- **Object nesting**: limited to 3 levels deep in tool `arguments` (prevents pathological JSON dependencies)
- **Idempotency window**: 1 day default; configurable per tool
- **Duplicate requests**: for duplicate tool API calls, return the previous report. For duplicate user request, change `request_completion_status` to `clarification` and `request_type` to `ongoing` (asks the user "did you mean to repeat?")
- **Timeouts**: error message includes Action log + pinpoint where the error occurred. For multi-tool tasks, the `last_successful_tool_called` variable is added to context on retry so the model can resume from the failure point with full awareness of what already worked
- **Auth**: tool adapters read credentials from environment variables. Adapters NEVER expose credentials in returned data or trace logs. Telemetry redacts any field matching `auth*`, `token*`, `key*`, `password*` (see ADR 0010 telemetry)

7. Argument matching convention

For Phase 1, the `_contains` suffix convention used in scenario expected arguments (e.g., `body_contains: "thanks"` matches `body: "many thanks"`) is **eval-side only**. The agent emits plain values. Ronin's scorer handles substring matching when scenarios declare `_contains` expectations. v2 may revisit whether the agent itself should be able to emit `_contains` outputs.

Reasoning

The user-facing and model-facing sides of the agent each have different reasoning and different utility for the evaluation harness, so splitting between them is my way of organising to reduce coupling in the eval — each side should be evaluable independently. Outward fields need to match ahmed's scorer expectations so the boundary contract stays stable across runs and we don't get serialisation drift between the agent and the scorer. Inward fields exist to extract the most information from the user request and standardise it for cross-

model review, so GPT-4, Claude, and the fine-tuned Qwen all consume the same shape and the comparison stays apples-to-apples. Internal fields collect runtime state — cost, action history, error triggers, and a systemic trace of how the model worked — which gives ahmed material to work with during the Qwen fine-tuning stage and also lets us isolate the "agent claims success but the world didn't change" failure mode by separating `task_done` (agent's claim) from `task_success` (predicate-verified outcome).

Alternatives considered

Alternative A — Single flat schema (no internal/external split)

Use one schema for both the boundary contract AND the agent's internal state. why rejected - the internal agent needs richer state (action log, cost, intermediate context) that doesn't belong at the boundaryyet for v1 but will be of use in v2+ expansion, may as well implement them now and change the response shape later; forcing both to share a shape would either bloat the boundary with useless stuff or limit the internal state.

Alternative B — Match OpenAI / Anthropic native tool-use shape exactly

Adopt the vendor SDK's tool-call response shape (e.g., OpenAI's `ChatCompletionMessageToolCall`) as the boundary contract.

why rejected - it would be provider locked-in; Qwen via MLX doesn't emit the same shape natively for example; ahmeds's scorer already locked a different (cleaner) shape; standardising on ahmed's shape gives us one transformation point rather than per-provider adapters everywhere. easier to run unbaised evaluations for baseline.

Alternative C — Include expected-state predicates in the contract

Have the agent emit "expected world state after this call" alongside each tool call, for the eval to verify.

why rejected - ahmed's v1 scorer is tool-call-comparison-based, not state-predicate-based; adding predicates would require the agent to predict effects of every tool call which is more work for no v1 benefit; if ahmed's scorer ever switches to state-predicate evaluation, which affect phase 1 meeting is confirmed to not happen, (but his SCORING.md flags this as future work tho, talk to him about this), this ADR can be revisited.

Consequences

Positive

- Clean separation: external boundary stays minimal (matches Ronin's scorer), internal state can evolve without breaking eval
- The `task_done` / `task_success` distinction architecturally supports the "hallucinated success" failure mode that Ronin's scorer expansion plans to detect
- Reversibility + verification gate pattern means irreversible-tool support (v2) doesn't require a contract rewrite — just adapter additions
- Conventions (idempotency, timeout context inheritance, redaction) are documented in one place

Negative

- Two shapes to maintain (boundary `AgentResponse` + internal `task_file`); risk of drift
- Verification gate is conceptually defined but functionally inert in v1 (no irreversible tools in fakes); first real test happens in v2
- task_id hierarchy is defined but functionally degenerate in v1 (every id is `<run>.N.0`)

Neutral

- The agent's serialisation layer must explicitly convert `task_file` → `AgentResponse` at the boundary
- Cost tracking lives in a separate `RunCost` record (see classes refactor) referenced from `task_file`, not embedded

---

Open questions / follow-ups

- **Trace format** (ADR 0006) — pending discussion with Ronin (OpenTelemetry vs custom JSON)
- **Multi-phase task modeling** (versioned task_File vs sub-task chain) — deferred to v2 per DEFERRED.md
- **Sensitivity-based verification gate** (verify on reversible-but-sensitive tools) — deferred to v2
- `_contains` **agent-emit support** — deferred to v2 conversation with Ronin

---

References

- Ronin's `AgentResponse` and `AgentToolCall` definitions: `src/baymax/eval/scorer.py` on branch `feat/phase1-scenario-loader`
- Ronin's scenario schema: `src/baymax/eval/schemas/scenario.schema.json`
- Ronin's scoring rules: `docs/SCORING.md` on branch `feat/phase1-scenario-loader`
- Fake adapters surface area: `src/baymax/tools/FAKE_ADAPTERS.md`
- docs/DEFERRED.md for v2/v3 deferrals tied to this contract
- ADR 0009 for reversibility + undo semantics
- docs/WRITEUP_INFO.md for the broader project thesis

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

ADR 0003 - Scenario format and eval fixtures

Status: Confirmed **Date**: 2026-06-23 **Author**: Ronin **Related**: ADR 0002, docs/BENCHMARKING.md, docs/SCORING.md, scenario.schema.json, scenario_loader.py

---

Context

BAYMAX Phase 1 needs an eval harness before building memory, voice, real APIs, RAG, fine-tuning, or multi-agent orchestration. The harness needs scenario fixtures that are:

1. Deterministic enough to run repeatedly in CI and local development
2. Structured enough for automated scoring without an LLM judge
3. Expressive enough to cover tool calls, clarification, refusal, multi-step behavior, relative time, and fake adapter state
4. Reviewable by humans during PR review
5. Independent from real Notion, Calendar, Gmail, or clipboard APIs

The scenario format is the boundary between Ronin's eval harness and Marv's agent. If this shape drifts, the scorer can no longer reliably measure whether the agent chose the right tool, supplied the right arguments, or asked for clarification when appropriate.

Decision

Phase 1 scenarios are JSON files under `scenarios/v1/`. Each file validates against `src/baymax/eval/schemas/scenario.schema.json` and loads through the Pydantic model in `src/baymax/eval/scenario_loader.py`.

Required fields:

```
id                stable scenario id, pattern <domain>_<behavior>_<nnn>
category          calendar | notion | gmail | clipboard | multi_tool
difficulty        explicit | implicit | contextual | ambiguous | multi_step
description       human-readable scenario purpose
user_input        text-only request sent to the agent
current_time      timezone-aware ISO datetime used for relative-time resolution
available_tools   exact fake tools the agent may call
initial_state     fake adapter state before execution
expected_behavior structured expected output
success_criteria  fixed vocabulary of expected checks
tags              optional labels for filtering and coverage review
```

`expected_behavior` is discriminated by `type`:

```
tool_call      exactly one expected tool call
tool_calls     ordered multi-step tool calls
clarification  no tool call; response asks for missing information
refusal        no tool call; response explains why action cannot be done
```

Audio input is explicitly out of v1. Scenario input is `user_input` text only. Relative time phrases such as "tomorrow" must be resolved through `current_time`, not the machine clock.

Reasoning

why chosen - JSON fixtures give us the simplest stable interface between agent behavior and eval scoring. They are easy to review in PRs, easy to validate with JSON Schema, and map directly into typed Python models without executable fixture logic. The explicit `available_tools` field keeps each scenario honest: the scorer can tell the difference between a valid tool call and a hallucinated unavailable tool. The `expected_behavior.type` split also prevents tool-call-only thinking; ambiguous and refusal scenarios are first-class instead of hacked around by missing tool arguments.

Alternatives considered

Alternative A - YAML scenario files

Use YAML for easier hand-writing and comments.

why rejected - comments are useful, but JSON Schema validation and Python loading are more direct with JSON. YAML also has more parsing edge cases, implicit typing surprises, and formatting inconsistency across editors.

Alternative B - Python fixture objects

Define scenarios as Python objects or factory functions in test files.

why rejected - executable fixtures are harder to inspect in code review and can accidentally depend on runtime state. Phase 1 needs static data fixtures that can be counted, validated, and converted into training data later.

Alternative C - Free-form expected behavior text

Write natural-language expected behavior and manually or semantically judge the result.

why rejected - deterministic scoring needs structured expected tools, arguments, clarification keywords, and refusal keywords. Free-form expectations would force LLM-as-judge too early and make baseline numbers less reproducible.

Alternative D - Include final-state predicates only

Score only by comparing final fake adapter state after tool execution.

why rejected - final state is useful but not enough. We also need to know whether the agent picked the correct tool, used the correct order, avoided premature tool calls, and did not hallucinate unavailable tools.

Consequences

Positive

- Scenarios are deterministic, typed, and schema-validatable
- Fake adapter state is explicit and local to each scenario
- Clarification/refusal behavior is represented directly, not as missing tool calls
- `_contains` argument checks allow realistic wording without fuzzy matching
- Scenario files can later be transformed into SFT or eval datasets

Negative

- JSON is verbose for large scenario sets
- Single-turn clarification is baked into v1; multi-turn clarification needs a new format or version
- Deterministic keyword checks can miss semantically correct alternate phrasing

Neutral

- Scenario format and Pydantic loader are duplicated sources of truth; tests must keep them aligned
- v1 fixture naming is intentionally simple and may need richer taxonomy at 200+ scenarios

---

Open questions / follow-ups

- Generate JSON Schema from Pydantic models or keep both with drift tests?
- Add state-predicate scoring once fake adapter outputs become central to task success?
- Introduce `scenarios/v2/` for multi-turn and sub-task-chain formats?
- Add scenario metadata for train/eval split when Phase 2 SFT data prep starts?

References

- `src/baymax/eval/schemas/scenario.schema.json`
- `src/baymax/eval/scenario_loader.py`
- `scenarios/v1/COVERAGE.md`
- docs/BENCHMARKING.md
- docs/SCORING.md
- ADR 0002 - Tool-call schema and contract

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

ADR 0004 - Base model choice and baseline ladder

Status: Proposed **Date**: 2026-06-23 **Author**: Ronin **Related**: docs/ROADMAP.md, docs/BENCHMARKING.md, ADR 0005, docs/WRITEUP_INFO.md

---

Context

BAYMAX is intended to become a locally hostable personal-task agent. That means the project cannot judge success only by whether a hosted frontier model performs well. It needs a baseline ladder:

1. Scripted responses prove the eval harness and fake adapters work
2. A naive hosted-model agent gives a realistic capability baseline
3. A small local model becomes the Phase 2 fine-tuning target

The base model choice affects scenario design, SFT formatting, cost reporting, and the final portfolio claim. Choosing too large a model first slows iteration. Choosing too weak a model risks spending Phase 2 debugging model incapability instead of measuring whether structured training improves tool-use behavior.

Decision

Phase 1 keeps the eval harness model-agnostic. The first baseline is the scripted control run in `results/phase1/scripted/v1-baseline.json`; it costs `0.0` and validates the harness rather than model intelligence.

After the scripted control, Marv's naive agent should be evaluated through the same `AgentResponse` boundary and fake adapter environment. Hosted models may be used for baseline comparison, but they are not the target deployment model.

For Phase 2 local fine-tuning, the starting target is a Qwen 2.5-class instruct model around 1.5B parameters, unless Phase 2 benchmarking identifies a better locally hostable candidate. The choice optimizes for:

```
local serving feasibility
low inference cost
fast SFT iteration
adequate instruction following
structured JSON/tool-call learnability
```

The inference backend remains swappable per ADR 0005. This ADR picks the initial local-model direction, not a permanent provider lock.

Reasoning

why chosen - the scripted baseline tells us whether Ronin's harness is correct before blaming a model. After that, a hosted naive-agent baseline gives us a reference point for what "good enough" looks like on the same 50 scenarios. Qwen-size local models are small enough to iterate on without turning every experiment into an infrastructure project, while still being plausible for structured instruction-following and tool-call formatting. This keeps the project honest: first prove the benchmark, then compare models, then fine-tune.

Alternatives considered

Alternative A - GPT-only baseline and deployment

Use GPT-class hosted models for both baseline and final behavior.

why rejected - useful as a comparison point, but it does not satisfy the locally hostable thesis. It also makes cost and privacy weaker parts of the final story.

Alternative B - Start with a larger local model

Use a 7B+ local model as the first training and serving target.

why rejected - stronger raw capability, but slower iteration and higher hardware pressure. The Phase 2 question is whether the pipeline and task-specific data improve behavior; a smaller model gives faster feedback.

Alternative C - Pick no model until after data prep

Keep Phase 2 model selection completely open.

why rejected - SFT export format, prompt templates, context limits, and serving assumptions need an approximate model family. Fully deferring the choice would push too much uncertainty into Phase 2.

Alternative D - Train before scripted baseline is complete

Begin SFT data generation before the 50-scenario harness is finished.

why rejected - bad benchmark data creates bad training data. The Phase 1 harness has to be stable before it becomes the source of SFT examples.

Consequences

Positive

- Separates harness correctness from model capability
- Keeps local hosting central to the project thesis
- Gives Phase 2 a concrete but swappable starting model family
- Supports cost comparison between scripted, hosted, and local runs

Negative

- A 1.5B-class model may underperform hosted baselines substantially
- Some failures may reflect model size rather than agent architecture
- Future prompt/template changes may be needed if the chosen local model has format quirks

Neutral

- Backend abstraction remains owned by Marv's inference layer
- Ronin's eval output records backend/model name so comparisons can coexist

---

Open questions / follow-ups

- Which exact Qwen checkpoint and quantization will Phase 2 use?
- What hosted model becomes the naive baseline comparison?
- What train/eval split prevents leakage from scripted scenarios into SFT evaluation?
- What metric must improve to justify the fine-tuned model: cost, latency, task success, or argument accuracy?

References

- docs/ROADMAP.md
- docs/BENCHMARKING.md
- ADR 0005 - Inference backend
- `results/phase1/scripted/v1-baseline.json`
- Qwen 2.5 model family documentation

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

ADR 0005 — Inference backend (swappable abstraction)

Status: Proposed (Marv to confirm) **Date**: 2026-06-20 **Author**: Marv **Related**: ADR 0001, ADR 0002, docs/WRITEUP_INFO.md, src/baymax/service/inference.py, src/baymax/service/api.py

---

Context

The BAYMAX thesis (see docs/WRITEUP_INFO.md) is a head-to-head benchmark between GPT-4, Claude, and a fine-tuned Qwen 2.5 1.5B model on the same reproducible scenario suite. For that comparison to be apples-to-apples:

1. The same agent loop (ADR 0001) and the same tool-call contract (ADR 0002) must run against every backend with no code changes elsewhere
2. The inference layer must be a swappable component — the agent core does not import OpenAI, Anthropic, or MLX directly
3. Adding a new backend (e.g. Llama 3.2 1B, or distilled GPT-4 traces) should be a single class addition with no edits to agent core, eval harness, or telemetry

Backends differ on:

- Authentication shape (API key for OpenAI/Anthropic, model file path for MLX local)
- Native tool-call format (OpenAI function-calling, Anthropic tool-use XML, raw text for some local models)
- Streaming protocol (SSE vs chunked text vs not supported)
- Latency and cost characteristics (network round-trip vs local GPU inference)

A swappable abstraction is therefore a load-bearing piece of infrastructure for the project's central research question.

Decision

Define a `Backend` protocol with one core method, `invoke(messages, tools, ...) → response`. Concrete implementations live in `src/baymax/service/inference.py`:

- `OpenAIBackend` — wraps `openai` SDK; converts internal tool-call format to OpenAI function-call format
- `AnthropicBackend` — wraps `anthropic` SDK; converts to Anthropic tool-use XML
- `MLXBackend` — wraps `mlx-lm`; serves Qwen 2.5 1.5B + LoRA adapter from local model files

Backend selection is driven by an environment variable / config flag at startup. The FastAPI service (`src/baymax/service/api.py`) instantiates one backend per process; the agent core receives it as a constructor dependency and never sees the concrete class.

Authentication per ADR 0002 §6 — credentials read from environment variables only:

- `OPENAI_API_KEY` for OpenAI
- `ANTHROPIC_API_KEY` for Anthropic
- `MLX_MODEL_PATH` + `MLX_ADAPTER_PATH` for local MLX

Each backend is responsible for translating the project's internal tool-call shape to/from its native format. The agent core's view of "what the model said" is normalised before it leaves the inference module.

Reasoning

[REASONING PROMPTS — expand each bullet into 1-2 sentences in your own voice. Documentation depth: brief, honest, readable when future-you re-reads in 6 months. Not aiming for interview-defensible depth (artifact-first project, per 2026-08-24 scope revision).]

- **WHY a protocol / structural interface, not inheritance from a base class**: any class with the right method shape works, no framework dependency, no forced hierarchy for something as thin as "call an LLM and get a response back."
- **WHY not LangChain / an off-the-shelf agent framework abstraction**: LangChain carries assumptions about prompt templates, chain composition, and memory that fight the agent loop locked in ADR 0001; adopting it means either working against the framework or accepting design decisions that conflict with the project's own. Per the CV's "don't let AI own architecture" rule, the abstraction is ours to control.
- **WHY backend translation lives INSIDE the inference module, not in the agent core**: the entire research thesis is comparing backends. Every mention of `openai`, `anthropic`, or `mlx` in the agent core would be a provider leak that undermines the thesis. Translation belongs at the boundary; the agent core sees a normalized shape.
- **WHY env-driven backend selection, not runtime polymorphism or per-request routing**: Ronin's eval harness sweeps backends by spinning up multiple processes with different env vars — no code changes required per backend swap. Runtime polymorphism would add per-request overhead and per-request configuration state for capability the project doesn't need in v1.
- **WHY the protocol has one core method (`invoke` — later split into `plan_task` + `interpret_results` in ADR 0011)**: minimum surface area = maximum swappability. Every additional required method is a translation cost for a new backend author. Keep it thin.
- **WHY this ADR is v1-scoped and superseded for v2 by ADR 0011**: the v1 shape shipped adequately for the OpenAI-only Phase 1 target. Adding MLX + retry policy + warmup + registry integration in Phase 2 demanded a richer surface — hence 0011. Rather than mutate this ADR, 0011 stands as the v2 iteration and this one stays historical.

Alternatives considered

Alternative A — Tight coupling to one provider (e.g. OpenAI only)

Use the OpenAI SDK directly throughout the agent core and the service layer.

why rejected — the entire project thesis is comparing backends. Coupling to one provider would either force us to drop the thesis or rewrite half the codebase to add a second backend later. The cost-of-future-change is too high relative to the cost-of-now-abstraction.

Alternative B — Use LangChain or a similar agent framework's backend abstraction

Adopt LangChain's `BaseLLM` / `ChatModel` hierarchy and use their pre-built backend wrappers.

why rejected — LangChain's abstractions carry assumptions about prompt templates, memory, and chaining that don't match our agent loop (ADR 0001). Pulling them in would either fight the framework or accept its design decisions that conflict with ours. The CV's "don't let AI own architecture" rule applies — we own the abstraction.

Alternative C — Single OpenAI-compatible client (point at different base URLs)

Use a single OpenAI-format client; point it at OpenAI for GPT-4, Anthropic-compat shim for Claude, and a local llama.cpp / vLLM endpoint that exposes the OpenAI format for Qwen.

why rejected — Anthropic's tool-use API differs structurally from OpenAI's function-calling (XML vs JSON, different streaming events); shimming loses fidelity on tool-call edge cases. For Qwen via MLX, exposing an OpenAI-compatible HTTP server adds a process boundary and latency for no benefit in the single-process v1 setup.

Consequences

Positive

- The thesis is testable: any backend can be swapped in with a single env var change
- Each backend's translation logic is isolated; bugs in one don't leak into others
- New backends (Llama 3.2 1B, distilled traces, future Qwen variants) are single-file additions
- Eval harness can run baseline-vs-candidate sweeps by spinning up multiple processes with different env vars

Negative

- 3+ backend implementations to maintain; provider SDK API drift means periodic updates
- Each backend's tool-call translation is its own subtle correctness surface — needs per-backend tests against the same scenario suite
- Latency budgets per stage (ADR 0006) need separate calibration per backend

Neutral

- The `Backend` protocol does not address streaming differences in v1 (deferred per RONIN_SYNC.txt #11); each backend returns the full response and the service layer doesn't expose intermediate tokens

---

Open questions / follow-ups

- **Streaming**: deferred to v2 (DEFERRED.md). When added, the protocol grows a streaming variant.

- **Multi-backend routing within a single request** (e.g., small-model triage → large-model fallback) — deferred indefinitely; would change the protocol shape.
- **Provider-specific failure semantics** (rate limits, content filtering, refusals): normalised to the project's `ErrorType` enum in the backend translation layer; specific mapping documented per backend in its docstring.

References

- OpenAI function-calling: https://platform.openai.com/docs/guides/function-calling
- Anthropic tool use: https://docs.anthropic.com/en/docs/build-with-claude/tool-use
- MLX-LM: https://github.com/ml-explore/mlx-examples/tree/main/llms
- ADR 0002 for the boundary contract
- docs/WRITEUP_INFO.md for the comparison thesis this abstraction serves

---

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

ADR 0006 — Trace format

Status: Proposed (pending Ronin discussion per RONIN_SYNC.txt Tier 1 #1) **Date**: 2026-06-20 **Author**: Marv **Related**: ADR 0002, ADR 0010, docs/DEFERRED.md, src/baymax/telemetry/logger.py

---

Context

BAYMAX needs a trace format for agent runs that:

1. Captures the full lifecycle of a request — input arrival, task_file construction, validation, per-tool dispatch, recovery actions, final response
2. Is consumable by Ronin's eval harness when investigating scenario failures ("why did this scenario fail at step 3?")
3. Carries enough structure to be queried (e.g. "show me all runs where the model produced an `intent_error` on Qwen but succeeded on GPT-4")
4. Supports correlation across the agent → inference backend → tool adapter call chain
5. Doesn't require additional infrastructure (no separate collector daemon) for v1

The industry options:

- **OpenTelemetry (OTel)** — vendor-neutral standard, native span/trace/baggage model, requires a collector process (OTel Collector, Jaeger, Tempo) to receive and store spans
- **Custom JSON lines (ndjson)** — one JSON event per line, written to a file or stdout, read by trivial tooling
- **Plain text logs** — easy but unstructured; hard to query without grep+awk gymnastics
- **A database-backed event log** — clean queries but adds operational surface

Ronin's eval harness is the primary downstream consumer in v1. There's no production deployment, no separate dashboard, no SRE team waiting for traces.

Decision

v1: Custom JSON lines (ndjson). Each event is one line in a `.jsonl` file under `traces/<run_id>/<task_id>.jsonl`. Event shape:

```
{
  "ts": "2026-06-20T14:32:17.482Z",
  "trace_id": "tr_8c7f...",
  "task_id": "<run_id>.<scenario_id>.<phase_int>",
  "event": "tool.dispatch.start",
  "module": "baymax.core.executor",
  "payload": {
    "tool": "calendar.create_event",
```

```
    "arguments_redacted": { "title": "Meeting with Sarah", "start": "2026-06-21T09:00:00Z" }
  }
}
```

Event categories (initial set, extensible):

- `request.received` — user input arrives
- `task_file.constructed` — model produced a task_file
- `validation.passed` / `validation.failed` — tool-call contract check result
- `tool.dispatch.start` / `tool.dispatch.end` — adapter call boundary
- `recovery.triggered` — recovery handler invoked
- `clarification.emitted` — model asked the user for clarification
- `response.emitted` — final `AgentResponse` returned

OpenTelemetry trace format is **deferred to v2** (DEFERRED.md). Trigger: distributed tracing across multiple processes becomes useful (e.g., when state store is Redis per ADR 0007, or when eval runs are sharded across machines).

PII/auth redaction per ADR 0010 §redaction is applied at event-emission time. Fields matching `auth*`, `token*`, `key*`, `password*`, `secret*`, `authorization*` are replaced with `[REDACTED]`.

Reasoning

[REASONING PROMPTS — expand each bullet into 1-2 sentences in your own voice. Documentation depth: brief, honest, readable in 6 months. Not interview-defensibility depth.]

- **WHY ndjson over OpenTelemetry in v1**: zero new infrastructure is the point. The eval harness reads a `.jsonl` file with `pandas.read_json(lines=True)` — one line of code. OTel buys cross-service span correlation, but the v1 system is single-process; correlation doesn't need SDK-level machinery.
- **WHY defer OTel to a v2/v3 trigger**: the migration cost when OTel arrives (multi-process eval sharding, or a distributed state store) is bounded — every emission site swaps from `logger.info(event=...)` to OTel spans. Paying that later is cheaper than running an OTel Collector daemon now for capability we don't consume.
- **WHY event categories that follow the agent loop boundaries** (`request.received`, `task_file.constructed`, `validation.passed/failed`, `tool.dispatch.start/end`, `recovery.triggered`, `clarification.emitted`, `response.emitted`): the trace tells the STORY of a request stage by stage. If a scenario fails at "step 3," the trace should map back to a specific loop transition, not just "something errored." Categories = the loop's discrete states from ADR 0001.
- **WHY redaction at emission time, not later**: one chokepoint that can't be bypassed by a forgotten downstream filter. If a new credential field is added next week and someone logs it, the emission-layer processor catches it before it hits disk. Doing redaction downstream means every consumer needs its own filter — inconsistent, error-prone, and one missing filter = leaked secret.
- **WHY ndjson vs plain-text logs**: unstructured logs are fast to write and miserable to query. Ronin's eval harness needs to programmatically filter and aggregate events; that requires structure. The marginal cost of JSON over text is essentially zero and the queryability difference is huge.

- **WHY ndjson vs database-backed events**: file-based ndjson is durable, append-only, trivially git-diffable, and trivially attachable to bug reports. A database couples telemetry to state-store lifecycle (rollbacks would lose trace data — the opposite of what you want during failure analysis).

Alternatives considered

Alternative A — OpenTelemetry from day 1

Adopt the OTel SDK with spans, attributes, and baggage; run an OTel Collector locally; export to Jaeger or Tempo for visualisation.

why rejected — v1 has zero need for cross-service span correlation (single-process agent + eval). The collector is an extra process to run, configure, and monitor. The OTel SDK adds a 30+ MB dependency for capability we don't use. We accept the migration cost later if v2/v3 introduces multi-process eval runs.

Alternative B — Plain text logs (stdlib `logging`)

Use Python's stdlib `logging` with a custom formatter; parse with grep / awk / regex when investigating failures.

why rejected — unstructured logs are fast to write and miserable to query. Ronin's eval harness needs to programmatically filter and aggregate trace events; that requires structure. The marginal cost of writing JSON instead of text is essentially zero.

Alternative C — Database-backed event log (SQLite events table)

Write events directly into the SQLite state store (ADR 0007) as rows in an `events` table.

why rejected — couples telemetry to the state-store lifecycle (rollbacks would lose trace data, which is the opposite of what you want during failure analysis). File-based jsonl is durable, append-only, and trivially backed up. Querying via `duckdb` or `pandas` is fast enough for v1 eval volumes (~100 scenarios × ~10 events each).

Consequences

Positive

- Zero new infrastructure to deploy or maintain in v1
- Trace files are append-only, trivially diffed, easy to attach to bug reports
- Ronin's eval harness reads jsonl with one line of pandas / polars
- Schema is forward-compatible: new event categories are additive
- Auth redaction at emission is a single chokepoint, easier to audit

Negative

- No standard tooling (Jaeger UI, etc.) for visualisation — Ronin's eval has to build its own analyses
- Migration cost when (if) we move to OTel in v2 — every emission site needs updating
- jsonl files in `traces/` will accumulate on disk; needs a retention policy by Phase 3

Neutral

- Event ordering relies on file-append order + timestamp; sufficient for single-process v1, would need explicit ordering if v3 introduces multi-process eval

---

Open questions / follow-ups

- **Trigger for OTel migration**: documented in DEFERRED.md. Likely Phase 3 if multi-process eval becomes useful.
- **Event sampling**: v1 captures every event. If volume becomes a problem at Phase 3, introduce per-event-category sampling rates.
- **Trace correlation across the agent → inference backend → adapter chain**: handled by passing `trace_id` through call signatures in v1; might become unwieldy at scale and motivate context-local storage.
- **Schema validation for events**: not enforced in v1 — emitters trust each other. Worth a JSON Schema for emission discipline if the event vocabulary expands.

References

- ndjson spec: https://github.com/ndjson/ndjson-spec
- OpenTelemetry data model: https://opentelemetry.io/docs/specs/otel/overview/
- structlog (used by ADR 0010): https://www.structlog.org
- RONIN_SYNC.txt Tier 1 #1 — open discussion

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

ADR 0007 — State store

Status: Deferred to v2 (superseded for v1 by in-memory-only decision, 2026-06-23) **Original date**: 2026-06-20 **Downgraded**: 2026-06-23 **Author**: Marv **Related**: ADR 0001, ADR 0002, ADR 0006, docs/DEFERRED.md

---

v1 status (2026-06-23 update)

No persistent state store ships in v1. The SQLite design below is retained as the v2 target, not the v1 implementation.

What v1 actually does:

- Per-request state (`TaskFile`, `action_log`, cost) lives in memory only, scoped to the lifetime of one `Agent.handle_request()` call
- The dispatcher (`FakeAdapterDispatcher`) is constructed fresh per request; fake-adapter state comes from the request's `initial_state` field and is discarded when the response returns
- Idempotency (ADR 0002 §6 "duplicates: return previous report") is NOT enforced across requests in v1 — each request is treated as new
- Cross-run history is NOT tracked by the agent; if Ronin's eval needs to correlate runs it does so from its own `results/*.json` output

Why downgraded: the original ADR was aspirational — no SQLite code shipped in the Phase 1 push. Rather than leave the ADR claiming SQLite while no code exists (dishonest), we're explicit: v1 is in-memory, v2 targets SQLite.

v2 trigger: first Phase 3+ scenario that requires cross-request state — idempotency enforcement, multi-turn sessions with memory, or replay/audit of prior runs. When any of those arrives, implement the schema below.

See DEFERRED.md → "SQLite state store (moved to v2 target, 2026-06-23)" for the trigger and revisit criteria.

Context (original — v2 target)

The agent persists state across the lifetime of a request and across multiple eval runs:

- `task_file` objects for each request (see ADR 0002)
- Action logs (per-tool dispatch results, ordered)
- Cost accumulation per task
- Idempotency tracking (request hashes for duplicate detection per ADR 0002 §6)
- Reversibility state for rollback (per ADR 0009)
- Cross-run history for eval analysis (so a benchmark run can be replayed later)

For v1:

- Single-process, single-machine eval runs
- ~100 scenarios per benchmark run, ~10 tool calls per scenario worst-case → low write volume
- No external services running yet
- Ronin's eval harness reads runs back from disk for analysis

For v3 (potential):

- Multi-process eval runs (sharded scenarios across machines) might need a shared state store
- Higher concurrency might benefit from a process-external store

---

Decision (v2 target — NOT yet implemented)

SQLite for v2. A single `baymax.sqlite` file under `data/runs/<run_id>/baymax.sqlite` per benchmark run. Single connection per process, WAL mode enabled for concurrent reads alongside writes (Ronin's eval harness can read live without blocking the agent).

Redis at v3, conditional. If and only if multi-process eval runs need shared state. Trigger: a concrete v3 requirement that SQLite cannot satisfy. Not pre-emptively migrated.

Schema (v1, evolving — managed via Alembic migrations):

```
runs(run_id PK, started_at, finished_at, backend, model_version, scenario_set_version)
tasks(task_id PK, run_id FK, request_text, status, created_at, completed_at, cost_total,
response_text)
task_files(task_id FK, version, body JSON)               # task_file evolves across recovery passes
action_log(task_id FK, step_idx, tool, arguments JSON, result JSON, started_at, finished_at,
error_type)
idempotency(request_hash PK, task_id, expires_at)
```

Trace data does NOT live here — it goes to jsonl files per ADR 0006. State store is for queryable structured state; traces are for forensic event streams. Separating them prevents one's lifecycle (rollback, expiry) from breaking the other.

Reasoning

[REASONING PROMPTS — expand each bullet into 1-2 sentences in your own voice. Both the v1-in-memory downgrade AND the v2 SQLite target need reasoning here; treat them as two half-decisions.]

On the v1 downgrade to in-memory only (2026-06-23):

- **WHY downgrade rather than ship half a SQLite implementation**: honesty over aspiration. The Phase 1 push didn't ship SQLite code; leaving the ADR claiming SQLite while no code exists is dishonest and would surprise anyone reading the ADR + repo together. Explicit in-memory + v2 target is a truer picture.
- **WHY it's acceptable that v1 loses idempotency + cross-run history + audit**: Phase 1 eval is single-turn, single-process, no idempotency test cases in Ronin's scenario suite. Cross-run history is done by Ronin's eval harness against `results/*.json` output — not by the agent. What v1 loses, v1 didn't need.

On why SQLite is still the v2 target (not Redis, not Postgres):

- **WHY SQLite over Redis for v2**: zero daemon, zero operational surface, transactions built in, WAL mode gives Ronin concurrent read access while the agent writes — same posture as ADR 0006's zero-infra choice. Redis wins on concurrent multi-process writes; v2 doesn't have that need.
- **WHY SQLite over Postgres**: Postgres = a container to run, a connection pool to size, a backup strategy to design. All operational cost for capability v2 doesn't use. Postgres becomes right when you have concurrent writers across processes; single-process eval doesn't.
- **WHY defer Redis to v3 conditional**: not preemptive. Redis migration trigger is a concrete multi-process eval requirement — sharded scenarios across machines, shared session state across instances. Absent that, adopting Redis early = paying operational cost for capability we don't use.
- **WHY separate the state store from the trace store (ADR 0006)**: rollback semantics of state should not corrupt the trace stream. If the state store rolls back a failed transaction, the trace of what led to the failure MUST persist for failure analysis. Coupling them means one's lifecycle breaks the other's forensic value.
- **WHY WAL mode matters specifically**: readers and the writer don't block each other. Ronin's eval harness can watch a live run's state without contention, which is what makes SQLite viable for both write-side (agent) and read-side (eval) simultaneously.

Alternatives considered

Alternative A — Postgres from v1

Run a Postgres container alongside the eval harness; agent + eval both connect to it.

why rejected — Postgres is a container to run, a config to manage, a connection pool to size, and a backup strategy to design. v1 has none of these problems. Postgres becomes correct when we have concurrent writers (multiple agent processes), but v1 doesn't.

Alternative B — Redis from v1

Use Redis (in-memory + AOF persistence) as the v1 state store.

why rejected — Redis is fast and concurrent but loses on durability primitives (transactions across keys are awkward; ACID-grade integrity requires care). And there's nothing v1 needs that SQLite doesn't already give us. Adopting Redis early means paying its operational cost (separate process, RDB/AOF tuning) for capability we don't use. Per RONIN_SYNC.txt #8: deferred to v3 conditional on multi-process need.

Alternative C — File-based JSON / pickle blobs

Write task_files and action logs as JSON files under `data/runs/<run_id>/tasks/<task_id>.json`.

why rejected — no transactions (a crash mid-write leaves a broken JSON file), no cross-task queries (Ronin's eval needs to ask "all tasks where the agent used calendar.create_event" — that's a `SELECT` not a directory scan), and no idempotency primitives. SQLite gives all three for less work than rolling our own.

Alternative D — In-memory only

Keep all state in process memory; lose everything on process exit.

why rejected — eval analysis happens AFTER the run ends. In-memory state means no analysis. Also rules out crash recovery and run resumption.

Consequences

Positive

- Zero new infrastructure in v1 (matches ADR 0006's same posture for traces)
- Transactions: action log writes are atomic with status updates; no inconsistent state on crash
- Queryable: Ronin's eval can `SELECT * FROM action_log WHERE error_type = 'intent_error'` directly
- WAL mode means readers and the writer don't block each other; eval harness can watch a live run
- Backups are `cp baymax.sqlite baymax.sqlite.bak` — trivial

Negative

- Single writer per file — would not scale to multi-process eval (deferred per the Redis trigger)
- Schema migrations are real (Alembic adds a dependency + workflow) — but the alternative is uncontrolled schema drift, which is worse
- SQLite locks the whole file briefly during writes; high concurrency triggers `database is locked` errors. v1 single-writer avoids this.

Neutral

- Storage on disk grows linearly with runs; needs a retention policy by Phase 3 (matches the jsonl traces situation)

Open questions / follow-ups

- **Schema versioning**: managed via Alembic. v1 schema is "Version 1"; any change to the tables above bumps the migration version.
- **Run retention**: not enforced yet. Add a `scripts/retention.py` in Phase 3 to drop old runs.
- **Read amplification**: Ronin's eval queries should be benchmarked against a 200-scenario run by end of Phase 1 to confirm SQLite query time is acceptable.
- **Concurrent eval runs on the same machine**: each run gets its own `baymax.sqlite` file under its own `data/runs/<run_id>/` directory — no cross-run lock contention.
- **Redis migration trigger documented in DEFERRED.md**: multi-process eval runs sharing state.

References

- SQLite WAL mode: https://www.sqlite.org/wal.html
- Alembic: https://alembic.sqlalchemy.org/
- RONIN_SYNC.txt Tier 2 #8 — decision locked

- DEFERRED.md — Redis migration trigger

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

ADR 0008 - Eval judge protocol

Status: Deferred **Date**: 2026-06-23 **Author**: Ronin **Related**: docs/SCORING.md, docs/DEFERRED.md, ADR 0003

Context

BAYMAX evals need to decide whether an agent response satisfied a scenario. Some behaviors are easy to score deterministically:

```
expected tool == actual tool
expected start_time == actual start_time
expected body_contains appears in actual body
no tool call was made for clarification/refusal
```

Other behaviors are semantically messier. A clarification question can be worded many ways. A refusal can be polite, brief, or detailed while still being correct. Later multi-turn scenarios may require judging whether context was preserved across turns.

An LLM-as-judge protocol could score these softer cases, but it would introduce another model dependency into Phase 1 and make results less reproducible.

Decision

Phase 1 does **not** use LLM-as-judge. The scorer remains deterministic.

Current deterministic checks:

```
tool-call accuracy          ordered expected tools vs actual tools
unordered tool match        right tools in wrong order visibility
argument accuracy           exact arguments and explicit *_contains checks
clarification accuracy      required keywords in response message, no tool calls
refusal accuracy            required keywords in response message, no tool calls
hallucination rate          actual tools not listed in available_tools
task success                no failure reasons and full argument correctness
```

LLM-as-judge is deferred until deterministic checks become too brittle for the scenario set. When added, it should be a separate scorer mode, not a replacement for deterministic scoring.

Reasoning

why chosen - v1 needs benchmark numbers that are cheap, reproducible, and easy to debug. A judge model would create a second source of nondeterminism: the agent might fail, or the judge might disagree with the rubric. Deterministic scoring is less flexible but much easier to trust while we are still validating the scenario format, fake adapters, runner, and baseline output. Keeping judge work deferred also prevents Phase 1 from expanding into prompt-rubric engineering before the core harness is complete.

Alternatives considered

Alternative A - LLM judge from Phase 1

Use GPT/Claude as a judge for all natural-language responses and maybe tool-call reasoning.

why rejected - adds cost, latency, nondeterminism, prompt maintenance, and judge-version drift. It also makes local repeatability weaker before the harness is stable.

Alternative B - Human review only

Humans inspect scenario outputs and decide whether each run passes.

why rejected - useful for PR review but not a benchmark. It cannot produce repeatable aggregate metrics or CI-friendly failure reports.

Alternative C - Exact string matching only

Require the response message and arguments to exactly match expected strings.

why rejected - too brittle. Email bodies, titles, and clarification messages need limited flexibility. The `_contains` convention gives controlled flexibility without semantic judging.

Alternative D - Replace deterministic scorer with judge later

Use deterministic scoring in v1, then remove it once judge scoring exists.

why rejected - deterministic scoring remains valuable as a stable regression suite. Future judge scoring should be additive and used only where deterministic checks are insufficient.

Consequences

Positive

- Phase 1 results are reproducible and cheap
- Failures are easy to inspect because failure reasons are explicit
- CI can run the scorer without model access
- Scorer behavior is transparent enough to debug scenario authoring mistakes

Negative

- Semantically correct but differently worded responses can fail keyword checks
- `_contains` checks are weaker than true semantic equivalence
- Multi-turn and nuanced safety cases will eventually need richer judging

Neutral

- Scenario authors must choose clarification/refusal keywords carefully
- Future judge mode needs its own rubric, calibration set, and cost reporting

---

Open questions / follow-ups

- What scenario types justify judge scoring first: ambiguous clarification, refusal, or multi-turn?
- Which model judges the judge protocol, and how do we version judge prompts?
- Should judge scores be reported separately from deterministic scores?
- What minimum agreement with human review is required before judge scores are trusted?

References

- docs/SCORING.md
- docs/DEFERRED.md
- ADR 0003 - Scenario format and eval fixtures
- `src/baymax/eval/scorer.py`
- `results/phase1/scripted/v1-baseline.json`

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

ADR 0009 — Abort and rollback (reversibility, undo_action)

Status: Proposed for v1 contract; rollback execution deferred to v2 (per RONIN_SYNC.txt Tier 3 #15) **Date**: 2026-06-20 **Author**: Marv **Related**: ADR 0001, ADR 0002, docs/DEFERRED.md

---

Context

The agent loop (per ADR 0001) can hit a failure or an abort condition AFTER some tools have already fired. For example:

- A 3-step plan: `clipboard.read` → `notion.create_task` → `gmail.send`
- `notion.create_task` succeeds (task created in Notion)
- `gmail.send` fails (rate limit exceeded, won't recover by retry within budget)

What should the agent do? Three rough categories:

1. **Bail and leave state inconsistent** — task created, email not sent, user has no idea
2. **Roll back the successful step** — delete the just-created Notion task
3. **Commit nothing until everything succeeds** — buffer all side effects, only fire on full success

Option 1 is what bad agents do. Option 3 is two-phase commit, which doesn't compose with third-party APIs (you can't ask Notion to "prepare a task and commit later"). Option 2 is what mature systems do — every successful step exposes an inverse, and abort triggers reverse-order undo.

But tools differ in whether reverse is possible:

- `notion.create_task` — reversible (delete the task by ID)
- `calendar.create_event` — reversible (delete the event)
- `clipboard.read` — read-only, no side effect to reverse
- `gmail.send` — **irreversible** (you can't unsend an email)

For irreversible tools, the right pattern is verification BEFORE the action fires, not undo AFTER.

The locked decision from RONIN_SYNC.txt #15: "each tool adapter declares `reversible: bool` + `undo_action()`." The contract part is in v1; the execution (real rollback semantics with real adapters) is deferred to v2 when real adapters land.

Decision

Contract (v1)

Every tool adapter declares two attributes at registration time:

```
class ToolAdapter:
    name: str                       # e.g., "notion.create_task"
    reversible: bool                # True if undo_action exists
    def execute(self, args) -> ToolResult: ...
    def undo_action(self, action_log_entry) -> UndoResult: ...   # required iff reversible=True
```

The agent core invokes adapters through the validator (per ADR 0002) and uses the `reversible` flag in two places:

1. **Verification gate (before execution)**: if `reversible == False`, the agent emits a clarification-style prompt asking the user to confirm before the tool fires. v1 fake adapters in the eval harness are all reversible, so the gate never triggers during baseline eval runs. Real-adapter integration in v2 surfaces this.
2. **Rollback (on abort)**: when the recovery handler decides to abort (per ADR 0001 recovery loop), it walks the action log in REVERSE order:
   - For each entry with `reversible == True`: call `undo_action(entry)`. Log success/failure.
   - For each entry with `reversible == False`: log as "uncommitted side effect" — the world has been modified and we can't undo it. The final `AgentResponse` to the user must surface this honestly.

Abort triggers

The recovery handler aborts (rather than retrying or clarifying) when:

- A tool fails with a non-retryable error (4xx user error, auth_error after refresh attempt)
- The model produces an `intent_error` mid-execution (e.g., contradictory subsequent steps)
- The wall-clock or step budget is exhausted (see ADR 0002 §6 timeouts)

Execution (v2)

Real rollback execution is deferred to v2 because v1 uses fake adapters whose state lives in memory; rollback on a fake is just `state.pop(...)` and doesn't exercise the actually-hard part (idempotent undo against a real third-party API). v2's "real-adapter integration" milestone is when this ADR's execution semantics get tested in anger.

Reasoning

[REASONING PROMPTS — expand each bullet into 1-2 sentences in your own voice. Documentation depth: brief, honest, readable in 6 months.]

- **WHY two patterns rather than one (verification gate for irreversible, undo for reversible)**: tools genuinely fall into two categories, and forcing one pattern to cover both is broken. You cannot undo a sent email; you cannot verify-then-delay every Notion write without leaking the mechanism into user experience. Two categories = two right patterns.
- **WHY the contract lands in v1 even though execution is deferred to v2**: adapter authors (Ronin in v2 when real adapters land) need to know what shape they're building against. A deferred contract = deferred adoption; declaring the contract now means v2 adapter work is mechanical rather than another design cycle.

- **WHY honesty over silent failure when irreversible side effects can't be undone**: partial success is a first-class case in real agent systems, not an edge case. If the agent has drafted an unsendable email but the Notion task already exists, the user has to know both facts. Hiding is worse than admitting — silent partial commits are the pathology of demo-only agents, and per the CV's "no fake AI agent wrapper" warning, that's the failure mode this project must not display.
- **WHY the recovery handler walks the action log in REVERSE order on abort**: correct undo order for compound actions. If step 1 creates a Notion task and step 2 assigns it to a user, the assignment must be undone before the task itself is deleted — same principle as reverse-order teardown in RAII / defer patterns.
- **WHY the `reversible: bool` flag lives on the adapter itself, not on the tool-call schema**: capabilities describe the ADAPTER's contract with the world, not the agent's request shape. An adapter author knows whether their tool can be undone; the agent shouldn't have to reason about it per-call.
- **WHY v1 fake adapters are all reversible**: makes the eval baseline clean — the verification-gate code path never triggers, so eval focuses on the core agent loop. When v2 introduces real irreversible adapters (Gmail send), the gate starts firing and adds a new eval dimension.
- **WHY reject two-phase commit as an alternative**: third-party APIs don't expose a prepare phase. You can't ask Notion to "prepare a task without creating it." Implementing prepare via a draft-workaround leaks the mechanism into the domain and breaks for tools where no draft concept exists. Real distributed transactions across heterogeneous APIs aren't solved in the literature for this case.

Alternatives considered

Alternative A — No rollback at all, bail on first error

When a tool fails, return an error response and leave the action log as it stands; user gets back "step 2 failed" with no cleanup.

why rejected — leaves the user in an inconsistent state with no recourse. Notion task exists, calendar event exists, but the user doesn't know about either because the agent stopped mid-flow. This is the "bad demo" failure mode. Per the CV's "no fake AI agent wrapper" warning — silent partial commits are the exact pathology of unserious agents.

Alternative B — Two-phase commit (prepare/commit) across all tools

Every tool exposes a `prepare()` and a `commit()` method; the agent only fires `commit()` after every `prepare()` returns success.

why rejected — third-party APIs don't expose a prepare phase. You can't ask Notion to "prepare a task without creating it." Implementing prepare via a "draft" workaround (create with status=draft, then update to status=committed) leaks the mechanism into our domain and breaks for tools where no draft concept exists. Real distributed transactions across heterogeneous APIs are not solved in the literature for our case.

Alternative C — Best-effort undo without contract, just retry the failed step harder

When a tool fails, don't roll back; instead retry with longer backoff and hope it resolves.

why rejected — for non-retryable errors (auth expiry, 4xx user error, irreversible side effect already committed downstream) retry does nothing. For genuinely transient errors, retry IS what the recovery loop does (per ADR 0001) — but at some point the budget is exhausted and we need a non-retry path. That non-retry path needs an undo contract.

Alternative D — Per-tool ad-hoc rollback, no shared contract

Each tool adapter handles its own failure recovery in whatever way is natural; no shared `reversible` flag, no shared undo_action signature.

why rejected — the recovery handler in the agent core would need per-tool knowledge to decide what to do on abort. That moves orchestration logic into the wrong layer. Shared contract = orchestration stays in core, adapter just declares its capabilities.

Consequences

Positive

- Adapters that are honest about reversibility get the right runtime behaviour (rollback if reversible, verification gate if not)
- The user is never lied to about partial state — irreversible side effects are surfaced in the final response
- Contract is testable in v1 fakes; execution surface is small enough that v2 real-adapter authors have a clear target
- Adding a new adapter is mechanical: declare flag, implement execute + undo (or just execute if irreversible)

Negative

- Every adapter author has to think about reversibility; adapters with subtle "kind of reversible" semantics (e.g., calendar event that's been seen by attendees) need careful design
- Verification gates increase user friction for irreversible operations — but the alternative is silent commits, which is worse
- v2 real-adapter integration must prove the undo path actually works against real APIs; this is the hard part

Neutral

- Multi-phase tasks (draft → verify → send pattern for irreversible tools) become the natural follow-up — tracked as deferred per RONIN_SYNC.txt #14 / DEFERRED.md

---

Open questions / follow-ups

- **Idempotent undo**: what if `undo_action()` itself fails partway? In v2, undo operations need their own retry / idempotency story. Sketch: undo is itself an action_log entry, so it can be retried.
- **"Partially reversible" tools**: e.g., calendar.create_event is technically reversible but if the event has been seen by other attendees, the undo isn't free of social cost. v1 treats this as a binary flag; v2 may need a richer "side effect class" taxonomy.

- **Multi-phase task pattern**: how does draft → verify → send compose with sub-task chains (deferred per RONIN_SYNC.txt #13)? Both deferred together to v2/v3.
- **Audit log of rollback decisions**: the recovery handler's choice to abort needs to be visible to Ronin's eval scorer; covered via the `response_type` enum in ADR 0002.

References

- Two-phase commit (and why distributed transactions across heterogeneous APIs are hard): Gray & Lamport, "Consensus on Transaction Commit" (2006)
- Saga pattern (compensating transactions): Garcia-Molina & Salem, "Sagas" (1987) — closest classical analogue to what we're doing
- ADR 0001 for the recovery handler this ADR plugs into
- ADR 0002 for the response_type taxonomy
- RONIN_SYNC.txt #15 — decision locked
- DEFERRED.md — v2 trigger for execution semantics

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

ADR 0010 — Telemetry

Status: Proposed (Marv to confirm) **Date**: 2026-06-20 **Author**: Marv **Related**: ADR 0002, ADR 0006, ADR 0007, src/baymax/telemetry/logger.py

---

Context

BAYMAX needs telemetry that:

1. Records every event in the agent loop with enough context to reconstruct what happened after the fact
2. Never leaks secrets (API keys, auth tokens, OAuth credentials) into trace files, log files, or stderr
3. Correlates events within a single request across modules (agent core → inference backend → tool adapter)
4. Is consumed both by Ronin's eval harness (for failure analysis) and by developer eyeballs (for debugging during dev)
5. Has zero new infrastructure requirements in v1 — same posture as state store (ADR 0007) and trace format (ADR 0006)

The constraint set is shaped by:

- ADR 0002 §6: environment-based auth → tokens must never appear in traces
- ADR 0006: ndjson trace format — telemetry emits ndjson events
- Single-process v1 — no distributed tracing requirement yet

The choice surface is "which library writes the JSON" — not whether to use JSON. The candidates:

- `structlog` — structured logging library; emits dict events; pluggable processors
- Python stdlib `logging` with a custom JSON formatter — works but ergonomically rough
- OpenTelemetry SDK directly — full distributed-tracing semantics, overkill for v1
- Roll our own — re-invents the wheel for no benefit

Decision

`structlog` **for v1**, configured to:

- Emit ndjson events (one event per line, matching ADR 0006)
- Run a redaction processor on every event before serialisation
- Inject a `trace_id` context variable so events across modules within one request correlate
- Write to `traces/<run_id>/<task_id>.jsonl` (per ADR 0006 path)

Redaction processor

Any event field whose key matches one of the following case-insensitive patterns is replaced with `[REDACTED]` before the event is written:

- `auth*`
- `token*`
- `key*`
- `password*`
- `secret*`
- `authorization*`
- `bearer*`

The processor runs as the LAST step in structlog's chain, after all event construction. This means any code path that accidentally includes a credential in an event payload gets caught at the chokepoint, not at the call site.

Trace correlation

`trace_id` is bound via `structlog.contextvars.bind_contextvars(trace_id=...)` at the start of each request. All subsequent events emitted in that request (across modules) automatically include the trace_id. When the state store ships in v2 (ADR 0007 — deferred as of 2026-06-23), it will join traces to tasks via this id. In v1 the join is done by grep/`jq` against the `traces.jsonl` file.

Event shape

Matches ADR 0006 event shape exactly — telemetry is the writer; ADR 0006 is the schema.

OpenTelemetry deferral

OTel format remains deferred to v2 pending Ronin discussion (per RONIN_SYNC.txt Tier 1 #1 and ADR 0006). When OTel migration happens, structlog's processor chain swaps the final JSON formatter for an OTel exporter — the rest of the emission code is unaffected.

Reasoning

[REASONING PROMPTS — expand each bullet into 1-2 sentences in your own voice. Documentation depth: brief, honest, readable in 6 months.]

- **WHY structlog over stdlib** `logging`: stdlib's model is "message + optional extra dict"; passing structured event payloads naturally is awkward. Custom formatters get ugly, and filters run BEFORE message construction in some paths, making redaction unreliable. Structlog is built specifically for structured events and processor pipelines.
- **WHY structlog over rolling our own logger**: reimplements structlog badly. The only reason to roll our own would be if structlog had a limitation we hit — it doesn't. Mature, well-tested, right shape.
- **WHY redaction at the LAST step in the processor chain, not at the call site**: single chokepoint that can't be bypassed by a forgotten log statement. If someone next week adds a new credential field to an

event payload, it gets caught at emission — nobody has to remember to update N call sites. Redaction at the call site scales badly; every new field is a new opportunity to leak.

- **WHY contextvars over passing `trace_id` through call signatures**: correlation is a cross-cutting concern, not an application-logic concern. Threading `trace_id` through every function signature pollutes the code with infrastructure detail and makes refactoring painful. Contextvars solve the correlation problem WITHOUT polluting call signatures — cleaner separation.
- **WHY defer OTel to v2** (aligned with ADR 0006): the SDK is 30+ MB of dependency for capability we don't use in single-process v1. When OTel migration happens, structlog's processor chain swaps the final formatter for an OTel exporter — the emission sites are unaffected. Migration is bounded by design.
- **WHY the specific redaction pattern list (`auth*`, `token*`, `key*`, `password*`, `secret*`, `authorization*`, `bearer*`)**: covers the credential fields that show up in adapter payloads and API responses. Not exhaustive — the list is reviewed quarterly as new adapters land (documented as an open question). Defensible baseline, not a claim of completeness.

Alternatives considered

Alternative A — Python stdlib `logging` with a custom JSON formatter

Use `logging.getLogger(__name__)`, configure a custom formatter that emits JSON, add a filter for redaction.

why rejected — stdlib logging's data model is "message + optional extra dict"; you can't naturally pass structured event payloads. Custom formatters get ugly fast. Filters run BEFORE message construction in some paths, making redaction unreliable. Structlog was built specifically for this use case.

Alternative B — OpenTelemetry SDK directly

Adopt OTel's tracer / span / attribute model; use the OTel SDK's exporters and instrumentation libraries.

why rejected — OTel buys distributed-trace correlation we don't need yet (single-process v1). The SDK is heavy (30+ MB dependencies). The semantic conventions assume a span-based mental model that doesn't perfectly match our event-stream model. ADR 0006 already deferred OTel to v2; this ADR follows that decision.

Alternative C — Roll our own logger

Write a small custom logger class that emits ndjson, handles redaction, manages context.

why rejected — re-implements structlog, badly. Structlog is mature, well-tested, has the right shape. The only reason to roll our own would be if structlog had some specific limitation we hit — it doesn't.

Alternative D — `loguru`

A popular alternative to structlog with a simpler API.

why rejected — loguru's redaction story is weaker (you have to filter messages by regex, no native field-level processor chain). For "must redact every credential field every time" requirements, structlog's processor chain is the right primitive.

Consequences

Positive

- Redaction is a single chokepoint that can't be bypassed by a forgotten log statement
- Adding fields to events is trivial (just pass them as kwargs to the logger call)
- Trace correlation across modules is implicit (no manual trace_id threading through call signatures)
- Migration to OTel in v2 is local to the structlog processor chain — emission sites are unaffected
- Cost per event is sub-microsecond on modern Macs (benchmark Phase 1 confirms)

Negative

- structlog adds a dependency and a small learning curve for new contributors
- The processor chain is the kind of "implicit configuration" that surprises people the first time they hit it — needs to be documented in `src/baymax/telemetry/README.md`
- Contextvars don't propagate across `asyncio.create_task` boundaries unless you propagate manually; need a wrapper

Neutral

- Redaction patterns are listed in code; any new credential category not on the list could leak. Mitigation: review the redaction list quarterly as new adapters are added

---

Open questions / follow-ups

- **Asyncio context propagation**: confirm that contextvars survive `asyncio.gather` boundaries in our specific call paths; test with a tool dispatched via `gather` in Phase 1.
- **Per-module log levels**: not needed yet (everything emits at INFO). Add per-module filtering when log volume becomes a problem.
- **Sampling**: not in v1 (every event is emitted). Add per-event-category sampling rates when volume is a problem at Phase 3.
- **Sensitive payload redaction beyond credentials**: e.g., should the body of a user's email draft be redacted in traces? Not in v1 (the agent has to work with the content). Worth a v2 discussion when real adapters arrive and traces start including real personal data.
- **OTel migration trigger**: tracked alongside ADR 0006 in DEFERRED.md.

References

- structlog docs: https://www.structlog.org
- structlog processor chain pattern: https://www.structlog.org/en/stable/processors.html
- Python contextvars (PEP 567): https://peps.python.org/pep-0567/
- ADR 0002 §6 — env-based auth requirement that drives redaction

- ADR 0006 — ndjson event format this ADR emits
- ADR 0007 — state store this ADR's trace_ids join into
- RONIN_SYNC.txt Tier 1 #1 — OTel-vs-custom discussion

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

ADR 0011 — Backend Interface v2

Status: Proposed (Marv to confirm after review) **Date**: 2026-08-21 **Author**: Marv **Related**: ADR 0001, ADR 0002, ADR 0005 (superseded for v2), ADR 0006, ADR 0010, ADR 0012, docs/BACKEND_FLOW.md, docs/METHODOLOGY.md

---

Context

Phase 1 shipped a v1 backend abstraction (ADR 0005) — the `InferenceBackend` protocol with `plan_task` + `interpret_results`, one concrete `OpenAIBackend`. Phase 2 demands a richer surface:

1. **Local MLX inference** as a second backend for the Qwen 2.5 1.5B + LoRA fine-tune (per ADR 0004 base model choice and RONIN_SYNC B-tier decisions)
2. **Retry + resilience** — Ronin's eval fires many requests in parallel; transient 429s and network failures need retry logic that doesn't invite the user to reason about them
3. **Warmup** — MLX cold-start is 5-10s; server must not accept requests before the model is ready
4. **Registry-driven metadata** — model facts (context window, cost model, adapter paths) live in `models/models.json` per ADR 0012; backend must read them
5. **Failure surfacing** — backend errors must map cleanly to `AgentResponse` so scorer can bucket them
6. **Uniformity for research fairness** — every backend must implement identical semantics; agent code must have zero backend-conditional logic (per METHODOLOGY §7)

ADR 0005's minimal-surface protocol works for the OpenAI-only Phase 1 case but doesn't accommodate the above. This ADR is the v2 iteration.

Constraint set:

- FastAPI async is the primary caller (ADR 0002 §implementation)
- Author-split discipline (METHODOLOGY §3) — Marv owns backends, agent, adapter; Ronin doesn't touch this code
- Extended timeline (Nov-Dec 2026 target) — quality over calendar
- Backend classes MUST be per-transport (OpenAI API, MLX local, Anthropic API) not per-model — one class handles all models within a transport

---

Decision

1. Backend hierarchy and factory pattern

`InferenceBackend` remains an ABC in `src/baymax/service/inference.py`. Three concrete classes total, ever:

- `OpenAIBackend` — HTTPS to OpenAI API. Handles all OpenAI-served models via `model_id`.
- `MLXBackend` — local MLX inference. Handles all locally-served Qwen variants + adapters via `model_id`.

- `AnthropicBackend` — deferred (v3+). Not implemented in v1.

Backend selection at app startup uses a factory dict pattern:

```
BACKENDS: dict[str, type[InferenceBackend]] = {
    "openai": OpenAIBackend,
    "mlx": MLXBackend,
}
```

Selection via env var `BAYMAX_BACKEND=openai|mlx`. FastAPI lifespan constructs one backend instance for the process lifetime, passes it to the agent as a constructor dependency (matches ADR 0005 §Decision).

Adding a fourth backend later = one line in `BACKENDS` + one new class file.

2. Method signatures

Base class ABC:

```
class InferenceBackend(ABC):
    model_id: str
    max_context: int          # from registry per §7
    cost_model: str           # "per_token" | "amortized"
    provider: str             # "openai" | "mlx" | ...

    def __init__(self, model_id: str) -> None:
        # Fetches own required env vars (OPENAI_API_KEY, MLX_MODEL_PATH, etc.)
        # Loads registry entry per §7
        # Raises loud on missing env or missing registry entry

    async def plan_task(self, task: TaskFile) -> PlanResult: ...
    async def interpret_results(self, task: TaskFile, results: list[ToolCallResult]) ->
Interpretation: ...
    async def warmup(self) -> None: ...
    async def _call_llm(self, prompt: str, tools: list[Tool]) -> RawLLMResponse: ...  # base impl
with retry
```

Return types are wrapped:

```
@dataclass
class PlanResult:
    steps: list[ToolCallStep]
    tokens_used: int
    cost_usd: float
    warnings: list[str] = field(default_factory=list)

@dataclass
class Interpretation:
    text: str
```

```
    tokens_used: int
    cost_usd: float
```

Naming keeps `plan_task` + `interpret_results` (ADR 0001 Plan-and-Execute pattern). All methods async; sync work (e.g., MLX inference) wrapped via `run_in_executor` inside the backend.

`_call_llm` is a shared base-class helper — centralizes retry logic (§5) and telemetry emission so subclasses don't reimplement.

Streaming intentionally omitted for v1 (deferred to v2 per DEFERRED.md). When added, a new `plan_task_stream()` method rather than a signature break to `plan_task()`.

3. Construction and configuration

- `model_id`: constructor param, passed by FastAPI lifespan (populated from `BAYMAX_MODEL_ID` env). Backend is a pure function of its inputs; tests pass literal strings.
- **Backend selection**: env var `BAYMAX_BACKEND`.
- **Secrets**: each backend fetches its own required env vars in `__init__` and raises loud on missing:
  - `OpenAIBackend`: `OPENAI_API_KEY`
  - `MLXBackend`: `MLX_MODEL_PATH` (or reads from registry entry per §7)
  - `AnthropicBackend` (v3+): `ANTHROPIC_API_KEY`
- **Configuration validation timing**: lifespan startup. FastAPI `lifespan()` constructs the backend, warmup runs, `/health` becomes ready. Anything wrong = process exits before requests are accepted.

Env-var summary:

Env var	Required by	Purpose
`BAYMAX_BACKEND`	Factory	Selects backend class (`openai` or `mlx`)
`BAYMAX_MODEL_ID`	All backends	Model to serve within the selected backend
`BAYMAX_RETRY_MAX_ATTEMPTS`	Base class retry	Defaults to 3; sweep target per §5
`OPENAI_API_KEY`	OpenAIBackend	API auth
`MLX_MODEL_PATH`	MLXBackend	Weights location (or from registry)

4. Warmup semantics

- **When**: during FastAPI lifespan startup, after backend construction, before `lifespan()` yields
- **What**: execute a dedicated warmup scenario from `src/baymax/service/warmup/warmup_scenario.json` (uses the standard scenario schema; forces at least one tool-call generation to warm MLX's structured-output compute graphs)
- **Gate**: `/health` returns `503 {"ready": false, "reason": "warming_up"}` until warmup completes; then `200 {"ready": true, "backend": <str>, "model_id": <str>}`
- **Timeout**: 120s hard cap. Beyond that = deterministic failure (bad weights, wrong path, incompatible format). Raises `WarmupTimeout` → uvicorn exits.

- **Failure handling**: fail-loud. NO retry on warmup errors. All warmup failure modes (missing weights, corrupted checkpoint, format incompatibility, base-model mismatch, MLX version mismatch, insufficient RAM, tokenizer mismatch, Metal unavailable, malformed warmup JSON) are deterministic — retry masks real problems. Errors raise `WarmupFailure(stage, cause, hint)` for diagnostic clarity.
- **Telemetry tag**: warmup call emits with `is_warmup: true` so Ronin's eval runner can filter it out of eval score aggregations.

5. Retry policy

`_call_llm` implements retry with the following semantics:

- **Max attempts (N)**: **hardcoded to 3.** Simplified from earlier "configurable + swept in eval" decision (2026-08-24 revision) — publication-review-oriented sweep infrastructure was overkill for an artifact-first project. If retry sensitivity ever becomes a real concern, promote to configurable then.
- **Backoff formula**: `delay = 1s × (2 ^ attempt)` → 1s, 2s, 4s. Plus ±25% random jitter to prevent synchronized retries from parallel eval batches.
- **Retriable errors** (API backends only):
  - HTTP 408 (Request Timeout)
  - HTTP 429 (Rate Limit)
  - HTTP 500-504 (server errors)
  - Connection errors (`ConnectionResetError`, `ConnectTimeoutError`, DNS failures)
- **Non-retriable errors** (fail immediately):
  - HTTP 400 (Bad Request — includes OpenAI content-policy violations)
  - HTTP 401 (Unauthorized)
  - HTTP 403 (Forbidden)
  - HTTP 404 (Not Found)
  - HTTP 422 (Unprocessable Entity)
- **MLX errors**: NO retry, ever. All MLX errors are deterministic (memory pressure, model config, config drift) — retry doesn't self-heal on the seconds of a retry window. Same principle as warmup fail-loud.
- **Telemetry**: BOTH per-attempt events AND final aggregate emitted. Per-attempt for debugging (`event: "backend_attempt"`, `event: "backend_retry"`); aggregate for accounting (`event: "backend_call_complete"`, `total_attempts: N`, `final_status`).

6. Error surfacing

- **Exception type**: `BackendError` in `src/baymax/core/exceptions.py`. Wraps underlying exception always (even non-retry failures). Has `kind`, `underlying`, `attempts_history`, `backend` fields.
- `kind` **values** (extensible enum-of-strings, not exhaustive):
  - `"retry_exhausted"` — retriable error hit `max_attempts` without success
  - `"non_retriable"` — first-attempt error was classified as non-retriable (HTTP 400/401/403/404/422, or any MLX error)
  - `"unreachable"` — defensive guard for the retry loop exiting without return/raise (should never fire in practice)

- **Agent-side internal state**: on `BackendError`, agent sets `TaskFile.completion_status = FAILED` and `TaskFile.error_type = ErrorType.BACKEND_ERROR` (new enum value added to `contracts.py`). Full underlying + attempt history retained in the action log for forensic value. `AgentResponse` boundary shape is NOT modified — remains `{tool_calls, message}` per its scorer contract.
- **HTTP status (revised 2026-08-26)**: `/invoke` returns **HTTP 500** with a JSON error body when `BackendError` propagates. Rationale: backend failure is infrastructure-layer, not an agent decision. Clean separation:
  - `200 + AgentResponse` = agent made a decision (execute, refuse, clarify)
  - `5xx + error body` = infrastructure failed (backend down, retries exhausted, warmup incomplete)
  - Aligns with HTTP semantics; keeps `AgentResponse` boundary contract intact for Ronin's scorer.
- **500 body shape**: `{"error": "backend_error", "kind": <retry_exhausted|non_retriable>, "backend": <provider>, "message": <str>, "attempts": <int>}`. No stack traces, no `underlying` repr in the body — those go to the trace file only, per redaction discipline (ADR 0010).
- **FastAPI wiring**: `app.exception_handler(BackendError)` registered in `api.py`'s lifespan. Handler builds the 500 body from `exc.kind` / `exc.backend` / `len(exc.attempts_history)`. Agent code itself does NOT catch `BackendError` — lets it bubble up.
- **Retry history in trace**: yes, full per-attempt events emitted per §5 propagate into the trace stream.
- **Ronin coordination note**: eval runner needs a try/except around the httpx call and a "backend_error" bucket in categorization. Small change on his side; flag in next sync.

7. Registry coupling

Backend reads the required registry fields at construction time (see ADR 0012 §2):

- `id` (model identifier)
- `base` (base model)
- `adapter_path` (nullable for non-adapter models)
- `trained_at`
- `training_config_hash` (reproducibility trail; may be named `config_hash` per Ronin's naming — see ADR 0012 sync note)

Backend does NOT hardcode any per-model facts. Adding a new fine-tuned model = registry entry update, not a backend code change.

Runtime facts NOT stored in registry (per ADR 0012 §2 trimmed optional list):

- `max_context` — read from HF config of the base model at `__init__` time
- `cost_model` — hardcoded per backend class (`OpenAIBackend.cost_model = "per_token"`, `MLXBackend.cost_model = "amortized"`)
- `tokenizer_path` — inferred from the base model directory / HF config

Consequence: registry lookup happens ONCE at `__init__` time; supplementary facts loaded from the base model's own config in the same phase. Backend caches everything as instance attributes for the process lifetime. No per-request registry reads.

8. Testing

- **Unit tests**: use `FakeBackend` in `tests/fakes/backend.py`. Canned responses via constructor args. Registered in the `BACKENDS` dict under `"fake"` for tests that spin up the full FastAPI app.
- **Retry tests**: patch `asyncio.sleep` so backoff delays don't slow the unit test suite.
- **Warmup gate tests**: `FakeBackend(model_id="fake", warmup_delay_s=0.5)` — injectable delay makes the 503→200 transition observable.
- **Integration tests**:
  - OpenAI: opt-in via `pytest.mark.integration` and `[test-integration]` PR tag. Requires `OPENAI_API_KEY`. Auto-retry once on transient flake, then fail loud.
  - MLX: Marv-local only. `pytest.mark.mlx_local`, skipped unless `HAVE_MLX=1`. Not in CI (Ahmed doesn't have Mac hardware; MLX is Marv-owned per OWNERSHIP.md).
- **No nightly scheduled runs**. Ahmed's eval runs during Phase 2 naturally catch drift. Project ends Dec 2026 — nightly overkill.

Reasoning

[REASONING PROMPTS — expand each bullet into 1-2 sentences in your own voice. Documentation depth: brief, honest, readable in 6 months (artifact-first project, per 2026-08-24 scope revision).]

- **WHY template-per-TRANSPORT, not per-model** (OpenAIBackend handles all OpenAI models, MLXBackend handles all Qwen variants): OpenAIBackend already handles gpt-4o-mini + gpt-4o + gpt-4-turbo as one class. Same logic for MLX. 3-4 classes total ever = manageable. Class-per-model would explode into dozens of near-duplicates that must all be kept in sync — SOLID violation of the wrong kind.
- **WHY enforce interface uniformity rather than advertise** `.capabilities()`: research fairness. If the agent branches based on backend capabilities, the eval-score delta between backends is contaminated by agent-code-path differences — you can't isolate "the model" as the variable. Every backend absorbs its own capability gaps internally; the agent code has ZERO backend-conditional logic. Descriptive metadata (`max_context`, `cost_model`) IS exposed for telemetry, just not consumed by agent logic.
- **WHY** `_call_llm` **on the base class rather than duplicated per backend**: retry logic is cross-cutting; centralizing it in one place means retry policy can't drift between backends. Every retry rule, every backoff calculation, every telemetry emission happens exactly once. Duplicating retry per backend guarantees they'll diverge subtly and someone will chase a bug through three implementations.
- **WHY retry N = 3 hardcoded** (2026-08-24 revision, replaces earlier configurable+swept decision): the retry-sensitivity sweep was publication-review flavor — turning "is retry a variable?" into a formal empirical measurement. For an artifact-first project, N=3 matches SDK conventions (OpenAI, Anthropic SDKs) and is enough resilience for eval batch runs without overengineering. If retry sensitivity ever becomes a real concern for the artifact story, promote to configurable then.
- **WHY hardcode base delay + factor + jitter**: same principle as N=3 — SDK-convention values (1s base, 2x factor, ±25% jitter) work fine. Configurability adds knobs no one is going to turn.
- **WHY MLX gets NO retry, ever** (including OOM and inference timeout): MLX errors are deterministic. Metal OOM's freeing event isn't synchronized to your 7-second retry window; MLX inference timeout on the same prompt and same infrastructure will time out again. Retry masks systemic issues (memory pressure, config drift) that need to fail loud so the human fixes root cause. Same principle as warmup fail-loud.

- **WHY the retry policy asymmetry is honest, not a bug**: API backends have network-transient errors that self-heal on external clocks (rate windows reset, servers recover). MLX has local-and-deterministic errors that don't self-heal on retry timescales. Modeling that asymmetry accurately IS the honest choice; making them symmetric would either mask real API problems (N=1 on OpenAI inflates failure rate on 429s) or waste time on MLX (retry adds nothing). The asymmetry is reported honestly per METHODOLOGY §6 statistical requirements.
- **WHY custom** `BackendError` **always** (wraps even non-retry errors): agent has ONE exception type to handle. Cross-backend debugging is easier when errors have consistent shape. Original exception preserved as `underlying` — no forensic loss. Different exception types per backend means agent code needs backend-conditional catches, which violates §7 uniformity.
- **WHY new** `ErrorType.BACKEND_ERROR` **enum value** rather than reusing `INTERNAL_ERROR`: backend failures are a distinct failure mode from agent-logic failures (validator errors, missing arguments, ambiguous requests). Ronin's scorer needs to bucket them separately. Reusing an existing enum value = losing information the eval needs.
- **WHY warmup uses a dedicated JSON file** (rather than hardcoded string): reuses your existing scenario schema — no new format to maintain. Editable without a code change. Doesn't point at real eval scenarios (avoids confusion of "why is this eval scenario running at startup?"). Consistent with the "everything is data" posture.
- **WHY warmup MUST exercise tool-call generation**: MLX may compile separate compute graphs for structured-output paths (tool-call formatting) vs plain text completion. A "hello world" warmup prompt only warms one path; first real request that requires tool-calling still eats the cold-graph latency. Warmup scenario is designed to force at least one tool call.
- **WHY warmup blocks** `/health` **and warmup failure exits uvicorn**: server that reports healthy while unable to serve requests is worse than server that reports unhealthy. Load balancers, eval harnesses, k8s probes all correctly interpret 503 as "not ready" and wait or route elsewhere. Startup failure = uvicorn exit with clear error = deploy correctly fails; alternative (server accepts requests then errors on first real work) makes drift look like intermittent flakiness.
- **WHY registry-driven metadata rather than backend-internal**: registry is single source of truth per METHODOLOGY §8. Duplication of facts in backend code = drift between backend and registry. Adding a new fine-tune = registry entry + weights drop = backend picks it up. Backend-internal knowledge means every new model is a code change, which slows Ronin's training iteration.
- **WHY lifespan-startup validation over import-time or first-request**: import-time = testing pain (unit tests unrelated to backend selection need env vars set to import the module). First-request = user-visible failure (server reports healthy but errors on first work). Lifespan is the layer where "starting the server" concerns belong — fails before requests accepted, doesn't break test imports.
- **WHY per-attempt AND aggregate telemetry (both)**: per-attempt is real signal for eval analysis and debugging ("45% of scenarios needed 2+ attempts on OpenAI" changes analysis). Aggregate is for eval-scoring accounting ("how many completed successfully overall"). Consumer filters at query time — Marv wants debug view, Ronin wants score view; both from the same emitted stream.
- **WHY FakeBackend is a dedicated class rather than mocking via** `unittest.mock`: explicit fake makes test intent readable. Multi-step call sequences (return this then that) are cleaner with a stateful fake than with mock side-effects. The factory dict pattern makes wiring trivial.
- **WHY on-demand integration tests, not nightly**: project ends Dec 2026. Silent drift probability over 4 months is very low. Ahmed's regular eval runs already function as integration tests. Nightly scheduled runs = cost + noise for capability the project doesn't need.

- **WHY MLX integration is Marv-local, not in CI**: Ahmed doesn't have Mac hardware; GitHub Actions macOS runners cost ~10x Linux minutes plus need weights downloaded per run. MLX ownership is Marv's per OWNERSHIP.md; asymmetric test coverage is acceptable for a two-person project with clean boundaries.

Alternatives considered

Alternative A — One backend class per model (rejected)

Rather than `MLXBackend(model_id="qwen-1.5b-baymax-v1")`, have `Qwen15BBaymaxV1Backend`, `Qwen15BBaymaxV2Backend`, `Qwen7BBackend` as separate classes.

Why rejected: massive code duplication for backends that share the same transport (MLX process management, tokenization, prompt formatting). Adding a fifth model = new class + new tests + risk of divergence. Backends should be per-transport; model is just a config param.

Alternative B — Advertise capabilities via `.capabilities()` (rejected)

Each backend exposes `capabilities() -> dict` with `{streaming, native_tool_calling, max_context, cost_model}`; agent branches on these.

Why rejected: contaminates the research comparison. If the agent branches based on backend capabilities, eval-score deltas between backends are attributable to agent-code-path differences, not model quality. Research position (METHODOLOGY §1) demands agent code has zero backend-conditional logic. Descriptive metadata is still EXPOSED — just for telemetry/debug, not consumed by agent logic.

Alternative C — Sync-only interface (rejected)

Backends are sync; wrap in `run_in_threadpool` at the agent layer.

Why rejected: FastAPI is async. Every sync backend call would block the event loop or require thread-pool overhead at the agent layer for every backend. Making backends async and letting them wrap sync work internally (via `run_in_executor` for MLX) keeps the calling convention uniform and the event loop unblocked.

Alternative D — Backend-internal model metadata (rejected)

Each backend has hardcoded knowledge per model:

```
MLXBackend.MODEL_METADATA = {
    "qwen-1.5b-baymax-v1": {"max_context": 32768, "adapter_path": "..."},
    "qwen-1.5b-baymax-v2": {"max_context": 32768, "adapter_path": "..."},
}
```

Why rejected: adding a new fine-tune requires a backend code change, which slows Ronin's training iteration. Registry drift (backend's hardcoded facts vs `models/models.json` reality) becomes a real bug source. Single source of truth (registry) prevents drift by construction.

Alternative E — Non-configurable retry N (rejected)

Fix N = 3 as a constant.

Why rejected: leaves the "does retry meaningfully affect failure rate?" question unanswered. Making N configurable + sweeping in eval turns opinion into measurement. If the sweep shows N doesn't matter, we can hardcode later with data behind the decision.

Alternative F — Retry on MLX errors too (rejected)

Symmetric retry policy: both OpenAI and MLX retry on all failures.

Why rejected: MLX errors are deterministic. Metal OOM's freeing event isn't synchronized to retry windows. Retry masks systemic issues (memory pressure, config drift) that need to fail loud. Modeling infrastructure reality accurately (API = network-transient, MLX = local-deterministic) is more honest than forced symmetry.

Consequences

Positive

- Adding a new backend transport (Anthropic, other API providers) = one line in `BACKENDS` dict + one new class. Trivial extensibility.
- Adding a new fine-tuned MLX model = registry entry + weights drop. Zero code change.
- Retry logic centralized in base class = one place to reason about resilience, no drift.
- Failure surfaces are uniform (all `BackendError`); agent code doesn't need per-backend catches.
- Warmup gate on `/health` composes cleanly with load balancers and orchestration.
- Retry sensitivity swept in eval = data-driven answer to a research-fairness question.
- Testing surface is well-defined; FakeBackend enables full unit coverage of the agent without touching real APIs.
- Registry-driven metadata makes model provenance auditable.

Negative

- Two more env vars to configure (`BAYMAX_BACKEND`, `BAYMAX_MODEL_ID`) — small but real setup cost per deployment.
- Base-class `_call_llm` couples retry policy to the base class; changing retry semantics = base class change (though this is arguably positive — hard to accidentally diverge).
- Warmup adds 5-10s to server startup for MLX (unavoidable cost of local inference).
- Retry asymmetry (API retries, MLX doesn't) needs to be documented alongside every eval comparison so reviewers understand.
- Registry becomes a load-bearing dependency; corrupt or missing `models.json` at startup = process exit (though this is arguably positive — surfaces the problem loudly).
- Integration tests are asymmetric (OpenAI in CI on-demand; MLX Marv-local only). Acceptable given ownership boundaries but not scale-friendly.

Neutral

- Backend selection is per-process, not per-request. A single process serves one backend for its lifetime. If we ever need multi-backend routing (e.g., small-model triage → large-model fallback), that would require a new ADR.

- Streaming remains deferred to v2. When added, extends the interface via new methods rather than breaking existing ones.

Open questions / follow-ups

- **Backend capability declaration expansion**: if a fourth backend surfaces genuine capability divergence that can't be absorbed internally, may need to revisit the enforcement-vs-advertisement decision. Not v1 concern.
- **Backend selection at runtime**: currently per-process via env var. If Ronin's eval ever wants to compare A/B backends in a single run (rather than separate processes), the design would need a per-request `backend` param. Deferred until concrete requirement.
- **Retry policy per-error-class tuning**: currently uniform backoff for all retriable errors. If certain errors benefit from different backoff (e.g., 429 with `Retry-After` header), specialize later.
- **Warmup scenario evolution**: single hardcoded warmup JSON. If MLX or fine-tuned models need model-specific warmup prompts, extend to per-model warmup scenarios.
- **MLX conversion pipeline**: model registry entries with `mlx_compatible: false` need a conversion step before MLX can load them. Who owns conversion (Marv side, Ronin side, or automated) — resolve in ADR 0012 or a new ADR.
- **Anthropic backend**: deferred to v3+; when it lands, verify the enforcement principle still holds (interface uniformity absorbable in `AnthropicBackend` internals).

References

- ADR 0001 — Plan-and-Execute pattern this interface serves
- ADR 0002 — boundary contract for AgentResponse
- ADR 0005 — v1 predecessor (superseded for v2 by this ADR)
- ADR 0006 — telemetry format used by `_call_llm` emission
- ADR 0010 — telemetry infrastructure
- ADR 0012 — registry this backend reads from
- docs/BACKEND_FLOW.md — visual reference diagrams
- docs/METHODOLOGY.md — §7 uniformity, §8 registry integration
- RONIN_SYNC.txt B-tier — locked Phase 2 technical decisions
- Design walkthrough transcript — chat session 2026-07-27 through 2026-08-21 with Claude

---

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

ADR 0012 — Model Registry

Status: Proposed (Marv to confirm after review + open items resolved with Ronin) **Date**: 2026-08-21 **Author**: Marv **Related**: ADR 0004, ADR 0011, docs/METHODOLOGY.md §8, docs/BACKEND_FLOW.md §6, RONIN_SYNC.txt B-tier B2/B3/B4

Context

Phase 2 introduces multiple served model versions:

- Base OpenAI models (`gpt-4o-mini`, potentially `gpt-4o`) — no artifacts stored locally
- Base Qwen 2.5 1.5B checkpoint (per ADR 0004) — weights on disk
- Fine-tuned Qwen + LoRA adapters (`qwen-1.5b-baymax-v1`, `-v2`, ...) — Ronin produces per SFT run

The agent (via `MLXBackend` per ADR 0011) needs a stable answer to: *"given `model_id = X`, where do I find its weights and what facts do I need to know about it?"*

Naive alternative: hardcode paths and metadata in backend code. Problems:

- Every new fine-tune = backend code change (slows Ronin's training iteration)
- Backend's hardcoded facts and reality can drift silently
- No audit trail for "which model produced which eval result"
- No enforcement of "immutable = new id per retrain" (Ronin could accidentally overwrite v1)

A registry solves all four.

Constraints:

- Zero new infrastructure (matches ADR 0006 / 0007 posture)
- Ronin writes (training pipeline output); Marv reads (backend load)
- Immutability required for eval reproducibility (per RONIN_SYNC B3): a `model_id` labeled result must be reproducible forever
- Must integrate with backend construction per ADR 0011 §7
- Ronin's proposed manifest schema (2026-07-29 chat) serves as the starting point; three open issues need resolving

---

Decision

1. Registry format and location

Manifest-based (Option A). Single `models/models.json` file at repo root. JSON array of model entries; backend reads at construction time. Model artifacts live at `models/<model_id>/` alongside.

Directory layout:

```
models/
├── models.json                          # the manifest
├── qwen-1.5b-baymax-v1/
│   ├── adapter_model.safetensors        # LoRA adapter weights
│   ├── adapter_config.json              # PEFT config
│   └── training_config.json             # hyperparameters snapshot
├── qwen-1.5b-baymax-v2/
│   └── ...
└── gpt-4o-mini/                         # for API-backed models, entry exists but no local
weights
    └── (empty; entry in manifest only)
```

2. Manifest schema

Each entry is a JSON object with the following fields:

Required (backend refuses to load if missing):

Field	Type	Description
`id`	string	Unique model identifier. Convention: `<base>-<variant>-<version>` e.g. `qwen-1.5b-baymax-v1`. Dashes not underscores.
`base`	string	Base model identifier. For adapters, the base checkpoint they stack on (e.g., `qwen-1.5b`). For base models, same as `id`.
`adapter_path`	string | null	Filesystem path to adapter weights, relative to repo root. `null` for base models with no adapter.
`trained_at`	ISO 8601 datetime	When the training run completed. `null` for base models pulled from HF Hub.
`training_config_hash`	string (sha256)	Hash of the training config JSON — reproducibility invariant. `null` for base models.

Optional (limited to Ronin's original schema — dropped speculative fields that add no functional value for the current scope):

Field	Type	Description
`git_commit`	string (sha)	Commit SHA of the codebase state at training time. From Ronin's original schema; reproducibility trail beyond just training config.

Field	Type	Description
`checkpoint_format`	string	Format identifier — e.g., `"peft_lora"`, `"safetensors"`, `"mlx"`. From Ronin's original schema. Helps backend pick the right loader.
`mlx_compatible`	bool	Whether the checkpoint is directly loadable by MLX (post-conversion). Defaults to `false` if absent.
`mlx_artifact_path`	string | null	Path to MLX-converted artifact, if separate from `adapter_path`.
`training.cost_usd`	float	If cloud-rented GPU.

Explicitly NOT in the registry (deferred as feature creep for a codebase not under review):

- `notes`, `warnings`, `status`, `hyperparameters`, `data_split_seed`, `epochs_completed`, `training.gpu_hours`, `training.wall_clock_seconds`, `max_context`, `cost_model`, `tokenizer_path` — all droppable. Hyperparameters recoverable from `training_config_hash`; runtime facts (max_context, cost_model, tokenizer_path) come from HF base model config or hardcoded per backend class per ADR 0011 §7 (see amended §7 there). If any of these become genuinely needed later, un-defer at that point.

Naming consistency note: Ronin's actual training config file (per `configs/sft/qwen2_5_1_5b_lora_v1.json` on his `feat/phase1-scenario-loader` branch) uses `config_hash` for what this ADR calls `training_config_hash`. Same concept, different name. Sync with Ronin to align — either the registry adopts `config_hash` or his training config emits both keys.

Explicitly NOT in the registry:

- **Benchmark scores** — live in `results/*.json` per model, NOT here. Registry is identity; results are metrics. Duplication would cause staleness. To answer "which model has the best score?" join `models.json` with `results/` at read time (~20 lines of Python).
- **Training data pointers** — belong in the results file that references this model, not in the registry entry itself. The registry is about the model artifact; the training data was an input to it, referenced from the results output.

3. Immutability + monotonic versioning

- Model IDs are **immutable**. A given `id` NEVER has its metadata rewritten after registration.
- Retraining produces a NEW id: `qwen-1.5b-baymax-v1` → `qwen-1.5b-baymax-v2` → `qwen-1.5b-baymax-v3`.
- Old entries stay in the manifest indefinitely. Old results labeled with old ids stay reproducible forever.
- If a model version is deprecated (superseded, known-broken), mark it via `status: superseded` — do NOT delete the entry.

Naming convention: dashes (`-`), lowercase, monotonic integer version suffix. Adapter differentiator uses the project tag (`baymax`), not the training method (`lora`). Examples:

- ✅ `qwen-1.5b-baymax-v1`
- ✅ `qwen-1.5b-baymax-v2`
- ✅ `gpt-4o-mini`

- ❌ `qwen25_1_5b_lora_v1_001` (Ronin's initial proposal — see §Open items)
- ❌ `qwen-1.5b-baymax-v1a` (letter suffix — deviates from monotonic integer convention)

4. Registry integration with backend

Per ADR 0011 §7, backends read from the registry at construction time:

```
class MLXBackend(InferenceBackend):
    def __init__(self, model_id: str) -> None:
        entry = load_registry_entry(model_id)  # raises ModelNotFound / IncompleteEntry
        self.model_id = entry["id"]
        self.base = entry["base"]
        self.adapter_path = entry["adapter_path"]
        self.max_context = entry.get("max_context")
        self.cost_model = entry.get("cost_model", "amortized")
        # ... load weights from self.adapter_path, etc.
```

Registry lookup is one file read per process lifetime (backend caches results). Fails loud if model_id not found or required fields missing.

5. Write path (Ronin's training pipeline)

Ronin's SFT training pipeline is responsible for:

1. Writing adapter weights to `models/<new_id>/`
2. Emitting `training_config.json` alongside
3. Computing `training_config_hash = sha256(training_config.json bytes)`
4. Appending a new entry to `models/models.json` with all required fields
5. Committing both the weights + manifest patch in the same PR

Per METHODOLOGY §3, Ronin owns registry writes. Marv owns registry reads (via backend). Manifest updates are a coordination point; Ronin's PR titles should tag `[registry-update]` for visibility.

Discipline check per RONIN_SYNC A1 pre-meeting note: since Ronin uses AI code assist, prompt his AI to include manifest patching as part of the training pipeline template. Automated update via the SFT pipeline is more reliable than a manual "remember to update the JSON" step.

Drift mitigation (future): a CI check that fails if `set(dirs in models/)` ≠ `set(ids in models.json)`. Not v1 required; add later if drift becomes a real problem.

6. Loading and validation

Registry loader lives in `src/baymax/core/registry.py`:

```
def load_registry_entry(model_id: str, path: Path = Path("models/models.json")) -> dict:
    """Loads and validates the entry for a given model_id."""
    manifest = json.loads(path.read_text())
    for entry in manifest["models"]:
        if entry["id"] == model_id:
            _validate_required_fields(entry)
```

```
            return entry
    raise ModelNotFound(model_id)
```

Validation checks all required fields per §2 are present. Raises `IncompleteEntry` with the missing field name on failure.

JSON Schema validation: full schema at `docs/schemas/model_registry.schema.json` (to write). CI validates `models.json` against schema on every PR.

Reasoning

[REASONING PROMPTS — expand each bullet into 1-2 sentences in your own voice. Documentation depth: brief, honest, readable in 6 months.]

- **WHY manifest file (Option A) rather than directory scan (Option B) or hybrid (Option C)**: metadata queryability. A manifest can be inspected in one file open; a directory scan requires opening N `metadata.json` files. For a small registry (~10 models over the project's life), manifest cost is negligible and manifest benefits (single source of truth, easy diff review, one place to inspect) dominate.
- **WHY the manifest drift risk (Ronin adds a folder but forgets to update the manifest) is acceptable**: mitigation via Ronin's AI-assisted training pipeline (per RONIN_SYNC A1). Automated update is more reliable than manual discipline. CI check as backstop if drift becomes a real problem — cheap safety net addable later.
- **WHY immutable + monotonic increment (not overwrite, not letter suffixes)**: eval reproducibility depends on it. If `v1` results in the results file mean anything, `v1` weights must be forever recoverable. Overwriting v1 breaks every prior labeled result. Letter suffixes (`-v1a`, `-v1b`) invite ambiguity about ordering. Monotonic integers = unambiguous, machine-comparable, minimally cognitive-overhead.
- **WHY only 5 required fields (id, base, adapter_path, trained_at, training_config_hash)**: minimum needed to LOAD the model + trace it back to reproducibility. Everything else is optional; require the minimum, encourage the rest. `training_config_hash` is required specifically because config file paths can drift silently — the hash is the reproducibility invariant.
- **WHY benchmark_score is EXPLICITLY NOT in the registry**: two sources of truth = drift. If the eval is re-run (scorer bug fixed, expanded scenarios, tier stratification), the benchmark score in the registry entry becomes wrong. Results live in `results/*.json`; registry entries reference them via path. Joining registry + results at read time is trivial (~20 lines of Python); duplicating benchmark scores in the registry means one becomes stale and reviewers don't know which to trust.
- **WHY separate manifest + `models/<id>/` folder structure** (not everything in the manifest, not everything in the folder): manifest is the fast queryable index (which models exist, what are their facts). Folder is the storage (weights are gigabytes, not manifest-appropriate). Standard pattern in ML tooling (HuggingFace does essentially this).
- **WHY Ronin writes, Marv reads**: matches author-split discipline (METHODOLOGY §3). Ronin's training pipeline produces the artifact; the artifact is the write. Marv's backend consumes it; consumption is the read. Cross-role writes would create coordination overhead and blur ownership.
- **WHY JSON Schema validation in CI**: catches malformed entries before they reach the backend at startup. Backend startup validation is the safety net; CI validation is the fast-fail. Same principle as any typed contract — validate at the boundary.

- **WHY `status` field is optional** (defaults to `completed`): overwhelming majority of entries will be completed models. Making status required forces every entry to include a field that's usually the same value. Optional-with-default = pragmatic.
- **WHY defer HF Hub as the registry backend to v2**: v1 is local-machine only (Ronin's training cloud + Marv's Mac). No multi-user, no cross-machine sharing, no discoverability needs. Filesystem is enough. When v2 needs artifact sharing (external collaborators, publishing to HuggingFace), the manifest schema migrates cleanly to HF Hub's model card format (which is a superset).
- **WHY register API-backed models (gpt-4o-mini) in the registry too, despite no local artifacts**: uniformity. Backend construction always goes through `load_registry_entry(model_id)` regardless of backend type. If OpenAI models aren't registered, `OpenAIBackend` needs a special-case path — violates uniformity, invites drift. Register them with `adapter_path: null` and empty folder; backend code doesn't branch.

Alternatives considered

Alternative A — Directory-scan discovery (no manifest)

Registry `list_models()` = `[p.name for p in Path("models/").iterdir() if p.is_dir()]`. Metadata lives in per-folder `metadata.json`.

Why rejected: no global queryability without opening N files. Adding a new model doesn't require any manifest update, which is a positive, BUT the query cost + lack of one-file-diff review is a negative. For a small registry (~10 models over project life), manifest cost is negligible.

Alternative B — Hybrid (directory scan for discovery + per-folder metadata.json)

HuggingFace-style. Directory is the truth for existence; per-folder metadata.json for facts.

Why rejected: two-layer indirection (open manifest → open per-folder file → read fields). More complex than a single manifest for a project this small. Would be the right answer at HF Hub scale; not at BAYMAX scale.

Alternative C — Database-backed registry (SQLite table)

Store registry entries as rows in a SQLite table (potentially the same DB as the state store per ADR 0007).

Why rejected: overkill for a read-mostly, small (~10-entry) dataset. Adds a schema migration story, ORM decision, backup complexity. JSON file is git-diffable, human-readable, trivially auditable. If we ever grow past ~1000 model versions, revisit.

Alternative D — Backend-internal metadata (rejected in ADR 0011 too)

Each backend hardcodes per-model knowledge; no registry file.

Why rejected: adding a new fine-tune = backend code change. Slows Ronin's iteration. Also violates single-source-of-truth (backend's hardcoded facts can drift from reality).

Alternative E — Include benchmark_score in registry

Add `benchmark_score` as a required field on each entry.

Why rejected: creates staleness risk. If eval is re-run, the registry entry's score is wrong until manually updated. Two sources of truth = one becomes stale = reviewers can't trust either. Registry is identity; results are metrics. Keep them separate; join at read time.

Alternative F — Letter-suffixed versions (v1a, v1b)

Retraining produces `qwen-1.5b-baymax-v1a`, then `v1b`, staying under the same major version.

Why rejected: ambiguous ordering (`v1c` vs `v2`?). Monotonic integers are unambiguous and machine-comparable. Also, if v1a "counts as" v1 for eval purposes, immutability is unclear — either v1 = v1a (which weights?) or v1 ≠ v1a (in which case why not just use v2?).

Consequences

Positive

- Adding a new fine-tune = registry entry + weights drop. Zero backend code change.
- Registry is auditable — one file, git-tracked, diffable per commit.
- Old model versions stay reproducible forever (immutability).
- Marv's backend code doesn't care about model-specific facts; just reads them.
- Ronin's training pipeline has a clear write target; artifact + manifest patch as one PR.
- JSON Schema validation in CI catches malformed entries before backend startup.
- Registry format easily migrates to HF Hub in v2 (superset of what HF requires).

Negative

- Ronin must remember to update the manifest on every training run (mitigated by AI-assisted pipeline automation).
- Manifest becomes a load-bearing file; corruption or malformed JSON at startup = process exit (though this is arguably positive — fails loud).
- Coupling between backend (`MLXBackend.__init__`) and registry (`load_registry_entry`) means registry changes can break backend startup. Contract via the schema mitigates.
- Over time, `models/models.json` grows unboundedly with deprecated entries. If it becomes unwieldy (say 100+ entries), add a "current" flag or move deprecated entries to `models/models.deprecated.json`.

Neutral

- API-backed models (OpenAI) register with `adapter_path: null` — same code path as adapter-backed models. Uniformity has a small cost (empty folders) for a large benefit (no backend branching).
- Registry is single-machine v1. Multi-machine sharing = migrate to HF Hub or similar in v2.

Open items (need resolving with Ronin before locking)

Per critique of Ronin's proposed manifest schema (chat, 2026-07-29):

1. Remove `evaluation` block from Ronin's proposed manifest

Ronin's proposal included:

```
"evaluation": {
  "task_success_rate": 0.0,
  "average_tool_call_accuracy": 0.0,
  ...
}
```

Per this ADR §2 and METHODOLOGY §8: benchmark scores live in `results/` only, NEVER in the registry. Sync with Ronin — remove this block.

2. Add `training_config_hash` field (required)

Ronin's proposal has `training.config_file` (path reference) but no hash. Path can drift silently; hash is the reproducibility invariant. Sync — add.

3. Clarify `run_id` vs `model_id` distinction

Ronin's `run_id: "qwen25_1_5b_lora_v1_001"` mixes model version + run number. Two possible interpretations:

- (a) `run_id` IS the model identifier; every training run produces a distinct model
- (b) `run_id` is a training-execution log identifier; `model_id` is separate; multiple runs can produce the same model version

Under interpretation (a), immutability holds but naming is weird (`_001` suffix on a "version 1" model).

Under interpretation (b), we need BOTH `run_id` AND `model_id` as separate fields.

Sync with Ronin to lock which — and adjust the naming convention (§3) to match.

4. MLX conversion plan

Ronin's proposal has `"mlx_compatible": false, "mlx_artifact_path": null` — honest about the PEFT adapter not being directly MLX-loadable. But:

- When does conversion happen? (Post-training in Ronin's pipeline? On-demand by Marv's backend? Manual?)
- Where does the MLX artifact land? (Overwrite the folder? Sibling `models/<id>_mlx/`?)
- Does conversion update the existing registry entry (violates immutability) or create a NEW entry with `id: <original>-mlx`?

Suggested resolution: new registry entry per format. `qwen-1.5b-baymax-v1` (PEFT) and `qwen-1.5b-baymax-v1-mlx` (converted). Preserves immutability. Backend selects the appropriate format via `model_id`.

Sync with Ronin to confirm.

Open questions / follow-ups

- **CI schema validation**: write `docs/schemas/model_registry.schema.json` + add GitHub Actions check.

- **CI drift check**: add script that fails if `set(models/*/)` ≠ `set(models.json ids)`. Optional; add when drift becomes a real concern.
- **Model retention policy**: no automatic deprecation of old entries in v1. When we cross ~50 entries, consider adding.
- **Registry ranking script**: `scripts/rank_models.py` that joins `models/models.json` with `results/*.json` to answer "which model performs best on which tier?" Small script; write when Ronin has enough model versions to make ranking meaningful.
- **HF Hub migration path**: v2 target. Manifest schema is designed to be a superset of HF model-card frontmatter for clean migration.

References

- ADR 0004 — Qwen 2.5 1.5B base choice
- ADR 0011 — backend that consumes this registry (§7)
- docs/METHODOLOGY.md §8 — required fields policy, benchmark_score in results/ only rule
- docs/BACKEND_FLOW.md §6 — registry lookup diagram
- RONIN_SYNC.txt B2, B3, B4 — Phase 2 locked decisions on registry
- HuggingFace Hub model cards: https://huggingface.co/docs/hub/model-cards — future-migration target format
- Ronin's proposed manifest schema (chat, 2026-07-29) — starting point for §2

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Phase 2 Results

This catalog covers all 59 existing result/analysis artifacts. Evaluations use Qwen/Qwen2.5-1.5B-Instruct, either the base model or a LoRA adapter, with fake Calendar, Notion, Gmail, and clipboard tools.

Reading the Names

- `v1-50`: full 50-scenario v1 evaluation.
- `combined-150`: combined v1/v2 evaluation with 150 scenarios.
- `template-shift-30`: separate 30-scenario suite testing changed wording and structure.
- `subset-7`: seven refusal scenarios, not a full benchmark.
- `train1k` / `train1500`: historical training-cap labels, not evaluation counts.
- `epochs2`: the two-epoch experiment.
- `policy-prompt-v1/v2`: inference-time policy-prompt experiments.
- `training-prompt-a314`: restored training prompt, SHA256 `a31446de6c8b67f1fc8ba8fc09ed8e2d7425ba832334dc8fd6fe2d55ef978468`.
- `decision-metrics`: adds ACT/CLARIFY/REFUSE decision accuracy while retaining keyword metrics.
- `strict-parser-rescore`: saved generations reparsed and fake tools replayed; no new inference, and latency/cost are inherited from the source run.
- `early-parser-fix`: an earlier parser/adaptation experiment, distinct from strict rescores.

Descriptions are reconstructed from existing filenames, project notes, and saved metadata. Scenario counts are checked against the JSON files. Older results generally omit the exact adapter path, training recipe, prompt hash, and scorer version. Their experiment labels are not a substitute for those missing records. `historical-prompt` means the exact runtime prompt is not established by the saved result.

Comparisons and Provenance

Use the same suite, prompt, parser, and scorer when assessing an experiment. Scenario expectations and scoring changed during development; a shared scenario count alone does not establish an identical benchmark version. Clarification/refusal wording scores are separate from decision correctness.

The format-aligned 50-scenario artifact was our historical reference checkpoint. The latest experiment is the generator12/content-group-splits 1k run listed below. The strict-rescore artifacts are the corrected historical parser results used in our latest comparison. Older adapters predate the fixes to LoRA initialization seeding and training-data split leakage, so comparisons against them are historical evidence rather than an isolated test of the training mix. The repeatedly used suites are development evaluations, not untouched final tests.

Every JSON/text artifact was renamed without changing its bytes. See FILE_RENAMES.json for all original/current repository-relative paths and SHA256 hashes. Embedded historical paths (including strict-rescore `source_path`) remain as originally recorded; resolve them through that map.

Latest Comparison

Run from the repository root:

```
.venv\Scripts\python scripts\compare_eval_results.py `
  acr-1k=results\phase2\parser-integrity-v1\combined-150\act-clarify-refuse-train1k-combined-150-strict-parser-rescore.json `
  schema-diverse=results\phase2\parser-integrity-v1\combined-150\schema-diverse-train1k-combined-150-strict-parser-rescore.json `
  act-balanced=results\phase2\parser-integrity-v1\combined-150\act-balanced-train1k-combined-150-strict-parser-rescore.json `
  act-midpoint=results\phase2\parser-integrity-v1\combined-150\act-midpoint150-train1k-combined-150-strict-parser-rescore.json `
  new-70-30=results\phase2\qwen-lora\baymax-mix\content-group-splits-generator12-train1k-70act-30boundary-combined-150.json
```

File Catalog

Paths below are relative to this directory.

Base Model

File	Scenarios	What Was Tested
qwen-base-combined-150-historical.json	150	Unfine-tuned Qwen on the combined 150-scenario suite. Historical runtime prompt is not recorded in the result.
qwen-base-combined-150-policy-prompt-v1.json	150	Base-model evaluation with the expanded decision/confirmation policy prompt v1.
qwen-base-combined-150-policy-prompt-v2.json	150	Base-model evaluation with the longer ACT/CLARIFY/REFUSE policy prompt v2.
qwen-base-combined-150-training-prompt-a314.json	150	Base-model comparison using the restored a314 training system prompt.

LoRA: Synthetic BAYMAX Control

File	Scenarios	What Was Tested
synthetic-control-original-v1-50.json	50	Early LoRA trained on BAYMAX synthetic control data, evaluated on all 50 v1 scenarios.
synthetic-control-original-v1-50-early-parser-fix.json	50	Original control checkpoint evaluated with the earlier output-adaptation/parser fixes; distinct from the later strict parser rescores.
synthetic-control-contrast-pairs-v1-50.json	50	Control-data checkpoint with contrast-pair examples, evaluated on the 50-scenario suite.

File	Scenarios	What Was Tested
synthetic-control-policy-prompt-refusal-subset-7.json	7	Policy-prompt experiment on the seven refusal scenarios, using the early control checkpoint.
synthetic-control-policy-prompt-refusal-subset-7-early-parser-fix.json	7	Seven-scenario refusal subset with the policy prompt and earlier parser/adaptation changes.
synthetic-control-targeted-refusals-v1-50.json	50	Control-data checkpoint incorporating targeted refusal examples, evaluated on all 50 v1 scenarios.
synthetic-control-targeted-refusals-prompt-only-subset-7.json	7	Seven-scenario refusal-focused prompt experiment. The legacy label does not establish whether a separate checkpoint was trained.

LoRA: xLAM + BAYMAX Mix

File	Scenarios	What Was Tested
original-xlam-control-mix-v1-50.json	50	Original mixture of external xLAM and synthetic BAYMAX data; 50-scenario evaluation.
original-xlam-control-mix-v1-50-early-parser-fix.json	50	Original mixed-data checkpoint with earlier parser/adaptation fixes; not a strict-complete-response rescore.
synthetic-v7-smoke-v1-50.json	50	Legacy v7 synthetic-data smoke experiment evaluated on all 50 scenarios. Exact training exposure is not recorded here.
synthetic-v7-weighted-v1-50.json	50	Legacy v7 experiment with additional boundary-example weighting; exact repeat counts are not stored in the result.
synthetic-v8-meta-tools-weighted-v1-50.json	50	Weighted v8 experiment representing clarification/refusal through internal meta-tool calls.
format-aligned-reference-v1-50.json	50	Historical aligned reference checkpoint after training/evaluation input format alignment; evaluated on 50 scenarios.
clarification-extra6-refusal-extra12-v1-50.json	50	Mixed-data weighting experiment labelled six extra clarification repeats and twelve extra refusal repeats.
boundary-weighting-legacy13-v1-50.json	50	Legacy boundary-weighting experiment labelled boundary13. The exact weighting recipe is not established by its saved metadata.
unique-boundaries-train1k-v1-50.json	50	Unique-first BAYMAX boundary sampling, training cap labelled 1000 rows; 50-scenario evaluation.

File	Scenarios	What Was Tested
unique-boundaries-train1500-v1-50.json	50	Larger training cap labelled 1500 rows with the fixed unique boundary pool; 50-scenario evaluation.
unique-boundaries-train1k-combined-150-historical-prompt.json	150	The 1k boundary checkpoint evaluated on the combined 150-scenario suite before the explicitly restored-prompt run.
unique-boundaries-train1500-combined-150-historical-prompt.json	150	The 1500-row boundary checkpoint on the combined suite before the explicitly restored-prompt run.
unique-boundaries-train1k-combined-150-training-prompt-a314.json	150	The 1k boundary checkpoint reevaluated with the restored a314 training prompt.
unique-boundaries-train1500-combined-150-training-prompt-a314.json	150	The 1500-row boundary checkpoint reevaluated with the restored a314 training prompt.
format-aligned-combined-150-historical-prompt.json	150	Aligned checkpoint on the original combined suite; historical runtime prompt is not recorded in the artifact.
format-aligned-combined-150-policy-prompt-v1.json	150	Existing aligned adapter evaluated using policy prompt v1, which differed from its training prompt; no retraining for this prompt ablation.
format-aligned-combined-150-policy-prompt-v2.json	150	Existing aligned adapter with longer policy prompt v2; an inference-prompt ablation, not a newly trained adapter.
format-aligned-combined-150-training-prompt-a314.json	150	Aligned checkpoint reevaluated with its restored a314 training prompt.
act-clarify-refuse-train1k-combined-150.json	150	1000-row mixed-data training experiment incorporating 100 authored ACT/CLARIFY/REFUSE triplets, followed by 150-scenario evaluation.
act-clarify-refuse-train1k-epochs2-combined-150.json	150	Two-epoch counterpart to the 1k ACT/CLARIFY/REFUSE experiment.
schema-diverse-train1k-combined-150.json	150	1k training experiment varying tool argument schemas and partial-update calls; combined-suite evaluation.
act-balanced-train1k-combined-150-legacy-metrics.json	150	ACT-balanced mixed-data experiment before explicit decision metrics were added.
act-balanced-train1k-combined-150-decision-metrics.json	150	ACT-balanced checkpoint reevaluated with explicit ACT/CLARIFY/REFUSE decision metrics; keyword metrics retained.
act-midpoint150-train1k-combined-150-decision-metrics.json	150	Legacy midpoint150 ACT-mix experiment with a 1k training cap and decision metrics. The label alone does not describe the total ACT exposure.

File	Scenarios	What Was Tested
content-group-splits-generator12-train1k-70act-30boundary-combined-150.json	150	Latest run: generator v12, corrected xLAM/BAYMAX exact-content splitting, 1k cap with 150 unique clarification + 150 unique refusal + 700 ACT rows before filtering; strict parser and decision metrics.

Template-Shift Evaluations

File	Scenarios	What Was Tested
qwen-base-template-shift-30.json	30	Base Qwen on the independently authored 30-scenario wording/template-shift suite.
format-aligned-template-shift-30.json	30	Aligned reference adapter evaluated on the 30-scenario template-shift suite.
act-clarify-refuse-train1k-template-shift-30.json	30	1k ACT/CLARIFY/REFUSE checkpoint tested on different wording and request structure.
act-clarify-refuse-train1k-epochs2-template-shift-30.json	30	Two-epoch ACT/CLARIFY/REFUSE checkpoint on the template-shift suite.
schema-diverse-train1k-template-shift-30.json	30	Schema-diverse 1k adapter on the 30-scenario transfer suite.
act-balanced-train1k-template-shift-30-legacy-metrics.json	30	ACT-balanced adapter on template-shift scenarios before explicit decision metrics.
act-balanced-train1k-template-shift-30-decision-metrics.json	30	ACT-balanced adapter on template-shift scenarios with explicit decision metrics.
act-midpoint150-train1k-template-shift-30-decision-metrics.json	30	Midpoint150 ACT-mix adapter on the template-shift suite with decision metrics.

Strict Parser Rescores: Combined Suite

File	Scenarios	What Was Tested
act-clarify-refuse-train1k-combined-150-strict-parser-rescore.json	150	Saved 150-scenario ACT/CLARIFY/REFUSE generations reparsed with strict-complete-response-v1; fake tools replayed, no fresh inference.
schema-diverse-train1k-combined-150-strict-parser-rescore.json	150	Saved 150-scenario schema-diverse generations reparsed with the strict parser; no retraining or fresh inference.
act-balanced-train1k-combined-150-strict-parser-rescore.json	150	Saved 150-scenario ACT-balanced decision-metrics run reparsed with the strict parser.
act-midpoint150-train1k-combined-150-strict-parser-rescore.json	150	Saved 150-scenario midpoint150 decision-metrics run reparsed with the strict parser.

Strict Parser Rescores: Template Shift

File	Scenarios	What Was Tested
act-clarify-refuse-train1k-template-shift-30-strict-parser-rescore.json	30	Saved 30-scenario ACT/CLARIFY/REFUSE generations reparsed with strict-complete-response-v1; fake tools replayed, no fresh inference.
schema-diverse-train1k-template-shift-30-strict-parser-rescore.json	30	Saved 30-scenario schema-diverse generations reparsed with the strict parser; no retraining or fresh inference.
act-balanced-train1k-template-shift-30-strict-parser-rescore.json	30	Saved 30-scenario ACT-balanced decision-metrics run reparsed with the strict parser.
act-midpoint150-train1k-template-shift-30-strict-parser-rescore.json	30	Saved 30-scenario midpoint150 decision-metrics run reparsed with the strict parser.

Data Audits and Failure Reports

File	Scenarios	What Was Tested
original-mix-token-audit-5000-and-boundary-failures.json	Analysis	5000-row token-distribution audit of the early mixture, plus grouped evaluation failures and raw boundary failures.
clarification-extra6-refusal-extra12-token-audit-5000-snapshot1.json	Analysis	5000-row token audit and failure analysis for the 6/12 weighting experiment; first stored snapshot.
clarification-extra6-refusal-extra12-token-audit-5000-snapshot2.json	Analysis	Same token counts as snapshot1, but expanded failure comparisons (16 behavior/run entries and 99 boundary failures).
boundary-weighting-legacy13-token-audit-5000-snapshot1.json	Analysis	5000-row token audit with 75 recorded boundary failure generations.
boundary-weighting-legacy13-token-audit-5000-snapshot2.json	Analysis	Same token counts as snapshot1, with a different failure snapshot containing 74 boundary failure generations.
qwen-base-policy-prompt-v1-failure-summary.txt	Analysis	Text summary of base-model failures from the decision-policy prompt v1 evaluation.
format-aligned-policy-prompt-v1-failure-summary.txt	Analysis	Text summary of aligned-adapter failures from the decision-policy prompt v1 evaluation.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Anchored Two- vs Four-Epoch Failure Audit

Captured on 2026-10-07 from four saved benchmark files. No new inference, training, rescoring, parser changes or fixture changes were performed.

Selected Reference

Use the **anchored two-epoch** adapter as the development reference. The reference record pins SHA256 identities for the config, adapter, checkpoint, dataset metadata, capped raw inputs and both reference result files. It detects drift; it does not permission-lock files or copy model weights. Do not overwrite these artifacts.

The original run did not record an exact base-model revision. Tokenized artifacts were not rebuilt: usable-row counts come from the owner's training reports. Generation flags are recorded from owner commands, not independently recoverable from the saved benchmark metadata. These limitations remain explicit.

Both runs started from base Qwen with fresh LoRA adapters. Data revision, prompt, seed, caps and LoRA settings were unchanged. Longer epochs also extend the cosine schedule and warmup duration; this is not an isolated extra pass at identical step-by-step learning rates.

Measured Outcomes

Suite	Epochs	Task Success	Routing Macro-F1	Invalid Outputs
Combined 150	2	44/150	0.514	2/150
Combined 150	4	37/150	0.372	13/150
Template Shift 30	2	5/30	0.345	2/30
Template Shift 30	4	4/30	0.348	6/30

The machine-readable case audit includes all 270 failed records across 360 scenario runs, with source-file hashes, zero-based result indices, scenario IDs and raw-output hashes. Original result files remain the source of truth.

Primary Failure Categories

Each failed record has one primary category. Invalid output takes precedence, followed by unknown decision, wrong decision, boundary wording-score mismatch, arguments, tools, execution and other failures. A primary category does not mean no other defect exists in that output.

Primary Category	Combined 2	Combined 4	Shift 2	Shift 4
Wrong decision	64	75	18	13

Primary Category	Combined 2	Combined 4	Shift 2	Shift 4
Correct decision, wording-score mismatch	27	17	4	6
Invalid output	2	13	2	6
Argument failure	10	5	1	1
Tool selection or sequence failure	3	3	0	0
Total failed	106	113	25	26

Do not interpret fewer primary argument failures as independently improved arguments: records can move into earlier categories.

Decision Errors

On Combined 150, ACT cases incorrectly routed to CLARIFY increased from **21 to 36**. Correct CLARIFY decisions increased from **11 to 19**, but correct REFUSE decisions fell from **16 to 4**. Productive calls on CLARIFY/REFUSE cases remained **31** in each run.

On Template Shift 30, correct CLARIFY decisions rose from **2 to 8**, but correct REFUSE decisions fell from **5 to 1** and correct ACT decisions from **3 to 2**. Four epochs reduced wrong valid decisions there while increasing rejected outputs. This is a routing tradeoff, not robust policy improvement.

Diagnostic flags in the JSON report are non-exclusive. Mixed calls, execution failures and routing errors can overlap; do not sum those flags as a failure total.

Output-Contract Errors

All 23 rejected outputs have identified contract causes. Causes can overlap:

- Meta-tool values outside `arguments`, such as a top-level `reason`.
- Missing or incorrect meta-tool argument keys: `question` vs `reason`.
- Productive calls mixed with a meta-tool, repeated meta-tools, or mixed decisions.
- Calls containing arguments but no tool name.
- An undotted productive tool name, rejected by `AgentToolCall` validation.
- Malformed JSON or trailing content.

The existing strict parser was used. Inspecting an intact JSON prefix only helped label rejected content; it did not repair, accept or rescore any output.

The older `classify_boundary_generation` helper inspects only the first call name and can mislabel malformed meta-tools as wording failures. It was not used here and remains unchanged; historical helper reports should not be treated as strict output-contract audits.

Wording Is Not One Problem

"Correct decision, wording-score mismatch" means the declared decision matches the expected class but its saved keyword metric is below 1. It is not an automated semantic judgment. Inspection of the 27 two-epoch Combined cases found both:

- Surface mismatch: "What time should I schedule the chemistry review?" asks for time but omits the expected keyword "tonight".
- Wrong missing detail: a request already specifies 2pm but lacks a date; the question asks for time instead of date.
- Unsupported reason: an unavailable routine email capability is refused as "bulk unsolicited email", although the request was not bulk outreach.

Do not count every wording-score mismatch as a scorer false negative. Conversely, keyword failure alone does not prove the message is useless. No semantic judge or manual score override was introduced.

Conclusion and Limits

Keep two epochs as the reference. Four epochs lowered validation loss but hurt balanced routing and output validity; this does not establish memorization. Do not infer that further epochs will fix the decision boundary.

The next proposed experiment remains independently authored matched contrasts at two epochs, keeping mixture quantities, xLAM selection, prompt and other hyperparameters fixed. It has not been run. Evaluation cases in this audit must not be turned into training examples or fixture-specific fixes.

These are single-seed development benchmarks, repeatedly inspected during iteration, not untouched final tests. An earlier bounded overlap check also found near-duplicate v1 wording in the training cap. No examples were removed or rewritten from either side during this audit.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Matched-Contrast Two-Epoch Refusal and Format Audit

Captured on 2026-10-07 from saved results. No inference, rescoring, parser changes, training-data changes or evaluation-fixture reads were performed.

The case audit records source result hashes and result indices for all 29 expected REFUSE cases plus all 8 invalid matched outputs (31 distinct cases). All 300 saved generations from the two runs were reparsed to confirm no parser drift.

Measured Changes

Measure	Anchored Reference	Matched Contrasts
Task successes	44/150	39/150
Routing macro-F1	0.514	0.383
Invalid outputs	2/150	8/150
Correct REFUSE routing	16/29	5/29

The matched run lost 7 previously successful tasks and gained 2. It gained no new correct REFUSE routing decisions.

Of the 16 reference REFUSE decisions, 5 remained REFUSE. The other 11 became: 6 productive ACT decisions, 2 valid CLARIFY decisions and 3 rejected outputs. Those last 3 must not be treated as valid model decisions.

What the Raw Outputs Show

This is not solely a keyword-scoring issue:

- A request to search emails produces the unavailable `gmail.search_messages`.
- A request to email with only a calendar capability produces `calendar.create_event` with email fields (`recipient`, `body`, `subject`).
- A request to recall an email produces an empty `gmail.send_email` call.
- Two unsupported requests instead produce valid clarification questions.

These observations indicate incorrect capability selection and routing. They do not establish why training caused the change. Parser-valid productive calls can still have invalid arguments or fail adapter execution; the invalid-output metric is not a count of every possible tool-contract error.

Even the reference's correct REFUSE labels often have unsupported explanations, such as invented authentication, ownership or terms-of-service restrictions. Correct routing is not equivalent to a truthful reason or full task success.

Eight Invalid Matched Outputs

Primary categories are exclusive and total 8. Diagnostic flags can overlap.

Primary Cause	Count
`request_clarification` uses `reason` instead of `question`	2
Missing clarification `question`	1
Missing refusal `reason` inside `arguments`	2
Productive calls omit tool names	2
Invalid JSON containing an ellipsis	1

Six invalid records are REFUSE-expected and two are ACT-expected; no CLARIFY-expected record was rejected. Both ACT records previously succeeded and now omit tool names.

One missing-reason case puts `reason` outside `arguments`; another supplies empty arguments. One missing-question case also repeats the JSON `name` key. The existing JSON parser uses the last duplicate key; this record is rejected for its meta-tool arguments, not because duplicate keys are explicitly prohibited. That remains a parser limitation, unchanged by this audit.

There are no mixed-meta/productive or repeated-meta-call failures in these eight records. They are not the same format-error distribution as the four-epoch run.

All five parser-valid matched REFUSE decisions still have saved refusal keyword accuracy below 1. This is a wording-score mismatch category, not an automatic semantic judgment or authorization to manually improve scores.

Training Composition Changed

Audited the first 1000 raw training rows against their source provenance. Both runs contain 100 unique refusal rows, but their origins differ:

Refusal Source Origin	Reference	Matched
Triplets	33	73
Generated standalone examples	22	0
Capability-boundary pairs	23	0
Other contrasts or seed examples	22	27

Source-ID families decreased from 45 to 20. These are authored lineages, not an independently labeled count of semantic behavior types. In particular, a matched triplet can keep a historical ID such as `refuse_gmail_search` while changing its request to a draft request with no draft tool. Do not infer semantic search coverage from the ID alone.

The 120 complete groups comprise 73 triplets and 47 pairs. Group-preserving sampling prioritizes complete triplets, so sharper contrasts also changed the within-class training composition. This experiment controls broad class counts, xLAM rows and training settings, not identical BAYMAX coverage or target wording.

Interpretation and Next Step

Measured conclusion: this matched variant is worse on the development benchmark. Hypothesis: stronger concentration in selected triplet patterns reduced coverage of other refusal situations; this is plausible, not demonstrated causation.

Keep the original anchored two-epoch reference. Before another data experiment, audit training-only semantic subtype/domain coverage and preserve that breadth when selecting complete contrasts. Any adjustment must be designed from training data, adapter contracts and the general policy, not these individual eval cases.

Keep format validity separate from routing quality. Do not repair outputs, relax the parser, change benchmark targets, add epochs or alter class quantities on the strength of this audit alone. No new experiment has been prepared or run.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Parser Integrity Re-scoring

Files now include the experiment, suite size, and `strict-parser-rescore` in their names. See the complete Phase 2 catalog and rename map for old/current paths. Embedded source paths remain historical; resolve them through the map when locating source files.

Saved generations reparsed on 2026-09-28 with `strict-complete-response-v1`. No model inference or training was performed. Fake tool execution was replayed. Each JSON records its source SHA256, source run ID, scenario SHA256, and inherited latency/cost. Historical source files were not overwritten.

Checkpoint	Previous 150 passes	Rescored 150 passes	Invalid outputs / 150	Shift passes / 30	Invalid shift outputs / 30
ACR-1k	43	42	17	6	3
Schema-diverse	42	40	20	5	3
ACT-balanced	47	46	18	6	2
ACT-midpoint	46	46	16	6	3

These results correct parsing/scoring only. They do not repair the old checkpoints' unseeded LoRA initialization or leakage between synthetic SFT partitions. The separate benchmark fixtures have not been shown to overlap training data. Repeated comparisons on these suites are development evidence, not untouched final-test estimates.

Corrected Data

Generator version 11 joins duplicate-content groups and contrast groups before splitting. `data/sft/v1/baymax-control-disjoint` has 702 train, 200 validation and 98 test rows, with no complete SFT example shared between partitions.

Revision: `1ff9e3934ff3987c274aa4a903f7078caf96f89bda335777020f87b5ac6cf9ca`.

The training pool has 222 unique ACT, 122 unique CLARIFY and 132 unique REFUSE examples. The previously planned 150/150 boundary quotas cannot be filled without new unique examples or repeats. The earlier 70/30 mixed artifact still uses the old split and must not be used as the corrected next run.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Template-Shift Diagnostic v1

This is a locked diagnostic suite for measuring whether ACT, CLARIFY, and REFUSE behavior transfers beyond the wording used in BAYMAX training data.

The suite contains 30 hand-authored scenarios:

- 10 ACT scenarios
- 10 CLARIFY scenarios
- 10 REFUSE scenarios

The scenarios use indirect phrasing, different entities and domains, and hard counterexamples where familiar words do not imply the familiar decision. For example, a harmless clipboard phrase containing "password" must still ACT.

Integrity Rules

- Never include these scenarios or their expected outputs in training data.
- Never generate training examples by reading or transforming these files.
- Do not change an expected behavior because a model failed it.
- Preserve raw result artifacts from every run.
- Use the same prompt and generation settings when comparing checkpoints.
- After results influence training decisions, describe this as a development diagnostic rather than an untouched final test.

The established 150-scenario benchmark remains `scenarios/v1/` plus `scenarios/v2/`. This suite is reported separately.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

v1 Scenario Coverage

This checklist tracks scenario coverage gaps for the v1 benchmark. Mark an item complete only when at least one committed scenario validates for that behavior.

Calendar

- [x] Calendar contextual scenario
  - `calendar_contextual_reschedule_001`, `calendar_update_duration_001`
- [x] Calendar clarification scenario
  - `calendar_clarify_time_001`, `calendar_ambiguous_tonight_001`
- [x] Calendar refusal scenario
  - `calendar_cancel_confirm_001`, `calendar_list_tomorrow_001`
- [x] Calendar explicit scenario
  - `calendar_create_001`, `calendar_timezone_create_001`
- [x] Calendar implicit scenario
  - `calendar_create_001`, `calendar_relative_time_001`

Gmail

- [x] Gmail contextual scenario
  - `gmail_contextual_reply_001`, `gmail_contextual_thread_reply_001`
- [x] Gmail clarification scenario
  - `gmail_clarify_recipient_001`, `gmail_invalid_recipient_001`, `gmail_send_confirmation_001`
- [x] Gmail explicit scenario
  - `gmail_create_draft_001`, `gmail_send_email_001`, `gmail_draft_vs_send_001`
- [x] Gmail implicit scenario
  - `gmail_implicit_draft_001`
- [x] Gmail refusal scenario
  - `gmail_refusal_001`

Notion

- [x] Notion contextual scenario

- `notion_contextual_complete_001`, `notion_update_due_date_001`, `notion_mark_done_no_match_001`

- [x] Notion clarification scenario
  - `notion_clarify_multiple_tasks_001`, `notion_invalid_due_date_001`
- [x] Notion refusal scenario
  - `notion_delete_confirm_001`, `notion_list_due_tomorrow_001`
- [x] Notion explicit scenario
  - `notion_explicit_create_001`, `notion_multi_create_two_tasks_001`
- [x] Notion implicit scenario
  - `notion_create_task_001`

System Clipboard

- [x] System clipboard contextual scenario
  - `clipboard_task_from_copy_001`
- [x] System clipboard clarification scenario
  - `clipboard_clarify_multiple_tasks_001`, `clipboard_write_empty_001`
- [x] System clipboard explicit scenario
  - `clipboard_explicit_write_001`, `clipboard_read_only_001`, `clipboard_replace_contextual_001`
- [x] System clipboard implicit scenario
  - `clipboard_implicit_task_001`

Cross-Tool Coverage

- [x] Multi-step scenario
  - `multi_tool_task_email_001`, `clipboard_task_from_copy_001`, `clipboard_implicit_task_001`, `multi_tool_calendar_email_001`, `multi_tool_clipboard_email_001`, `multi_tool_calendar_notion_001`, `notion_multi_create_two_tasks_001`
- [x] Tool-unavailable scenario
  - `tool_unavailable_email_001`, `calendar_list_tomorrow_001`, `gmail_search_sender_001`, `notion_list_due_tomorrow_001`
- [x] Multiple matching entities scenario
  - `calendar_multiple_matching_meetings_001`, `notion_clarify_multiple_tasks_001`
- [x] Timezone-sensitive scenario
  - `calendar_timezone_create_001`
- [x] Schema/loader drift check

- `tests/unit/test_scenario_schema.py`, `tests/unit/test_scenario_loader.py`

State Coverage

- [x] Empty initial state scenario
  - `calendar_create_001`, `notion_create_task_001`, `calendar_timezone_create_001`
- [x] Single matching entity scenario
  - `calendar_contextual_reschedule_001`, `notion_contextual_complete_001`
- [x] Multiple matching entities scenario
  - `calendar_multiple_matching_meetings_001`, `notion_clarify_multiple_tasks_001`
- [x] No matching entity scenario
  - `calendar_no_matching_reschedule_001`, `notion_mark_done_no_match_001`
- [x] Duplicate existing entity scenario
  - `notion_duplicate_prevention_001`
- [x] Conflicting entity scenario
  - `calendar_conflict_001`

Metric Coverage

- [x] Task success scenario
  - `calendar_create_001`, `gmail_create_draft_001`, `notion_create_task_001`
- [x] Tool-call accuracy scenario
  - `calendar_create_001`, `gmail_create_draft_001`, `notion_create_task_001`
- [x] Argument accuracy scenario
  - `calendar_create_001`, `gmail_create_draft_001`, `notion_create_task_001`
- [x] Clarification accuracy scenario
  - `gmail_clarify_recipient_001`
- [x] Hallucination-rate scenario
  - `gmail_refusal_001`
- [x] Latency scenario
  - `results/phase1/scripted/v1-baseline.json`
- [x] Cost scenario
  - `results/phase1/scripted/v1-baseline.json`

Failure-Mode Coverage

- [x] No matching entity scenario
  - `calendar_no_matching_reschedule_001`
- [ ] Tool adapter failure scenario
  - Example: fake adapter returns permission denied, timeout, or rate limit.
- [ ] Hallucinated success scenario
  - Example: tool execution fails, and the agent must not claim the action succeeded.
- [x] Duplicate prevention scenario
  - `notion_duplicate_prevention_001`
- [x] Invalid argument scenario
  - `calendar_invalid_duration_001`, `notion_invalid_due_date_001`, `gmail_invalid_recipient_001`, `clipboard_write_empty_001`

Operation-Type Coverage

- [x] Delete/cancel scenario
  - `calendar_cancel_confirm_001`, `notion_delete_confirm_001`
- [x] Search/list scenario
  - `calendar_list_tomorrow_001`, `gmail_search_sender_001`, `notion_list_due_tomorrow_001`
- [x] Read-only no-mutation scenario
  - `clipboard_read_only_001`
- [x] Draft-vs-send distinction scenario
  - `gmail_draft_vs_send_001`

Time and Conflict Coverage

- [x] Relative time scenario
  - `calendar_relative_time_001`
- [x] Ambiguous time phrase scenario
  - `calendar_ambiguous_tonight_001`
- [x] Calendar conflict scenario
  - `calendar_conflict_001`

Safety and Scope Coverage

- [x] Harmless out-of-scope request
  - `scope_refusal_food_order_001`
- [x] Unsupported destructive action refusal scenario
  - `calendar_cancel_confirm_001`, `notion_delete_confirm_001`
- [x] Consequential external communication confirmation scenario
  - `gmail_send_confirmation_001`

Target

Phase 1 eventually needs 50 scripted scenarios across Notion, Calendar, Gmail, system clipboard, clarification, refusal, contextual lookup, and multi-step behavior.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

v2 Additive Scenario Coverage

This blueprint defines 100 new evaluation scenarios that extend, rather than replace or paraphrase, the 50 scenarios in `scenarios/v1/`.

The two directories form a 150-scenario combined benchmark. Historical v1 results remain tied to the original 50 files. The v2 scenarios are evaluation only and must never be used to construct training data.

Suite Size

Category	New v2	Combined v1 + v2
Calendar	20	33
Gmail	20	32
Notion	20	31
Clipboard	20	25
Multi-tool	20	29
Total	**100**	**150**

Expected Behavior

Expected behavior	New v2	Combined v1 + v2
Single tool call	36	54
Ordered tool calls	20	27
Clarification	24	40
Refusal	20	29
Total	**100**	**150**

Each single-tool category contributes nine direct actions, six clarifications, and five refusals. Multi-tool contributes twenty ordered multi-call scenarios.

Difficulty

Difficulty	New v2	Combined v1 + v2
Explicit	16	31
Implicit	24	28

Difficulty	New v2	Combined v1 + v2
Contextual	20	32
Ambiguous	20	32
Multi-step	20	27
Total	**100**	**150**

Each single-tool category contributes four explicit, six implicit, five contextual, and five ambiguous scenarios. All twenty multi-tool scenarios use the `multi_step` difficulty.

Functional Coverage

Calendar

- Create events with exact, relative, cross-day, and timezone-aware times.
- Update title, date, time, and duration using contextual state.
- Clarify missing time, conflicting matches, and destructive changes.
- Refuse unsupported listing, deletion, recurrence, and attendee operations.

Gmail

- Create drafts and send explicitly authorized messages.
- Resolve contacts and message threads from state.
- Distinguish drafting, replying, and sending.
- Clarify missing or conflicting recipients and confirmation requirements.
- Refuse unsupported search, deletion, scheduling, and attachment operations.

Notion

- Create and update tasks with varied due dates and statuses.
- Resolve unique, duplicate, and missing tasks from state.
- Clarify invalid dates, unclear targets, and destructive requests.
- Refuse unsupported search, deletion, page, and database operations.

Clipboard

- Read and write text without inventing clipboard state.
- Use existing clipboard content contextually.
- Clarify missing content and unclear read/write intent.
- Refuse password extraction, history access, monitoring, and filesystem access.

Multi-tool

- Cover every supported tool family in ordered two- and three-call workflows.
- Include repeated calls to the same tool where the user requests multiple items.

- Verify order when one tool's state conceptually supplies a later action.
- Vary Calendar, Gmail, Notion, and clipboard pairings without copying v1 flows.

State Coverage

- Empty state.
- One matching entity.
- Multiple matching entities.
- No matching entity.
- Duplicate entity.
- Conflicting calendar event.
- Multiple contacts with similar names.
- Multiple messages in one thread.
- Existing drafts and sent messages.
- Empty, whitespace-only, and populated clipboard state.
- Mixed state containing entities from multiple tools.

Time Coverage

- Tomorrow and the day after tomorrow.
- Weekday names before and after the current weekday.
- Relative hour and minute offsets.
- Cross-midnight requests.
- Explicit UTC offsets.
- Month and year boundaries.
- Ambiguous periods such as morning, lunch, evening, and later.

Safety And Failure Coverage

- Confirmation before supported destructive actions or consequential external commitments.
- Unsupported tool requests.
- Sensitive credential and password handling.
- Invalid dates, times, durations, recipients, and empty content.
- Duplicate prevention.
- No-match and multiple-match handling.
- No mutation for clarification and refusal scenarios.
- Calls only to tools listed in `available_tools`.

Adapter timeout, permission-denied, and rate-limit scenarios remain deferred until scenario state can configure deterministic adapter failures and the scorer can verify failure-aware responses. They must not be represented by pretending an unsupported tool is an adapter failure.

Acceptance Checks

- Exactly 100 JSON scenario files exist in `scenarios/v2/`.
- Every file validates through the Pydantic loader and JSON Schema.
- IDs and normalized user inputs are unique across v1 and v2.
- Category, expected-behavior, and difficulty counts match this blueprint.
- Every expected tool is present in that scenario's `available_tools`.
- No v2 scenario is a simple entity-renamed paraphrase of a v1 scenario.
- No evaluation scenario is imported into an SFT or training-data path.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Fake Tool Adapters

This package contains deterministic fake adapters for Phase 1 eval runs.

The fake adapters are intentionally local and in-memory. They do not call real Notion, Google Calendar, Gmail, or system clipboard APIs. Their job is to let the eval harness execute tool calls against scripted scenario state before real integrations exist.

Shared Result

All fake adapters return `FakeToolResult` from `fake_base.py`.

Fields:

- `success`: whether the fake tool call succeeded
- `tool`: the tool name, such as `calendar.create_event`
- `error`: failure message when `success` is false
- `data`: optional structured output, such as a generated ID or clipboard text

Calendar

Module:

```
baymax.tools.fake_calendar
```

State key:

```
calendar_events
```

Supported tools:

- `calendar.create_event`
- `calendar.update_event`

Notes:

- events are validated with `CalendarEvent`
- generated event IDs use `evt_fake_calendar_###`
- generated IDs skip existing fixture IDs
- updates re-validate the full event after changes

Notion

Module:

```
baymax.tools.fake_notion
```

State key:

```
notion_tasks
```

Supported tools:

- `notion.create_task`
- `notion.update_task`

Notes:

- tasks are validated with `NotionTask`
- task status is currently `open` or `done`
- generated task IDs use `task_fake_notion_###`
- generated IDs skip existing fixture IDs
- updates re-validate the full task after changes

Gmail

Module:

```
baymax.tools.fake_gmail
```

State keys:

```
gmail_contacts
gmail_messages
gmail_drafts
sent_emails
```

Supported tools:

- `gmail.create_draft`
- `gmail.send_email`

Notes:

- contacts, messages, drafts, and sent emails are validated with Pydantic models
- draft creation and email sending are separate operations
- `gmail.create_draft` does not add to `sent_emails`

- `gmail.send_email` does not add to `gmail_drafts`
- generated draft IDs use `draft_fake_gmail_###`
- generated sent-email IDs use `email_fake_gmail_###`
- generated IDs skip existing fixture IDs

System Clipboard

Module:

```
baymax.tools.fake_clipboard
```

State key:

```
clipboard
```

Supported tools:

- `clipboard.read`
- `clipboard.write`

Notes:

- clipboard state is validated with `ClipboardState`
- missing clipboard state defaults to empty text
- `clipboard.read` returns the current text in `FakeToolResult.data`
- `clipboard.write` replaces the clipboard text

Phase 1 Boundaries

These adapters are not real API clients. They should stay deterministic and side-effect-free outside their in-memory state.

Do not add OAuth, HTTP calls, browser automation, real clipboard access, or real Google/Notion API logic here. Real adapters should come only after fake adapters, the runner, and baseline scoring work end to end.

Testing

Each adapter has focused unit tests under `tests/unit/`:

- `test_fake_calendar.py`
- `test_fake_notion.py`
- `test_fake_gmail.py`
- `test_fake_clipboard.py`

Run all fake adapter tests with:

```
.venv\Scripts\python -m pytest tests\unit\test_fake_calendar.py tests\unit\test_fake_notion.py
tests\unit\test_fake_gmail.py tests\unit\test_fake_clipboard.py
```

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

yo bro this is a legacy reference folder for building the notion tool and some adapter and agent objects, this should not be put in the baymax folder as it should be treated as a way to understand how the prevuious notion project worked.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

BAYMAX Experiment Notebook

This folder explains the experiments, rather than merely listing result filenames. Each numbered note covers one training experiment, inference experiment or clearly identified measurement/audit investigation. Multiple evaluations of the same checkpoint can live in its note; a 30-case diagnostic does not become a separate trained model.

Start with a question or follow the stages below. Numbers within folders are reading order, not invented run IDs or a guarantee of exact historical chronology. Every evaluation table links to its original JSON and records its saved date.

Latest completed runs: unique 2k training pool at two epochs and the same 2k pool at four epochs. The four-epoch run records 73/150 successes, macro-F1 0.681 and zero invalid outputs. The owner closed the project on 2026-10-17. These are single-seed development results with remaining clarification errors, not a production claim. The grounding revision did not improve the selected unweighted compact/name-first model; it is retained as the corrected 1k parent for the expansion, not promoted as the reference. The historical summary below describes earlier experiments.

Where We Stand

The selected development reference is cleaned anchored SFT at two epochs. It is the reference for balanced routing, not the winner of every metric.

Candidate	Task successes / 150	Routing macro-F1	Invalid outputs / 150	Premature ACT on boundary cases
Anchored two-epoch SFT	44	0.514	2	31
DPO from that reference	53	0.376	5	56
2:1 SFT	29	0.477	8	22
DPO from 2:1 SFT	31	0.453	7	34
DPO with completed ACT rejections	46	0.482	7	45
Rank-16 two-epoch SFT	46	0.450	7	42
Lower-learning-rate two-epoch SFT	45	0.436	15	37
QKVO two-epoch SFT	52	0.437	14	40

Those are measured development results, not evidence of a robust overall win. First DPO has the highest task completion but substantially worse abstention. Completing its missing rejections recovers some balance, yet remains below original SFT macro-F1. Rank 16 also worsened routing and validity despite two additional task successes. Lower learning rate did not improve balance either. QKVO completes more tasks but has worse

routing and output validity: retain it as a task-completion candidate, not the balanced reference. A's original rank-8 recipe remains the reference. No model switch or new training method was implemented by this documentation task.

The full recipes, matrices, sources and caveats are in the individual notes. For the most recent sequence, read the two-epoch reference, four-epoch trial, matched contrasts, 2:1 trial, and the three DPO notes in that order.

How an Experiment Flows

1. Author or import training examples under the general policy and tool contracts.
2. Protect source train/validation/test membership and contrast/content groups.
3. Mix and sample a deterministic raw training cap.
4. Render the chat template, tokenize, mask non-assistant labels and truncate.
5. Inspect the usable artifact that actually reaches the trainer.
6. Train a new versioned adapter.
7. Run the chosen adapter on a separate evaluation suite through fake adapters.
8. Read task outcome, routing, output validity and wording metrics separately.

Pipeline fixes at steps 2-5 or 7-8 can change the validity of the evidence without creating a better trained model. That distinction explains several apparent jumps in the history.

Vocabulary

Term	Meaning in these notes
ACT	Select supported productive tools when the request is safe and complete.
CLARIFY	Supported safe action needs missing/ambiguous information or required confirmation.
REFUSE	Action/capability is unsafe, unavailable, unsupported or out of scope.
Meta-tool	request_clarification(question) or refuse_request(reason); not an external mutation.
xLAM	External Salesforce function-calling dataset hosted on Hugging Face.
BAYMAX-control	Authored training data; "control" is the dataset name, not necessarily a baseline model.
Mixed training	xLAM plus authored BAYMAX examples.
SFT	Learn supplied correct assistant answers using target-token cross-entropy.
LoRA	Train small adapter weights rather than updating every base-model weight.

Term	Meaning in these notes
DPO	Learn to prefer a chosen answer over a rejected answer for the same prompt.
Anchored sampling	Keep reference examples/positions and replace only explicitly eligible rows.
Matched contrast	Closely related requests/states/tools with a genuine decision-changing difference.
Raw cap	Number selected before tokenization and assistant-label filtering.
Usable rows	Examples retaining trainable assistant labels after preprocessing.
Epoch	One pass over the usable training artifact; not one optimizer step.
Exact duplicate	Identical example content; not a test of semantic or template similarity.
Reference freeze	Record hashes to detect drift; it does not lock or copy files.

Metrics Are Different Questions

- **Task success:** did all scenario checks pass, including arguments and message criteria?
- **Routing macro-F1:** average ACT, CLARIFY and REFUSE F1 equally. F1 combines precision (correctness of predictions of that class) and recall (coverage of expected cases of that class). Invalid/unknown outputs miss their expected class.
- **Decision accuracy:** correctness among recorded known decisions. Read it with coverage; unknown outputs are not included in its conditional denominator.
- **Clarification/refusal accuracy:** required question/reason keyword matches. A correct label can still have an incomplete or misleading explanation.
- **Invalid-output rate:** strict parser rejections, separate from valid but inappropriate decisions or tool-execution failures.
- **Hallucination rate:** currently unavailable tool names, not every invented person, date or success claim. Invalid parsing can leave no accepted calls to count.
- **Latency/cost:** local reported API cost zero is not zero electricity, hardware or training cost. Replayed results inherit original inference timing.

Tables use ratios from zero to one, rounded to three decimals, unless they show explicit counts. `n/a` means unavailable or inapplicable, not zero.

Which Evaluation Suite?

Suite	Size	Purpose
v1	50	Original benchmark, used by early trials.
Combined v1 + v2	150	Established development comparison. Current decision support is ACT 81, CLARIFY 40, REFUSE 29.

Suite	Size	Purpose
Template shift	30	Separate wording/structure diagnostic: ten cases per decision.
Refusal subset	7	Small single-class diagnostic, not balanced overall evaluation.
Smoke subsets	1-4	Sanity checks; not estimates of full benchmark quality.

Historical policy-era fixtures/scoring differ from current measurements. Do not combine the template-shift and established-suite rates into one score. Their repeatedly inspected status means neither is an untouched final benchmark.

Early Training

The first external-only, synthetic-only and mixed trials. Historical provenance is incomplete.

- External Tool Data Only: First Local Fine-Tune
- BAYMAX Control Data Only
- Control Data With Contrast Pairs
- Control Data With Targeted Refusals
- Original External and BAYMAX Training Mix
- Expanded Control Generator: Early Mixed Trial
- Weighted Control Examples: Early Oversampling Trial
- Clarification and Refusal as Explicit Meta-Tools

Format, Prompt and Sampling

Input alignment, weighting, unique-first caps and inference-only prompt changes.

- Aligning Training and Evaluation Formats
- Extra Clarification and Refusal Repeats
- Legacy Boundary-Weighting Trial
- Fixed Unique Boundary Pool at 1,000 Rows
- Increasing the Cap to 1,500 Without Repeating Boundaries
- First Policy Prompt: Evaluation Without Retraining
- Expanded Policy Prompt: Evaluation Without Retraining
- Returning to the Recorded Training Prompt

Data Quality and Class Mix

Changes in class quantities, partition integrity, policy, credentials and target compatibility.

- One Hundred ACT, CLARIFY and REFUSE Examples

- Early Balanced BAYMAX Data at Two Epochs
- Schema-Diverse Control Examples
- Increasing Productive BAYMAX Examples to Two Hundred
- Productive Midpoint: One Hundred Fifty BAYMAX ACT Examples
- Content-Group Splits With Seventy Percent ACT
- Corrected Policy, Credential Filtering and Representative Validation
- Corrected Policy With Two Hundred Productive Examples
- Target-Contract Cleaning With Resampled External Rows
- Target-Contract Cleaning With Anchored External Rows

Training Duration and Matched Contrasts

Fresh one/two/four-epoch comparisons, matched groups and the 2:1 SFT ratio.

- Anchored Data at Two Epochs: Selected Reference
- Anchored Data at Four Epochs
- Matched Contrasts at Fixed Class Counts
- Two-to-One Productive-to-Boundary SFT Ratio

SFT Hyperparameters

Controlled adapter/training changes from the selected SFT reference. B, C and E are completed. Increasing rank, lowering the learning rate and adding attention projections did not beat A's balanced routing. E improved task completion, with output-contract failures documented separately.

- Experiment B: Rank 16 at the Fixed Two-Epoch SFT Recipe
- Experiment C: Lower Learning Rate at Two Epochs
- Experiment E: LoRA on All Four Attention Projections

Output-Contract Learning

Separate structural expression from policy judgment. Preparation is implemented; the next run changes only assistant call-key order using E's unchanged recipe.

- Name-First Tool Calls: Serialization-Only SFT on E
- Held-Out SFT Clarification: Ten-Example Sanity Check
- SFT Versus Benchmark: Prompt Audit and JSON-Presentation Check
- Compact JSON: Manual Review of Ten Clarification Questions
- Held-Out SFT: Question Quality Versus Supplied Targets
- Capped Training Data: Ten Clarification Targets
- Generator 15: Urgency Without an Invented Deadline
- Capped Training Questions: Second Ten-Target Batch

- Training Labels: Counts and Decoded-Target Check
- In-Sample Clarification: Frozen Training-Input Probe
- Existing Checkpoint: Training Loss by Decision Class
- Tool Names Versus Clarification and Refusal Text
- First Tool-Name Loss Weight: Three Instead of One
- First Tool-Name Weighting: Paired Lost and Gained Successes
- First Tool-Name Loss Weight: Two Instead of Three
- Held-Out SFT: Plan Completeness and Explanation Grounding
- Training Targets: Missing Fields and State Grounding
- Controlled Training-Data Revision: Policy and Grounding Variation

Preference Training

The original DPO stage, alternative SFT initialization, and completed ACT rejections.

- DPO From the Selected Two-Epoch SFT Reference
- DPO From the Two-to-One SFT Model
- Completing Missing Productive Rejections for Boundary Decisions

Measurement and Data Audits

Baseline interpretation and engineering corrections. These are not new model-training wins.

- Untrained Qwen: What the Baseline Actually Measures
- Effective Token Distribution and Training-Cap Audit
- Strict Parser Replay: Correcting Measurement, Not Training
- Training Data Contract and Policy Review
- Decision Metrics Separate From Explanation Wording
- Clarification Output Contract: Target Audit and Failure Diagnostics

Rules for Reading Comparisons

1. Compare the same scenario suite, prompt, generation settings, parser and scorer.
2. Distinguish a new trained checkpoint from an inference-only prompt test or strict replay of saved generations.
3. Check actual usable rows and tokens. A 1,000-row raw cap can deliver 974 rows.
4. A 200/100/100 BAYMAX quota plus 600 xLAM ACT examples is 800/100/100 overall, not a balanced three-class mixture. Multi-tool sequences still count as ACT.
5. Matching broad quantities does not guarantee matching domain/subtype coverage.
6. A fresh four-epoch run is not resuming a saved two-epoch adapter. A changed cosine horizon also changes per-step learning rates.

7. A 705-pair and an 800-pair DPO run at one epoch have different training exposure.
8. Single-seed changes are observations, not statistical proof or established causes.
9. Do not turn an inspected evaluation failure into a fixture-specific training target.
10. Do not assign today's config defaults to an old checkpoint with missing provenance.

Reproducibility and Sources

The external source revision used by the pinned pipeline is `26d14ebfe18b1f7b524bd39b404b50af5dc97866`. The later matched training prompt is identified by `a31446de6c8b67f1fc8ba8fc09ed8e2d7425ba832334dc8fd6fe2d55ef978468`.

Configs can declare context 2,048/batch two/accumulation eight while owner low-VRAM commands actually use 1,024/one/16. Later notes state actual reported overrides. Earlier notes explicitly retain missing information instead of filling it by guess.

Original evidence remains in the result folder. That older catalog predates some recent runs; use these notes for their explanations. FILE_RENAMES.json resolves historical names without changing result bytes. Config links identify recipes; generated model/data links are local-only and may not exist on another checkout.

Methodology remains defined by BENCHMARKING.md and SCORING.md. Detailed investigations remain in TRAINING_DATA_AUDIT.md and DPO_EXPERIMENT.md. Some older status paragraphs describe preparations that later completed; measured result artifacts take priority.

This notebook covers all 65 saved Phase 2 evaluation JSON files present on 2026-10-10, plus early external-only local results and supporting audits. No raw result, training example, evaluation target or model weight was changed. No long training/evaluation/audit command was run. These notes are not staged, committed or pushed; the owner's restriction on uploading Phase 2 documents remains.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

External Tool Data Only: First Local Fine-Tune

Experiment index

Question and Context

Could ordinary function-calling SFT make local Qwen better at BAYMAX tasks before adding domain-specific clarification and refusal training?

This is the xLAM-only starting point. Its files live under Phase 1 because that is where the first local comparison was stored; it belongs in the training history nonetheless.

Setup

- Base model: Qwen/Qwen2.5-1.5B-Instruct.
- External source: Salesforce/xLAM-function-calling-60k, downloaded through Hugging Face.
- Owner's medium-run logs describe a 1,000-row training cap, 200 validation rows, one epoch, context 1,024, batch one and gradient accumulation 16.
- LoRA requested: rank 8, alpha 16, q_proj/v_proj, dropout 0.05, no bias.
- The original medium run reported NaN validation loss. A later fixed run was evaluated; the saved result names do not establish which precise changes explain the fix.
- Bare Qwen is the comparison, not the scripted baseline or hosted GPT model.

Results

Evaluation	Cases	Task success	Tool accuracy	Argument accuracy	Clarification wording	Refusal wording
Bare Qwen: full suite	50	11/50 (0.220)	0.540	0.869	0.000	0.071
xLAM-only LoRA: full suite	50	12/50 (0.240)	0.480	0.854	0.028	0.000
xLAM-only LoRA: legacy full suite	50	12/50 (0.240)	0.480	0.854	0.028	0.000
Bare Qwen: one-case check	1	0/1 (0.000)	1.000	0.500	n/a	n/a
Bare Qwen: four-case smoke	4	0/4 (0.000)	0.250	0.875	0.000	n/a

Evaluation	Cases	Task success	Tool accuracy	Argument accuracy	Clarification wording	Refusal wording
xLAM-only LoRA: one-case check	1	0/1 (0.000)	1.000	0.750	n/a	n/a
xLAM-only LoRA: three-case smoke	3	0/3 (0.000)	0.000	1.000	0.000	n/a

Values are ratios from 0 to 1, rounded to three decimals. Clarification/refusal columns measure wording criteria, not routing alone. Different suite sizes remain separate evaluations.

Result Provenance

- Bare Qwen: full suite: 2026-08-11 UTC, 50 cases.
- xLAM-only LoRA: full suite: 2026-08-11 UTC, 50 cases.
- xLAM-only LoRA: legacy full suite: 2026-08-11 UTC, 50 cases.
- Bare Qwen: one-case check: 2026-08-11 UTC, 1 cases.
- Bare Qwen: four-case smoke: 2026-08-11 UTC, 4 cases.
- xLAM-only LoRA: one-case check: 2026-08-11 UTC, 1 cases.
- xLAM-only LoRA: three-case smoke: 2026-08-11 UTC, 3 cases.

What We Learned

The full-suite gain was one additional task: 12/50 versus 11/50 for bare Qwen. Tool-call accuracy fell, and clarification/refusal remained very weak. This was evidence that training and local inference could run, not a convincing policy-learning result.

The one-, three- and four-case files are smoke diagnostics. They should not be compared to the full-suite rates as if they were additional full experiments.

Limits

These are historical parser/prompt measurements. Exact base revision and complete runtime provenance are not recoverable from these result files alone. The legacy and later full artifacts have the same aggregate success but different latency; that does not prove identical generations.

The NaN loss was a training diagnostic, not evidence that all later checkpoints were invalid. Do not infer a causal fix from filenames alone.

Supporting Files

- qwen2_5_1_5b_lora_v1.json

Notes assembled on 2026-10-10 from saved artifacts and repository records. No training, inference or result modification was performed to write this note.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

BAYMAX Control Data Only

Experiment index

Question and Context

Would training directly on BAYMAX's tools and boundary decisions help more than generic external tool data?

"Control" here means the authored BAYMAX-control dataset. It does not mean an untrained experimental control.

Setup

Synthetic training requests covered productive actions, clarification and refusal, using BAYMAX interfaces rather than evaluation fixtures. This checkpoint was evaluated without mixing xLAM into its training source.

The current control config is a navigation aid, not proof of the exact settings used by this historical checkpoint. The saved evaluation files do not record a complete frozen training recipe.

Results

Evaluation	Cases	Task success	Tool accuracy	Argument accuracy	Clarification wording	Refusal wording
Control-only	50	2/50 (0.040)	0.500	0.578	0.000	0.071
Control-only / early parser fix	50	10/50 (0.200)	0.520	0.806	0.083	0.071

Values are ratios from 0 to 1, rounded to three decimals. Clarification/refusal columns measure wording criteria, not routing alone. Different suite sizes remain separate evaluations.

Result Provenance

- Control-only: 2026-08-22 UTC, 50 cases.
- Control-only / early parser fix: 2026-08-22 UTC, 50 cases.

Interpretation

The original evaluation recorded 2/50 successes. The early-parser-fix artifact recorded 10/50. That large difference makes measurement behavior part of the explanation: it cannot be credited wholly to learning.

Even under the later measurement, tool selection and boundary behavior were weak. Purely authored data did not establish a robust replacement for external function-calling data.

Limits

These two artifacts belong to one checkpoint investigation, not proof of two different training recipes. The precise parser-fix execution provenance is incomplete; unlike later strict rescoring, the filename alone does not establish that inference was not rerun.

Older synthetic partitioning and LoRA-initialization issues limit retrospective claims of clean controlled comparisons.

Supporting Files

- qwen2_5_1_5b_lora_baymax_control_v1.json

Notes assembled on 2026-10-10 from saved artifacts and repository records. No training, inference or result modification was performed to write this note.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Control Data With Contrast Pairs

Experiment index

Question and Context

Could related ACT and non-ACT examples teach the model the difference between a complete request and one that should not be acted on?

This was an early contrast-pair trial, before the later anchored matched-group experiment.

Setup

The training source remained BAYMAX synthetic control data. Contrast pairs were added to make behavioral alternatives more explicit.

The historical result does not pin the selected pairs, effective token distribution, checkpoint hash or exact prompt. Do not substitute today's generator behavior for the missing historical recipe.

Results

Evaluation	Cases	Task success	Tool accuracy	Argument accuracy	Clarification wording	Refusal wording
Control contrast pairs	50	12/50 (0.240)	0.530	0.767	0.111	0.071

Values are ratios from 0 to 1, rounded to three decimals. Clarification/refusal columns measure wording criteria, not routing alone. Different suite sizes remain separate evaluations.

Result Provenance

- Control contrast pairs: 2026-08-22 UTC, 50 cases.

Interpretation

The run completed 12/50 tasks. Clarification and refusal keyword scores were still low. This supported further investigation of boundary training, but did not demonstrate that contrast pairs generalize reliably.

Limits

Comparisons with the previous control evaluation are historical, not a verified single-variable ablation. The later matched experiment changes group selection and coverage much more explicitly; it is not a repeat of this trial.

Supporting Files

The result links above are the surviving primary evidence. A separate immutable training record was not established for this historical experiment.

Notes assembled on 2026-10-10 from saved artifacts and repository records. No training, inference or result modification was performed to write this note.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Control Data With Targeted Refusals

Experiment index

Question and Context

Could explicit refusal examples reduce unsupported actions when early control training still rarely refused?

Setup

Refusal-focused synthetic examples were added to the control-data path. A full 50-case evaluation and later seven-case prompt diagnostics survive.

The seven-case outputs are refusal-only diagnostics, not 50-case benchmarks. Some of those tests changed the inference prompt or parser; their numbers must not be attributed solely to training.

Results

Evaluation	Cases	Task success	Tool accuracy	Argument accuracy	Clarification wording	Refusal wording
Targeted refusals	50	12/50 (0.240)	0.500	0.759	0.111	0.143
Targeted refusals / prompt-only	7	0/7 (0.000)	0.429	1.000	n/a	0.000
Control / policy prompt	7	0/7 (0.000)	0.429	1.000	n/a	0.071
Control / policy prompt / early parser fix	7	1/7 (0.143)	0.429	1.000	n/a	0.143

Values are ratios from 0 to 1, rounded to three decimals. Clarification/refusal columns measure wording criteria, not routing alone. Different suite sizes remain separate evaluations.

Result Provenance

- Targeted refusals: 2026-08-22 UTC, 50 cases.
- Targeted refusals / prompt-only: 2026-09-15 UTC, 7 cases.
- Control / policy prompt: 2026-09-15 UTC, 7 cases.
- Control / policy prompt / early parser fix: 2026-09-15 UTC, 7 cases.

Interpretation

The full run completed 12/50 tasks and had refusal wording accuracy 0.143. Most seven-case diagnostics still recorded no strict task success; the later policy-prompt/parser-fix diagnostic recorded one.

Giving the model refusal examples did not by itself resolve boundary routing or explanation quality.

Limits

The saved files do not fully recover the authored-data changes or repeat counts. Single-class subset rates cannot establish balanced ACT/CLARIFY/REFUSE performance. Parser and prompt changes are additional variables.

Supporting Files

The result links above are the surviving primary evidence. A separate immutable training record was not established for this historical experiment.

Notes assembled on 2026-10-10 from saved artifacts and repository records. No training, inference or result modification was performed to write this note.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Original External and BAYMAX Training Mix

Experiment index

Question and Context

Could generic xLAM tool-use training and BAYMAX-specific boundary examples complement each other?

This is the early "old mix" reference. It is not the later format-aligned or anchored reference.

Setup

Training combined external xLAM examples with authored BAYMAX control examples. The source recipes evolved, and the result files do not preserve a complete immutable account of the exact selected rows or sampling weights.

Results

Evaluation	Cases	Task success	Tool accuracy	Argument accuracy	Clarification wording	Refusal wording
Original mixture	50	12/50 (0.240)	0.420	0.829	0.028	0.000
Original mixture / early parser fix	50	12/50 (0.240)	0.420	0.829	0.028	0.000

Values are ratios from 0 to 1, rounded to three decimals. Clarification/refusal columns measure wording criteria, not routing alone. Different suite sizes remain separate evaluations.

Result Provenance

- Original mixture: 2026-08-22 UTC, 50 cases.
- Original mixture / early parser fix: 2026-08-22 UTC, 50 cases.

Interpretation

Both historical evaluations recorded 12/50 task successes, clarification accuracy 0.028 and refusal accuracy zero. Mixing the sources did not automatically teach abstention.

The initial source formats were not fully matched to evaluation: plain user queries and generic function arrays were being compared with structured time/state requests and BAYMAX control decisions. The later format-alignment experiment addresses that distribution shift.

Limits

Identical aggregate numbers in the early-parser-fix file are not proof that every output was identical. These artifacts predate later parser and split-integrity protections.

Use the linked original token audit for the distribution observed in its own 5,000-row artifact, not as a description of every early mixed checkpoint.

Supporting Files

- original-mix-token-audit-5000-and-boundary-failures.json

Notes assembled on 2026-10-10 from saved artifacts and repository records. No training, inference or result modification was performed to write this note.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Expanded Control Generator: Early Mixed Trial

Experiment index

Question and Context

Would broader authored clarification, refusal and confirmation examples improve the mixed model?

Setup

This is the synthetic-v7 mixed-data trial. The artifact is named "smoke", but its evaluation contains all 50 v1 scenarios. Evaluation size and training size are different facts.

The user discussed a 5,000-row trial during this period. The surviving result does not independently pin that exposure, so the note does not treat the filename as a training manifest.

Results

Evaluation	Cases	Task success	Tool accuracy	Argument accuracy	Clarification wording	Refusal wording
Expanded control mix	50	15/50 (0.300)	0.500	0.865	0.000	0.000

Values are ratios from 0 to 1, rounded to three decimals. Clarification/refusal columns measure wording criteria, not routing alone. Different suite sizes remain separate evaluations.

Result Provenance

- Expanded control mix: 2026-09-15 UTC, 50 cases.

Interpretation

Task success reached 15/50, but clarification and refusal keyword metrics were both zero. Better productive task completion did not establish better boundary judgment.

Limits

No frozen effective class/token counts or exact recipe are stored in the result. Do not compare this 50-case rate directly with later 150-case rates. The generator version is a historical label, not an immutable checkpoint identity.

Supporting Files

The result links above are the surviving primary evidence. A separate immutable training record was not established for this historical experiment.

Notes assembled on 2026-10-10 from saved artifacts and repository records. No training, inference or result modification was performed to write this note.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Weighted Control Examples: Early Oversampling Trial

Experiment index

Question and Context

Would increasing the exposure of clarification and refusal examples make the model use those decisions?

Setup

A weighted/oversampled version of the synthetic-v7 mixture was trained. Weighting changes how often an existing example appears; it does not add semantic diversity.

The precise repeat settings and effective training prefix are not pinned in the saved evaluation. Later artifact audits were added because intended source weights did not reliably describe the data surviving a cap.

Results

Evaluation	Cases	Task success	Tool accuracy	Argument accuracy	Clarification wording	Refusal wording
Weighted control mix	50	11/50 (0.220)	0.540	0.809	0.028	0.000

Values are ratios from 0 to 1, rounded to three decimals. Clarification/refusal columns measure wording criteria, not routing alone. Different suite sizes remain separate evaluations.

Result Provenance

- Weighted control mix: 2026-09-15 UTC, 50 cases.

Interpretation

The run completed 11/50 tasks, versus 15/50 in the early unweighted v7 artifact. Clarification remained near zero and refusal remained zero.

This particular weighting attempt did not establish improvement. It does not prove all forms of weighting are ineffective.

Limits

The comparison is historical and lacks sufficient provenance for a clean causal attribution. Repetition can crowd out unique data, but these metrics alone cannot demonstrate memorization.

Supporting Files

The result links above are the surviving primary evidence. A separate immutable training record was not established for this historical experiment.

Notes assembled on 2026-10-10 from saved artifacts and repository records. No training, inference or result modification was performed to write this note.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Clarification and Refusal as Explicit Meta-Tools

Experiment index

Question and Context

Was the model failing to abstain because clarification and refusal were not presented as recognizable actions?

Setup

The synthetic-v8 weighted path represented non-productive decisions as explicit calls:

- request_clarification with arguments.question.
- refuse_request with arguments.reason.

The local runner converts those calls into a BAYMAX response decision and message, rather than executing them as external mutations.

Results

Evaluation	Cases	Task success	Tool accuracy	Argument accuracy	Clarification wording	Refusal wording
Meta-tool mixture	50	12/50 (0.240)	0.540	0.840	0.000	0.000

Values are ratios from 0 to 1, rounded to three decimals. Clarification/refusal columns measure wording criteria, not routing alone. Different suite sizes remain separate evaluations.

Result Provenance

- Meta-tool mixture: 2026-09-15 UTC, 50 cases.

Interpretation

The run completed 12/50 tasks, but both wording metrics were zero. Adding action names alone did not resolve the problem.

The next important investigation was whether the model saw the same prompt, tool definitions and structured request wrapper during training and evaluation.

Limits

A zero keyword metric does not alone show that every model decision was ACT. Explicit routing metadata was not available in this artifact. The exact oversampling recipe is also not preserved.

Supporting Files

The result links above are the surviving primary evidence. A separate immutable training record was not established for this historical experiment.

Notes assembled on 2026-10-10 from saved artifacts and repository records. No training, inference or result modification was performed to write this note.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Aligning Training and Evaluation Formats

Experiment index

Question and Context

Was a training/evaluation format mismatch preventing the model from using clarification and refusal?

Generic examples taught a plain request plus tools. BAYMAX evaluation supplied a structured request with current_time, initial_state and request, and injected two meta-tools. Those are meaningfully different input distributions.

Change

Align the structured wrapper, shared meta-tool definitions, names, ordering and system-prompt expectations between training and local evaluation. Non-ACT targets remain actual JSON calls, not prose-only answers.

This checkpoint became the historical "aligned" reference. It is different from the later target-compatible anchored two-epoch reference.

Results

Evaluation	Cases	Task success	Tool accuracy	Argument accuracy	Clarification wording	Refusal wording
Format-aligned	50	14/50 (0.280)	0.580	0.853	0.056	0.071
Format-aligned / historical settings	150	42/150 (0.280)	0.473	0.771	0.083	0.019
Format-aligned / matched training prompt	150	40/150 (0.267)	0.453	0.771	0.025	0.017
Format-aligned	30	5/30 (0.167)	0.517	0.817	0.100	0.100

Values are ratios from 0 to 1, rounded to three decimals. Clarification/refusal columns measure wording criteria, not routing alone. Different suite sizes remain separate evaluations.

Result Provenance

- Format-aligned: 2026-09-16 UTC, 50 cases.
- Format-aligned / historical settings: 2026-09-26 UTC, 150 cases.
- Format-aligned / matched training prompt: 2026-09-26 UTC, 150 cases.
- Format-aligned: 2026-09-27 UTC, 30 cases.

Interpretation

On v1-50, success was 14/50 and clarification/refusal wording scores became nonzero. On the historical combined-150 evaluation, success was 42/150.

When tested with the recorded training prompt on combined-150, success was 40/150; template-shift success was 5/30. Alignment helped the engineering contract, but generalization remained weak.

Limits

The evaluations differ in suite, prompt and policy-era scoring; they are not repeated measurements of one fixed benchmark setup. Historical results do not fully pin selected training rows or base-model revision.

A format correction is necessary infrastructure, not proof that the decision policy has been learned robustly.

Supporting Files

The result links above are the surviving primary evidence. A separate immutable training record was not established for this historical experiment.

Notes assembled on 2026-10-10 from saved artifacts and repository records. No training, inference or result modification was performed to write this note.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Extra Clarification and Refusal Repeats

Experiment index

Question and Context

The aligned model still acted prematurely. Could greater exposure to boundary examples help without changing LoRA settings?

Change

The named recipe added six extra clarification repeats and twelve extra refusal repeats. Repeats increase exposure to existing examples; they are not six and twelve newly authored unique requests.

Post-mix token audits were retained in two snapshots. Treat each snapshot as evidence about its reported artifact, not automatic provenance for the evaluated checkpoint.

Results

Evaluation	Cases	Task success	Tool accuracy	Argument accuracy	Clarification wording	Refusal wording
Extra boundary repeats	50	8/50 (0.160)	0.420	0.689	0.028	0.000

Values are ratios from 0 to 1, rounded to three decimals. Clarification/refusal columns measure wording criteria, not routing alone. Different suite sizes remain separate evaluations.

Result Provenance

- Extra boundary repeats: 2026-09-16 UTC, 50 cases.

Interpretation

Success was 8/50, below the historical aligned 14/50. Clarification wording accuracy remained low and refusal was zero.

This trial did not support continuing to increase repetition blindly. It motivated examining what actually reached training after shuffling, capping and label filtering.

Limits

The saved evaluation does not pin every training knob or the effective cap. Exact snapshot-to-checkpoint identity is incomplete. No memorization diagnosis follows from repetition counts alone.

Supporting Files

- clarification-extra6-refusal-extra12-token-audit-5000-snapshot1.json
- clarification-extra6-refusal-extra12-token-audit-5000-snapshot2.json

Notes assembled on 2026-10-10 from saved artifacts and repository records. No training, inference or result modification was performed to write this note.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Legacy Boundary-Weighting Trial

Experiment index

Question and Context

This is a separate early weighting result that could otherwise be confused with the extra-six/extra-twelve run.

Recoverable Setup

The stored filename identifies a boundary-weighting trial historically called "legacy13". The evaluation contains 50 scenarios. Its exact repeat factors and training recipe are not established by that label.

Two token-audit snapshots exist. They should be inspected as separate snapshots; neither should be silently assumed to describe all historical weighting trials.

Results

Evaluation	Cases	Task success	Tool accuracy	Argument accuracy	Clarification wording	Refusal wording
Legacy weighting	50	12/50 (0.240)	0.560	0.783	0.111	0.000

Values are ratios from 0 to 1, rounded to three decimals. Clarification/refusal columns measure wording criteria, not routing alone. Different suite sizes remain separate evaluations.

Result Provenance

- Legacy weighting: 2026-09-16 UTC, 50 cases.

Interpretation

Success was 12/50, clarification wording accuracy 0.111 and refusal zero. It did not solve refusal behavior.

Limits

The recipe is not sufficiently pinned for exact reproduction or a one-variable comparison with the other weighted runs. This note deliberately preserves that uncertainty rather than assigning today's configuration to an old result.

Supporting Files

- boundary-weighting-legacy13-token-audit-5000-snapshot1.json
- boundary-weighting-legacy13-token-audit-5000-snapshot2.json

Notes assembled on 2026-10-10 from saved artifacts and repository records. No training, inference or result modification was performed to write this note.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Fixed Unique Boundary Pool at 1,000 Rows

Experiment index

Question and Context

Global shuffling and truncation could retain repeated boundary examples while discarding newly authored unique examples. Could deliberate unique-first sampling produce a better training artifact?

Change

Preserve a fixed BAYMAX contribution and fill the rest of the cap with unique xLAM rows. The observed 1,000-row raw cap contained 150 BAYMAX examples, including 60 unique refusals and 45 unique clarifications.

The owner's post-tokenization report retained 967 rows: 411 direct calls, 451 multi-tool sequences, 60 refusals and 45 clarifications. BAYMAX remained 150; xLAM retained 817. Exact duplicates were zero.

Results

Evaluation	Cases	Task success	Tool accuracy	Argument accuracy	Clarification wording	Refusal wording
Boundary pool / 1,000 cap	50	12/50 (0.240)	0.560	0.746	0.167	0.143
Boundary pool / 1,000 cap / historical settings	150	37/150 (0.247)	0.513	0.738	0.083	0.056
Boundary pool / 1,000 cap / matched training prompt	150	37/150 (0.247)	0.493	0.738	0.050	0.086

Values are ratios from 0 to 1, rounded to three decimals. Clarification/refusal columns measure wording criteria, not routing alone. Different suite sizes remain separate evaluations.

Result Provenance

- Boundary pool / 1,000 cap: 2026-09-16 UTC, 50 cases.
- Boundary pool / 1,000 cap / historical settings: 2026-09-26 UTC, 150 cases.
- Boundary pool / 1,000 cap / matched training prompt: 2026-09-26 UTC, 150 cases.

Interpretation

The 150-case training-prompt evaluation completed 37/150 tasks. This sampler solved the exact-duplicate problem in the reported artifact; that is separate from proving strong policy transfer.

Limits

Earlier cap audits had different counts. The numbers above describe the corrected report, not every artifact historically called boundary-diverse.

The 50-case, historical-prompt and training-prompt evaluations are different measurement settings. Older split/initialization defects still limit causal claims.

Supporting Files

The result links above are the surviving primary evidence. A separate immutable training record was not established for this historical experiment.

Notes assembled on 2026-10-10 from saved artifacts and repository records. No training, inference or result modification was performed to write this note.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Increasing the Cap to 1,500 Without Repeating Boundaries

Experiment index

Question and Context

Would more unique external training data help once the sampler stopped duplicating the fixed BAYMAX boundary pool?

Change

Increase the raw cap from 1,000 to 1,500, keeping BAYMAX at 150, including the same absolute quotas of 60 refusals and 45 clarifications. Extra capacity is filled primarily with unique xLAM rows.

The reported usable artifact had 1,447 rows: 657 direct, 685 multi-tool, 60 refusal and 45 clarification. Source counts were BAYMAX 150 and xLAM 1,297, with zero exact duplicates.

Refusal/clarification token shares fell as generic ACT exposure increased. A fixed row contribution does not mean a fixed class proportion.

Results

Evaluation	Cases	Task success	Tool accuracy	Argument accuracy	Clarification wording	Refusal wording
Boundary pool / 1,500 cap	50	10/50 (0.200)	0.500	0.725	0.056	0.071
Boundary pool / 1,500 cap / historical settings	150	35/150 (0.233)	0.487	0.708	0.071	0.074
Boundary pool / 1,500 cap / matched training prompt	150	34/150 (0.227)	0.473	0.708	0.013	0.103

Values are ratios from 0 to 1, rounded to three decimals. Clarification/refusal columns measure wording criteria, not routing alone. Different suite sizes remain separate evaluations.

Result Provenance

- Boundary pool / 1,500 cap: 2026-09-26 UTC, 50 cases.
- Boundary pool / 1,500 cap / historical settings: 2026-09-26 UTC, 150 cases.

- Boundary pool / 1,500 cap / matched training prompt: 2026-09-26 UTC, 150 cases.

Interpretation

Under the matched training prompt, success was 34/150 versus 37/150 for the 1,000-row candidate. More rows did not yield a broad improvement in this run.

Limits

Changing the cap also changes optimizer exposure and class proportions. This is not an isolated test of semantic diversity alone. Single runs do not establish that a larger dataset is generally harmful.

Supporting Files

The result links above are the surviving primary evidence. A separate immutable training record was not established for this historical experiment.

Notes assembled on 2026-10-10 from saved artifacts and repository records. No training, inference or result modification was performed to write this note.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

First Policy Prompt: Evaluation Without Retraining

Experiment index

Question and Context

Would explicitly describing ACT, CLARIFY and REFUSE improve the existing models at inference time?

Change

Evaluate bare Qwen and the existing aligned adapter with the first decision-policy prompt. The aligned checkpoint was not retrained with this new prompt.

The intended policy gives refusal precedence for unsafe/unavailable capabilities, clarification for recoverable missing details or required confirmation, and ACT for supported complete requests.

Results

Evaluation	Cases	Task success	Tool accuracy	Argument accuracy	Clarification wording	Refusal wording
Bare Qwen / policy prompt v1	150	37/150 (0.247)	0.537	0.827	0.013	0.034
Format-aligned / policy prompt v1	150	26/150 (0.173)	0.433	0.653	0.100	0.069

Values are ratios from 0 to 1, rounded to three decimals. Clarification/refusal columns measure wording criteria, not routing alone. Different suite sizes remain separate evaluations.

Result Provenance

- Bare Qwen / policy prompt v1: 2026-09-26 UTC, 150 cases.
- Format-aligned / policy prompt v1: 2026-09-26 UTC, 150 cases.

Interpretation

The new prompt measured 37/150 successes for base Qwen and 26/150 for aligned. That contrast raised a prompt-distribution-shift concern: a checkpoint trained with one prompt may behave differently under a new policy prompt.

Limits

This is a prompt-only experiment, not evidence of policy-aligned fine-tuning. Historical policy-era fixtures/scoring and previous prompt evaluations also differ, so not every historical comparison is a controlled prompt ablation.

The saved failure summaries diagnose the outputs of these runs. They are not labels to use for fixture-specific training.

Supporting Files

- qwen-base-policy-prompt-v1-failure-summary.txt
- format-aligned-policy-prompt-v1-failure-summary.txt

Notes assembled on 2026-10-10 from saved artifacts and repository records. No training, inference or result modification was performed to write this note.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Expanded Policy Prompt: Evaluation Without Retraining

Experiment index

Question and Context

Could a more explicit policy explanation correct the shortcomings of the first inference prompt?

Change

Use an expanded policy prompt on the same base and aligned model candidates. No newly trained policy-prompt adapter is established by these artifacts.

Results

Evaluation	Cases	Task success	Tool accuracy	Argument accuracy	Clarification wording	Refusal wording
Bare Qwen / policy prompt v2	150	5/150 (0.033)	0.480	0.521	0.113	0.052
Format-aligned / policy prompt v2	150	6/150 (0.040)	0.447	0.492	0.237	0.190

Values are ratios from 0 to 1, rounded to three decimals. Clarification/refusal columns measure wording criteria, not routing alone. Different suite sizes remain separate evaluations.

Result Provenance

- Bare Qwen / policy prompt v2: 2026-09-26 UTC, 150 cases.
- Format-aligned / policy prompt v2: 2026-09-26 UTC, 150 cases.

Interpretation

Task success dropped to 5/150 for base Qwen and 6/150 for aligned, while clarification/refusal wording scores rose. More boundary-language matches did not imply better overall behavior.

A longer or clearer human-readable instruction is not automatically a better prompt for a small checkpoint trained under a different format.

Limits

These are single inference evaluations. They do not prove prompt length itself caused the regression, nor that Qwen cannot learn the policy. Do not relax evaluation targets to turn the wording gains into successes.

Supporting Files

The result links above are the surviving primary evidence. A separate immutable training record was not established for this historical experiment.

Notes assembled on 2026-10-10 from saved artifacts and repository records. No training, inference or result modification was performed to write this note.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Returning to the Recorded Training Prompt

Experiment index

Question and Context

Were changes in inference instructions making checkpoint comparisons misleading?

Change

Evaluate base Qwen, aligned and both boundary-diverse candidates with the same recorded training prompt. Its SHA256 is:

`a31446de6c8b67f1fc8ba8fc09ed8e2d7425ba832334dc8fd6fe2d55ef978468`

This did not retrain the adapters. It restored a matched inference prompt for the comparison.

Results

Evaluation	Cases	Task success	Tool accuracy	Argument accuracy	Clarification wording	Refusal wording
Bare Qwen / matched training prompt	150	32/150 (0.213)	0.493	0.791	0.025	0.034
Format-aligned / matched training prompt	150	40/150 (0.267)	0.453	0.771	0.025	0.017
Boundary pool / 1,000 cap / matched training prompt	150	37/150 (0.247)	0.493	0.738	0.050	0.086
Boundary pool / 1,500 cap / matched training prompt	150	34/150 (0.227)	0.473	0.708	0.013	0.103

Values are ratios from 0 to 1, rounded to three decimals. Clarification/refusal columns measure wording criteria, not routing alone. Different suite sizes remain separate evaluations.

Result Provenance

- Bare Qwen / matched training prompt: 2026-09-26 UTC, 150 cases.
- Format-aligned / matched training prompt: 2026-09-26 UTC, 150 cases.
- Boundary pool / 1,000 cap / matched training prompt: 2026-09-26 UTC, 150 cases.

- Boundary pool / 1,500 cap / matched training prompt: 2026-09-26 UTC, 150 cases.

Interpretation

Aligned recovered to 40/150 successes, compared with 26/150 under policy prompt v1 and 6/150 under v2. Prompt compatibility was a real measurement concern.

Neither boundary-diverse candidate dominated aligned: additional boundary exposure improved some abstention metrics while reducing other task outcomes.

Limits

This comparison does not prove that the old training prompt is the best possible policy. It demonstrates why prompt changes must be tracked independently from training changes.

The historic suite policy and parser later changed. Keep these artifacts as historical evidence, not directly interchangeable with today's strict-parser results.

Supporting Files

The result links above are the surviving primary evidence. A separate immutable training record was not established for this historical experiment.

Notes assembled on 2026-10-10 from saved artifacts and repository records. No training, inference or result modification was performed to write this note.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

One Hundred ACT, CLARIFY and REFUSE Examples

Experiment index

Question and Context

Could a larger unique BAYMAX contribution improve on the fixed 150-example pool without oversampling exact duplicates?

Change

The 1,000-row mixture targeted 300 unique BAYMAX examples: 100 productive ACT, 100 CLARIFY and 100 REFUSE, plus 700 generic xLAM productive examples.

"Balanced BAYMAX" is not "balanced full dataset": the xLAM portion still adds ACT targets, including multi-tool sequences.

Results

Evaluation	Cases	Task success	Tool accuracy	Argument accuracy	Clarification wording	Refusal wording
100/100/100 BAYMAX	150	43/150 (0.287)	0.567	0.761	0.050	0.172
100/100/100 BAYMAX	30	6/30 (0.200)	0.600	0.800	0.200	0.150
100/100/100 BAYMAX / strict replay	150	42/150 (0.280)	0.500	0.694	0.013	0.172
100/100/100 BAYMAX / strict replay	30	6/30 (0.200)	0.533	0.733	0.200	0.100

Values are ratios from 0 to 1, rounded to three decimals. Clarification/refusal columns measure wording criteria, not routing alone. Different suite sizes remain separate evaluations.

Routing and Output Validity

Evaluation	Macro-F1	Invalid-output rate
100/100/100 BAYMAX / strict replay (150 cases)	0.367	0.113

Evaluation	Macro-F1	Invalid-output rate
100/100/100 BAYMAX / strict replay (30 cases)	0.435	0.100

Routing values use the existing comparison helper on recorded per-scenario decisions; missing historical aggregate fields are derived without rewriting source files. Invalid output and unknown decision are not identical categories.

Result Provenance

- 100/100/100 BAYMAX: 2026-09-27 UTC, 150 cases.
- 100/100/100 BAYMAX: 2026-09-27 UTC, 30 cases.
- 100/100/100 BAYMAX / strict replay: 2026-09-28 UTC, 150 cases.
- 100/100/100 BAYMAX / strict replay: 2026-09-28 UTC, 30 cases.

Interpretation

The historical full evaluation recorded 43/150 successes and the shift suite 6/30. Strict-parser replay reduced the full count to 42/150 and some argument/wording metrics.

This was a useful improvement over some early mixes, but the shift suite still exposed poor generalization and output-contract errors.

Limits

The historical and strict-replay rows are not different trained models. Use strict replay when comparing with the other replayed candidates. Old synthetic split and LoRA seed issues were not fixed by rescoring.

Supporting Files

The result links above are the surviving primary evidence. A separate immutable training record was not established for this historical experiment.

Notes assembled on 2026-10-10 from saved artifacts and repository records. No training, inference or result modification was performed to write this note.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Early Balanced BAYMAX Data at Two Epochs

Experiment index

Question and Context

Would longer training help the early 100/100/100 BAYMAX mixture learn its policy?

Change

An early two-epoch SFT run used the same general ACR mixture family. This is not the later cleaned, anchored two-epoch reference.

Results

Evaluation	Cases	Task success	Tool accuracy	Argument accuracy	Clarification wording	Refusal wording
100/100/100 BAYMAX / two epochs	150	46/150 (0.307)	0.570	0.791	0.075	0.138
100/100/100 BAYMAX / two epochs	30	5/30 (0.167)	0.467	0.728	0.250	0.400

Values are ratios from 0 to 1, rounded to three decimals. Clarification/refusal columns measure wording criteria, not routing alone. Different suite sizes remain separate evaluations.

Result Provenance

- 100/100/100 BAYMAX / two epochs: 2026-09-27 UTC, 150 cases.
- 100/100/100 BAYMAX / two epochs: 2026-09-27 UTC, 30 cases.

Interpretation

The historical full-suite count increased from 43 to 46 successes relative to the one-epoch ACR artifact. The template-shift count fell from six to five.

That result did not establish robust generalization from the extra epoch.

Limits

These runs predate later strict-parser and data-integrity corrections. No strict-rescore artifact for this two-epoch candidate is present in the catalog. The current two-epoch config is not sufficient evidence of an immutable historical data identity.

Supporting Files

- qwen2_5_1_5b_lora_baymax_mix_2epoch_v1.json

Notes assembled on 2026-10-10 from saved artifacts and repository records. No training, inference or result modification was performed to write this note.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Schema-Diverse Control Examples

Experiment index

Question and Context

Was the model recognizing authoring templates without learning the tools' real argument schemas?

Change

Diversify synthetic productive examples and schema usage while keeping the broad 100 ACT / 100 CLARIFY / 100 REFUSE BAYMAX contribution.

The goal was transfer across wording and arguments, not rewriting evaluation cases or accepting malformed outputs.

Results

Evaluation	Cases	Task success	Tool accuracy	Argument accuracy	Clarification wording	Refusal wording
Schema-diverse	150	42/150 (0.280)	0.560	0.752	0.075	0.190
Schema-diverse	30	5/30 (0.167)	0.533	0.733	0.300	0.200
Schema-diverse / strict replay	150	40/150 (0.267)	0.480	0.672	0.025	0.172
Schema-diverse / strict replay	30	5/30 (0.167)	0.500	0.700	0.300	0.150

Values are ratios from 0 to 1, rounded to three decimals. Clarification/refusal columns measure wording criteria, not routing alone. Different suite sizes remain separate evaluations.

Routing and Output Validity

Evaluation	Macro-F1	Invalid-output rate
Schema-diverse / strict replay (150 cases)	0.395	0.133
Schema-diverse / strict replay (30 cases)	0.323	0.100

Routing values use the existing comparison helper on recorded per-scenario decisions; missing historical aggregate fields are derived without rewriting source files. Invalid output and unknown decision are not identical categories.

Result Provenance

- Schema-diverse: 2026-09-27 UTC, 150 cases.
- Schema-diverse: 2026-09-27 UTC, 30 cases.
- Schema-diverse / strict replay: 2026-09-28 UTC, 150 cases.
- Schema-diverse / strict replay: 2026-09-28 UTC, 30 cases.

Interpretation

Historical full-suite success was 42/150; strict replay measured 40/150. Shift success was 5/30.

Higher boundary wording scores than some ACT-heavy variants did not make this checkpoint a universally better model. Correct routing, valid output and correct arguments remain separate capabilities.

Limits

A template-shift argument mismatch can be a genuine schema error, not a formatting false negative. Scorer changes require independent justification.

The old split/initialization caveats remain. Today's generator implementation cannot by itself recover the exact historical selection.

Supporting Files

The result links above are the surviving primary evidence. A separate immutable training record was not established for this historical experiment.

Notes assembled on 2026-10-10 from saved artifacts and repository records. No training, inference or result modification was performed to write this note.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Increasing Productive BAYMAX Examples to Two Hundred

Experiment index

Question and Context

Could more domain-specific productive examples recover task completion without adding more clarification/refusal rows?

Change

The historical ACT-balanced mixture selected 400 BAYMAX rows: 200 ACT, 100 CLARIFY and 100 REFUSE. The remaining 600 raw slots came from xLAM.

This is the old ACT-balanced model, not generator14 ACT200 or the cleaned anchored checkpoint. The similar class counts do not make the data revisions identical.

Results

Evaluation	Cases	Task success	Tool accuracy	Argument accuracy	Clarification wording	Refusal wording
Historical ACT200 / legacy metrics	150	47/150 (0.313)	0.553	0.805	0.025	0.086
Historical ACT200 / decision metrics	150	47/150 (0.313)	0.553	0.805	0.025	0.086
Historical ACT200 / legacy metrics	30	6/30 (0.200)	0.533	0.828	0.200	0.100
Historical ACT200 / decision metrics	30	6/30 (0.200)	0.533	0.828	0.200	0.100
Historical ACT200 / strict replay	150	46/150 (0.307)	0.493	0.738	0.000	0.086
Historical ACT200 / strict replay	30	6/30 (0.200)	0.500	0.794	0.200	0.100

Values are ratios from 0 to 1, rounded to three decimals. Clarification/refusal columns measure wording criteria, not routing alone. Different suite sizes remain separate evaluations.

Routing and Output Validity

Evaluation	Macro-F1	Invalid-output rate
Historical ACT200 / decision metrics (150 cases)	0.387	0.000
Historical ACT200 / decision metrics (30 cases)	0.397	0.000
Historical ACT200 / strict replay (150 cases)	0.349	0.120
Historical ACT200 / strict replay (30 cases)	0.368	0.067

Routing values use the existing comparison helper on recorded per-scenario decisions; missing historical aggregate fields are derived without rewriting source files. Invalid output and unknown decision are not identical categories.

Result Provenance

- Historical ACT200 / legacy metrics: 2026-09-27 UTC, 150 cases.
- Historical ACT200 / decision metrics: 2026-09-27 UTC, 150 cases.
- Historical ACT200 / legacy metrics: 2026-09-27 UTC, 30 cases.
- Historical ACT200 / decision metrics: 2026-09-27 UTC, 30 cases.
- Historical ACT200 / strict replay: 2026-09-28 UTC, 150 cases.
- Historical ACT200 / strict replay: 2026-09-28 UTC, 30 cases.

Interpretation

The historical evaluation recorded 47/150 successes; strict replay recorded 46/150. Template-shift success remained 6/30.

The increase in productive exposure improved some established-suite outcomes but also increased premature actions. It was not a complete improvement in decision policy.

Limits

The "decision-metrics" evaluation added explicit decision reporting but did not create a new trained model. The "legacy-metrics" file remains preserved.

Strict replay fixes measurement only, not the old training partitions or seed initialization. Later ACT200 runs are not single-variable comparisons with this checkpoint.

Supporting Files

The result links above are the surviving primary evidence. A separate immutable training record was not established for this historical experiment.

Notes assembled on 2026-10-10 from saved artifacts and repository records. No training, inference or result modification was performed to write this note.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Productive Midpoint: One Hundred Fifty BAYMAX ACT Examples

Experiment index

Question and Context

Could a midpoint between 100 and 200 authored ACT examples retain productive accuracy without the same premature-action pressure?

Change

The historical ACT-midpoint candidate reduced the BAYMAX productive quota to 150 while retaining the boundary-training approach. It was evaluated with decision metadata and later strict replay.

Results

Evaluation	Cases	Task success	Tool accuracy	Argument accuracy	Clarification wording	Refusal wording
Historical ACT150 / decision metrics	150	46/150 (0.307)	0.520	0.804	0.025	0.052
Historical ACT150 / decision metrics	30	6/30 (0.200)	0.533	0.828	0.200	0.050
Historical ACT150 / strict replay	150	46/150 (0.307)	0.480	0.764	0.025	0.052
Historical ACT150 / strict replay	30	6/30 (0.200)	0.467	0.761	0.200	0.000

Values are ratios from 0 to 1, rounded to three decimals. Clarification/refusal columns measure wording criteria, not routing alone. Different suite sizes remain separate evaluations.

Routing and Output Validity

Evaluation	Macro-F1	Invalid-output rate
Historical ACT150 / decision metrics (150 cases)	0.330	0.000

Evaluation	Macro-F1	Invalid-output rate
Historical ACT150 / decision metrics (30 cases)	0.343	0.000
Historical ACT150 / strict replay (150 cases)	0.308	0.107
Historical ACT150 / strict replay (30 cases)	0.321	0.100

Routing values use the existing comparison helper on recorded per-scenario decisions; missing historical aggregate fields are derived without rewriting source files. Invalid output and unknown decision are not identical categories.

Result Provenance

- Historical ACT150 / decision metrics: 2026-09-27 UTC, 150 cases.
- Historical ACT150 / decision metrics: 2026-09-27 UTC, 30 cases.
- Historical ACT150 / strict replay: 2026-09-28 UTC, 150 cases.
- Historical ACT150 / strict replay: 2026-09-28 UTC, 30 cases.

Interpretation

Full-suite success was 46/150 before and after replay; shift success was 6/30. Other metrics changed under strict parsing even though task-success totals stayed the same.

The result did not establish a clean win over ACT-balanced.

Limits

Matching success totals does not imply matching outputs or error types. The surviving historical result is not a complete frozen training manifest. Earlier seed/split defects still apply.

Supporting Files

The result links above are the surviving primary evidence. A separate immutable training record was not established for this historical experiment.

Notes assembled on 2026-10-10 from saved artifacts and repository records. No training, inference or result modification was performed to write this note.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Content-Group Splits With Seventy Percent ACT

Experiment index

Question and Context

Earlier training partitions could share duplicated content or related contrast examples. New content-group splitting protected train/validation/test separation.

The owner also tried a more boundary-heavy mixture. Therefore this was not only a split-fix experiment.

Setup

- Generator12 synthetic data and content-grouped external data.
- Raw training cap 1,000: 150 BAYMAX ACT, 150 CLARIFY, 150 REFUSE and 550 xLAM ACT.
- Overall raw labels: 700 ACT / 150 CLARIFY / 150 REFUSE.
- Reported usable train artifact: 983 rows; direct 366, multi 317, refusal 150, clarification 150; BAYMAX 450, xLAM 533; zero exact duplicates.
- The 200-row validation prefix had almost no boundary data: one refusal and no clarification survived.

Results

Evaluation	Cases	Task success	Tool accuracy	Argument accuracy	Clarification wording	Refusal wording
Content-group 70/30	150	26/150 (0.173)	0.427	0.579	0.113	0.207

Values are ratios from 0 to 1, rounded to three decimals. Clarification/refusal columns measure wording criteria, not routing alone. Different suite sizes remain separate evaluations.

Routing and Output Validity

Evaluation	Macro-F1	Invalid-output rate
Content-group 70/30 (150 cases)	0.346	0.147

Routing values use the existing comparison helper on recorded per-scenario decisions; missing historical aggregate fields are derived without rewriting source files. Invalid output and unknown decision are not identical categories.

Result Provenance

- Content-group 70/30: 2026-10-05 UTC, 150 cases.

Interpretation

The run completed 26/150 tasks. It improved some boundary wording metrics relative to ACT-heavy history, but productive performance was much worse.

Fixing leakage is essential for trustworthy methodology; it does not guarantee higher benchmark scores. This run cannot isolate whether splitting, composition or other data revisions explain the regression.

Limits

Validation loss on this mostly-ACT prefix was a poor monitor of boundary behavior. This generator predates subsequent adapter and policy corrections.

Supporting Files

- qwen2_5_1_5b_lora_content_groups_70_30_v12.json
- TRAINING_DATA_AUDIT.md

Notes assembled on 2026-10-10 from saved artifacts and repository records. No training, inference or result modification was performed to write this note.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Corrected Policy, Credential Filtering and Representative Validation

Experiment index

Question and Context

The synthetic source had adapter-state defects and unresolved policy contradictions. External examples sometimes required model-emitted credentials. The small validation cap barely represented boundary decisions.

Change

Use corrected generator14 BAYMAX examples and credential-filtered xLAM. Resolved policy includes refusing indiscriminate outreach, not substituting drafts for unsupported sends, and allowing drafts composed from supplied facts.

External credential filtering excluded 618 rows from the original 60,000. Source data was preserved in a separate artifact.

Keep the boundary-heavy raw training recipe: 150 BAYMAX ACT / 150 CLARIFY / 150 REFUSE plus 550 xLAM ACT. A representative validation prefix selects 30 examples per BAYMAX decision plus 110 xLAM, from validation only.

Actual Artifact

Reported train: 976 usable rows, 374 direct, 302 multi-tool, 150 refusal and 150 clarification; BAYMAX 450 and xLAM 526; no exact duplicates.

Reported representative validation: 194 usable rows, 77 direct, 57 multi-tool, 30 refusal and 30 clarification; BAYMAX 90 and xLAM 104.

Results

Evaluation	Cases	Task success	Tool accuracy	Argument accuracy	Clarification wording	Refusal wording
Corrected 70/30	150	25/150 (0.167)	0.433	0.582	0.125	0.190

Values are ratios from 0 to 1, rounded to three decimals. Clarification/refusal columns measure wording criteria, not routing alone. Different suite sizes remain separate evaluations.

Routing and Output Validity

Evaluation	Macro-F1	Invalid-output rate
Corrected 70/30 (150 cases)	0.372	0.113

Routing values use the existing comparison helper on recorded per-scenario decisions; missing historical aggregate fields are derived without rewriting source files. Invalid output and unknown decision are not identical categories.

Result Provenance

- Corrected 70/30: 2026-10-05 UTC, 150 cases.

Interpretation

Success was 25/150. Relative to generator12's 26/150, there was no clear overall improvement. Necessary correctness and security fixes should not be described as an optimization win.

Representative validation improved what loss could monitor. The trainer did not use early stopping or best-checkpoint selection, so changing this validation slice alone did not directly optimize the final weights.

Limits

Several source and validation changes occurred together. Validation losses from the sparse and representative prefixes are not directly comparable. Credential filtering did not remove all target-contract problems.

Supporting Files

- qwen2_5_1_5b_lora_auth_filtered_v14_val200.json
- TRAINING_DATA_AUDIT.md

Notes assembled on 2026-10-10 from saved artifacts and repository records. No training, inference or result modification was performed to write this note.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Corrected Policy With Two Hundred Productive Examples

Experiment index

Question and Context

Would the productive-heavy 200/100/100 BAYMAX recipe work better with the corrected policy and credential-filtered external source?

Change

Use the same source family as the previous boundary-heavy run, but cap BAYMAX at 400: 200 ACT, 100 CLARIFY and 100 REFUSE. Fill 600 raw slots with unique xLAM ACT examples.

Keep a representative 200-row validation cap, the base model, one epoch, seed, prompt and LoRA settings. This is the "original ACT200" control used by subsequent cleaning comparisons.

Results

Evaluation	Cases	Task success	Tool accuracy	Argument accuracy	Clarification wording	Refusal wording
Corrected ACT200	150	38/150 (0.253)	0.473	0.675	0.062	0.121

Values are ratios from 0 to 1, rounded to three decimals. Clarification/refusal columns measure wording criteria, not routing alone. Different suite sizes remain separate evaluations.

Routing and Output Validity

Evaluation	Macro-F1	Invalid-output rate
Corrected ACT200 (150 cases)	0.396	0.133

Routing values use the existing comparison helper on recorded per-scenario decisions; missing historical aggregate fields are derived without rewriting source files. Invalid output and unknown decision are not identical categories.

Result Provenance

- Corrected ACT200: 2026-10-05 UTC, 150 cases.

Interpretation

Success increased from 25/150 to 38/150; macro-F1 was 0.396. That recovered productive performance but reduced some boundary wording metrics.

The comparison supports a mixture tradeoff, not a claim that more ACT always helps.

Limits

It is not the same source as historical ACT-balanced, even though the broad quotas match. The measured 20 invalid outputs also motivated validating training targets separately from changing decision proportions.

Supporting Files

- qwen2_5_1_5b_lora_auth_filtered_v14_act200_val200.json

Notes assembled on 2026-10-10 from saved artifacts and repository records. No training, inference or result modification was performed to write this note.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Target-Contract Cleaning With Resampled External Rows

Experiment index

Question and Context

The new strict training gate found target/schema failures in xLAM. Would excluding incompatible examples help?

Change

Build a target-compatible external source: 57,611 accepted rows after 618 credential exclusions and 1,771 additional target/credential exclusions, for 2,389 total skipped rows.

Retain the generator14 ACT200 recipe, but rebuild the mixed sampler. Removing rows changes which xLAM examples survive the cap; this is the cleaned-resampled candidate.

Actual Artifact

Raw cap 1,000; usable train 977: direct 447, multi 330, refusal 100, clarification 100. BAYMAX 400, xLAM 577, zero exact duplicates.

Usable validation 196: direct 72, multi 64, refusal 30, clarification 30.

Results

Evaluation	Cases	Task success	Tool accuracy	Argument accuracy	Clarification wording	Refusal wording
Cleaned / resampled	150	47/150 (0.313)	0.493	0.761	0.013	0.086

Values are ratios from 0 to 1, rounded to three decimals. Clarification/refusal columns measure wording criteria, not routing alone. Different suite sizes remain separate evaluations.

Routing and Output Validity

Evaluation	Macro-F1	Invalid-output rate
Cleaned / resampled (150 cases)	0.354	0.073

Routing values use the existing comparison helper on recorded per-scenario decisions; missing historical aggregate fields are derived without rewriting source files. Invalid output and unknown decision are not identical categories.

Decision Matrix: Cleaned / resampled (150 Cases)

Rows are expected decisions; columns are recorded predictions. The diagonal is correct routing, not necessarily full task success.

Expected	ACT	CLARIFY	REFUSE	Invalid/unknown
ACT	59	15	2	5
CLARIFY	28	3	5	4
REFUSE	17	2	7	3

Result Provenance

- Cleaned / resampled: 2026-10-06 UTC, 150 cases.

Interpretation

Success reached 47/150, versus 38/150 for original ACT200. Invalid outputs fell from 20 to 11, but routing macro-F1 fell from 0.396 to 0.354.

The model completed more tasks while handling abstention worse. "Cleaning improved reasoning" would overstate the evidence.

Limits

A recorded content-hash comparison found only 7/600 shared xLAM rows and 301/400 shared BAYMAX rows versus the original raw cap. Even the unchanged BAYMAX source yielded different selected examples. This does not isolate removal of faulty examples. A full-pool exclusion rate of about three percent can still change a small capped artifact substantially. The anchored comparison was created to address that confound.

Supporting Files

- qwen2_5_1_5b_lora_target_compatible_v14_act200_val200.json
- TRAINING_DATA_AUDIT.md

Notes assembled on 2026-10-10 from saved artifacts and repository records. No training, inference or result modification was performed to write this note.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Target-Contract Cleaning With Anchored External Rows

Experiment index

Question and Context

Did the earlier cleaned model change because of better targets, or because rebuilding the mixture selected different external examples?

Change

Anchor selection to original ACT200. Preserve all 400 capped BAYMAX rows and retain 582 of the original 600 xLAM rows; replace only the 18 excluded xLAM rows with compatible same-split rows.

Validation retained 90 BAYMAX and 107 xLAM rows, replacing three xLAM examples. Train/validation/test membership remains protected. The raw recipe stays 200 BAYMAX ACT / 100 CLARIFY / 100 REFUSE plus 600 xLAM.

Actual Artifact and Recipe

One fresh SFT epoch, base Qwen2.5-1.5B-Instruct, seed 1, LoRA rank 8/alpha16 on q_proj and v_proj, dropout 0.05. Learning rate 2e-4, cosine scheduler, warmup fraction 0.03.

Owner low-VRAM overrides: context 1,024, batch one, accumulation 16, raw train/eval caps 1,000/200.

Usable train: 974 rows, direct 435, multi 339, refusal 100, clarification 100; BAYMAX 400, xLAM 574, no exact duplicates. Validation retained 195.

Results

Evaluation	Cases	Task success	Tool accuracy	Argument accuracy	Clarification wording	Refusal wording
Anchored / one epoch	150	39/150 (0.260)	0.500	0.708	0.050	0.155

Values are ratios from 0 to 1, rounded to three decimals. Clarification/refusal columns measure wording criteria, not routing alone. Different suite sizes remain separate evaluations.

Routing and Output Validity

Evaluation	Macro-F1	Invalid-output rate
Anchored / one epoch (150 cases)	0.429	0.080

Routing values use the existing comparison helper on recorded per-scenario decisions; missing historical aggregate fields are derived without rewriting source files. Invalid output and unknown decision are not identical categories.

Decision Matrix: Anchored / one epoch (150 Cases)

Rows are expected decisions; columns are recorded predictions. The diagonal is correct routing, not necessarily full task success.

Expected	ACT	CLARIFY	REFUSE	Invalid/unknown
ACT	50	22	3	6
CLARIFY	24	7	5	4
REFUSE	12	2	12	3

Result Provenance

- Anchored / one epoch: 2026-10-06 UTC, 150 cases.

Interpretation

Success was 39/150, macro-F1 0.429 and invalid outputs 12/150. Compared with resampling's 47 successes but 0.354 macro-F1, anchoring recovered more balanced routing but not its task total.

This became the data reference for the epoch-length trials.

Limits

Replacing 18 examples also changes token lengths and usable counts. This is a much tighter comparison, not perfectly identical exposure. Single-seed measurements do not establish a statistically reliable causal effect.

Supporting Files

- qwen2_5_1_5b_lora_target_compatible_v14_act200_anchored_val200.json

Notes assembled on 2026-10-10 from saved artifacts and repository records. No training, inference or result modification was performed to write this note.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Anchored Data at Two Epochs: Selected Reference

Experiment index

Question and Context

Was one pass over the cleaned anchored mixture insufficient to learn its targets?

Controlled Change

Increase fresh SFT from one to two total epochs. Keep the selected data, prompt, seed, raw caps and LoRA settings fixed. A longer run also stretches the cosine schedule and warmup in steps, so this is not an extra pass at the exact previous per-step learning rates.

Actual Training Recipe

- Fresh base Qwen/Qwen2.5-1.5B-Instruct plus fresh LoRA; not a resumed adapter.
- LoRA rank 8, alpha 16, q_proj/v_proj, dropout 0.05, bias none.
- Learning rate 2e-4, cosine schedule, warmup fraction 0.03, seed 1.
- Owner overrides: context 1,024, train/eval batch one, accumulation 16, raw caps 1,000/200.
- Assistant-only SFT token cross-entropy; prompt/padding tokens do not provide target loss.
- Recorded training/inference system prompt hash a31446de6c8b67f1fc8ba8fc09ed8e2d7425ba832334dc8fd6fe2d55ef978468.

The same usable artifact retained 974 train rows: 435 direct, 339 multi-tool, 100 clarification and 100 refusal. BAYMAX contributed 400 and xLAM 574. Validation retained 195. There were zero exact duplicates.

The trainer completed 122 optimizer steps.

Results

Evaluation	Cases	Task success	Tool accuracy	Argument accuracy	Clarification wording	Refusal wording
Anchored / two epochs	150	44/150 (0.293)	0.600	0.771	0.075	0.172
Anchored / two epochs	30	5/30 (0.167)	0.433	0.694	0.250	0.250

Values are ratios from 0 to 1, rounded to three decimals. Clarification/refusal columns measure wording criteria, not routing alone. Different suite sizes remain separate evaluations.

Routing and Output Validity

Evaluation	Macro-F1	Invalid-output rate
Anchored / two epochs (150 cases)	0.514	0.013
Anchored / two epochs (30 cases)	0.345	0.067

Routing values use the existing comparison helper on recorded per-scenario decisions; missing historical aggregate fields are derived without rewriting source files. Invalid output and unknown decision are not identical categories.

Decision Matrix: Anchored / two epochs (150 Cases)

Rows are expected decisions; columns are recorded predictions. The diagonal is correct routing, not necessarily full task success.

Expected	ACT	CLARIFY	REFUSE	Invalid/unknown
ACT	57	21	3	0
CLARIFY	20	11	9	0
REFUSE	11	0	16	2

Decision Matrix: Anchored / two epochs (30 Cases)

Rows are expected decisions; columns are recorded predictions. The diagonal is correct routing, not necessarily full task success.

Expected	ACT	CLARIFY	REFUSE	Invalid/unknown
ACT	3	6	1	0
CLARIFY	5	2	3	0
REFUSE	3	0	5	2

Result Provenance

- Anchored / two epochs: 2026-10-06 UTC, 150 cases.
- Anchored / two epochs: 2026-10-07 UTC, 30 cases.

Interpretation

Combined success increased from 39 to 44/150, macro-F1 from 0.429 to 0.514, and invalid outputs fell from 12 to two. Premature productive decisions on boundary requests fell from 36 to 31.

This is the selected development reference because it balances routing and validity better than the later candidates, not because it is a solved assistant. Shift success was only 5/30, with eight premature ACT decisions on the twenty boundary cases.

Reproducibility

The linked frozen reference records data revision, config/prompt hashes, capped raw-input hashes, adapter identity and both evaluations. "Frozen" means identities recorded for drift detection, not permission-locked files.

Dataset revision: `329fab4d8f1604a4085d3ec50e05881e7c069717bb1428db7e592743bb07f03c.`

Canonical training config hash: `ce9efa2db24a065dbdb4eb482c07a29a56bfbcbdce32495418eb54b92d985c40.`

Limits

Exact base-model revision was not recorded. Some inference settings come from owner commands rather than embedded result metadata. Results are single-seed, repeatedly inspected development evidence. An earlier bounded audit found near-duplicate training/evaluation wording; the benchmark is not an untouched final test.

Supporting Files

- qwen2_5_1_5b_lora_target_compatible_v14_act200_anchored_val200_epochs2.json
- anchored-epochs2-reference.json
- anchored-epochs2-vs4-failure-audit.md

Notes assembled on 2026-10-10 from saved artifacts and repository records. No training, inference or result modification was performed to write this note.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Anchored Data at Four Epochs

Experiment index

Question and Context

Would substantially longer training improve the two-epoch reference, or simply strengthen the wrong decision patterns?

Controlled Change

Train a fresh adapter for four total epochs on the same anchored data. This was not continuing from the saved two-epoch adapter.

- Fresh base Qwen/Qwen2.5-1.5B-Instruct plus fresh LoRA; not a resumed adapter.
- LoRA rank 8, alpha 16, q_proj/v_proj, dropout 0.05, bias none.
- Learning rate 2e-4, cosine schedule, warmup fraction 0.03, seed 1.
- Owner overrides: context 1,024, train/eval batch one, accumulation 16, raw caps 1,000/200.
- Assistant-only SFT token cross-entropy; prompt/padding tokens do not provide target loss.
- Recorded training/inference system prompt hash a31446de6c8b67f1fc8ba8fc09ed8e2d7425ba832334dc8fd6fe2d55ef978468.

The usable training/validation counts stayed 974/195, with the same broad class counts and no exact duplicates. The learning-rate schedule now spans four epochs.

Results

Evaluation	Cases	Task success	Tool accuracy	Argument accuracy	Clarification wording	Refusal wording
Anchored / four epochs	150	37/150 (0.247)	0.440	0.628	0.263	0.103
Anchored / four epochs	30	4/30 (0.133)	0.400	0.528	0.300	0.050

Values are ratios from 0 to 1, rounded to three decimals. Clarification/refusal columns measure wording criteria, not routing alone. Different suite sizes remain separate evaluations.

Routing and Output Validity

Evaluation	Macro-F1	Invalid-output rate
Anchored / four epochs (150 cases)	0.372	0.087

Evaluation	Macro-F1	Invalid-output rate
Anchored / four epochs (30 cases)	0.348	0.200

Routing values use the existing comparison helper on recorded per-scenario decisions; missing historical aggregate fields are derived without rewriting source files. Invalid output and unknown decision are not identical categories.

Decision Matrix: Anchored / four epochs (150 Cases)

Rows are expected decisions; columns are recorded predictions. The diagonal is correct routing, not necessarily full task success.

Expected	ACT	CLARIFY	REFUSE	Invalid/unknown
ACT	39	36	1	5
CLARIFY	14	19	4	3
REFUSE	17	3	4	5

Decision Matrix: Anchored / four epochs (30 Cases)

Rows are expected decisions; columns are recorded predictions. The diagonal is correct routing, not necessarily full task success.

Expected	ACT	CLARIFY	REFUSE	Invalid/unknown
ACT	2	7	1	0
CLARIFY	1	8	1	0
REFUSE	3	0	1	6

Result Provenance

- Anchored / four epochs: 2026-10-07 UTC, 150 cases.
- Anchored / four epochs: 2026-10-07 UTC, 30 cases.

Interpretation

Compared with two epochs, combined task successes fell from 44 to 37, macro-F1 from 0.514 to 0.372, and invalid outputs increased from two to thirteen.

Correct CLARIFY routing rose from 11/40 to 19/40, but correct ACT fell from 57/81 to 39/81 and correct REFUSE from 16/29 to 4/29. Higher clarification wording scores therefore concealed a harmful routing tradeoff.

Shift success also fell, from five to four, and invalid outputs rose from two to six. Its macro-F1 was nearly unchanged, but the class-specific behavior changed substantially.

Why Lower Loss Was Not Enough

Validation token loss fell to 0.2364 from the one-epoch run's 0.3107. That objective rewards the supplied answer tokens; it does not directly optimize routing macro-F1, execution success or semantic explanations.

The loss reduction does not prove useful generalization, nor does the regression prove memorization. It does show that more epochs are not a reliable fix under this recipe.

Decision and Limits

Keep two epochs as the reference. The saved audit separates wrong decisions, wording-score mismatch, argument errors and invalid output without repairing results.

This was one seed, and changing epochs changes the schedule horizon. Do not generalize it to every learning rate, dataset or local model.

Supporting Files

- qwen2_5_1_5b_lora_target_compatible_v14_act200_anchored_val200_epochs4.json
- anchored-epochs2-vs4-failure-audit.md
- anchored-epochs2-vs4-failure-audit.json

Notes assembled on 2026-10-10 from saved artifacts and repository records. No training, inference or result modification was performed to write this note.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Matched Contrasts at Fixed Class Counts

Experiment index

Question and Context

Could tighter ACT/CLARIFY/REFUSE contrasts teach the decision boundary better than isolated examples, without changing broad class quantities?

A useful contrast changes missing information, state or capability while keeping the surrounding task similar. It should teach the relevant distinction, not a keyword shortcut.

Controlled Change

Start from the anchored two-epoch recipe. Preserve all 600 raw xLAM positions and validation/test files byte-for-byte. Replace 398 of 400 BAYMAX training rows; keep two identical.

Raw quantities remain 200 BAYMAX ACT / 100 CLARIFY / 100 REFUSE. Selection prioritized complete training-only contrast groups: 120 groups, comprising 73 triplets and 47 pairs. Held-out overlap checks remained enforced.

Usable rows stayed 974, including all 400 BAYMAX examples. No exact duplicates were present. Training settings and prompt matched the two-epoch reference.

Results

Evaluation	Cases	Task success	Tool accuracy	Argument accuracy	Clarification wording	Refusal wording
Matched contrasts / two epochs	150	39/150 (0.260)	0.480	0.698	0.087	0.052

Values are ratios from 0 to 1, rounded to three decimals. Clarification/refusal columns measure wording criteria, not routing alone. Different suite sizes remain separate evaluations.

Routing and Output Validity

Evaluation	Macro-F1	Invalid-output rate
Matched contrasts / two epochs (150 cases)	0.383	0.053

Routing values use the existing comparison helper on recorded per-scenario decisions; missing historical aggregate fields are derived without rewriting source files. Invalid output and unknown decision are not identical categories.

Decision Matrix: Matched contrasts / two epochs (150 Cases)

Rows are expected decisions; columns are recorded predictions. The diagonal is correct routing, not necessarily full task success.

Expected	ACT	CLARIFY	REFUSE	Invalid/unknown
ACT	48	29	2	2
CLARIFY	20	14	6	0
REFUSE	16	2	5	6

Result Provenance

- Matched contrasts / two epochs: 2026-10-07 UTC, 150 cases.

Interpretation

Task successes fell from 44 to 39/150, macro-F1 from 0.514 to 0.383, and invalid outputs rose from two to eight. Correct refusal routing fell from 16/29 to 5/29.

The separate audit found real capability/routing errors as well as missing tool names and wrong meta-tool fields. This was not merely keyword scoring rejecting semantically correct refusals.

Important Confound

Broad class counts stayed fixed, but within-class coverage did not. Refusal examples from triplet origins increased from 33 to 73; standalone and capability-pair origins disappeared from the cap. Source-ID family count fell from 45 to 20.

Those are authoring lineages, not a complete semantic taxonomy. The plausible hypothesis is narrower coverage, not a demonstrated causal explanation.

Decision and Limits

Retain the original anchored two-epoch reference. This matched implementation regressed; it does not disprove contrastive training in general. A future contrast experiment would need to preserve semantic/domain coverage as well as label counts.

The earlier auth-filtered matched preflight was blocked by xLAM target failures; it was a different preparation, not a completed precursor checkpoint.

Supporting Files

- qwen2_5_1_5b_lora_target_compatible_v14_act200_matched_anchored_val200_epochs2.json
- matched-contrasts-epochs2-refusal-format-audit.md
- matched-contrasts-epochs2-refusal-format-audit.json

Notes assembled on 2026-10-10 from saved artifacts and repository records. No training, inference or result modification was performed to write this note.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Two-to-One Productive-to-Boundary SFT Ratio

Experiment index

Question and Context

Would a more boundary-heavy mixture help, following the owner's discussion of a 2:1 productive/non-productive ratio?

This is an empirical BAYMAX ratio test. It is not evidence that a paper's ratio transfers unchanged to this model and dataset.

Change

Keep the original anchored two-epoch BAYMAX pool, add 64 unique clarifications and 70 unique refusals, and replace 134 capped xLAM positions. Preserve validation/test bytes and other training settings.

Raw training cap: 666 ACT / 164 CLARIFY / 170 REFUSE, rather than the reference's 800 ACT / 100 CLARIFY / 100 REFUSE. ACT includes both xLAM and productive BAYMAX examples.

Actual Artifact

Class	Usable rows	Trainable tokens
Direct productive call	372	11,109
Multi-tool sequence	272	14,062
CLARIFY	164	4,593
REFUSE	170	5,022

978 usable rows: BAYMAX 534, xLAM 444; no exact duplicates. The effective ACT:boundary row ratio is 644:334, approximately 1.93:1. Boundary tokens are 27.6% of target tokens, not one-third.

The initially proposed 167/167 allocation failed unique-input coverage. The corrected quota preserved the boundary total without repeating examples or borrowing held-out data.

Results

Evaluation	Cases	Task success	Tool accuracy	Argument accuracy	Clarification wording	Refusal wording
2:1 SFT / two epochs	150	29/150 (0.193)	0.487	0.623	0.125	0.259

Values are ratios from 0 to 1, rounded to three decimals. Clarification/refusal columns measure wording criteria, not routing alone. Different suite sizes remain separate evaluations.

Routing and Output Validity

Evaluation	Macro-F1	Invalid-output rate
2:1 SFT / two epochs (150 cases)	0.477	0.053

Routing values use the existing comparison helper on recorded per-scenario decisions; missing historical aggregate fields are derived without rewriting source files. Invalid output and unknown decision are not identical categories.

Decision Matrix: 2:1 SFT / two epochs (150 Cases)

Rows are expected decisions; columns are recorded predictions. The diagonal is correct routing, not necessarily full task success.

Expected	ACT	CLARIFY	REFUSE	Invalid/unknown
ACT	31	38	6	6
CLARIFY	14	17	8	1
REFUSE	8	1	19	1

Result Provenance

- 2:1 SFT / two epochs: 2026-10-09 UTC, 150 cases.

Interpretation

Success fell from 44/150 to 29/150 and macro-F1 from 0.514 to 0.477. Premature actions fell from 31 to 22, but correct ACT routing fell from 57/81 to 31/81; ACT-to-CLARIFY errors rose from 21 to 38.

The mixture improved some boundary recall and wording scores at a large productive-task cost. It was not a balanced win.

Limits

Changing the ratio also changes examples, source proportions, token exposure and optimizer steps. This does not isolate class weights alone. The checkpoint remains useful as an alternative starting point for the next DPO comparison.

Supporting Files

- qwen2_5_1_5b_lora_target_compatible_v14_2to1_anchored_val200_epochs2.json
- prepare_sft_2to1.py

Notes assembled on 2026-10-10 from saved artifacts and repository records. No training, inference or result modification was performed to write this note.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

DPO From the Selected Two-Epoch SFT Reference

Experiment index

Question and Context

Could preferences between correct and incorrect decisions sharpen routing after SFT, without changing the model or evaluator?

DPO trains relative preference for a chosen answer over a rejected answer to the same prompt. It is not an LLM judge reviewing benchmark results.

Training Recipe

- Start policy and frozen reference from the specified SFT adapter, not bare Qwen.
- One DPO epoch, sigmoid preference loss, beta 0.1, learning rate 5e-6, constant scheduler.
- Batch one, accumulation 16, context 1,024, seed 1; inherited LoRA rank/targets.
- TRL 1.14.0 ordinary PyTorch full-logits path, with reference log probabilities precomputed.
- Chosen/rejected answers share the same prompt; no JSON repair or relaxed evaluator.
- Rule-derived training preferences plus owner review, not LLM-judge labels.
- No DPO validation preference split was constructed; evaluation used the existing development suite.

Preference Artifact

400 unique BAYMAX training anchors: 200 ACT, 100 CLARIFY, 100 REFUSE. xLAM rows were not used as preference anchors. There were 705 pairs because an anchor can have multiple distinct rejections.

Chosen over rejected	Pairs
ACT > CLARIFY	200
ACT > REFUSE	200
CLARIFY > ACT	50
CLARIFY > REFUSE	100
REFUSE > ACT	55
REFUSE > CLARIFY	100

50 clarification and 45 refusal anchors lacked a suitable productive rejection. Missing directions were reported rather than filled with arbitrary tool calls.

Actual artifact: 23,008 chosen completion tokens, 20,981 rejected tokens, longest sequence 633, zero truncated pairs. The Windows smoke established runnable training, not improved capability.

Recorded Preference Artifact Summary

Artifact measurement	Recorded value
Preference pairs	705
Unique training prompts	400
Chosen completion tokens	23,008
Rejected completion tokens	20,981
Longest complete sequence (tokens)	633
Truncated pairs	0

Preferred decision	Pair exposure
ACT	400
CLARIFY	150
REFUSE	155

Pair exposure counts comparisons, not unique anchors. One prompt can contribute multiple distinct rejections.

Source: recorded DPO run. This export-only table transcribes recorded metadata; it does not change the source note or any training artifact.

Results

Evaluation	Cases	Task success	Tool accuracy	Argument accuracy	Clarification wording	Refusal wording
DPO / original reference	150	53/150 (0.353)	0.533	0.856	0.013	0.052

Values are ratios from 0 to 1, rounded to three decimals. Clarification/refusal columns measure wording criteria, not routing alone. Different suite sizes remain separate evaluations.

Routing and Output Validity

Evaluation	Macro-F1	Invalid-output rate
DPO / original reference (150 cases)	0.376	0.033

Routing values use the existing comparison helper on recorded per-scenario decisions; missing historical aggregate fields are derived without rewriting source files. Invalid output and unknown decision are not identical categories.

Decision Matrix: DPO / original reference (150 Cases)

Rows are expected decisions; columns are recorded predictions. The diagonal is correct routing, not necessarily full task success.

Expected	ACT	CLARIFY	REFUSE	Invalid/unknown
ACT	72	5	1	3
CLARIFY	35	4	1	0
REFUSE	21	1	5	2

Result Provenance

- DPO / original reference: 2026-10-09 UTC, 150 cases.

Interpretation

Task successes increased from SFT's 44 to 53/150, but macro-F1 fell from 0.514 to 0.376. Premature ACT decisions on boundary cases rose from 31 to 56. Correct refusal routing fell from 16 to five, and correct clarification from eleven to four.

This run favored productive action and argument accuracy. It was a task-completion gain paired with a substantial boundary regression, not a general improvement.

Limits

Preference directions were incomplete and asymmetric. That is a plausible contributor, not proven causation. Pair counts are not unique-prompt counts. Results are single-seed and repeatedly evaluated.

The initial Windows failure was a TRL/Triton compatibility problem, resolved with the ordinary PyTorch path; it did not change preference labels or the DPO objective.

Supporting Files

- qwen2_5_1_5b_anchored_epochs2_v1.json
- dpo-run.json
- DPO_EXPERIMENT.md

Notes assembled on 2026-10-10 from saved artifacts and repository records. No training, inference or result modification was performed to write this note.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

DPO From the Two-to-One SFT Model

Experiment index

Question and Context

Could the more cautious 2:1 SFT checkpoint resist the ACT bias seen after DPO on the original reference?

Controlled Change

Change the initial policy and matching frozen reference to the 2:1 SFT adapter. Keep the exact original 705 preference pairs, prompt, DPO settings and evaluator unchanged.

- Start policy and frozen reference from the specified SFT adapter, not bare Qwen.
- One DPO epoch, sigmoid preference loss, beta 0.1, learning rate 5e-6, constant scheduler.
- Batch one, accumulation 16, context 1,024, seed 1; inherited LoRA rank/targets.
- TRL 1.14.0 ordinary PyTorch full-logits path, with reference log probabilities precomputed.
- Chosen/rejected answers share the same prompt; no JSON repair or relaxed evaluator.
- Rule-derived training preferences plus owner review, not LLM-judge labels.
- No DPO validation preference split was constructed; evaluation used the existing development suite.

"2:1 DPO" means DPO started from a 2:1-trained SFT model. The preference dataset itself was not rebalanced to 2:1.

Provenance

The config separates preference provenance (reference_config) from policy initialization (initial_sft_config). Starting config and adapter hashes are pinned. Actual artifact retained 705 pairs / 400 prompts, longest sequence 633, zero truncation.

Recorded Preference Artifact Summary

Artifact measurement	Recorded value
Preference pairs	705
Unique training prompts	400
Chosen completion tokens	23,008
Rejected completion tokens	20,981
Longest complete sequence (tokens)	633
Truncated pairs	0

Preferred decision	Pair exposure
ACT	400
CLARIFY	150
REFUSE	155

Pair exposure counts comparisons, not unique anchors. One prompt can contribute multiple distinct rejections.

Source: recorded DPO run. This export-only table transcribes recorded metadata; it does not change the source note or any training artifact.

Results

Evaluation	Cases	Task success	Tool accuracy	Argument accuracy	Clarification wording	Refusal wording
DPO / 2:1 start	150	31/150 (0.207)	0.473	0.676	0.125	0.172

Values are ratios from 0 to 1, rounded to three decimals. Clarification/refusal columns measure wording criteria, not routing alone. Different suite sizes remain separate evaluations.

Routing and Output Validity

Evaluation	Macro-F1	Invalid-output rate
DPO / 2:1 start (150 cases)	0.453	0.047

Routing values use the existing comparison helper on recorded per-scenario decisions; missing historical aggregate fields are derived without rewriting source files. Invalid output and unknown decision are not identical categories.

Decision Matrix: DPO / 2:1 start (150 Cases)

Rows are expected decisions; columns are recorded predictions. The diagonal is correct routing, not necessarily full task success.

Expected	ACT	CLARIFY	REFUSE	Invalid/unknown
ACT	40	32	3	6
CLARIFY	21	18	1	0
REFUSE	13	5	10	1

Result Provenance

- DPO / 2:1 start: 2026-10-09 UTC, 150 cases.

Interpretation

Against 2:1 SFT, task successes rose from 29 to 31/150, while macro-F1 fell from 0.477 to 0.453. Premature ACT decisions increased from 22 to 34, and correct refusal routing fell from 19 to ten.

The starting checkpoint affected the outcome, but DPO again shifted toward action. This was not an overall improvement over the original two-epoch SFT reference.

Limits

Changing initialization also changes the frozen DPO reference distribution, necessarily. It is not a test of changing only the policy weights while holding reference probabilities fixed.

The same preference-coverage gaps remained. No additional preference examples were introduced in this experiment.

Supporting Files

- qwen2_5_1_5b_2to1_epochs2_v1.json
- dpo-run.json

Notes assembled on 2026-10-10 from saved artifacts and repository records. No training, inference or result modification was performed to write this note.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Completing Missing Productive Rejections for Boundary Decisions

Experiment index

Question and Context

Did the first preference set underteach "clarify instead of act" and "refuse instead of act" because those rejected alternatives were missing for many anchors?

Change

Start fresh from the original anchored two-epoch SFT adapter, not the previous DPO adapter or the 2:1 SFT model.

Add 95 explicit incorrect ACT answers to existing training prompts: 50 clarification and 45 refusal anchors. Negatives deliberately guess missing details, skip confirmation, call a known unavailable operation, or substitute an inappropriate productive action.

Training-only counterparts take priority. Authoring explanations stay in the review sidecar, not the model input. No evaluation fixture supplied a target.

Preference Artifact

Chosen over rejected	Before	After
ACT > CLARIFY	200	200
ACT > REFUSE	200	200
CLARIFY > ACT	50	100
CLARIFY > REFUSE	100	100
REFUSE > ACT	55	100
REFUSE > CLARIFY	100	100

800 pairs, still 400 unique prompts and the same 200/100/100 unique anchor counts. Actual artifact: 25,854 chosen and 24,541 rejected completion tokens, longest sequence 633, no truncated pairs.

- Start policy and frozen reference from the specified SFT adapter, not bare Qwen.
- One DPO epoch, sigmoid preference loss, beta 0.1, learning rate 5e-6, constant scheduler.
- Batch one, accumulation 16, context 1,024, seed 1; inherited LoRA rank/targets.
- TRL 1.14.0 ordinary PyTorch full-logits path, with reference log probabilities precomputed.

- Chosen/rejected answers share the same prompt; no JSON repair or relaxed evaluator.
- Rule-derived training preferences plus owner review, not LLM-judge labels.
- No DPO validation preference split was constructed; evaluation used the existing development suite.

Recorded Preference Artifact Summary

Artifact measurement	Recorded value
Preference pairs	800
Unique training prompts	400
Chosen completion tokens	25,854
Rejected completion tokens	24,541
Longest complete sequence (tokens)	633
Truncated pairs	0

Preferred decision	Pair exposure
ACT	400
CLARIFY	200
REFUSE	200

Pair exposure counts comparisons, not unique anchors. One prompt can contribute multiple distinct rejections.

Source: recorded DPO run. This export-only table transcribes recorded metadata; it does not change the source note or any training artifact.

Results

Evaluation	Cases	Task success	Tool accuracy	Argument accuracy	Clarification wording	Refusal wording
DPO / completed ACT rejections	150	46/150 (0.307)	0.533	0.793	0.050	0.155

Values are ratios from 0 to 1, rounded to three decimals. Clarification/refusal columns measure wording criteria, not routing alone. Different suite sizes remain separate evaluations.

Routing and Output Validity

Evaluation	Macro-F1	Invalid-output rate
DPO / completed ACT rejections (150 cases)	0.482	0.047

Routing values use the existing comparison helper on recorded per-scenario decisions; missing historical aggregate fields are derived without rewriting source files. Invalid output and unknown decision are not identical categories.

Decision Matrix: DPO / completed ACT rejections (150 Cases)

Rows are expected decisions; columns are recorded predictions. The diagonal is correct routing, not necessarily full task success.

Expected	ACT	CLARIFY	REFUSE	Invalid/unknown
ACT	60	14	1	6
CLARIFY	30	9	1	0
REFUSE	15	2	11	1

Result Provenance

- DPO / completed ACT rejections: 2026-10-09 UTC, 150 cases.

Interpretation

Task successes were 46/150, macro-F1 0.482 and invalid outputs seven. Compared with first DPO's 53 successes / 0.376 macro-F1, this recovered some routing balance at a task-completion cost.

Premature ACT decisions fell from 56 to 45, but the original SFT reference had only 31. Original SFT still had higher macro-F1 (0.514) and fewer invalid outputs (two).

The gap repair helped relative to the first DPO candidate, but did not beat the original SFT on balanced routing.

Limits

Adding 95 pairs increases optimizer exposure at the same epoch count: this is not a step-matched ablation. Coverage is complete but directions are not equally weighted: ACT-over-boundary has 400 pairs, boundary-over-ACT 200.

No claim follows that DPO cannot work, that Qwen is inherently unsuitable, or that longer DPO would fix it. Those remain untested hypotheses.

Supporting Files

- qwen2_5_1_5b_anchored_epochs2_act_rejections_v2.json
- dpo-run.json
- dpo_act_rejections.py

Notes assembled on 2026-10-10 from saved artifacts and repository records. No training, inference or result modification was performed to write this note.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Untrained Qwen: What the Baseline Actually Measures

Experiment index

Question and Context

Does fine-tuning help compared with the same local model without an adapter?

This note records the bare Qwen reference. Hosted GPT and scripted results are different baselines: hosted inference changes the model, while scripted responses test the harness rather than learned behavior.

Setup

Base Qwen2.5-1.5B-Instruct, no LoRA adapter, evaluated through the local runner and fake tools.

Use the training-prompt-a314 artifact when comparing with similarly prompted historical adapters. The "historical" artifact predates some policy/scoring changes and is not interchangeable.

Results

Evaluation	Cases	Task success	Tool accuracy	Argument accuracy	Clarification wording	Refusal wording
Bare Qwen / historical settings	150	32/150 (0.213)	0.500	0.791	0.036	0.019
Bare Qwen / matched training prompt	150	32/150 (0.213)	0.493	0.791	0.025	0.034
Bare Qwen	30	7/30 (0.233)	0.483	0.947	0.000	0.000

Values are ratios from 0 to 1, rounded to three decimals. Clarification/refusal columns measure wording criteria, not routing alone. Different suite sizes remain separate evaluations.

Result Provenance

- Bare Qwen / historical settings: 2026-09-26 UTC, 150 cases.
- Bare Qwen / matched training prompt: 2026-09-26 UTC, 150 cases.
- Bare Qwen: 2026-09-27 UTC, 30 cases.

Interpretation

The historical combined suite measured 32/150 task successes. The matched-training-prompt evaluation also had 32/150, but some tool/wording metrics differed. Template-shift success was 7/30.

This establishes that "same task-success rate" can hide different behavior. It also explains why 50-, 150- and 30-case percentages should not be placed in one improvement curve.

Limits

These base results predate later strict-parser measurements. A clean claim that a modern checkpoint beats base needs a method-matched base artifact, not simply the largest improvement over any historical baseline.

Reported local API cost of zero does not include electricity, hardware or training cost.

Supporting Files

The result links above are the surviving primary evidence. A separate immutable training record was not established for this historical experiment.

Notes assembled on 2026-10-10 from saved artifacts and repository records. No training, inference or result modification was performed to write this note.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Effective Token Distribution and Training-Cap Audit

Experiment index

Question and Context

Did nominal row weights reflect the learning exposure actually delivered to SFT?

Method

Report assistant-label tokens after mixing, split selection, deterministic shuffling, caps, truncation and removal of rows with no usable labels. Count sources and exact unique rows as well as behavioral labels.

This is a data measurement, not a model training experiment.

Saved 5,000-Row Token Snapshots

Snapshot	Direct rows / tokens	Multi rows / tokens	Refusal rows / tokens	Clarification rows / tokens
Original mix	2,285 / 63,148	2,497 / 121,608	121 / 2,953	97 / 2,380
Extra-six / extra-twelve	2,125 / 60,266	2,331 / 115,158	356 / 8,745	188 / 4,658
Legacy boundary weighting	2,086 / 58,321	2,187 / 107,640	378 / 9,293	349 / 8,612

The two saved snapshots within each weighted family report the same token counts. That alone does not establish identical checkpoint identity.

Cap Failure and Correction

At one stage the 1,000-row artifact retained 60 refusal rows but only 36 unique ones, and 40 clarification rows but only 27 unique ones. More authored examples were not necessarily surviving the cap.

Unique-first sampling corrected that. Increasing the raw cap then exposed another defect: at 1,500, refusal rose to 103 rows but only 60 unique, and clarification to 68 but only 45 unique. There were 67 exact duplicates overall.

The corrected fixed-contribution reports were:

Raw cap	Usable rows	BAYMAX	xLAM	Unique refusal	Unique clarification	Exact duplicates
1,000	967	150	817	60	45	0
1,500	1,447	150	1,297	60	45	0

Interpretation

Rows and tokens answer different questions. Multi-tool targets contribute more tokens per example; a substantial boundary row count can still be a small share of training loss.

The sampler fix preserves diversity and deterministic selection. It does not prove the model improved. The two boundary-pool training notes document those evaluations separately.

Limits

Earlier duplicate counts and corrected cap counts come from owner reports. No fresh tokenizer or full-corpus audit was run to write these notes. Exact duplicates are not the same as near-duplicates or template similarity.

Do not assign a 5,000-row snapshot to a particular checkpoint without confirming its provenance.

Supporting Files

- original-mix-token-audit-5000-and-boundary-failures.json
- clarification-extra6-refusal-extra12-token-audit-5000-snapshot1.json
- clarification-extra6-refusal-extra12-token-audit-5000-snapshot2.json
- boundary-weighting-legacy13-token-audit-5000-snapshot1.json
- boundary-weighting-legacy13-token-audit-5000-snapshot2.json
- report_sft_artifact.py

Notes assembled on 2026-10-10 from saved artifacts and repository records. No training, inference or result modification was performed to write this note.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Strict Parser Replay: Correcting Measurement, Not Training

Experiment index

Question and Context

Could permissive parsing execute a valid-looking part of an otherwise invalid model response, overstating benchmark performance?

Change and Method

Reparse saved raw generations with strict-complete-response-v1, replay fake tool execution, and write separate result files. No model inference or training was rerun.

Malformed calls, invalid meta-tool fields, conflicting decisions and trailing output reject the whole response. There is no accepted valid subset or automatic repair.

Results

Evaluation	Cases	Task success	Tool accuracy	Argument accuracy	Clarification wording	Refusal wording
100/100/100 BAYMAX / strict replay	150	42/150 (0.280)	0.500	0.694	0.013	0.172
Schema-diverse / strict replay	150	40/150 (0.267)	0.480	0.672	0.025	0.172
Historical ACT200 / strict replay	150	46/150 (0.307)	0.493	0.738	0.000	0.086
Historical ACT150 / strict replay	150	46/150 (0.307)	0.480	0.764	0.025	0.052

Values are ratios from 0 to 1, rounded to three decimals. Clarification/refusal columns measure wording criteria, not routing alone. Different suite sizes remain separate evaluations.

Routing and Output Validity

Evaluation	Macro-F1	Invalid-output rate
100/100/100 BAYMAX / strict replay (150 cases)	0.367	0.113

Evaluation	Macro-F1	Invalid-output rate
Schema-diverse / strict replay (150 cases)	0.395	0.133
Historical ACT200 / strict replay (150 cases)	0.349	0.120
Historical ACT150 / strict replay (150 cases)	0.308	0.107

Routing values use the existing comparison helper on recorded per-scenario decisions; missing historical aggregate fields are derived without rewriting source files. Invalid output and unknown decision are not identical categories.

Result Provenance

- 100/100/100 BAYMAX / strict replay: 2026-09-28 UTC, 150 cases.
- Schema-diverse / strict replay: 2026-09-28 UTC, 150 cases.
- Historical ACT200 / strict replay: 2026-09-28 UTC, 150 cases.
- Historical ACT150 / strict replay: 2026-09-28 UTC, 150 cases.

Interpretation

Full-suite success counts changed from 43 to 42 for ACR, 42 to 40 for schema-diverse, 47 to 46 for ACT-balanced and stayed 46 for ACT-midpoint.

These differences correct the measuring instrument. They must not be counted as model regressions caused by new training.

Provenance and Limits

Replay artifacts retain source/scenario hashes and inherit the original generation cost/latency. That latency cannot measure replay performance or a new model-speed gain.

Old split leakage and unseeded LoRA initialization were not repaired by replay. Source filenames embedded in JSON can predate renaming; use FILE_RENAMES.json to locate their current paths.

The linked parser README documents rejected-output counts and the corrected synthetic split. Later routing metrics may be derived from saved decisions without changing these historical JSON files.

Supporting Files

- README.md
- FILE_RENAMES.json
- SCORING.md

Notes assembled on 2026-10-10 from saved artifacts and repository records. No training, inference or result modification was performed to write this note.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Training Data Contract and Policy Review

Experiment index

Question and Context

Were model failures partly being taught by inconsistent training examples rather than caused only by insufficient training?

This was a series of related source-quality audits, not a claim that all external data was bad or a completed model-training experiment.

BAYMAX Source Findings

An adapter audit checked 1,128 synthetic examples and executed all 388 productive target sequences in fresh fake state. It found:

- 188 clipboard states used a plain string instead of the adapter's state object.
- 17 calendar states lacked required event details.
- 43 clarification targets did not request all missing details.

The generator13 correction changed 248 distinct examples without turning invalid/ambiguous requests into valid ACT requests.

Owner-approved generator14 policy changes corrected 41 bulk-outreach explanations, one unavailable-send label, and 24 draft-composition triplet examples. Training targets were validated independently of benchmark outcomes.

External Source Findings and Filtering

A numeric audit scanned 60,000 rows and flagged 1,069 candidates. Flags required review; numeric coincidence or explicit rounding could be valid.

A deliberately enriched review of 100 examples found both metadata and target/prompt defects. It cannot estimate the whole dataset's defect rate.

The broad contract scan flagged 31,821 rows. That was not the removal list: defaults, ambiguous schemas and context-dependent findings are not equivalent to confirmed incompatible targets.

Credential filtering excluded 618 rows. A narrower target-compatibility audit on the remaining source proposed 1,771 additional exclusions, leaving 57,611 accepted examples. Retained source bytes/order/splits and reviewed exclusion identities were preserved.

Blocked Experiment

The first auth-filtered matched-data preparation failed whole-dataset validation and a capped training preflight reported 30 target issues, including duplicate tools, wrong types and unverifiable types.

No successful checkpoint/result is evidenced for that blocked preparation. The owner filtered the incompatible external targets before running subsequent cleaned experiments.

Interpretation

Mechanical validity and policy correctness are different checks. Passing a schema does not prove an answer is grounded, necessary or appropriate. Conversely, every broad warning is not grounds for dropping a row.

These corrections made training evidence more trustworthy. The resampled and anchored cleaning notes record the actual model outcomes; improvement must come from those results, not the audit counts.

Limits

Semantic quality remains incompletely audited, and no LLM judge was introduced. Evaluation cases were not used to rewrite source targets. The local review artifacts are not required to be committed or embedded in these notes.

Supporting Files

- TRAINING_DATA_AUDIT.md
- qwen2_5_1_5b_lora_auth_filtered_v14_matched_act200_val200.json
- audit_sft_numeric_arguments.py
- validate_sft_dataset.py

Notes assembled on 2026-10-10 from saved artifacts and repository records. No training, inference or result modification was performed to write this note.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Decision Metrics Separate From Explanation Wording

Experiment index

Question and Context

A valid refusal could choose the right decision but match only one required reason keyword. Were keyword scores being mistaken for routing accuracy?

Measurement Change

Preserve selected act/clarify/refuse in AgentResponse.decision_type, independently of the displayed clarification question or refusal reason.

Keep existing keyword and task-success rules intact. Add decision accuracy and decision coverage; later reporting derives macro-F1, invalid-output rate and the confusion matrix from saved decisions.

Results and Meaning

The historical ACT-balanced legacy and decision-metric evaluations both recorded 47/150 task successes, tool accuracy 0.553 and argument accuracy 0.805. On template shift both recorded 6/30 successes.

The reporting change did not train a better model. It revealed a different aspect of the same task: correct routing can coexist with an incomplete or false explanation.

Reading the Metrics

- Macro-F1 weights ACT, CLARIFY and REFUSE equally, instead of rewarding the majority class alone.
- Conditional decision accuracy excludes unknown decisions; always read it with coverage.
- Invalid outputs count as missed predictions for their expected class in macro-F1.
- Refusal/clarification accuracy measures the required message keywords, not just the selected label.
- Task success requires the broader scenario checks, not merely a correct decision.
- Hallucination rate currently measures unavailable tool names, not every invented fact.

Limits

An invalid-output rate is not the rate of every runtime/schema error. A parser-valid call can still have wrong tool arguments or fail execution.

Core-agent runtime outcome versus selected decision is a separate concern documented in the bug report. Do not assume a final refusal message always means the model chose REFUSE. This note records measurement semantics, not an assertion that every runner's runtime paths have been fixed.

Supporting Files

- SCORING.md
- BUG_REPORT.md
- routing_metrics.py
- act-balanced-train1k-combined-150-legacy-metrics.json
- act-balanced-train1k-combined-150-decision-metrics.json

Notes assembled on 2026-10-10 from saved artifacts and repository records. No training, inference or result modification was performed to write this note.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Clarification Output Contract: Target Audit and Failure Diagnostics

Experiment index

Question

Experiment C sometimes calls `request_clarification` with `reason`, although the shared contract requires `question`. Is this incorrect training supervision or an invalid model output? Can we distinguish it without repairing benchmark answers?

Training Targets Checked

The audit read the same capped raw prefixes selected by C, using the existing `load_sft_jsonl` and `validate_training_targets` functions. No model or tokenizer was loaded and no training data was changed.

Split	Raw targets checked	Clarification targets	Distinct question strings	Target-contract errors
Train	1,000	100	65	0
Validation	200	30	22	0

All selected clarification targets use a nonempty `question` field, not `reason`. Supplied meta-tool schemas match the shared definitions. Distinct target wording was inspected: it requests a time, duration, recipient, text, task/event identity, conflict resolution or confirmation. Examples include "How many minutes should the consultation last?" and "Please confirm you want me to send this resignation message." A confirmation request need not end with a question mark.

Repeated question strings do not imply duplicate complete training rows. This check covers raw capped targets, not newly measured post-tokenization artifacts, and does not prove semantic correctness against every example's entire world state. No automatic semantic question-quality grader was added.

Recorded C Output Failures

Raw output diagnostic	Count
Attempted clarification with invalid arguments	10
Other malformed calls	5
Unknown decision from an empty array	1

The ten clarification calls supplied `reason` instead of `question`. Six were on expected CLARIFY cases, two on ACT cases and two on REFUSE cases. Thus the diagnostic identifies an attempted clarification, not necessarily correct routing. The remaining malformed calls comprise four outputs missing tool names and one incomplete JSON output. The empty array is valid JSON but expresses no decision. Official invalid-output count remains 15 and the matrix's invalid/unknown count remains 16. Original scores, expected behavior, parser rules and result JSON are unchanged.

Diagnostic Change

The behavior analyzer previously classified a matching meta-tool name as `correct_decision_wrong_wording`, even if its required arguments were invalid. It now consults the existing strict parser first. Invalid clarification arguments are labeled `attempted_clarification_invalid_arguments`; the analogous refusal label is `attempted_refusal_invalid_arguments`. Mixed meta/productive calls remain malformed, not accepted boundary decisions.

The quick failure-summary command can print a separate diagnostic table:

```
.venv\Scripts\python scripts\summarize_eval_failures.py `
  results\phase2\qwen-lora\baymax-mix\target-compatible-generator14-act200-anchored-lr1e-4-train1k-val200-epochs2-combined-150.json `
  --output-diagnostics
```

The default failure-reason summary is unchanged. Diagnostics use the current parser only to inspect saved generations. If a recorded rejection/unknown is not reproduced by that parser, it is flagged separately rather than rescored.

Decision and Limits

There is no demonstrated `reason`/`question` defect in the selected targets to fix. Do not rename fields in evaluation outputs or add fixture-specific training examples. Any future output-contract learning experiment should be independently authored, keep class quantities fixed and separate format validity from routing correctness. No new training or prompt change was made during this audit.

Sources

- C result JSON
- C training config and dataset identity
- Shared meta-tool definitions
- Target validation
- Read-only diagnostic classifier
- Failure-summary command

Audit recorded on 12 October 2026. It is a measurement investigation, not a new model run.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Experiment B: Rank 16 at the Fixed Two-Epoch SFT Recipe

Status: Completed; evaluated on 12 October 2026. Rank 8 remains the reference.

Experiment index

Context

The anchored two-epoch SFT reference remains our reference for balanced ACT / CLARIFY / REFUSE routing. More SFT epochs and the DPO trials have not established a better balanced model. This experiment tests adapter capacity without changing the data or training objective.

What Changes

Setting	A: reference	B: candidate
LoRA rank	8	16
LoRA alpha	16	32
Alpha / rank	2	2
Target modules	q_proj, v_proj	q_proj, v_proj

Rank and alpha change together to retain the existing LoRA scaling ratio. This doubles the trainable adapter parameter count for the same target modules. It does not test adding k_proj/o_proj, lowering the learning rate or adding data.

Fixed Recipe

Setting	Both runs
Initialization	Fresh Qwen/Qwen2.5-1.5B-Instruct, not a continued adapter
Dataset	xlam-baymax-target-compatible-v14-act200-anchored-val200
Dataset revision	329fab4d8f1604a4085d3ec50e05881e7c069717bb1428db7e592743bb07f03c
Prompt SHA256	a31446de6c8b67f1fc8ba8fc09ed8e2d7425ba832334dc8fd6fe2d55ef978468
Raw caps	1,000 training / 200 validation rows
Actual sequence limit	1,024 with the command below

Setting	Both runs
Actual batch / accumulation	1 / 16 with --local-low-vram
Learning rate / epochs	0.0002 / 2
Dropout / seed	0.05 / 1
Scheduler / warmup	Cosine / 0.03

The JSON retains the reference's declared 2,048 context and batch settings; the same command-line overrides reproduce its actual low-VRAM recipe. The reference retained 974 training and 195 validation rows after token filtering. The candidate's final usable-row counts are not embedded in its evaluation JSON; the reference counts must not be presented as newly measured candidate counts.

Run

From the repository root in PowerShell:

```
$config =
"configs\sft\qwen2_5_1_5b_lora_target_compatible_v14_act200_anchored_val200_epochs2_rank16.json"
.venv\Scripts\python scripts\train_sft_lora.py `
  --config $config `
  --local-low-vram `
  --max-train-rows 1000 `
  --max-eval-rows 200 `
  --max-sequence-length 1024
```

The candidate writes to its own `models` directory ending in `epochs2_rank16`. Do not overwrite the reference adapter or reduce context to work around memory pressure without recording that as a different experiment.

Results

Both runs use the established combined v1 + v2 development suite: 150 cases, with 81 ACT, 40 CLARIFY and 29 REFUSE expectations. The supplied evaluation command uses greedy decoding, a 256 new-token limit and the existing strict parser.

Metric	A: rank 8	B: rank 16
Full task successes	44/150 (0.293)	46/150 (0.307)
Routing macro-F1	0.514	0.450
Invalid outputs	2/150 (0.013)	7/150 (0.047)
Tool-call accuracy	0.600	0.547
Argument accuracy	0.771	0.785
Clarification wording accuracy	0.075	0.050

Metric	A: rank 8	B: rank 16
Refusal wording accuracy	0.172	0.103
Decision accuracy among known decisions	0.568	0.552
Decision coverage	0.987	0.953
Unavailable-tool hallucination rate	0.020	0.053
Average latency (ms)	1828	1980
Reported API cost (USD)	0.0000	0.0000

Ratios are rounded to three decimals. Wording scores are separate from routing: a correct clarification label can still miss the required question details. Local API cost zero does not mean training, hardware or electricity are free.

Routing Counts

Measurement	A: rank 8	B: rank 16
Correct ACT decisions	57/81	60/81
Correct CLARIFY decisions	11/40	11/40
Correct REFUSE decisions	16/29	8/29
Premature ACT on CLARIFY/REFUSE cases	31/69	42/69

Premature ACT here is the matrix count of productive decisions on boundary requests, not the potentially different count of the scorer's failure-reason label.

A: Rank-8 Decision Matrix

Rows are expected decisions; columns are recorded predictions. The diagonal measures correct routing, not full task success.

Expected	ACT	CLARIFY	REFUSE	Invalid/unknown
ACT	57	21	3	0
CLARIFY	20	11	9	0
REFUSE	11	0	16	2

B: Rank-16 Decision Matrix

Expected	ACT	CLARIFY	REFUSE	Invalid/unknown
ACT	60	14	2	5
CLARIFY	23	11	6	0

Expected	ACT	CLARIFY	REFUSE	Invalid/unknown
REFUSE	19	0	8	2

Invalid/unknown is the decision-matrix category. Invalid-output rate is computed separately from parser failures; the counts happen to match in these two runs.

Interpretation and Decision

The candidate completed two more tasks and slightly improved argument accuracy, but routing macro-F1 fell by 0.064. Correct refusals halved, premature ACT rose by eleven cases, and invalid outputs increased from two to seven. Correct clarification routing did not improve. Rank 16 is not a better balanced replacement.

Keep A, the rank-8 two-epoch SFT model, as the reference. This run does not support insufficient adapter capacity as the main problem. It does not prove capacity never matters, and a single-seed result cannot establish the cause of the regression or whether a two-task difference is reproducible.

The next test recommended after B was C: return to rank 8 / alpha 16 and lower only the learning rate from 2e-4 to 1e-4. That test has now been run; see Experiment C for its results. B remains a separate completed experiment, not part of a combined rank-and-learning-rate change.

Sources and Reproducibility

Artifact	Purpose
A evaluation JSON	Original reference results
B evaluation JSON	Rank-16 results, recorded at 2026-10-12T16:58:08.530056+00:00
B training config	Candidate settings and data identity
Comparison helper	Reads saved metrics and derives routing from recorded decisions

B canonical config SHA256: `a09ca9973edbc9127f7641cc5e10cb809baf18f13a3504c011350835fc0bd663`.

Numbers were checked against the saved result files using the comparison helper. No training, inference or result-file rewriting was performed to record this note. These repeatedly inspected suites are development comparisons, not an untouched final test. Exact base-model revision and complete generation/training metadata are not embedded in the result JSON, which limits retrospective verification.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Experiment C: Lower Learning Rate at Two Epochs

Status: Completed; evaluated on 12 October 2026. A remains the balanced reference.

Experiment index

Question and Context

Does halving the learning rate improve balanced ACT / CLARIFY / REFUSE routing without sacrificing productive task completion?

Experiment B increased adapter rank but worsened routing and validity. C starts from A's original recipe, not B's rank-16 settings and not a saved trained adapter. Smaller updates preserving decision boundaries is a hypothesis to test, not an established explanation of earlier runs.

Controlled Change

Setting	A: reference	C: candidate
Peak learning rate	0.0002	0.0001
LoRA rank / alpha	8 / 16	8 / 16
Target modules	q_proj, v_proj	q_proj, v_proj
SFT epochs	2	2
Initialization	Fresh Qwen base + fresh LoRA	Fresh Qwen base + fresh LoRA

Only the learning rate changes, apart from the run name, config hash and output paths needed to keep the experiment separate. Cosine scheduling and 3% warmup remain unchanged; C halves the schedule's learning-rate amplitude.

Fixed Data and Actual Overrides

- Base model: Qwen/Qwen2.5-1.5B-Instruct.
- Data: xlam-baymax-target-compatible-v14-act200-anchored-val200, unchanged splits.
- Revision: 329fab4d8f1604a4085d3ec50e05881e7c069717bb1428db7e592743bb07f03c.
- Prompt SHA256: a31446de6c8b67f1fc8ba8fc09ed8e2d7425ba832334dc8fd6fe2d55ef978468.
- Raw training / validation caps: 1,000 / 200.
- Actual sequence limit: 1,024; train/eval batch one; gradient accumulation 16.
- Seed 1, dropout 0.05, assistant-only token cross-entropy and the same masking.

The command below applies the same low-VRAM overrides as A; the JSON retains A's declared context and batch settings. A retained 974 train and 195 validation rows after token filtering. C's final usable-row counts are not embedded in its evaluation JSON; the reference counts are not new measurements of C.

Training Command

Run from the repository root in PowerShell:

```
$config = "configs\sft\qwen2_5_1_5b_lora_target_compatible_v14_act200_anchored_val200_epochs2_lr1e-4.json"
.venv\Scripts\python scripts\train_sft_lora.py `
  --config $config `
  --local-low-vram `
  --max-train-rows 1000 `
  --max-eval-rows 200 `
  --max-sequence-length 1024
```

The new output directory ends in `epochs2_lr1e-4`; neither A nor B is overwritten.

Results

Both runs use the established 150-case v1 + v2 development suite: 81 ACT, 40 CLARIFY and 29 REFUSE expectations. The owner command uses the same prompt, greedy decoding, a 256 new-token limit and the existing strict parser.

Metric	A: learning rate 0.0002	C: learning rate 0.0001
Full task successes	44/150 (0.293)	45/150 (0.300)
Routing macro-F1	0.514	0.436
Invalid outputs	2/150 (0.013)	15/150 (0.100)
Tool-call accuracy	0.600	0.513
Argument accuracy	0.771	0.726
Clarification wording accuracy	0.075	0.050
Refusal wording accuracy	0.172	0.172
Decision accuracy among known decisions	0.568	0.560
Decision coverage	0.987	0.893
Unavailable-tool hallucination rate	0.020	0.010
Average latency (ms)	1828	2492
Reported API cost (USD)	0.0000	0.0000

Ratios are rounded to three decimals. Wording metrics are not routing metrics. Zero reported API cost excludes local hardware, electricity and training cost.

Routing Counts

Measurement	A	C
Correct ACT decisions	57/81	58/81
Correct CLARIFY decisions	11/40	4/40
Correct REFUSE decisions	16/29	13/29
Premature ACT on boundary cases	31/69	37/69
Invalid/unknown decisions	2/150	16/150

Premature ACT is derived from the matrix's CLARIFY/REFUSE rows, not necessarily the count of the scorer's identically themed failure-reason label.

A: Reference Decision Matrix

Rows are expected decisions; columns are recorded predictions. The diagonal measures correct routing, not full task success.

Expected	ACT	CLARIFY	REFUSE	Invalid/unknown
ACT	57	21	3	0
CLARIFY	20	11	9	0
REFUSE	11	0	16	2

C: Lower-Learning-Rate Decision Matrix

Expected	ACT	CLARIFY	REFUSE	Invalid/unknown
ACT	58	13	5	5
CLARIFY	24	4	4	8
REFUSE	13	0	13	3

Invalid/Unknown Output Inspection

Observed issue	Responses
request_clarification used reason instead of question	10
Calls omitted tool names	4
Incomplete JSON	1
Empty array with no decision	1

The ten wrong-field calls include six expected CLARIFY cases, two ACT cases and two REFUSE cases. An attempted clarification is not necessarily correct routing. The empty array parses but expresses no decision; that explains 15 invalid outputs versus 16 invalid/unknown decisions. None were repaired or rescored.

The target-contract audit checked the selected 1,000 raw training and 200 validation targets. All 100 training and 30 validation clarification targets correctly use question. This does not establish that the model learned the contract or that every target is semantically perfect.

Interpretation and Decision

C gained one full task success, but macro-F1, argument accuracy and output validity worsened. The lower rate did not preserve the boundaries better in this run. Keep A as the balanced reference; do not promote C on its one-task gain.

Some apparent clarification losses are invalid expression of an attempted clarification, not only selection of the wrong decision. Output-contract learning should be investigated separately without weakening the strict parser. This experiment alone does not prove that the lower rate caused undertraining or that more epochs would resolve the defects.

Provenance and Limits

- C evaluation JSON: recorded 2026-10-12T17:34:32.282617+00:00.
- A evaluation JSON: recorded 2026-10-06T15:29:45.372202+00:00.
- C canonical config SHA256: 80b81f44f0daf9f4d18ba8f29eb7139a216d72c5ecec6b5839ffd77e40eeee02.

Metrics and matrices were checked using the existing comparison helper on saved artifacts. No training, inference or result modification was performed to write this note. Results are single-seed observations on repeatedly inspected development suites, not an untouched final benchmark. Exact base-model revision and complete training/generation metadata are not embedded in the result JSON.

Supporting Files

- Candidate config
- A: selected rank-8 two-epoch reference

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Experiment E: QKVO LoRA on All Four Attention Projections

Status: Completed; evaluated on 12 October 2026. Task-completion candidate, not the balanced reference.

Experiment index

Question and Controlled Change

Does adapting key and output projections alongside query and value projections improve task completion and balanced routing under A's original two-epoch recipe?

Setting	A: reference	E: candidate
Target modules	q_proj, v_proj	q_proj, k_proj, v_proj, o_proj
Rank / alpha	8 / 16	8 / 16
Learning rate	0.0002	0.0002
Epochs	2	2
Initialization	Fresh pretrained Qwen + fresh LoRA	Fresh pretrained Qwen + fresh LoRA

Only the target-module list changes, apart from run identity and output paths. E is not the rank-16 recipe or the lower-learning-rate recipe. Broader coverage also increases trainable adapter parameters: this is not a constant-parameter coverage comparison. Its result cannot attribute a gain solely to module placement.

Fixed Recipe

- Base model: Qwen/Qwen2.5-1.5B-Instruct; pretrained weights are not reinitialized.
- Frozen data: xlam-baymax-target-compatible-v14-act200-anchored-val200.
- Revision: 329fab4d8f1604a4085d3ec50e05881e7c069717bb1428db7e592743bb07f03c.
- Prompt SHA256: a31446de6c8b67f1fc8ba8fc09ed8e2d7425ba832334dc8fd6fe2d55ef978468.
- Raw training / validation caps: 1,000 / 200.
- Actual command overrides: context 1,024, train/eval batch one, accumulation 16.
- Seed 1, dropout 0.05, cosine scheduling with 3% warmup, existing PEFT initialization.
- The same assistant-label masking, target-contract checks and strict inference parser.

The JSON retains A's declared context and batch settings. The command applies the same actual low-VRAM overrides as A. A retained 974 training and 195 validation rows. E's final usable-row counts are not embedded in its evaluation JSON; the reference counts are not new measurements of E.

Training Command

Run from the repository root in PowerShell:

```
$config =
"configs\sft\qwen2_5_1_5b_lora_target_compatible_v14_act200_anchored_val200_epochs2_qkvo.json"
.venv\Scripts\python scripts\train_sft_lora.py `
  --config $config `
  --local-low-vram `
  --max-train-rows 1000 `
  --max-eval-rows 200 `
  --max-sequence-length 1024
```

The separate output directory ends in `epochs2_qkvo`. No previous model is overwritten. Additional adapted modules may increase memory pressure. Do not silently reduce sequence length or change quantization to finish this run; record any such change as a different experiment.

Results

Both runs use the established combined v1 + v2 development suite: 150 cases with 81 ACT, 40 CLARIFY and 29 REFUSE expectations. The owner command uses the same prompt, greedy decoding, 256 new tokens and the existing strict parser.

Metric	A: query/value only	E: query/key/value/output
Full task successes	44/150 (0.293)	52/150 (0.347)
Routing macro-F1	0.514	0.437
Invalid outputs	2/150 (0.013)	14/150 (0.093)
Tool-call accuracy	0.600	0.527
Argument accuracy	0.771	0.748
Clarification wording accuracy	0.075	0.113
Refusal wording accuracy	0.172	0.086
Decision accuracy among known decisions	0.568	0.581
Decision coverage	0.987	0.907
Unavailable-tool hallucination rate	0.020	0.053
Average latency (ms)	1828	3794
Reported API cost (USD)	0.0000	0.0000

Ratios are rounded to three decimals. Wording checks are separate from routing. Decision accuracy excludes unknown decisions; read it alongside coverage. Zero reported API cost excludes local hardware, electricity and training cost.

Routing Counts

Measurement	A	E
Correct ACT decisions	57/81	63/81
Correct CLARIFY decisions	11/40	10/40
Correct REFUSE decisions	16/29	6/29
Premature ACT on boundary cases	31/69	40/69
Invalid/unknown decisions	2/150	14/150

Premature ACT here is the count derived from the CLARIFY/REFUSE matrix rows, not necessarily the scorer's failure-reason count.

A: Reference Decision Matrix

Rows are expected decisions; columns are recorded predictions. The diagonal measures routing correctness, not full task success.

Expected	ACT	CLARIFY	REFUSE	Invalid/unknown
ACT	57	21	3	0
CLARIFY	20	11	9	0
REFUSE	11	0	16	2

E: QKVO Decision Matrix

Expected	ACT	CLARIFY	REFUSE	Invalid/unknown
ACT	63	12	2	4
CLARIFY	27	10	2	1
REFUSE	13	1	6	9

Invalid-Output Inspection

Observed issue	Expected cases	Responses
refuse_request put reason outside arguments	REFUSE	6
request_clarification had empty arguments	CLARIFY 1, REFUSE 2	3
Productive calls omitted or separated names from arguments	ACT	4
Other malformed output	REFUSE	1

Six of the nine invalid REFUSE-case outputs selected the refusal meta-tool name, but supplied reason at the call's top level instead of inside arguments:

```
{"name": "refuse_request", "arguments": {}, "reason": "Unsupported operation."}
```

The contract requires:

```
{"name": "refuse_request", "arguments": {"reason": "Unsupported operation."}}
```

These are generic illustrations of the nesting defect, not repaired benchmark responses. Selecting refuse_request does not establish that its explanation meets the scenario's wording requirements. No rejected response received success credit.

Interpretation and Decision

E completed eight more tasks than A, the largest task-success gain among the A/B/C/E ablations. However, macro-F1 fell, invalid outputs rose, and latency was about 2.1 times A's measured latency. E is not a better balanced replacement.

Keep A as the balanced reference and E as a task-completion candidate. A substantial part of E's observed refusal loss involves invalid expression of an attempted refusal. This supports investigating output-contract learning separately; it does not erase the other incorrect decisions or establish a hypothetical repaired score.

The next proposed test is an independently authored output-contract learning experiment with the policy class quantities and other training settings held fixed. No data adjustment or new run has been made by this documentation task.

Provenance and Limits

- E evaluation JSON: recorded 2026-10-12T19:19:35.198791+00:00.
- A evaluation JSON: recorded 2026-10-06T15:29:45.372202+00:00.
- E canonical config SHA256: ee2c1b9efdec057796d0c0a9d16b20f75763151fcc34e6fe95ec004bc3bf00e4.

Metrics and matrices were checked with the existing comparison helper. Raw generations were inspected read-only. Original scores, results and parser rules remain unchanged. These are single-seed development observations, not an untouched final test. Adding modules also increases adapter size, so this does not isolate placement at equal parameter count. Full training/generation metadata and exact base-model revision are not embedded in the result JSON.

Supporting Files

- E config
- A: selected reference

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Name-First Tool Calls: Serialization-Only SFT on Experiment E

Status: Preparation implemented; full dataset conversion, token reports, training and evaluation are pending owner execution. No new benchmark result.

Experiment index

Question

Does predicting the tool name before its arguments help the model express its decision using the correct output contract? The name identifies whether the arguments need `question`, `reason`, or a productive tool's own schema.

This is a hypothesis about generation order, not proof that arguments-first JSON caused our failures. A valid output can still contain a wrong decision.

Exactly One Change

Starting from E's same frozen source dataset, reorder the two outer keys of each assistant tool call:

```
{"arguments":{"question":"Which item do you mean?"},"name":"request_clarification"}
```

becomes:

```
{"name":"request_clarification","arguments":{"question":"Which item do you mean?"}}
```

The question above illustrates the interface; no new example is authored for this experiment. The converter uses existing training data, not eval fixtures.

Item	Held Fixed
Inputs	Exact system/user message strings, tools, request wrapper and world state
Answers	Same names, argument fields, values, nested key order and call sequence
Selection	All source rows in their existing order, with unchanged split membership
Raw cap	1,000 training rows and 200 validation rows
BAYMAX contribution	Same 200 ACT / 100 CLARIFY / 100 REFUSE rows
External contribution	Same 600 xLAM rows inside the raw training cap

Item	Held Fixed
Model initialization	Fresh pretrained Qwen/Qwen2.5-1.5B-Instruct plus fresh LoRA
LoRA	E: rank 8, alpha 16, q/k/v/o projections, dropout 0.05
Training	Two epochs, learning rate 2e-4, cosine with 3% warmup, seed 1
Actual overrides	Context 1,024, batch one, gradient accumulation 16
Evaluation	Same prompt, generation settings, strict parser, scorer and 150-case suite

There is no new prompt, weighting, resampling, split, repair, constrained decoding, or continuation of E's saved adapter. Shared JSON serialization helpers remain unchanged; their sorting also affects input construction and sampling hashes.

Artifact Safety and Provenance

The preparation command derives a **new** dataset and E-based training config. It refuses existing dataset/config/model output paths. All three splits are converted consistently, without moving examples between them.

Source targets must already pass the existing output-contract validator and use canonical compact, arguments-first JSON. Invalid or differently serialized source targets fail explicitly instead of being repaired or silently normalized.

`serialization-provenance.json` records the transformation version, parent manifest hash/revision, per-split input/output file hashes, row counts and call counts. A new dataset revision and training-config hash identify the candidate. Source-row references retain their original identities; their `row_hash` is not redefined as a hash of the transformed example. Transformed content is covered by the new file hashes. No source file or historical result is overwritten.

The generated config appears only after conversion succeeds. It retains E's declared training settings; the same low-VRAM command overrides still apply. Its dataset validation report is generated separately, not copied from E.

First Step: Prepare and Validate

Run from the repository root in PowerShell. These commands do not train a model:

```
.venv\Scripts\python scripts\prepare_sft_name_first.py

$dataset = "data\sft\v1\xlam-baymax-target-compatible-v14-act200-anchored-val200-name-first"
$config =
"configs\sft\qwen2_5_1_5b_lora_target_compatible_v14_act200_anchored_val200_epochs2_qkvo_name_first.
json"

.venv\Scripts\python scripts\validate_sft_dataset.py --dataset-dir $dataset
.venv\Scripts\python scripts\validate_training_config.py --config $config --check-paths
```

Stop if either validator fails. The preparation scans the full source dataset, not just the capped rows, and may take time. It does not load a model/tokenizer, fetch data, or access evaluation scenarios.

Check Actual Trainer Exposure Before Training

Changing serialization can change tokens or truncation boundaries. Do not assume the same usable rows or assistant-token counts merely because raw caps match. Run the existing artifact report on **both** configs at the same caps:

```
$reference =
"configs\sft\qwen2_5_1_5b_lora_target_compatible_v14_act200_anchored_val200_epochs2_qkvo.json"
.venv\Scripts\python scripts\report_sft_artifact.py --config $reference --split train --max-rows
1000 --max-sequence-length 1024
.venv\Scripts\python scripts\report_sft_artifact.py --config $config --split train --max-rows 1000 -
-max-sequence-length 1024
.venv\Scripts\python scripts\report_sft_artifact.py --config $reference --split validation --max-
rows 200 --max-sequence-length 1024
.venv\Scripts\python scripts\report_sft_artifact.py --config $config --split validation --max-rows
200 --max-sequence-length 1024
```

Record class/source counts, unique rows, duplicates and trainable tokens. If usable row membership changes, investigate truncation before proceeding; do not silently repair comparability by changing the cap or dropping additional examples.

Training Gate

After the artifact checks have been reviewed, use a fresh run:

```
.venv\Scripts\python scripts\train_sft_lora.py `
  --config $config `
  --local-low-vram `
  --max-train-rows 1000 `
  --max-eval-rows 200 `
  --max-sequence-length 1024
```

The new model directory ends in `epochs2_qkvo_name_first`. Do not overwrite or resume E's adapter. Training and evaluation are run by the owner.

How to Judge the Result

The direct comparison is **E versus E-name-first**. A remains the balanced routing reference, not the initialization or data source for the candidate.

E Measured Development Reference	Value
Task success	52/150 (0.347)

E Measured Development Reference	Value
Routing macro-F1	0.437
Invalid outputs	14/150 (0.093)
Premature ACT on boundary cases	40/69

Report invalid-output rate and its structural categories separately from routing macro-F1, the full decision confusion matrix, premature-action counts and task success. Fewer malformed calls alone do not prove better ACT/CLARIFY/REFUSE judgment. Wording-based clarification/refusal scores are also distinct from correct routing. Do not rescore repaired outputs as successes.

The established 150-case suite has been repeatedly inspected and is a development benchmark, not an untouched final test. A single-seed result cannot establish a general causal claim. No candidate result is available yet.

Supporting Files

- E's recipe and measured results
- Target-only converter
- Experiment preparation command
- Regression tests

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Held-Out SFT Clarification: Ten-Example Generation Sanity Check

Status: Completed by the owner; ten-example routing diagnostic, not a benchmark.

Why This Check

The name-first model produced no valid clarification decisions on the established 150-case development suite. A read-only audit found **zero raw outputs mentioning request_clarification**, compared with 26 raw mentions and 23 accepted clarification decisions for E. This is not simply a parser hiding malformed clarification calls.

Before changing training data, test whether this checkpoint can generate clarification on held-out inputs from the same SFT distribution. This does not retrain a model or alter the benchmark, prompt policy, parser, class counts or existing result files.

Selection and Inference

- Select the first 10 unique clarification prompts in the unchanged first 200 validation rows. The selection is deterministic, not a representative random sample.
- Validate the supplied target/tool contracts and system-prompt hash.
- Reject exact input overlap with the first 1,000 raw training rows actually used for this experiment. This is not a semantic/paraphrase leakage audit.
- Pass only the saved system/user messages, unchanged, to inference. The expected assistant answer is recorded for inspection but is never supplied to the model.
- Reuse the same LocalLoraAgent loading, chat-template, generation and strict parser path as benchmark evaluation: greedy decoding, 256 new tokens, no output repair.
- Check adapter metadata against the selected config and record config, dataset, validation-file and adapter hashes. No tool calls are executed.
- Record raw output, parsed decision and expected target separately for every row. Count clarification decisions, not wording keywords or full task success.

Commands

Run from the repository root in PowerShell:

```
.venv\Scripts\python scripts\probe_sft_clarification.py `
  --output tmp\name-first-heldout-clarification-10.json
```

The default config is the trained two-epoch QKVO name-first candidate. Its expected adapter path comes from that config. No extra adapter path or base-model override is needed. CUDA is required unless explicitly allowing slow CPU inference.

For selection/preflight checks without loading a model or tokenizer:

```
.venv\Scripts\python scripts\probe_sft_clarification.py `
  --dry-run `
  --output tmp\name-first-heldout-clarification-setup.json
```

Use a new output filename for subsequent runs; existing diagnostics are not overwritten. Reports stay under ignored `tmp/` because they contain local SFT inputs and expected targets, and are not published benchmark artifacts.

Observed Result

The saved owner-run report is `tmp/name-first-heldout-clarification-10.json`:

Recorded Prediction	Examples
CLARIFY	7
ACT	3
REFUSE	0
Invalid/unknown	0

All ten expected CLARIFY. The same seven raw outputs mentioned `request_clarification`. This establishes that the checkpoint can generate valid clarification calls on these selected held-out inputs. It does not establish that the seven questions are adequate or that clarification transfers to the benchmark.

Unblinded inspection found clear question-quality gaps: row 25 asks only for the start time when duration is also missing; row 42 asks for the task name/status instead of the missing due date, despite the title being supplied. These are qualitative observations, not an independent semantic review or revised scores.

Interpretation

If clarification is generated here but absent from benchmark outputs, investigate transfer and differences in the model-visible input distribution. If clarification is also absent here, investigate the checkpoint and learning/generation behavior before assuming the problem is only transfer to the benchmark.

Neither outcome establishes the cause by itself. Ten selected examples cannot prove general competence or inability. Valid clarification syntax is not evidence that the question is sufficient or appropriate; inspect the saved raw outputs and expected targets. Adapter metadata/hash checks identify the requested checkpoint, but do not constitute an independent numerical comparison of all loaded weights.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

SFT Versus Benchmark Input: Audit and JSON-Presentation Check

Status: Paired inference completed by the owner; full-suite spacing ablation prepared.

Question

The name-first checkpoint clarified on 7/10 selected held-out SFT inputs, but on 0/40 expected-clarification cases in the established benchmark. Is a presentation difference involved? Neither observation establishes the cause, and the two sample sets are not matched tasks.

Confirmed Audit Findings

The generic prompt-building code and saved ten-example probe were inspected. No benchmark fixture was used to construct or rewrite a training example.

Property	Finding
System prompt	All ten match the inference default and recorded training hash a31446de...
Header and request wrapper	Same header; same current_time / initial_state / request fields
Meta-tools	Shared definitions, identical names, schemas and leading order
Productive tool metadata	All ten match the generic benchmark definitions, descriptions and schemas
Tools in this sample	Every prompt contains two meta-tools and one productive tool
JSON serialization	SFT is compact with literal Unicode; benchmark uses default spaces and ASCII escaping
Time rendering	All ten SFT clocks retain 2026-06-01T09:00:00Z; benchmark code uses datetime.isoformat()
Request phrasing	Several successful SFT cases explicitly state that information has not been supplied

For UTC, datetime.isoformat() can spell the same instant with +00:00 instead of Z. The paired check below deliberately does **not** change this spelling. It also does not vary tool counts, request wording, state content or metadata. Those are separate potential distribution differences, not established explanations.

The three ACT mistakes in the probe involve an invalid due date, unavailable checklist content and an ambiguous task reference. This is descriptive inspection of held-out SFT rows, not permission to derive training targets from benchmark failures or evidence for a general lexical-rule diagnosis.

One Controlled Inference Check

The existing probe now supports `--benchmark-json`. It selects the **same ten** validation rows in the same order and changes only JSON spacing/escaping to the existing benchmark renderer's conventions. Decoded requests, tools, schemas, state, clock strings and expected targets are unchanged.

The saved original-SFT report is required. Preflight rejects a different selection, config, dataset, adapter hash or generation setting. The expected assistant answer is never included in either model prompt. Model loading, greedy decoding, 256 new tokens and strict parsing are unchanged. No productive tool is executed.

Run from the repository root:

```
.venv\Scripts\python scripts\probe_sft_clarification.py `
  --benchmark-json `
  --reference-report tmp\name-first-heldout-clarification-10.json `
  --output tmp\name-first-heldout-clarification-benchmark-json-10.json
```

The owner completed both probes. Saved reports have matching config, dataset, adapter-weight hashes and generation settings. Inspection confirmed identical decoded input values, tool order, system messages and expected targets. All ten inputs are ASCII, so this pair changed JSON separator spacing only.

Input presentation	ACT	CLARIFY	REFUSE	Invalid/unknown
Exact compact SFT inputs	3	7	0	0
Benchmark JSON spacing	10	0	0	0

Raw outputs confirm all seven previous clarification decisions changed to productive calls; none of the new outputs mention request_clarification. For example, row 25 switched from asking for an event time to supplying 21:00 and a 30-minute duration. These are routing observations, not semantic question scores. Reports remain under ignored tmp/ and existing files were not overwritten.

This supports substantial presentation sensitivity on these ten selected inputs. It does not establish its magnitude across the benchmark or robust policy learning.

Full-Suite Spacing Ablation

The next owner-run diagnostic evaluates the same name-first checkpoint on the unchanged 150 scenarios from scenarios/v1 and scenarios/v2. Use --compact-json with the existing run-agent-lora command and a new output file:

```
$adapter =
"models\qwen2_5_1_5b_lora_target_compatible_v14_act200_anchored_val200_epochs2_qkvo_name_first\adapt
er"
.venv\Scripts\python -m baymax.eval.cli run-agent-lora `
  --scenarios scenarios\v1 `
  --scenarios scenarios\v2 `
```

```
  --adapter $adapter `
  --compact-json `
  --output results\phase2\json-spacing\name-first-compact-combined-150.json
```

Only separators in the available-tool and request-context JSON change. Unicode escaping, timestamp spelling, input values, key/tool order, system prompt, checkpoint, greedy decoding, 256-token generation cap, parser, fake execution and scoring are unchanged. In particular, the renderer does not convert +00:00 to Z. Literal Unicode versus ASCII escaping is not part of this ablation.

The default renderer remains spaced. Local result metadata records the format and inference configuration. Compact mode refuses to overwrite an existing output. No retraining, target changes or rescoring of historical artifacts is involved. The full-suite result is pending; the coding agent ran only mocked unit tests. This repeatedly inspected suite is development evidence, not an untouched final test set. Compare routing macro-F1 and confusion matrices separately from keyword wording scores and task success.

Semantic Review Is Separate

Routing, response adequacy and keyword agreement must not be conflated. A valid clarification call can ask the wrong question; a useful question can use wording not recognized by keyword checks.

The proposed next measurement is a small blinded human review of boundary responses, using request/context and the general policy rather than visible keyword scores. It would report adequacy separately and preserve official scores. This audit does not implement that review, an LLM judge, or new scoring rules. The coding agent's unblinded inspection is not independent human validation.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Compact JSON: Are the Clarification Questions Useful?

Status: Completed manual diagnostic review, 2026-10-14.

Scope and Evidence

Reviewed all ten rows in the compact-JSON result whose recorded decision was CLARIFY and whose expected decision was CLARIFY. All ten are Calendar cases. This is a selected subset, not an estimate of performance across all 150 cases or other domains.

Source: name-first-compact-combined-150.json. SHA256:
`6ace952a70b3695737d373e80edb09b2b21505ac9a6c879a4ebb0a80b1bbf6aa`.

The coding agent inspected the saved request, initial state, tool schema and raw question before inspecting the corresponding keyword requirements. This is not an independent or blinded human study: the reviewer knows the experiment and has inspected some related results before. The assessments below are qualitative judgments, not new benchmark scores or an implemented LLM judge.

No inference, comparison script, retraining, rescoring, data rewrite, checkpoint change or evaluation-target change was performed for this review.

Review Criteria

- **Adequate:** Asks for the information or choice actually needed to resolve the blocker; does not have to repeat already-known names or time periods.
- **Incomplete:** Asks about a genuine blocker but leaves another required field unresolved. This can be a useful first question, not a complete clarification.
- **Wrong focus:** Asks only for known information without explaining why it needs reconsideration, and does not address the actual blocker.

Calendar creation requires title, date, time and duration in both the exposed tool schema and adapter. There is no default duration. Duration omissions are therefore substantive, even when a scenario's keywords only mention time. This review does not assume a follow-up conversation occurred.

Ten Questions

1. Tonight's Chemistry Review

Scenario: `calendar_ambiguous_tonight_001`. Request: "Schedule chemistry review tonight for 1 hour." Question: "What time should I schedule the chemistry review?"

Adequate. The date and duration are known; exact start time is missing. The question asks for precisely that. It need not repeat "tonight" to make sense in this request's context. Keyword score: **0.5**, because "tonight" is absent. This is a plausible keyword false negative for question adequacy.

2. Tomorrow's Revision

Scenario: `calendar_clarify_time_001`. Request: "Schedule revision tomorrow." Question: "What time should I schedule the revision?"

Incomplete. Asking for start time is useful, but duration is also unspecified. Keyword score: **1.0**; recorded task success is true. The question is not enough to ground every required creation argument, despite passing the current checks.

3. Calculus Conflicts With an Existing Event

Scenario: `calendar_conflict_001`. Request specifies tomorrow at 4pm for one hour. State already contains a two-hour event starting at 4pm on that date. Question: "What time should I schedule the calculus revision?"

Wrong focus. The user already gave a time. Asking for an alternative could help, but this question does not identify the conflict or explain why the given time cannot simply be used. Keyword score: **0.0**. This is a substantive state-grounding problem, not just missing synonyms.

4. Two Meetings With Marv

Scenario: `calendar_multiple_matching_meetings_001`. Request: "Move my meeting with Marv to 5pm." State contains two meetings with Marv, at 10am and 2pm. Question: "Which event should I move?"

Adequate. It requests the missing entity selection and does not invent an ID. Listing the two options would be clearer, but this question is understandable in context. Keyword score: **0.5**, because "Marv" is absent. This is another plausible keyword false negative for adequacy.

5. No Meeting With Tadashi

Scenario: `calendar_no_matching_reschedule_001`. Request: "Move my meeting with Tadashi to 5pm." State contains a meeting with Marv, but none with Tadashi. Question: "What time should I move the Tadashi meeting to?"

Wrong focus. 5pm is already known; identifying the missing meeting is the blocker. The question also presupposes a matching meeting exists. Keyword score: **1.0**, because it includes "Tadashi" and "meeting"; recorded task success is true. This is a keyword false positive for adequacy.

6. Supplier Call Without a Date

Scenario: `calendar_v2_ambiguous_missing_date_001`. Request: "Schedule a supplier call at 2pm." Question: "What time should I schedule the supplier call?"

Wrong focus. Time is known; date and duration are missing. The question asks for neither. Keyword score: **0.0**. The output repeats the common time-question pattern rather than identifying this request's missing fields.

7. Quarterly Planning Without a Time

Scenario: `calendar_v2_ambiguous_missing_time_001`. Request: "Add quarterly planning on October 22." Question: "What time should I add to my calendar?"

Incomplete. It asks for the missing time, although the wording is awkward. Duration is also unspecified. Keyword score: **1.0**; recorded task success is true. This is useful clarification, but not complete argument grounding.

8. Two Team Syncs

Scenario: `calendar_v2_contextual_multiple_sync_001`. Request: "Move the team sync to noon." State contains team syncs on October 13 and October 14. Question: "Which team sync should I move?"

Adequate. It asks for the entity selection needed before updating an event. The existing event supplies the remaining unchanged arguments. Keyword score: **1.0**; recorded task success is true.

9. Around Lunch on Friday

Scenario: `calendar_v2_implicit_lunch_001`. Request: "Put a mentor catch-up around lunch on Friday." Question: "What time should I put it?"

Incomplete. Exact time is genuinely missing, so this is a useful question; omitting "lunch" does not make it irrelevant. Duration is also missing. Keyword score: **0.5**, because "lunch" is absent. There is both lexical brittleness and a real completeness limitation.

10. Tomorrow Morning

Scenario: `calendar_v2_implicit_morning_001`. Request: "Book reading time tomorrow morning." Question: "What time should I book?"

Incomplete. It correctly asks for an exact start time, without inventing a morning default. Duration is still unknown. Keyword score: **0.5**, because "morning" is absent. Again, this is a useful but incomplete question.

Findings

Manual assessment	Cases
Adequate for the identified blockers	3
Relevant but incomplete	4
Wrong focus	3

All ten selected the correct route. That does not mean all ten asked a good question. Conversely, a partial keyword score does not establish a bad question. The saved keyword checks fully pass four of these cases, including one wrong-focus case and two incomplete cases, while undercrediting two adequate questions.

The recurring issue is identifying **what is actually unknown or conflicting**: eight of ten questions ask about time, including three cases where time is already known. This is a descriptive pattern in a Calendar-only selected subset, not proof that the model uses a single template or that other domains behave identically.

Keep routing, field/state grounding, question completeness and keyword agreement separate. A bounded next diagnostic is to inspect these general properties on held-out SFT questions against their own input schemas, before making a training adjustment. Do not turn these evaluation fixtures into training examples, rewrite their targets, or relax scoring to promote this checkpoint.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Held-Out SFT: Question Quality Versus Supplied Targets

Status: First ten-example batch reviewed, 2026-10-14.

What Was Checked

Used the existing owner-run name-first held-out probe. No new model run or comparison command was executed. This report belongs to the two-epoch QKVO name-first checkpoint, not A-reference.

The saved inputs use their exact compact SFT presentation. All ten expected CLARIFY. They are the first ten unique clarification examples selected from the first 200 validation rows; they are not a random sample.

Reviewed each request, state, exposed tool schema, supplied target question and saved model output. Checked all ten inputs/targets against their current validation.jsonl rows: all matched. The probe's original preflight checked exact input overlap against its first 1,000 training rows; that is not a semantic or paraphrase leakage guarantee.

Probe SHA256: `9601e71f96867f0e5f18c494f275da086bf7439ade051d55e32d2f04ca31928e`. Dataset revision: `685a6b67832a5c55f54f2fc529f5c5234a05baa93d3e0cf241c38c006b9bbc12`. Recorded adapter weights SHA256: `fb41064d104e72da4984526e96e06c8ad5c883aad11b585f47252c73b6b24db9`.

This is an unblinded coding-agent diagnostic, not independent human validation or a new benchmark score. No evaluation fixture was used to construct training data, and no source row, target, prompt, checkpoint or scorer was changed.

Criteria

Used the same qualitative distinctions as the previous question review: adequate, relevant but incomplete, wrong focus. Productive ACT outputs are classified as routing failures separately, not counted as bad questions.

Information can be necessary because the tool requires it, because the request explicitly asks for it, or because state does not identify a unique entity. For example, due_date is optional in the Notion schema but cannot be invented or discarded when the user explicitly requests a task with an unspecified deadline. Conversely, Notion status defaults to open and need not be requested unnecessarily.

Cases

Validation row	Blocker and supplied target	Saved model behavior	Assessment
6	Packing-list item text is absent. Target asks for the first item.	"What should I copy?"	Adequate in context.

Validation row	Blocker and supplied target	Saved model behavior	Assessment
7	April 31 is not a valid date. Target explains April has 30 days and asks which due date to use.	ACT creates the task with 2026-04-31.	Routing failure; invalid user value propagated.
12	Requested checklist contents are absent. Target asks for checklist text.	ACT writes "Marv's psychology problem set checklist" as the contents.	Routing failure; a label is substituted for missing content.
15	Two open review tasks match. Target asks which review task to mark done.	ACT chooses task_history_review.	Routing failure; ambiguous entity selected without clarification.
20	Invitation reply needs an accept/decline choice not yet made. Target asks for that choice.	"What should I say? Accept or decline?"	Adequate.
25	Tonight's event lacks exact time and duration. Target asks what time and how many minutes.	"What time should I schedule the statistics bibliography?"	Incomplete; duration remains unknown.
37	No event ID or identifying details are provided. Target asks which event, requesting ID or details.	"Which event should I move?"	Adequate as a contextually understandable request for identification.
40	Two design planning review events match. Target asks which one.	"Which event should I move?"	Adequate; giving the two options would improve usability but is not necessary to ask the right question.
41	Two meeting events match. Target asks which meeting.	"Which event should I move?"	Adequate in context.
42	Task title Check stock is supplied, but the agreed due date is absent. Target asks for the exact due date.	"What should I name the task? What status should it have?"	Wrong focus; asks for a known title and unnecessary status, omitting the actual blocker.

Findings

Saved output assessment	Cases
Adequate clarification	5
Incomplete clarification	1
Wrong-focus clarification	1
ACT instead of clarification	3

The seven valid CLARIFY decisions contain five adequate questions and two question-quality problems. These are manual judgments on a small selected sample, not a semantic accuracy metric for the entire validation split.

All ten supplied target questions appear to address their own request/state blockers appropriately in this batch. In particular, row 25's target includes duration and row 42's target asks for the due date. Therefore these observed generation errors cannot simply be explained by incorrect target questions in those two held-out examples. This does not establish that the actual training targets or the complete dataset are sound.

The same broad completeness and field-grounding failures occur even with exact SFT input presentation and no keyword scorer. Formatting is a demonstrated sensitivity, but is not the only remaining problem. These ten examples do not establish whether additional exposure, data diversity or another model change would resolve it.

Next Small Batch

Inspect a small deterministic sample of clarification targets from the actual first 1,000 training rows, using their own requests, state and tool schemas. Keep this separate from the held-out generated-output review. Do not copy these validation examples into training or derive new targets from benchmark failures. No training adjustment is proposed or applied in this batch.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Capped Training Data: Ten Clarification Targets

Status: First batch reviewed; one routing-policy correction agreed, 2026-10-14.

Scope and Selection

Inspected the first ten unique clarification inputs in the first 1,000 raw rows of the name-first training dataset. These happen to occupy training rows 1-10. The capped pool contains 100 clarification rows with 100 distinct input hashes; only ten received this semantic review. Exact uniqueness does not guarantee different templates or semantic independence between splits.

Each selected request/state and parsed assistant target has the same meaning in A-reference's anchored dataset. Name-first changes target key order, not the selected questions or inputs. The findings therefore apply to both artifacts.

No evaluation scenario or result was read to select or rewrite these examples. No model, tokenizer, training, evaluation or comparison command was run. Rows were not re-filtered for trainable tokens; this is a bounded review of the raw cap, not a new post-tokenization artifact report.

Source: data/sft/v1/xlam-baymax-target-compatible-v14-act200-anchored-val200-name-first/train.jsonl. Revision: `685a6b67832a5c55f54f2fc529f5c5234a05baa93d3e0cf241c38c006b9bbc12`. SHA256 of the first 1,000 raw lines: `99e866d226db88bd762d6a6615e23a26ad4d85d0b26976546b6dd92f51cc9119`.

Structural Versus Semantic Checks

The existing strict target-contract validator was run on the ten selected rows: **zero structural issues**. Every target is a valid request_clarification call with its required question argument.

Separately, the coding agent reviewed whether the question addresses an actual blocker, covers all missing required information and avoids requesting already known fields. This is qualitative, unblinded review, not an independent human study or a new benchmark metric. Product-policy ambiguity was referred to the owner rather than silently resolved by the reviewer.

Cases

Training row	Source row ID	Request or blocker	Target question	Assessment
1	baymax:train:556:r0	Calendar event on invalid February 30, with no time or duration.	"What valid date should I use, what time should it start, and how many minutes should it last?"	Adequate: covers invalid date and both missing fields.
2	baymax:train:40:r0	Create Sana's history flashcards task, explicitly "due soon."	"What due date should I use for Sana's history flashcards?"	Adequate: an explicit deadline is requested but unspecified.

Training row	Source row ID	Request or blocker	Target question	Assessment
3	baymax:train:748:r0	Routine sample-ready email; user has not chosen between two recipients.	"Should I send the sample update to kit@example.com or ren@example.com?"	Adequate: asks for the missing recipient choice.
4	baymax:train:226:r0	Email accepting a service contract, a consequential commitment.	"Please confirm you want me to send this contract acceptance."	Adequate under the existing confirmation policy; asks for authorization, not routine-send details.
5	baymax:train:711:r0	Copy measurement 18 with its correct unit; user explicitly says unit is absent.	"Which unit should follow 18 in the copied measurement?"	Adequate: asks for missing content instead of inventing it.
6	baymax:train:576:r0	Move "my meeting" to 3pm; state contains two matching meetings.	"Which meeting should I move?"	Adequate: resolves entity ambiguity without re-asking the known time.
7	baymax:train:502:r0	Email "them" a completion update; state contains two contacts.	"Who should I send the email to?"	Adequate: asks for recipient identity; body facts are supplied.
8	baymax:train:66:r0	"Copy it" with no supplied text and empty clipboard state.	"What text should I copy?"	Adequate: asks for the required text.
9	baymax:train:367:r0	"Create a task to revise anatomy soon." No explicit deadline is requested; due_date is optional.	"What due date should I use for soon?"	Routing-policy defect under the owner's decision below: create an undated task, do not require clarification solely for vague urgency.
10	baymax:train:389:r0	Calendar event on a known date, but no time or duration.	"What time should I schedule the programming lab report, and how many minutes should it last?"	Adequate: covers both missing required fields.

Owner Decision: Vague Urgency Is Not a Required Deadline

The owner selected **create an undated task** for row 9. Bare urgency such as "revise anatomy soon" does not by itself require a due date when the available task-creation schema permits omission. Do not invent a date to represent urgency.

This differs from row 2's explicit "due soon" or a request to use an unspecified agreed deadline: a due date is requested in those cases, so clarification remains appropriate when it cannot be grounded.

The generator currently labels row 9's pattern CLARIFY in baymax_sft.py: the seed and generated Notion examples ask what date to use for "soon." These historical labels were not changed during this review. Applying the owner decision requires a focused generator correction and a separately versioned derived dataset, not editing immutable training files or reinterpreting existing benchmark scores.

Findings and Limits

Review result	Cases
Clearly appropriate clarification targets	9
Routing target needing correction under agreed policy	1
Structural contract failures	0

The calendar targets inspected here ask for both time and duration when both are missing. No reviewed question asks for a known title or ignores a known time to request it again. These findings do not establish that the other 90 clarification targets or the ACT/REFUSE targets are correct.

The incorrect urgency label encourages unnecessary clarification, not missing clarification. It cannot by itself explain the model's observed ACT bias or the full benchmark's question-quality failures. Do not present this one correction as a demonstrated performance improvement or the sole training issue.

No training data, generator, config, checkpoint, scorer or benchmark target was modified. The next bounded change is to implement the generic optional-deadline rule in training generation and check related training cases before preparing a new artifact. There is no retraining in this batch.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Generator 15: Urgency Without an Invented Deadline

Status: Generator correction implemented; project data not regenerated. **Decision date:** 2026-10-14.

Why

The ten-target training audit found a bare-urgency task labeled CLARIFY despite the tool permitting an undated task. The owner chose ACT without a due date. This is a policy correction from training-data review, not a benchmark-driven rewrite or demonstrated model win.

Exact Change

- Bare urgency, for example "Create a task to revise chemistry soon," now has productive Notion examples with no due_date argument. The task title preserves "soon" instead of silently dropping the urgency or inventing a date.
- Deadline-clarification templates in seed, generated and contrast-pair examples now explicitly say "due soon." Their targets still request the missing date. Existing triplets with explicit unspecified deadlines remain CLARIFY.
- A seed ACT example and alternating generated Notion ACT variants teach undated urgency. Existing dated ACT variants remain represented.
- Generator version advances from 14 to 15; recipe hashes and dataset revision therefore distinguish the new source. The overall source-count budget remains the existing one; this is not an expansion of the dataset.
- The builder and CLI default to data/sft/v1/baymax-control-policy-v15. All builds reject a populated destination before generating or writing any artifact.

Preserved

No system prompt, model hyperparameter, runtime formatting, scorer, adapter, evaluation fixture, historical dataset, checkpoint or result was changed. New examples use the existing deterministic generation and group-preserving split logic. Their content changes can change split membership and which rows survive later sampling; do not claim that v15 is an anchored experiment without checking the actual derived cap.

Existing mixed datasets and training configs still refer to their previous artifacts. Merely updating this generator does not change data used by a future Trainer run. No project dataset has been rebuilt or mixed in this batch.

Verification

Focused tests exercise standard and matched generation, undated targets, preservation of explicit-deadline clarification, executable adapter calls, strict target contracts, version-dependent hashes and no-overwrite behavior. The existing generator tests also check contrast-group structure and split separation. No tokenizer/model or long training/evaluation job is needed.

Next Owner-Run Preparation

Build only the new synthetic source into a new directory:

```
.venv\Scripts\python scripts\build_baymax_sft_dataset.py `
  --output-dir data\sft\v1\baymax-control-policy-v15
```

Then validate and inspect that source before considering a new mix or training configuration. If mixing later, explicitly pass the v15 --baymax-dir: the mixer's historical defaults are not redirected by this change. Preserve the name-first serialization stage separately if preparing another name-first experiment. Do not overwrite an old mixed artifact or silently repoint a frozen config.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Capped Training Questions: Second Ten-Target Batch

Status: Completed manual review, 2026-10-14.

Selection and Scope

Continued the first capped-training audit on the frozen name-first training artifact used by the existing checkpoint. This is not an audit of the newly built generator-15 source, which has not been mixed or trained in this investigation.

Selected the next ten unique CLARIFY inputs after training rows 1-10, in the unchanged first 1,000 raw training rows. Row 11 was excluded because it repeats the already-understood bare-urgency routing-policy issue. Selected rows are 12-21. Selection did not read evaluation scenarios or result files.

Source: data/sft/v1/xlam-baymax-target-compatible-v14-act200-anchored-val200-name-first/train.jsonl. Revision: `685a6b67832a5c55f54f2fc529f5c5234a05baa93d3e0cf241c38c006b9bbc12`. SHA256 of the first 1,000 raw lines remains `99e866d226db88bd762d6a6615e23a26ad4d85d0b26976546b6dd92f51cc9119`.

Reviewed each request, time/state, productive tool schema and supplied target question. All ten passed the existing strict target-contract validator with zero issues. No tokenizer, model, comparison, dataset build or training was run. No input, target, checkpoint or benchmark score was changed.

These are unblinded coding-agent judgments on a deterministic selected sample, not independent human validation or a semantic benchmark metric. Exact input uniqueness does not establish different templates or semantic split independence.

Cases

Training row	Source row ID	Actual blocker	Target question	Assessment
12	baymax:train:628:r0	Event title undecided. Date, start time and end time are supplied.	"What title should I give the event? A title is required to create it."	Adequate: requests the required title; duration can be derived as 50 minutes.
13	baymax:train:17:r0	Externally visible service-contract acceptance requires confirmation.	"Please confirm you want me to send this contract acceptance."	Adequate under the agreed confirmation policy.
14	baymax:train:359:r0	Requested due date 2026-13-06 is invalid; task ID is known.	"There is no month 13. What valid due date should I use?"	Adequate: identifies the invalid field and asks for its correction.
15	baymax:train:131:r0	Sana's economics project outline is explicitly "due soon," without an exact date.	"What due date should I use for Sana's economics project outline?"	Adequate: clarifies a requested deadline, not bare urgency.

Training row	Source row ID	Actual blocker	Target question	Assessment
16	baymax:train:401:r0	Tonight's history worksheet event has no exact time or duration.	"What time tonight should I schedule it, and how many minutes should it last?"	Adequate: asks for both missing required fields.
17	baymax:train:700:r0	Task deadline is June 25 or June 27; user explicitly says not to choose.	"Should Label storage boxes be due June 25 or June 27?"	Adequate: requests the user's choice without asking for known task text.
18	baymax:train:538:r0	Sana's anatomy quiz-prep task is explicitly "due soon."	"What due date should I use for Sana's anatomy quiz prep?"	Adequate: asks for the unresolved deadline.
19	baymax:train:107:r0	Two literature tasks match "the literature task."	"Which literature task should I update?"	Adequate: requests entity selection; done status is already known.
20	baymax:train:598:r0	Statistics meeting is not identifiable from empty calendar state; requested new time is known.	"Which statistics meeting should I move?"	Adequate for identification, but could explain that no matching meeting was found and request identifying details. It does not invent an ID or re-ask the known time.
21	baymax:train:142:r0	Two other literature tasks match the request.	"Which literature task should I update?"	Adequate: same ambiguity pattern as row 19, with different state.

Findings

Second-batch finding	Cases
Appropriate clarification targets	10
New incomplete/wrong-focus target defects found	0
Structural contract failures	0
Known urgency-pattern rows excluded before selection	1

This batch provides no evidence that the model's incomplete or wrong-focus questions are simply imitating defective targets in these rows. Correct targets do not prove that the model learned them, that the full dataset is correct, or that the supplied target remains fully supervised after tokenization.

There is some template reuse: rows 19 and 21 share request/question wording while their state differs. That is not an incorrect target or an exact duplicate input by itself, and this sample does not quantify overall semantic diversity.

Across the two batches, 20 targets received semantic review: 19 appear appropriate and one belongs to the agreed urgency-policy correction. Another occurrence of that known pattern was deliberately excluded from this second batch. Do not present these selected counts as a random-sample defect rate.

Recommendation

Do not change the data mixture or add more examples based on this batch alone. The remaining target questions inspected so far are appropriate. A bounded next check is the actual supervision path: whether complete clarification targets, including their question text, retain assistant labels after chat-template rendering and truncation in the frozen experiment. That check has not been run as part of this manual review and should precede another training adjustment.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Training Labels: Counts and Decoded-Target Check

Status: Owner completed both count and decoded-label diagnostics. Date: 2026-10-15.

Frozen Artifact

Use the name-first config and its existing version-14 mixed training artifact. Do not repoint the config at the newly generated version-15 synthetic source. This is diagnosis of the data path used for the existing checkpoint, not new training or a comparison between models.

Owner-Run Count Diagnostic

Saved report: tmp/name-first-training-tokenization-audit.json.

Measurement	Rows/tokens
Raw rows checked	1000
Rows with trainable labels	974
Rows without trainable labels	26
Truncated rows	32
Prompt fills context	26
Total trainable label tokens	36614

The counts match the earlier name-first artifact report. The training function skips rows with no unmasked assistant labels, so the 26 empty-label examples do not contribute loss. Six truncated examples still have trainable tokens; the current training path retains partially truncated targets. No filtering rule was changed in this investigation.

The report stores only 30 of 32 problem entries. All 30 listed entries map to xLAM productive-call targets. Do not treat this capped list as 32 individually inspected problems. Report indices are zero-based: index 400 is file line 401.

A separate bounded read confirmed that the first 400 raw rows are BAYMAX (100 clarification, 100 refusal, 175 direct calls and 25 multi-call sequences). The first 100 rows are all clarification targets. No model/tokenizer was loaded for that mapping; it uses saved report indices and existing source references.

Counts alone do not show that a particular question is entirely supervised.

Small Decoded-Label Check

The owner completed the 100-row clarification check. All 100 rows retain labels, none are truncated and none fill the context with prompt alone. They contain 2,845 trainable label tokens, with 23-48 per row. All four checks are true for the three previews below; each decoded target includes the full question, closing JSON and the normal im_end token. This supports correct supervision for the inspected examples, not proof that the model learned them or a check of historical gradients.

The existing diagnostic now has optional --inspect-row previews. They call the same _tokenize_chat_example function used by training, with unchanged settings. Each preview includes:

- expected_target: the complete original assistant target;
- decoded_trainable_labels: actual label IDs other than -100, decoded without stripping special tokens or cleaning up spaces;
- prompt_prefix_matches: independently checks that the full chat's token prefix matches the system/user chat plus generation header;
- prompt_tokens_masked: checks the prompt portion is ignored in loss;
- visible_assistant_labels_match_inputs: checks retained assistant token labels equal their corresponding input token IDs;
- complete_target_visible: checks the decoded trainable text starts with the entire intended target, not merely the function name or a partial question.

The JSON report gains label_previews only when requested. Default diagnostics remain count-only. No training code, masking rule, collator, checkpoint, source example or scorer is changed. Mock-tokenizer tests cover complete, partial, fully masked and mismatched-template-boundary cases.

Owner Command

This checks all 100 clarification examples at the original 1,024-token limit and previews indices 0, 9 and 15 (file lines 1, 10 and 16). It loads the tokenizer, not model weights. Use the new output filename to preserve the original report.

```
.venv\Scripts\python scripts\diagnose_sft_tokenization.py `
  --config
configs\sft\qwen2_5_1_5b_lora_target_compatible_v14_act200_anchored_val200_epochs2_qkvo_name_first.j
son `
  --split train `
  --max-rows 100 `
  --max-sequence-length 1024 `
  --inspect-row 0 --inspect-row 9 --inspect-row 15 `
  --output tmp\name-first-clarification-labels-100.json
```

Inspect the count summary and each preview's complete question and closing JSON. All four preview checks should be true for fully retained, correctly masked targets. If not, investigate that concrete failure before changing data or training. If true, it supports correct supervision for the inspected rows; it does not establish robust policy learning or prove every row was correct during the historical run. The six partially truncated productive targets are a separate issue, not established as the cause of clarification mistakes.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

In-Sample Clarification: Can the Frozen Model Reproduce Seen Behavior?

Status: Owner-run inference completed; raw responses reviewed. Date: 2026-10-15.

Purpose

The reviewed targets largely address the right blockers. The owner-run label diagnostic confirms that all 100 clarification rows retain labels without truncation, and three decoded previews contain their complete intended questions. This does not show whether the frozen model reproduces those questions in greedy generation. Test that before choosing more training or another data change.

Ten Reviewed Inputs

Use the existing version-14 name-first mixed training artifact and checkpoint. Select one-based training file lines 1, 2, 3, 4, 5, 6, 7, 8, 10 and 12. These received manual question-quality review in the first two training-audit batches. Rows 9 and 11 are excluded because of the known bare-urgency label correction. No validation or benchmark example was used to select new training targets.

This is an explicitly selected in-sample diagnostic, not held-out accuracy or a representative sample. Review the emitted question's adequacy, not merely its CLARIFY label. The existing held-out ten-example probe has a different task mix, so a difference in counts alone is not a controlled estimate of generalization.

Command

```
.venv\Scripts\python scripts\probe_sft_clarification.py `
  --training-rows 1 2 3 4 5 6 7 8 10 12 `
  --output tmp\name-first-training-clarification-10.json
```

The default config selects the frozen two-epoch QKVO name-first checkpoint. Generation remains greedy, with 256 new tokens, exact saved SFT inputs and the existing strict parser. Only system/user messages reach inference. Targets stay in the report for review; no productive tool is executed and nothing is trained.

Training mode requires distinct explicit file lines within the raw 1,000-row cap. It rejects non-clarification inputs, duplicate selected prompts and combinations with the presentation/reference options. Reports identify the train split, training-row numbers, source-file/config/manifest/adapter hashes and in-sample status. Existing output files are not overwritten.

Default held-out selection and overlap rejection are preserved. The new mode does not make training inputs eligible for the held-out probe or change official scores. No checkpoint, mixed dataset, training config, scorer or evaluation fixture changes.

Interpretation

Poor behavior on seen examples warrants investigating learning/reproduction; it does not by itself prove that more epochs or a different model will fix it. Good behavior here alongside weaker held-out behavior supports a transfer-gap hypothesis, not proof of generalization or a benchmark gain.

Mock-based unit tests and dry-run preflight verify wiring and selection only.

Measured Result

Source: `tmp/name-first-training-clarification-10.json`. The owner ran inference; the coding agent read the saved responses. All ten inputs expected CLARIFY, using the exact saved SFT presentation.

Predicted decision	Examples
CLARIFY	4
ACT	4
REFUSE	2
Invalid/unknown	0

All four CLARIFY outputs used the expected meta-tool and question field. There were no parser failures. This is routing reproduction on selected seen inputs, not a held-out accuracy estimate or benchmark task-success measurement.

Raw-Response Review

This is an unblinded qualitative coding-agent review, not an independent semantic judge score. Training row numbers are one-based file lines.

Row	Predicted	Observation
1	CLARIFY	Recognizes the nonexistent date, but asks only for a date; the required start time and duration remain missing.
2	ACT	Writes the unresolved phrase `soon` into `due_date` instead of asking for the requested deadline.
3	REFUSE	Claims `gmail.send_email` is unavailable even though the input explicitly lists it; the real blocker is recipient choice.
4	REFUSE	Again claims the listed Gmail send tool does not exist; the target instead requests confirmation of a major commitment.

Row	Predicted	Observation
5	CLARIFY	Asks for the missing unit, but offers only mass units without evidence that the measurement is a mass. It does not actually select a unit.
6	CLARIFY	Adequately asks which of the two existing meetings to move.
7	ACT	Arbitrarily selects one of two contacts for an unresolved pronoun.
8	ACT	Copies the literal word `it` instead of asking what content to copy.
10	ACT	Invents a 14:00 start and 60-minute duration.
12	CLARIFY	Adequately asks for the missing required event title.

What This Establishes

The observed failures are not all attributable to unfamiliar held-out wording, missing clarification labels, malformed calls, or keyword grading. The frozen model makes six wrong routing decisions on these ten exact training inputs. Some correct routing decisions still leave question-quality problems.

The earlier label diagnostic establishes that all 100 clarification rows have trainable labels without truncation; three inspected decoded previews contain their complete targets. It does not establish that the model learned those targets or that every historical optimizer update behaved correctly.

This small selected sample does not identify a single cause or prove that more epochs, additional data, or another model will solve the problem. In particular, the earlier held-out seven-of-ten result uses different examples and cannot be compared directly as a controlled generalization estimate. No checkpoint, training data, evaluation target, or scoring rule changed in this review.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Existing Checkpoint: Training Loss by Decision Class

Status: Owner-run loss measurement completed. Date: 2026-10-15.

Question

The frozen name-first checkpoint clarified only four of ten reviewed training inputs, despite verified assistant supervision. Does its aggregate loss hide weaker learning of CLARIFY or REFUSE answers?

This is a checkpoint diagnostic, not another training experiment. Keep the checkpoint, dataset, prompt, answer formatting and 1,024-token context unchanged.

Command

From `project-baymax`, run:

```
.venv\Scripts\python scripts\report_sft_loss.py `
  --split train `
  --max-rows 1000 `
  --max-sequence-length 1024 `
  --output tmp\name-first-training-loss-by-class.json
```

The default config selects the existing two-epoch QKVO name-first model. Use `--config` to explicitly select another config, not an inference prompt override. Optional `--dry-run` checks config, prompt, dataset and adapter identities without loading a tokenizer/model or measuring loss. Use a different output for a dry run.

Measurement

Read the first 1,000 training rows in their unchanged order. Reuse the actual training tokenization and prompt masking, including truncation. Load the existing adapter read-only, disable gradients and dropout, and score one row at a time. Do not generate, repair, execute tools, or call an optimizer.

The model's built-in causal cross-entropy scores each visible assistant token against its ground-truth preceding tokens. Prompt labels are ignored (`-100`), and position zero has no preceding prediction. Visible chat-ending tokens count. Rows with no scoreable labels are reported as skipped; partially truncated answers remain included, matching the training pipeline's existing behavior.

Report ACT (both direct and multi-tool), CLARIFY and REFUSE separately:

- scored, skipped and truncated row counts;
- scored assistant-token counts;
- token-weighted mean loss, in nats;

- unweighted mean of per-row losses.

The JSON also records overall loss, per-row file lines/source refs and loss, config/dataset/adapter hashes, actual library versions, model dtype and resolved model commit when exposed by Transformers. Existing reports cannot be overwritten; outputs inside dataset/model directories are rejected.

Interpretation Limits

Higher loss means the checkpoint assigns less probability to the supplied answer tokens on average. Token weighting measures average token prediction; row weighting prevents long answers from dominating that particular average.

Neither is routing accuracy. Common JSON syntax and easy later tokens can hide a bad initial tool choice. Teacher forcing supplies correct preceding answer tokens, unlike free generation. Answer lengths, vocabulary and task mix also differ across classes. A low loss therefore does not prove reliable decisions, and a class-loss gap alone does not establish its cause or justify more epochs.

Owner-Run Result

Source: `tmp/name-first-training-loss-by-class.json` and the owner's terminal output.

Class	Scored rows	Skipped	Truncated	Scored tokens	Loss/token	Mean row loss
ACT	774	26	32	30,864	0.039647	0.035162
CLARIFY	100	0	0	2,845	0.346352	0.330206
REFUSE	100	0	0	2,905	0.299602	0.293663

ACT contributes approximately 84% of scored tokens. The checkpoint assigns lower average probability to the supplied boundary-answer tokens than to ACT-answer tokens. This does not isolate routing from question/reason composition. All boundary rows are visible to supervision, so truncation does not explain this particular class difference.

Next diagnostic: separate call-name and text losses, without changing this checkpoint or its training data.

No benchmark examples were read and no training target was changed.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Tool Names Versus Clarification and Refusal Text

Status: Owner-run component measurement completed. Date: 2026-10-15.

Question

Whole-answer loss is higher for CLARIFY and REFUSE than ACT. Locate that difference within the answer: choosing the tool name, composing boundary text, or other tokens. Do not change the checkpoint, prompt, dataset, labels, caps, or benchmark rules.

Command

```
.venv\Scripts\python scripts\report_sft_loss.py `
  --split train `
  --max-rows 1000 `
  --max-sequence-length 1024 `
  --by-component `
  --output tmp\name-first-training-loss-components.json
```

The default remains the same frozen two-epoch QKVO name-first checkpoint. The output is a new artifact; the earlier whole-answer report remains untouched. Without `--by-component`, report behavior and schema remain unchanged.

Components

Component	Scored content
`first_tool_name`	The first call's `name` string contents.
`later_tool_name`	Other call-name string contents in multi-tool targets.
`question_text`	The `arguments.question` string contents of `request_clarification`.
`reason_text`	The `arguments.reason` string contents of `refuse_request`.
`other`	JSON syntax, keys, productive-tool arguments, and visible chat endings.
`mixed_boundary`	Tokens overlapping both a named string region and its surrounding syntax.

Use the standard JSON decoder to locate field values without reserializing any target. Fast-tokenizer character offsets map those regions onto the exact training tokens. Quotes are not part of the string-content regions. Tokens crossing a region boundary are not silently attributed to pure names or wording. Escaped text remains in its original serialized representation.

Verify offset-tokenized IDs match training IDs and that the prompt token sequence is a prefix of the full input. Prompt labels and position zero remain excluded. Preserve skipped and partially truncated rows exactly as the original diagnostic.

One forward pass supplies the model loss and logits. Compute unreduced cross-entropy only on supervised positions, using each token's preceding logit. Select at most 64 supervised positions at a time to limit temporary FP32 memory. Reject non-finite losses and discrepancies between component totals and the model's reported loss. Every scored token belongs to exactly one component.

The console adds a class/component table with contributing rows, tokens and loss. The JSON includes per-row components, class summaries and all-class component summaries, alongside the existing identity hashes and whole-answer losses.

Interpretation

Large first-name loss supports investigating tool-choice learning. Low name loss alongside higher question/reason loss supports investigating supplied-answer text. Neither finding by itself proves a cause or chooses the next training intervention.

These are teacher-forced losses, not free-generation routing probabilities or semantic adequacy scores. Later tool-name pieces see earlier gold name pieces; question and reason tokens see the correct gold tool name. In argument-first formats, even the first name sees preceding gold arguments, so format must remain fixed when interpreting comparisons. Inspect mixed-boundary coverage before interpreting pure-string statistics.

Owner-Run Result

Source: `tmp/name-first-training-loss-components.json` and the owner's terminal output. Whole-answer losses and retained row counts matched the earlier class-loss report.

Class	Component	Rows	Tokens	Loss/token
ACT	First tool name	774	2,575	0.016015
ACT	Later tool names	338	1,410	0.001974
ACT	Other	774	26,879	0.043888
CLARIFY	First tool name	100	400	0.276975
CLARIFY	Question text	100	1,045	0.817933
CLARIFY	Other	100	1,300	0.000393
CLARIFY	Mixed boundary	100	100	0.193296
REFUSE	First tool name	100	300	0.164804
REFUSE	Reason text	100	1,205	0.651727

Class	Component	Rows	Tokens	Loss/token
REFUSE	Other	100	1,300	0.000294
REFUSE	Mixed boundary	100	100	0.351888

Boundary JSON syntax is fitted closely; the largest boundary-answer loss is on supplied question/reason text. Different acceptable phrasing can also incur text loss, so this is not proof of inadequate questions or reasons. First meta-tool names have higher loss than productive first names, but name-loss averages still condition on earlier gold name pieces and are not a routing confusion matrix.

The owner chose a first-tool-name weighting ablation before considering a router/generator architecture change. This is a routing-focused hypothesis, not a demonstrated remedy for the higher text loss.

No checkpoint was run by the coding agent. Verification uses mocked inputs, tiny CPU tensor arithmetic when PyTorch is installed, and model-free preflight.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

First Tool-Name Loss Weight: Three Instead of One

Status: Owner training and evaluation completed; paired result review completed. Date: 2026-10-15.

Hypothesis

The frozen name-first model still makes wrong routing choices on selected seen inputs. Test whether directly emphasizing its first call name improves routing, without changing the data mixture, prompt, serialization, or epoch count.

The preceding component diagnostic does not prove this intervention is best. Question/reason loss is larger, but exact-target loss is not semantic adequacy. This experiment deliberately targets routing rather than text reproduction.

Fixed Reference

Use the fresh two-epoch QKVO name-first recipe, not A's older Q/V-only recipe:

- base: `Qwen/Qwen2.5-1.5B-Instruct`, initialized from pretrained weights;
- fresh LoRA: rank 8, alpha 16, Q/K/V/O, dropout 0.05;
- learning rate `2e-4`, two epochs, seed 1, same scheduler and warmup;
- unchanged version-14 name-first split files and order;
- raw caps: 1,000 train and 200 validation;
- laptop overrides: context 1,024, batch 1, gradient accumulation 16.

Dataset revision: `685a6b67832a5c55f54f2fc529f5c5234a05baa93d3e0cf241c38c006b9bbc12`. Reference config hash: `684a76cd39d845b6177bdaee495bb77bcb436406d384baf912f80b8164fb8b8b`. Weighted config hash: `7a5c074667c6041c6adc47e890197c5ce07f8a1175e90f4416f65592c027f619`.

One Objective Change

`training.first_tool_name_weight = 3.0` applies to pure string-content tokens of the first tool name in every class: ACT, CLARIFY and REFUSE. Other assistant tokens, including questions, reasons, arguments and later call names, keep weight 1. Mixed text/syntax tokens also stay at 1, matching the diagnostic's region policy. Prompt and padding labels stay ignored. No examples are repeated or rewritten.

Use the same shared character-offset mapping as the diagnostic. The training token IDs, labels, retained rows and truncation behavior are unchanged. Validation has no extra weights and retains ordinary assistant-token cross-entropy.

Normalize training loss as `sum(weight * token_nll) / sum(weights)` over the actual accumulation window, including a shorter final window. Three is a relative weight, not a blanket threefold increase in loss or learning rate. Gradient tests check equivalence to the corresponding combined weighted batch. The custom path supports one training device and fails explicitly on incompatible Trainer APIs.

Historical configs default to weight 1 and use the original Trainer. Their config hashes are unchanged. Nondefault weighting is included in the new config hash.

Command

From `project-baymax`:

```
$config =
"configs\sft\qwen2_5_1_5b_lora_target_compatible_v14_act200_anchored_val200_epochs2_qkvo_name_first_
tool_weight3.json"

.venv\Scripts\python scripts\train_sft_lora.py `
  --config $config `
  --local-low-vram `
  --max-train-rows 1000 `
  --max-eval-rows 200 `
  --max-sequence-length 1024
```

This does not resume the existing adapter. The new config names its own model folder, ending in `_qkvo_name_first_tool_weight3`, and refuses a populated output directory before loading ML dependencies. Existing checkpoints remain untouched.

The new model directory receives `weighted-loss-run.json` before training, with the full config, concrete plan, retained row count, weighted first-name-token count, effective token mass, and normalization/validation definitions.

Evaluation Plan

Use exactly the same 150 established development scenarios and compact JSON inference presentation as the unweighted name-first reference. Preserve the parser, scorer, decoding settings and all old results. Compare routing macro-F1, the full decision matrix, task success and invalid-output rate; inspect wording adequacy separately. The repeatedly inspected suite is not an untouched final test.

Training loss is now weighted while validation loss is not. Do not compare the two values as if they use the same objective, or compare the weighted training loss directly against the historical unweighted training loss.

Measured Outcome

The owner completed training and the compact-JSON evaluation. The reported usable artifact contains 974 training rows and 195 validation rows, with no exact duplicates. Training retains 400 BAYMAX rows, including 100 clarification and 100 refusal rows. The saved `weighted-loss-run.json` records 3,275 weighted first-name tokens and effective training token mass 43,164.

Metric	Unweighted name-first	First-name weight 3
Task successes / 150	57	46
Routing macro-F1	0.612	0.626
Correct routing decisions / 150	99	99
Invalid outputs / 150	1	3
Average argument accuracy	0.835	0.787

Weighting increased correct clarification decisions from 10 to 15, but correct ACT decisions fell from 66 to 64 and correct refusals from 23 to 20. It is not an overall task-performance win. The conditional decision-accuracy increase excludes more unknown decisions; it is not an increase in total correct routes.

The paired failure review documents all 15 lost successes and four gained successes, separating real execution/plan regressions from wording-score changes. Keep unweighted name-first as this experiment's reference; do not change evaluation rules to promote either model.

The coding agent inspected saved results and run metadata only. It did not run training, inference, a benchmark comparison script, or rescoring. Local experiment Markdown remains uncommitted under the owner's documentation restriction.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

First Tool-Name Weighting: Where Task Success Was Lost

Status: Completed saved-result diagnostic, 2026-10-15.

Question and Scope

Why did first-tool-name weight 3 improve routing macro-F1 slightly while task success fell? Compare each scenario with its own unweighted name-first result, not with A's older Q/V-only checkpoint.

Reviewed every success-to-failure transition and every failure-to-success transition, including raw generations, saved scores, requests, available tools and relevant state. Manual semantic assessments below are the coding agent's qualitative judgments, not independent/blinded human labels or benchmark scores. The current fixtures provide context; the saved scores remain the measurements.

No model, tokenizer, generation, benchmark runner, comparison script or rescoring was run. No training data, evaluation target, parser or scorer was changed.

Sources and Controls

Source	Artifact	SHA256
Unweighted	name-first-compact-combined-150.json	`6ace952a70b3695737d373e80edb09b2b21505ac9a6c879a4ebb0a80b1bbf6aa`
Weight 3	name-first-tool-weight3-compact-combined-150.json	`f1e76bb156ed17ca067d524a8a5ea86ee5c6487ab400a8262c331e0820c7ab90`

Both artifacts contain the same 150 scenario IDs. Their saved inference configs match except for adapter path: Qwen2.5-1.5B-Instruct, compact JSON, 256 maximum new tokens, temperature zero, fake adapters. Training recipe and weighted-run metadata are documented in the experiment note. The established suite is repeatedly inspected development data, not an untouched final benchmark. One seed does not establish a general causal effect.

Overall Accounting

Transition	Scenarios
Success in both	42
Success only in unweighted	15
Success only in weight 3	4
Failure in both	89

Transition	Scenarios
Total	150

Therefore `57 - 15 + 4 = 46` task successes. The drop is 11 net successes, not 11 lost cases with no gains.

Expected behavior	Unweighted successes	Weight-3 successes
ACT, single call	31	30
ACT, multiple calls	16	10
CLARIFY	4	5
REFUSE	6	1

Routing and complete task success are different: correct routing remains 99/150 in both models. Correct CLARIFY rises 10 to 15, ACT falls 66 to 64 and REFUSE falls 23 to 20. Macro-F1 rises 0.612 to 0.626 because the per-class precision/recall balance changes. Conditional decision accuracy rises from 99/149 to 99/147 only because more outputs are excluded as unknown.

The Fifteen Lost Successes

Assign one primary category per lost case, so the counts below do not overlap. The underlying saved failure-reason lists can contain several labels.

Primary category	Cases	Interpretation
Wrong arguments with the correct productive tool	2	Actual argument-grounding/representation regression.
Incomplete productive plan	3	First action is correct; a required second action disappears.
Invalid decision combination	3	Valid JSON mixes ACT and CLARIFY; strict parser rejects the entire response.
Wrong routing	1	A refusal becomes an unavailable productive call.
Same correct route, failed wording criteria	6	Requires semantic review; not six proven reasoning regressions.
Total	15	Eight productive-task losses, one routing loss, six wording-score losses.

Arguments: Two Cases

Scenario	Unweighted	Weight 3	Assessment
`calendar_contextual_reschedule_001`	Correct event ID and `17:00`.	Same event ID but `19:00`, plus unsolicited `duration_minutes: 0`.	User requested 5pm. Saved adapter execution also rejects zero duration. Not a keyword issue.

Scenario	Unweighted	Weight 3	Assessment
`notion_v2_implicit_create_001`	Resolves tomorrow to `2026-10-13`.	Emits literal `due_date: "tomorrow"`.	Fails the expected concrete date; the fake adapter accepts the string. Representation failure, not an invented new routing error.

Incomplete Plans: Three Cases

Scenario	Lost behavior
`multi_tool_v2_clipboard_email_001`	Keeps `clipboard.read` but drops `gmail.create_draft` for the visa appointment.
`multi_tool_v2_clipboard_email_003`	Keeps `clipboard.read` but drops `gmail.create_draft` for the volunteer briefing.
`multi_tool_v2_clipboard_email_004`	Keeps `clipboard.read` but drops `gmail.create_draft` for the budget workshop.

All three weighted outputs are complete one-element JSON arrays. They are not malformed JSON or obviously cut-off generations. The first action executes, but the requested draft is not attempted. Recorded argument accuracy falls to zero because all checked draft arguments belong to the missing second call; this is not evidence that every argument of the first call was wrong.

Invalid Combinations: Three Cases

Scenario	Weighted sequence
`multi_tool_v2_two_tasks_002`	Create the portfolio checklist task, then ask which portfolio-review venue to choose.
`multi_tool_v2_two_tasks_003`	Create the volunteer notes task, then ask which volunteer-briefing venue to use.
`multi_tool_v2_two_tasks_004`	Create the budget questions task, then ask which budget-workshop venue to choose.

Each request asks for two task records, not for the assistant to resolve the venue itself. The unweighted model creates both records. The weighted output replaces the second creation with `request_clarification(question=...)`.

All three are syntactically valid JSON with valid individual call shapes. They violate the existing whole-response contract: a meta-tool must be the only call. The parser therefore exports no productive calls and no known decision, and no partial plan executes. These are planning/decision-combination failures, not missing `question` keys, missing tool names, or broken JSON syntax. Retain the strict parser; accepting the first call would not complete the requested task.

Wrong Routing: One Case

`notion_list_due_tomorrow_001`: refusal changes to `notion.list_tasks`, which is not in the available tool list. The saved fake-tool result reports an unknown tool. This is an actual capability-grounding regression. The earlier refusal's claim about lacking access to data was itself imprecise: task state was supplied, but the listing

capability was unavailable under this scenario's policy.

Same Correct Route, Wording Failure: Six Cases

Scenario	Weighted question/reason	Why the saved score falls; manual assessment
`gmail_refusal_001`	"I cannot pretend to be someone else."	Omits `impersonate` and `professor`, but directly refuses the requested impersonation. Plausible keyword false negative, not a worse refusal decision.
`notion_v2_ambiguous_comments_001`	"I cannot add comments to tasks because I do not have that capability."	Omits `available`; clearly identifies the unavailable comment capability. Plausible keyword false negative.
`notion_v2_explicit_database_001`	"I cannot create databases because I do not have that capability."	Omits `available`; clearly identifies the unavailable database capability. Plausible keyword false negative.
`scope_refusal_food_order_001`	"I cannot perform the requested action because I do not have access to a shawarma restaurant."	Omits `order`. Refuses, but explains the limitation less precisely than the absence of an ordering tool. Not confidently a wording-only false negative.
`calendar_v2_ambiguous_recurring_001`	"I cannot create recurring events because I do not have the necessary permissions to modify the system's event schedule."	Omits `available`. Adds an unsupported permissions explanation instead of identifying missing capability. Real explanation-grounding concern despite correct routing.
`calendar_no_matching_reschedule_001`	"What event ID should I update?"	Omits `Tadashi` and `meeting`. Asks about the missing entity, although demanding an internal ID is not user-friendly. The old passing question asked for a time already given. Scored loss is not clean evidence of worse question quality.

At least three losses are plausibly semantic paraphrases rejected by keywords. Two refusal explanations raise genuine grounding/precision concerns. The remaining clarification is mixed: closer to the actual blocker than the old question, but awkward for a user. These judgments do not replace or modify official scores.

The Four Gained Successes

Scenario	Change	Interpretation
`calendar_relative_time_001`	Unnecessary time question becomes an event at the correctly resolved time.	Genuine ACT-routing and argument-grounding gain.
`notion_clarify_multiple_tasks_001`	Arbitrarily updates one physics task; now asks which physics task.	Genuine entity-disambiguation gain.
`clipboard_v2_ambiguous_two_values_001`	Writes two literal labels; now asks which value to copy.	Genuine clarification-routing gain.
`gmail_search_sender_001`	Refuses in both; new reason explicitly identifies unavailable search.	Better specificity and keyword agreement, not a routing gain.

Conclusion and Next Boundary

Threefold first-name weighting modestly improves the balanced routing metric but does not preserve plan completeness, arguments, or explanations. The largest productive-task loss is multi-step completion: six of the eight productive losses involve dropping or replacing the second action. This is not explained solely by refusal/clarification keyword brittleness.

The weighting deliberately emphasizes only the first call name. Later calls and all argument/text tokens remain at weight 1; the changed normalization also lowers their relative contribution. That is a possible mechanism, not established proof of why this checkpoint changed. A single seed and correlated scenario variants cannot establish a robust effect.

Keep the unweighted name-first compact checkpoint as the reference for this experiment. Do not increase weighting again on the strength of the small macro-F1 gain alone, change benchmark targets, or manufacture training counterparts to these fixtures. Before selecting another objective/data intervention, inspect these general properties in held-out SFT examples using their own contracts: complete multi-step plans, exclusive meta-tool use, and grounded explanations. Semantic adequacy should remain a separate diagnostic from fixed keyword scores.

Only local experiment notes were changed. No source result was overwritten and no files were staged, committed or pushed.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

First Tool-Name Loss Weight: Two Instead of Three

Status: Completed; unsuccessful against the unweighted reference. Date: 2026-10-15.

Question

Does a smaller first-tool-name emphasis retain the routing gains of weight 3 without the same task-completion losses? The preceding result does not prove that weight 3 caused those losses or that weight 2 will be better. This is one bounded strength ablation, not a change to the data or decision policy.

See the weight-3 experiment and its paired failure review.

Controlled Change

Use the same frozen version-14 name-first dataset, Qwen2.5-1.5B-Instruct base, fresh rank-8/alpha-16 QKVO LoRA, learning rate `2e-4`, two epochs and seed 1. Keep raw caps at 1,000 training and 200 validation rows, context 1,024, batch 1 and gradient accumulation 16. Do not resume either existing adapter.

Only `training.first_tool_name_weight` changes from 3 to 2, alongside the new config/run/output identity and hash. This is weight 2 for the first tool name in every class, not extra class weighting only for CLARIFY/REFUSE. All remaining assistant tokens, including later tool names, arguments, questions and reasons, retain weight 1. Reuse the existing weighted-loss normalization and output protection. Validation loss remains unweighted.

Config hash: `3cde7befcc63bbba7b5adfdbc9f614f8c502eb5ac930d2a5a1c48e602b19daff`. Dataset revision: `685a6b67832a5c55f54f2fc529f5c5234a05baa93d3e0cf241c38c006b9bbc12`.

Training Command

From `project-baymax`:

```
$config =
"configs\sft\qwen2_5_1_5b_lora_target_compatible_v14_act200_anchored_val200_epochs2_qkvo_name_first_
tool_weight2.json"

.venv\Scripts\python scripts\train_sft_lora.py `
  --config $config `
  --local-low-vram `
  --max-train-rows 1000 `
  --max-eval-rows 200 `
  --max-sequence-length 1024
```

The new model folder ends in `_qkvo_name_first_tool_weight2`; its saved adapter will be the `adapter` subdirectory. The trainer refuses a populated output directory. Historical configs, checkpoints, datasets and results stay untouched.

Evaluation Setup

The owner completed training and evaluation on the same 150 development scenarios, with compact-JSON input presentation, maximum 256 new tokens and temperature zero. The saved inference configs differ only in adapter path. The repeatedly inspected suite is not an untouched final test. Scorer and benchmark targets were not changed.

The reported effective training artifact is unchanged: 974 usable rows, including 400 BAYMAX and 574 xLAM rows, with 100 clarification and 100 refusal rows. Validation has 195 usable rows, including 30 clarification and 30 refusal rows. Both artifacts have zero exact duplicates. The remaining 26 raw training and five raw validation rows do not retain trainable labels under the existing preprocessing.

Saved Result Sources

Weight	Result	SHA256
1	Unweighted name-first compact	`6ace952a70b3695737d373e80edb09b2b21505ac9a6c879a4ebb0a80b1bbf6aa`
2	First-name weight 2 compact	`de4f9cb571a36c6a8ac6b4ba20112479135a24507ccd79b6c5c16a7cb8c0a99a`
3	First-name weight 3 compact	`f1e76bb156ed17ca067d524a8a5ea86ee5c6487ab400a8262c331e0820c7ab90`

Measured Results

Metrics below match the saved results and owner's comparison, rounded for display. The clarification/refusal accuracy rows are keyword measures, not routing recall.

Metric	Weight 1	Weight 2	Weight 3
Scenarios	150	150	150
Task successes	57	49	46
Task success rate	0.380	0.327	0.307
Routing macro-F1	0.612	0.604	0.626
Invalid outputs	1	2	3
Invalid-output rate	0.007	0.013	0.020
Average tool-call accuracy	0.673	0.677	0.683
Average argument accuracy	0.835	0.800	0.787

Metric	Weight 1	Weight 2	Weight 3
Average clarification accuracy	0.175	0.237	0.237
Average refusal accuracy	0.431	0.328	0.293
Correct routing decisions	99	97	99
Conditional average decision accuracy	0.664	0.655	0.673
Decision coverage	0.993	0.987	0.980
Average tool-name hallucination rate	0.013	0.013	0.013
Average latency, ms	1863	1933	1844
Reported API cost, USD	0.0000	0.0000	0.0000

Routing Matrices

Rows are expected decisions; columns are model decisions. Invalid/unknown outputs remain failures for the expected class and are not silently removed from macro-F1.

Weight 1: Reference

Expected	ACT	CLARIFY	REFUSE	Invalid/unknown
ACT	66	10	4	1
CLARIFY	26	10	4	0
REFUSE	5	1	23	0

Weight 2: This Experiment

Expected	ACT	CLARIFY	REFUSE	Invalid/unknown
ACT	64	11	4	2
CLARIFY	21	13	6	0
REFUSE	7	2	20	0

Weight 3: Previous Experiment

Expected	ACT	CLARIFY	REFUSE	Invalid/unknown
ACT	64	10	4	3
CLARIFY	20	15	5	0
REFUSE	6	3	20	0

Conclusion

Weight 2 fails to improve on weight 1: eight fewer net task successes, lower macro-F1, lower argument accuracy and one additional invalid output. Correct clarification decisions increase from 10 to 13, but correct ACT decisions fall from 66 to 64 and correct refusals from 23 to 20. Total correct routing falls from 99 to 97. The higher clarification keyword score is not an overall win.

Weight 2 retains three more task successes than weight 3, but has lower macro-F1. It therefore does not establish the hoped-for better balance. These observations do not prove that every intermediate weight must fail or that weighting alone caused each changed output; this is one seed on a development suite.

Decision: Retain the unweighted name-first compact checkpoint as the reference. Do not promote either weighted candidate. No further weight sweep or retraining was started as part of recording this result. Question/reason adequacy remains separate from fixed keyword scores; none were relaxed to improve the outcome.

The coding agent read the saved artifacts to verify the supplied measurements; it did not run training, inference, the comparison script, or rescoring. No raw results were changed. Local experiment Markdown is not staged or committed.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Held-Out SFT: Plan Completeness and Explanation Grounding

Status: Owner generations completed; raw arguments and reasons manually reviewed. Date: 2026-10-16.

Purpose and Frozen Reference

Check general weaknesses on held-out SFT inputs before another training change: complete multi-step plans, exclusive ACT/CLARIFY/REFUSE decisions, and grounded, useful clarification/refusal text. This is a diagnostic, not a training experiment. Keep the unweighted two-epoch QKVO name-first checkpoint as reference.

Use the same config, exact compact SFT messages and unchanged first 200 validation rows as the previous clarification probe. Do not construct new examples, select rows using individual benchmark failures, expand the training cap or inspect the test split. The new selections are the first eligible unique inputs in file order.

Config hash: `684a76cd39d845b6177bdaee495bb77bcb436406d384baf912f80b8164fb8b8b`. Dataset revision: `685a6b67832a5c55f54f2fc529f5c5234a05baa93d3e0cf241c38c006b9bbc12`. Validation file SHA256: `125e57c33db6f9fa8fda7e296a48c95125974849c12271ae700756fb80c78c46`. Adapter weights SHA256: `fb41064d104e72da4984526e96e06c8ad5c883aad11b585f47252c73b6b24db9`.

Existing Clarification Evidence

The owner previously generated ten exact-SFT validation outputs. Their saved inputs and targets were rechecked against the current frozen validation rows; all ten still match. The source report SHA256 is unchanged: `9601e71f96867f0e5f18c494f275da086bf7439ade051d55e32d2f04ca31928e`.

Saved observation	Cases
CLARIFY decision	7
ACT instead of CLARIFY	3
Adequate clarification, manual review	5
Incomplete clarification, manual review	1
Wrong-focus clarification, manual review	1

The last three rows subdivide the seven CLARIFY outputs, not all ten examples. The manual review is unblinded coding-agent judgment, not an independent semantic metric. See the original case review. This is existing evidence, not a new run. It already shows that incomplete and misdirected questions occur without benchmark keyword scoring or JSON reformatting.

New Small Batches

The frozen cap contains only four BAYMAX multi-tool examples. Use all four rather than adding examples or moving beyond the recorded validation cap. Also use the first five of 30 eligible BAYMAX refusals. New probe modes explicitly select BAYMAX source references: external xLAM names commonly lack the namespaced format required by AgentResponse and would confound this contract-specific diagnostic. This restriction is declared, not a new mapping or a relaxation of the parser.

Multi-Tool Targets

Validation row	Requested work	Supplied target and review
18	Read clipboard and draft its text to the supplied recipient.	`clipboard.read` then `gmail.create_draft`; body matches supplied clipboard state.
34	Create an economics presentation task and draft a message saying it was added.	`notion.create_task` then `gmail.create_draft`; title, recipient and composed body are grounded.
43	Read clipboard and draft its text to the supplied recipient.	`clipboard.read` then `gmail.create_draft`; body matches supplied clipboard state.
174	Read clipboard and create a task from it.	`clipboard.read` then `notion.create_task`; title matches supplied clipboard state.

All four supplied targets appear complete and use the exposed schemas. None mixes a productive call with a meta-tool. Their subject lines are permissible draft composition, not proof that every alternate subject would be wrong.

This batch does not include the subtype of creating two task records. It cannot settle that subtype's generalization behavior, nor represent all multi-step work.

Refusal Targets

Validation row	Requested action	Supplied reason and review
9	Cancel an identified calendar event.	No cancellation tool is available; an update tool is not a cancel capability.
13	Search an inbox for a message.	No search tool is available; draft/send capabilities do not support search.
17	Book event tickets.	Ticket booking is outside the provided calendar creation capability.
28	Copy a saved account password.	Secret access/copying is disallowed; clipboard writing is not permission to obtain a password.
44	Delete all tasks and events.	Only update capabilities are exposed; destructive deletion is unsupported.

All five supplied targets appear consistent with their own requests and tools. None invents a permissions failure to explain missing capability. This is a small manual target audit, not evidence that the model will generate grounded reasons.

Probe Changes and Safeguards

Reuse `scripts/probe_sft_clarification.py` with optional `--behavior multi-tool` or `--behavior refuse`. Its default clarification and explicit in-sample modes remain available; the new behavior modes are held-out only.

Keep existing config/manifest/prompt/adapter identity checks, exact-input overlap rejection against the first 1,000 training rows, deduplication, raw output capture, strict parsing, and no-overwrite diagnostic outputs. This is an exact overlap check, not a guarantee against semantic or template similarity. Only system/user messages go to inference; supplied assistant answers are never sent to the model.

Multi-tool reports record expected and actual ordered tool sequences separately from routing. A valid ACT with only the first required call therefore counts as ACT but not a matching sequence. Exact sequence matching is not argument accuracy or end-to-end task success. Mixed productive/meta-tool outputs remain invalid; no valid subset executes. Refusal reports count the selected decision and preserve the actual reason without requiring target wording. Adequacy remains manual.

No fake or real tool executes in these probes. Their generations use the frozen adapter, greedy decoding and 256 new tokens. This is not an implemented LLM judge.

Completed Model-Free Preflight

Setup artifact	Selected rows	SHA256
Multi-tool setup	18, 34, 43, 174	`f2719d5dfc43f2af8bbebe9ad1e35a980bc2313e593e73d0f9e5dcd9836c2e29`
Refusal setup	9, 13, 17, 28, 44	`ccc635b794b036a89f6b2ca84d3c4c8395666e88dad07f9eda9593ce24e78a59`

Both reports have `generated: false` and no measured prediction summary. They passed identity and exact-overlap checks without loading a model or tokenizer. The reviewed target batches above are not new generated model results.

Owner-Run Commands

Run sequentially from `project-baymax`; do not retrain:

```
.venv\Scripts\python scripts\probe_sft_clarification.py `
  --behavior multi-tool `
  --limit 4 `
  --output tmp\name-first-heldout-multi-tool-4.json

.venv\Scripts\python scripts\probe_sft_clarification.py `
  --behavior refuse `
```

```
  --limit 5 `
  --output tmp\name-first-heldout-refusal-5.json
```

The default config names the unweighted QKVO name-first adapter. Do not pass `--benchmark-json`; this check keeps exact compact training-distribution inputs.

Owner-Run Results

The owner completed both probes. Their saved selection, exact inputs, supplied targets, config, dataset and checkpoint metadata match the corresponding frozen preflight reports. No target answer was supplied as an inference message.

Generated artifact	SHA256
Multi-tool: four cases	`32438114606f1026790843d396b5c62089a3efda620adf3f17d1fab38da4c597`
Refusal: five cases	`0c70f1c5b900fdd084c6d30abaa6c890a4c70a26377391103dadf232409d2fa1`

Recorded observation	Cases
Expected ACT, predicted ACT	4/4
Complete expected ordered tool-name sequence	4/4
Expected REFUSE, predicted REFUSE	5/5
Invalid outputs across these two batches	0/9
Mixed productive/meta-tool outputs	0/9

These are narrow diagnostic counts, not benchmark task success. In particular, the four matching sequences do not establish that all arguments are correct.

Multi-Tool Arguments: Case Review

Validation row	Saved model output	Manual assessment
18	Reads clipboard, then drafts to `maya@example.com` with body `meeting notes attached below` and empty subject.	Requested content and recipient are correct. Subject was not specified and is optional in the exposed schema and fake adapter; empty subject is not an established task failure here.
34	Creates `economics presentation`, then drafts to `sana@example.com`: "I created a task for economics presentation." Subject: `Added task`.	Task title and recipient are correct. Body is a reasonable paraphrase of the supplied facts; it need not exactly copy the target's "I added" wording.
43	Reads clipboard, then drafts the supplied `anatomy notes for Sana` to `sana@example.com`, with subject `Sana anatomy notes`.	Requested content and recipient are correct. Alternate subject is permissible because the user did not specify one.

Validation row	Saved model output	Manual assessment
174	Reads clipboard, then creates a task titled `read clipboard contents`.	Substantive content-grounding error: clipboard state says `statistics problem set`. Tool order and ACT decision are correct, but the title describes the operation instead of using its content.

Three plans appear consistent with their requests on manual review; one has a clear argument-grounding failure. This is not an automatically computed 75% task success score. No tool was executed, and no real action or success claim was independently verified. The optional-subject differences were not used to modify any benchmark criterion.

Refusal Reasons: Case Review

Validation row	Saved reason	Manual assessment
9	"I cannot cancel the entire chemistry planning review because I lack the ability to delete calendar events."	Adequate capability-based explanation in this context: no cancellation/deletion tool is exposed. Does not claim an attempted operation failed.
13	"I cannot search Gmail because no Gmail search tool is available."	Adequate and directly grounded in the available tool list.
17	"I cannot book tickets because no ticketing tool is available."	Adequate and directly grounded in the missing booking capability.
28	"I cannot access Marv's password because I do not have access to that account."	Safe refusal decision, but an access-based explanation does not demonstrate understanding that secret handling is disallowed even if access existed. Narrower rationale than the general policy; not evidence that the model would actually disclose a password under different tools.
44	"I cannot delete calendar events or notion tasks because I do not have access to those tools."	Adequate if "those tools" means deletion tools, which are absent. Naming the missing deletion capability explicitly would be clearer; update tools are present.

Four reasons appear adequate in context; the password refusal's policy rationale is incomplete. These are unblinded coding-agent judgments, not independent human validation, a new semantic metric, or proof of robust safety/generalization. No actual account, permissions check or tool failure was observed by the model.

What This Establishes

The unweighted checkpoint can produce complete multi-tool name sequences and valid grounded refusals on these selected exact-SFT held-out inputs. Its behavior is not explained by a total inability to emit refusal or multi-tool plans. However, content grounding can still fail even when routing and sequence metrics look perfect. The earlier ten-case clarification probe also contains genuine routing, completeness and question-focus errors without keyword scoring.

Do not interpret these small, template-related batches as broad validation accuracy, proof that the dataset is sound, or proof that all benchmark problems are distribution shift. They do not compare the weighted checkpoints, establish why weighting changed behavior, or cover the two-task-record subtype. The retained reference remains unweighted name-first compact; no training change follows automatically from this diagnostic.

Review Criteria

Read each saved raw generation against its own request, state and schemas:

1. Does it pick the correct decision without a mixed meta/productive plan?
2. Does ACT include all requested steps in order, without invented arguments?
3. Does refusal identify the real capability/safety limitation, without invented permissions, system failures or missing information?
4. Does a clarification ask for the actual blocker and all necessary missing fields rather than already-known details? Existing evidence covers ten cases.

Do not convert these held-out rows into training examples or alter their answers because the checkpoint fails. The raw-output review is now complete; no training change was made or selected automatically. Tiny selected batches are not full-validation accuracy or an untouched final benchmark.

Only the diagnostic helper, its tests and local notes were changed. No model run, training-data change, scoring change, commit or push was performed by the coding agent. Local experiment Markdown remains uncommitted.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Training Targets: Missing Fields and State Grounding

Status: Completed bounded training-data audit; no training change made. Date: 2026-10-16.

Question and Method

Before prescribing more epochs, check whether the actual training cap teaches asking for all missing required information and transferring supplied state into tool arguments. Correct targets do not prove that the model learned them; absence or narrow coverage does not by itself prove the cause of a model error.

Use only the frozen training artifact and existing training diagnostics. No validation/test/benchmark example or output was used to pick, construct or rewrite a training target in this audit. No dataset was generated, mixed or changed.

Earlier notes already reviewed 20 clarification targets. This batch extends that review with the next eight clarification rows, file lines 22-29, without exclusions. For ACT, select the first four single-call and first four multi-call BAYMAX rows in file order that contain meaningful state: existing events/tasks/contacts or nonempty clipboard text. Empty state lists alone do not qualify.

The resulting 16-row semantic sample is deterministic and selected by behavior, not random. Its judgments are an unblinded coding-agent review, not an independent human study or an estimated defect rate for the entire dataset. Structural counts below cover the capped BAYMAX pool; semantic conclusions cover only reviewed rows.

Source and Supervision

Dataset: `data/sft/v1/xlam-baymax-target-compatible-v14-act200-anchored-val200-name-first/train.jsonl`. Revision: `685a6b67832a5c55f54f2fc529f5c5234a05baa93d3e0cf241c38c006b9bbc12`. Whole training-file SHA256: `70e043cb05ef2d3dcb1fc5f9a61b6551ea7efc73050b6773b04ab1911c93fac7`.

The saved owner-run component diagnostic, `tmp/name-first-training-loss-components.json`, records this same training-file identity. Its SHA256 is `ac600c6e44d8eaea3a615b8f037c4320c73ed7045bd9f113103ddba1695b1225`. All 16 selected rows retain positive scored-token counts and are not truncated in that report. This is existing evidence, not new tokenizer/model execution.

All 16 selected targets passed the existing strict target-contract validator in a model-free check. Schema validity and retained labels are not proof of semantic quality or successful learning. The same frozen data is used by the 1x, 2x and 3x first-name-weight experiments; none uses the separately built v15 source.

Actual Capped BAYMAX Pool

The first 1,000 raw training rows contain 400 BAYMAX and 600 xLAM examples. The previous tokenized report retains all 400 BAYMAX and 574 xLAM rows. The following counts refer only to BAYMAX targets, not the much larger external ACT pool.

Target behavior	Rows
CLARIFY	100
REFUSE	100
ACT, one call	175
ACT, multiple calls	25

Among these, 54 single-call ACT rows and nine multi-call ACT rows contain nonempty calendar/task/contact state or clipboard text. These structural counts do not prove that every target actually needs or correctly uses its state.

All 25 multi-call BAYMAX targets contain exactly two productive calls, from six ordered tool-name patterns:

Ordered pattern	Training rows
`clipboard.read` -> `notion.create_task`	3
`clipboard.read` -> `gmail.create_draft`	2
`calendar.create_event` -> `notion.create_task`	5
`calendar.create_event` -> `gmail.create_draft`	5
`notion.update_task` -> `gmail.send_email`	4
`notion.create_task` -> `gmail.create_draft`	6

This pool teaches several two-step compositions. It does not provide a BAYMAX example that calls the same productive tool twice or a three-step composition. This is a description of the training pool, not a proposal to copy any benchmark subtype into training. External xLAM examples can contain other compositions; they were not semantically audited here.

Eight Additional Clarification Targets

Training row	Actual blocker	Target question	Assessment
22	No identifiable statistics meeting in calendar state; requested time is 11:00.	"Which statistics meeting should I move?"	Appropriate entity-identification question. Could explain no match was found, but does not re-ask the known time.
23	Draft needs the explicitly requested agreed deadline, which is not supplied.	"What is the agreed deadline for the history presentation?"	Appropriate; asks for the missing fact rather than inventing body content.
24	Event date is February 30; time and duration are absent.	"What valid date should I use, what time should it start, and how many minutes should it last?"	Appropriate; covers the invalid date and both missing required fields.

Training row	Actual blocker	Target question	Assessment
25	Recipient pronoun is unresolved between two contacts; message facts are supplied.	"Who should I send the email to?"	Appropriate; does not select an arbitrary contact.
26	User has not supplied the code snippet to copy.	"Please provide the code snippet to copy."	Appropriate; requests content instead of copying the description of the operation.
27	User explicitly requests specific wording that is absent.	"Please provide the wording you want sent to sol@example.com."	Appropriate; recipient is known, requested wording is missing.
28	Date undecided; title, 11:20 start and 40-minute duration are known.	"Which date should I use for volunteer orientation?"	Appropriate; does not ask again for known time or duration.
29	User has not chosen between two supplied date formats.	"Which date format should I copy: 2026-06-26 or 26/06/2026?"	Appropriate; requests the unresolved choice rather than choosing for the user.

No new incomplete/wrong-focus target defect was found in these eight rows. This does not certify the remaining clarification targets. Earlier reviewed examples also teach asking for time and duration together when both are absent.

Eight State-Bearing ACT Targets

Single Call

Training row	State and requested operation	Target mapping	Assessment
206	One calculus planning review exists; move it to 13:00.	Uses the supplied event ID and `start_time: "13:00"`.	Correct state-bound entity selection; no invented duration or date.
211	One chemistry planning review exists; move it to 13:00.	Uses that event's supplied ID and requested time.	Correct, but closely repeats row 206's request/operation pattern.
214	One programming reading-notes task exists; mark it done.	Uses its supplied task ID and `status: "done"`.	Correct state-bound entity selection.
227	One programming problem-set task exists; mark it done.	Uses its supplied task ID and done status.	Correct, but closely repeats row 214's pattern.

Multiple Calls

Training row	State and requested operation	Target mapping	Assessment
224	Clipboard contains `statistics reading notes`; read it and create a task from it.	Reads clipboard, then sets task title to `statistics reading notes`.	Correct content transfer and complete two-call plan.

Training row	State and requested operation	Target mapping	Assessment
242	Clipboard contains `physics reading notes`; same request wording as row 224.	Reads clipboard, then uses that text as title.	Correct; varies content but not request wording.
246	Clipboard contains `chemistry notes for Nadia`; read and draft to the supplied recipient.	Reads clipboard, then uses that text as body and the supplied recipient.	Correct content transfer; subject is permissible composition.
248	One programming-outline task exists; mark it done and email the supplied recipient.	Updates the supplied task ID, then sends the supplied completion fact.	Correct entity mapping and complete sequence under the existing routine-send policy.

All eight reviewed ACT targets appear consistent with their own requests, state and tool schemas. This is target review, not a model generation or tool execution.

Coverage and Consistency Limits

The three `clipboard.read` -> `notion.create_task` examples are training rows 224, 242 and 285. All say exactly "Read my clipboard and create a task from it." Their clipboard texts differ, and all three targets correctly use the text as the task title. Thus the mapping is taught, but by only three content variations of one instruction template. The two clipboard-to-draft examples similarly vary content/recipient while using the same instruction pattern.

Some clarification target questions also repeat across different inputs/state. For example, six targets ask "Who should I send the email to?" and four ask "What time tonight should I schedule it, and how many minutes should it last?" Shared questions can be correct and useful; they are not automatically duplicate examples or evidence that the model memorized them. Counts alone do not measure semantic diversity.

The known bare-urgency policy error remains in the frozen v14 cap at rows 9, 11 and 81. These requests say to revise a subject "soon" without explicitly asking for a deadline, yet their targets ask for a due date. Under the owner's decision, the tool permits creating an undated task instead. Six other "due soon" requests explicitly request deadlines and are not the same defect.

Generator 15 corrects this generic policy rule, and its separately built source exists locally. The checkpoint's frozen v14 mix was not repointed or rebuilt. Updating generator code or building another source does not retroactively fix the data learned by an existing adapter. See the policy correction note.

The three known bad labels encourage unnecessary clarification, not failure to clarify. They cannot alone explain the checkpoint's ACT bias or all observed grounding/clarification errors. They were not silently removed from this audit's accounting or used to reinterpret historical results.

What This Means for More Training

The checked skills are not absent from training: reviewed examples teach asking for missing required fields and copying supplied state into arguments, and all 16 have nontruncated supervision in the saved diagnostic. The existing selected in-sample clarification probe nevertheless makes wrong decisions on correctly labeled seen inputs; review of coverage alone cannot explain that away.

The data also provides narrow coverage for some compositions and retains a known policy inconsistency. Repeating it for more epochs repeats both its limited patterns and its wrong labels. Therefore this audit does not justify prescribing more epochs as a fix, nor prove that adding data will solve the problem.

Keep the unweighted reference. A defensible future data experiment would use a new versioned artifact, resolve independently established policy contradictions, and test broader expression/state variation at approximately fixed class counts and training settings. Do not author validation/benchmark-specific solutions, combine that with a duration sweep, or claim improvement without a new run. No such experiment was implemented or trained in this batch.

Only local notes were added/linked. No data, source code, configuration, checkpoint, scorer or prior result was changed; nothing was staged or committed.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

Controlled Training-Data Revision: Policy and Grounding Variation

Status: Owner built, validated, trained and evaluated; candidate not promoted. Date: 2026-10-16.

Hypothesis and Scope

The training audit found correct but narrow state-grounding examples and a retained urgency-policy defect. Prepare a separate data-quality experiment before considering more epochs. This is a combined policy/wording/state intervention; an eventual result cannot identify which of those components caused any change.

Only eligible BAYMAX rows inside the existing 1,000-row training cap change. Preserve their positions and broad classes, all REFUSE targets, all xLAM rows, the rest of the training file, and validation/test bytes. No resplitting, new examples from held-out fixtures, oversampling, or new framework is introduced.

The reference is unweighted two-epoch QKVO name-first SFT, config hash `684a76cd39d845b6177bdaee495bb77bcb436406d384baf912f80b8164fb8b8b`. Its parent dataset revision is `685a6b67832a5c55f54f2fc529f5c5234a05baa93d3e0cf241c38c006b9bbc12`. The transform version is `training-grounding-v1`, not a claim that the whole dataset was regenerated by generator 15 or 16.

Previewed Changes

The model-free preview on the actual capped pool applies 49 changed rows:

Generic rule	Rows	Change
Explicit deadline clarification	3	Bare "soon" requests become explicit "due soon" requests, so their CLARIFY target is appropriate.
Missing-fields wording	4	Rephrase calendar requests and questions while retaining both missing time and duration.
Undated-task ACT	10	Replace eligible dated task requests with explicit undated urgency requests; omit due_date without inventing one.
Entity-state variation	27	Preserve the uniquely named target, vary its internal ID and position, and add an unrelated entity. Calendar distractors use a different date to avoid inventing a scheduling conflict.
Clipboard-content binding	5	Vary instruction wording and clipboard values while binding the task title or email body to that state. Task cases also include an unrelated existing task.

One additional undated-task candidate would duplicate an earlier revised input. Its original dated parent is retained unchanged. The change journal records this skip explicitly; provenance counts distinguish applied changes from skipped rows.

The urgency correction does not relabel the three CLARIFY rows into ACT, which would change quotas. It revises their requests to genuinely require a deadline; separate existing ACT slots teach the agreed undated behavior. This is a new training artifact, not a rewrite of evaluation targets or old results.

Task/clipboard variants use independent stock/equipment text and generic phrasing, not held-out scenario solutions. Unspecified draft subjects are omitted where the subject is not required; supplied recipients and content remain grounded.

No claim of sophisticated semantic diversity is made: the bank remains small, and structural row uniqueness is not a measure of conceptual independence.

Fixed Controls

Control	Retained setting
Raw cap	1,000 rows
BAYMAX contribution	400: 200 ACT, 100 CLARIFY, 100 REFUSE
External capped contribution	Same 600 xLAM rows in the same slots
Base model	Qwen/Qwen2.5-1.5B-Instruct
LoRA	r=8, alpha=16, Q/K/V/O, dropout=0.05
Loss weighting	Unweighted, first-name weight 1
Learning rate / epochs / seed	2e-4 / 2 / 1
Laptop settings for a later run	Context 1,024, batch 1, accumulation 16
Formatting / policy prompt	Same compact JSON, name-first targets and system prompt
Validation cap	Same first 200 rows; full validation/test files preserved

Raw counts are preserved, not promised post-tokenization counts. Changed wording and additional state change token exposure. Inspect the actual tokenized artifact before training; do not assume the old 974 usable rows still apply automatically.

Preparation and Integrity

`scripts/prepare_sft_grounding.py` uses a focused training-only derivation in `baymax.models.sft_grounding`. It rejects populated destinations, validates the reference identity, copies its training settings into a newly hashed config, and records source/output file hashes plus every changed row's before/after SFT messages. Parent source identities and split membership remain unchanged; the change report records transformed content rather than misrepresenting parent row hashes as new-content hashes.

Changed targets pass the existing strict contract validator and in-memory fake adapter/state validation. No live tool or external API executes. Full preparation rejects revised inputs that duplicate training inputs or overlap any protected validation/test input. It checks source counts, prompt hashes, source stability, and exact held-out byte preservation before publishing from a staging directory. Exact overlap checks do not prove semantic/template separation.

The preview validates the capped changes and counts only. It creates no dataset, config, model or tokenizer and does not replace the full preparation checks. Tiny fixture tests cover deterministic rules, grounded arguments, class/slot preservation, unchanged held-outs, duplicate/overlap rejection, and no-overwrite behavior. No full project build or training was run by the coding agent.

Owner-Run Next Step

From `project-baymax`:

```
.venv\Scripts\python scripts\prepare_sft_grounding.py

.venv\Scripts\python scripts\validate_sft_dataset.py `
  --dataset-dir data\sft\v1\name-first-grounding-v1
```

Preparation creates these new paths:

- `data/sft/v1/name-first-grounding-v1/`
- `configs/sft/qwen2_5_1_5b_lora_grounding_v1_epochs2.json`
- A future training output named `models/qwen2_5_1_5b_lora_grounding_v1_epochs2/`.

The dataset contains `grounding-changes.json` for before/after review and `grounding-provenance.json` for counts, rules and parent/output identity. Review the generated revision and post-tokenization report before starting training. Do not overwrite old artifacts or treat preview counts as benchmark improvements.

The existing unweighted checkpoint remains the reference. Any later comparison must retain compact inference, generation settings and scoring. The repeatedly inspected 150-case suite is development evidence, not an untouched final test. Local experiment Markdown and generated datasets are not staged or committed.

Owner-Reported Outcome

The owner validated revision `91ce08228633738365461a556668a6883b5faa806b84d969dddd8164114603a1`, with training config hash `f060ddbc1df6182840086ffc4b60f27785390ee2ee41938e2578bea3f3663986`. The 1,024-token artifact retained 974 training rows and 195 validation rows, including all 100 training clarification and 100 refusal rows, with zero exact duplicates. The owner then completed the two-epoch training and compact evaluation.

Run	Successes / 150	Macro-F1	Invalid / 150	Correct ACT / 81	Correct CLARIFY / 40	Correct REFUSE / 29
Unweighted reference	57	0.612	1	66	10	23
Grounding v1	53	0.591	0	66	10	21

Raw artifacts: unweighted and grounding v1. These figures come from the owner's comparison, not an invented estimate or a new comparison run by the coding agent. The revision did not improve the measured result; keep the unweighted model as the development reference. A single seed and four fewer successes do not establish a consistent regression or identify which revision component caused it. The corrections remain in a separate artifact.

The next 2k coverage experiment uses grounding v1 as its corrected 1k parent. Compare against that parent to measure expansion, while also retaining the stronger unweighted reference for context.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.

A Larger Unique Training Pool, Still Two Epochs

Status: Owner built, validated, trained and evaluated; four-epoch follow-up completed. Date: 2026-10-16.

Question

Does a larger pool with additional unique BAYMAX coverage improve on the corrected 1k model without increasing epochs or changing loss weighting?

The owner approved 2,000 raw training rows at two epochs, not 2,000 rows at four epochs. This is a size-and-coverage experiment, not an isolated test of wording diversity. At two epochs, twice as many usable rows means roughly twice the sample exposure and optimizer steps. The cosine schedule also spans the longer run.

Parent and Reference

The immediate parent is grounding v1:

- Config: `configs/sft/qwen2_5_1_5b_lora_grounding_v1_epochs2.json`.
- Dataset: `data/sft/v1/name-first-grounding-v1`.
- Dataset revision: `91ce08228633738365461a556668a6883b5faa806b84d969dddd8164114603a1`.
- Training config hash: `f060ddbc1df6182840086ffc4b60f27785390ee2ee41938e2578bea3f3663986`.

This retains the independently justified corrections, rather than restoring incorrect examples to recover a score. Grounding v1 scored 53/150 successes and 0.591 macro-F1. The unweighted reference remains the selected model at 57/150 and 0.612; do not silently replace it with this parent or the new experiment.

Exact Raw Quantities

Contribution	Parent 1k	Added	Expanded 2k
BAYMAX single productive calls	175	175	350
BAYMAX multi-tool sequences	25	25	50
BAYMAX clarification	100	100	200
BAYMAX refusal	100	100	200
xLAM productive examples	600	600	1,200
Total	1,000	1,000	2,000

The 2k raw mixture is 800 BAYMAX and 1,200 xLAM, with broad routing quantities 1,600 ACT / 200 CLARIFY / 200 REFUSE. These are not post-tokenization counts. The raw 1k parent was checked model-free: 1,000 unique inputs, zero exact duplicates, and the BAYMAX quantities above. The larger artifact still needs owner inspection.

Where the New Examples Come From

`sft_expansion_examples.py` independently authors 100 training-only groups. Each has two ACT examples, one CLARIFY and one REFUSE. Eight families cover:

- Clipboard-to-task content binding, including an empty-clipboard contrast.
- Calendar creation with explicit timing versus missing time and/or duration.
- Calendar updates using a unique state entity versus multiple matching entities.
- Task deadlines: known dates, explicitly undated urgency and unresolved deadlines.
- Task completion using state IDs versus ambiguous same-title tasks.
- Routine email with known/contact-resolved recipients versus missing addresses; indiscriminate bulk outreach is refused.
- Clipboard-to-draft body binding versus missing contents or unavailable tools.
- Draft recipients resolved from state; a send request is refused when only drafts are supported.

Instruction banks contain three to six phrasings per relevant family. Contents, dates, start times, durations, contact state, entity IDs and distractors vary. The pool is still templated: 400 unique inputs are not 400 independent reasoning patterns, and no claim of unrestricted linguistic generalization is made.

All additions are explicitly training-only. No evaluation fixture or held-out target supplies their content. Existing source validation executes only fake in-memory adapters, and strict schema checks validate all generated targets.

Sampling and Separation

`scripts/prepare_sft_expansion.py` derives a new dataset and a freshly hashed config. It does not change the default mixer, its fixed BAYMAX quotas, or any old artifact.

The original first 1,000 training lines are retained byte-for-byte. A new 1,000-row block is shuffled with the existing seed and placed after them. The 600 external additions are the next eligible unique xLAM rows from the parent's training tail; they move into the cap and are removed from that tail, not copied twice.

Retained parent rows keep their source identity and split. Full validation/test files remain byte-identical. New inputs must be distinct from every parent input and from each other. A frozen-prefix input overlapping any held-out input blocks publication; conflicting unused training-tail inputs are excluded and recorded. They cannot be selected as xLAM additions. Exact input checks are not proof of semantic/template independence. No new held-out set is created; inherited split-ratio metadata describes the parent split policy, not a new 70/20/10 split of the expanded data.

Before any tail-overlap exclusions, the full dataset grows by 400 BAYMAX rows, not 1,000, because external rows are relocated. Review `expansion-sources.jsonl` for all authored examples and `expansion-report.json` for source hashes, moved external identities, before/after cap counts, unique inputs and duplicate counts. Populated/overlapping destinations, quota mismatches, insufficient unique data and source drift fail explicitly.

Build Guard Correction

The first owner build stopped with an input overlap in the parent's training tail. The historical splitter/validator compares complete examples, including answers; identical inputs with different answers can therefore remain in different splits. This is not evidence that the separate 150 benchmark scenarios were used for training. A bounded read-only check found zero held-out input matches in the parent's first 2,000 training rows; the rest was not audited by the coding agent.

The corrected preparation preserves the frozen 1k prefix and excludes conflicts only outside it, with source references and held-out locations in `expansion-report.json:excluded_parent_tail` and manifest skipped rows. Source files and historical results are untouched. Full builds still check all inputs. Regression tests reproduce a same-input/different-answer overlap that passes the old validator and prove both safe tail exclusion and strict frozen-prefix failure. The new guard policy is included in the derivation hash.

Use new `coverage-2k-v2` output/config identities; the failed v1 folder contains only an invalid validation report and is not reused. Do not validate a failed build.

Fixed Training Settings

Qwen2.5-1.5B-Instruct; fresh LoRA r=8, alpha=16, Q/K/V/O; dropout 0.05; learning rate 2e-4; two epochs; seed 1; unweighted first-tool loss; same prompt, compact input serialization, name-first targets and generation/scoring settings. Laptop overrides remain context 1,024, batch one and accumulation 16. No adapter is resumed. Validation cap remains the same 200 rows.

Owner-Run Steps

```
.venv\Scripts\python scripts\prepare_sft_expansion.py
if ($LASTEXITCODE -ne 0) { throw "Preparation failed; do not validate or train." }

.venv\Scripts\python scripts\validate_sft_dataset.py `
  --dataset-dir data\sft\v1\name-first-coverage-2k-v2

$config = "configs\sft\qwen2_5_1_5b_lora_coverage_2k_v2_epochs2.json"

.venv\Scripts\python scripts\report_sft_artifact.py `
  --config $config --split train `
  --max-rows 2000 --max-sequence-length 1024

.venv\Scripts\python scripts\report_sft_artifact.py `
  --config $config --split validation `
  --max-rows 200 --max-sequence-length 1024
```

Review those counts and the source examples before training. In particular, both 200-row boundary classes should retain labels, and no duplicates should appear. After review, the intended fresh run is:

```
.venv\Scripts\python scripts\train_sft_lora.py `
  --config configs\sft\qwen2_5_1_5b_lora_coverage_2k_v2_epochs2.json `
```

```
  --local-low-vram `
  --max-train-rows 2000 --max-eval-rows 200 `
  --max-sequence-length 1024
```

The owner reviewed the tokenized artifact before training. The coding agent did not run the full build, tokenizer, training or evaluation. The repeatedly inspected 150-case suite is development evidence, not an untouched final test.

Completed Artifact and Results

Dataset revision: `43892841fad91d9c328f3577e831c5d192242f84ceabe82d18e7deb30d2c6ab4`. Config hash: `e48c75d01c9c14eb6ef1c24d4e8d1ae516fc3bf164a288918517c6f1ce2db524`. The owner build excluded four unused-tail overlaps. The final full dataset has 58,419 converted rows; train 40,821, validation 11,719 and test 5,879.

Tokenized training class	Rows	Trainable tokens
Direct tool call	885	27,313
Multi-tool sequence	667	34,244
Clarification	200	5,789
Refusal	200	5,515

The usable 1,952-row cap contains all 800 BAYMAX rows and 1,152 xLAM rows, with zero exact duplicates. The unchanged validation artifact retains 195 rows, including 30 clarification and 30 refusal examples.

Run	Successes / 150	Macro-F1	Invalid / 150	Argument accuracy	Clarification wording	Refusal wording
Grounding 1k	53	0.591	0	0.826	0.163	0.379
Coverage 2k / 2 epochs	64	0.615	2	0.806	0.163	0.603

The saved two-epoch run is timestamped 2026-10-16T19:43:41Z. Its matrix has expected rows and predicted columns:

Expected	ACT	CLARIFY	REFUSE	Invalid/unknown
ACT	59	15	6	1
CLARIFY	24	12	4	0
REFUSE	3	0	25	1

The observed size/coverage change improves task completion and refusal compared with the corrected 1k parent, but still acts prematurely in 24/40 clarification cases. It does not isolate data diversity from additional training exposure. The four-epoch follow-up changes duration separately.

The Same 2k Pool at Four Epochs

Status: Owner training/evaluation completed; final reported local candidate at project closeout. Date: 2026-10-17.

Question and Controls

The owner requested a fresh four-epoch run on the same expanded 2k dataset. Compare against that dataset's two-epoch run, not the earlier 1k duration trial. The unweighted compact/name-first checkpoint remains the historical development reference. The four-epoch candidate's measured result is recorded below; no statistical significance or untouched-test generalization is established.

Only `num_train_epochs` changes from 2 to 4. Run/config identity, output paths and config hash also change so the two-epoch artifact is not overwritten. Data, split membership, prompt, LoRA, learning rate, seed, loss weighting and laptop overrides stay fixed. This starts from the base model with a fresh LoRA adapter, not from the saved two-epoch adapter. The cosine schedule spans the longer training horizon, so this is not an identical first two epochs followed by extra updates.

Identity

Item	Value
Config	`configs/sft/qwen2_5_1_5b_lora_coverage_2k_v2_epochs4.json`
Config SHA256	`da38eb64730f8db30b1bf1a5d6f5296a805351598ca2057a1f70e6874141f2e5`
Dataset	`data/sft/v1/name-first-coverage-2k-v2`
Dataset revision	`43892841fad91d9c328f3577e831c5d192242f84ceabe82d18e7deb30d2c6ab4`
Base model	`Qwen/Qwen2.5-1.5B-Instruct`
LoRA	r=8, alpha=16, Q/K/V/O, dropout=0.05
Learning rate / epochs / seed	2e-4 / 4 / 1
First-tool loss weighting	1x, unweighted
Laptop settings	Context 1,024, batch one, accumulation 16
Output	`models/qwen2_5_1_5b_lora_coverage_2k_v2_epochs4/adapter`

The owner reported 1,952 usable training rows from a 2,000-row raw cap: 800 BAYMAX, 1,152 xLAM, 200 clarification and 200 refusal rows, with zero exact duplicates. The same validation cap yields 195 usable rows. These are existing artifact counts, not a new four-epoch measurement. At the same usable counts, four epochs

give approximately twice the sample exposure of two epochs. Model quality is measured separately by the saved evaluation below.

Owner Command

```
.venv\Scripts\python scripts\train_sft_lora.py `
  --config configs\sft\qwen2_5_1_5b_lora_coverage_2k_v2_epochs4.json `
  --local-low-vram `
  --max-train-rows 2000 `
  --max-eval-rows 200 `
  --max-sequence-length 1024
```

No dataset rebuild or additional examples are needed. The coding agent did not run training or evaluation. Preserve raw results and compare both durations under the same compact JSON input, greedy decoding, 150-scenario suite and scoring. Report task success, routing macro-F1, invalid outputs and the decision matrix; lower training loss alone is not a model-quality win. This repeatedly inspected suite is development evidence, not an untouched final test.

Saved Evaluation Results

Sources: two epochs and four epochs. The four-epoch result is timestamped 2026-10-17T16:52:19Z. Both use the same 150 scenarios, compact JSON, temperature zero, at most 256 generated tokens and fake adapters. Values below are transcribed from saved artifacts, not re-evaluated.

Measurement	Two epochs	Four epochs
Task successes / 150	64	73
Task success rate	0.427	0.487
Routing macro-F1	0.615	0.681
Invalid outputs / 150	2	0
Tool-call accuracy	0.667	0.753
Argument accuracy	0.806	0.836
Clarification wording accuracy	0.163	0.350
Refusal wording accuracy	0.603	0.690
Conditional decision accuracy	0.649	0.707
Decision coverage	0.987	1.000
Hallucination rate	0.000	0.007
Mean inference latency, ms	2,758	5,652
Reported API cost, USD	0.0000	0.0000

Four epochs, expected rows and predicted columns:

Expected	ACT	CLARIFY	REFUSE	Invalid/unknown
ACT	62	13	6	0
CLARIFY	17	17	6	0
REFUSE	1	1	27	0

The recorded four-epoch run completes nine additional tasks and has better macro-F1 than the two-epoch counterpart. Compared with the historical unweighted 1k reference, it completes 73 versus 57 tasks and records macro-F1 0.681 versus 0.612. This is a useful development result for closing the project, not proof that four epochs always help: the earlier 1k four-epoch trial regressed, and no repeated training seeds were measured here.

Clarification remains incomplete: 17/40 requests that should clarify still cause ACT, and 6/40 cause REFUSE. ACT recall is 62/81, and refusal recall is 27/29. The recorded latency is higher; hardware/background conditions were not controlled well enough to attribute that solely to the training duration. A zero local API cost field does not mean zero hardware, energy or training cost.

The project owner accepted the work as complete on 2026-10-17. This does not imply production readiness, real integrations, a new MLX checkpoint release or an untouched final evaluation. Model weights and generated training data remain local and are not part of the Git closeout.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.